
DATA ANALYST

Série 2 — Volume IV

Visualisation et communication

Graphiques · Tableaux de bord · Python
Storytelling · Présenter à un décideur

Faire comprendre et utiliser les enseignements,
avec une boutique pour fil rouge.

Table des matières

Introduction : un enseignement que personne ne comprend n'a aucune valeur	1
Un résultat juste, trois présentations	1
Explorer ou expliquer	2
Le lecteur et la décision d'abord	3
De l'analyse à la décision : cinq maillons	3
Ne pas tromper	4
Comment lire ce volume	4
Carte du volume, données et environnement	6
Carte du volume	6
Les jeux de données	7
Le fichier des villes	8
Les fichiers de vérité	8
Régénérer les données	8
L'environnement	9
Les captures d'écran : ce que ce livre montre, et ce qu'il ne montre pas	9
Le style visuel du livre	10
Conventions	11
Chapitre 1 : Principes de visualisation	12
Le chemin de ce chapitre	13
Les données du chapitre	13
1.1 Choisir le bon graphique	14
1.1.1 On part de la question, pas du graphique	14
1.1.2 Ce que l'œil sait faire : la hiérarchie des encodages	15
1.1.3 Le même jeu de données, dit de quatre façons	16
1.1.4 Le catalogue : huit graphiques à connaître	17
1.1.5 Ce qu'il vaut mieux éviter, et pourquoi	20
1.1.6 Une méthode en cinq questions	21
1.2 Clarté, simplicité et mise en page	21
1.2.1 Retirer ce qui ne dit rien	21
1.2.2 Ordonner et étiqueter directement	22
1.2.3 Le titre qui dit la conclusion	23
1.2.4 Axes, unités, alignement et hiérarchie visuelle	24
1.2.5 Les petits multiples	25
1.2.6 Penser à la salle : lisibilité en projection	26
1.2.7 Un seul message : redessiner en cinq gestes	26
1.3 ⊕ Théorie des couleurs, accessibilité, systèmes de design pour tableaux de bord	27
1.3.1 Trois familles de palettes	27

1.3.2 Une couleur, un sens (et un seul)	28
1.3.3 Le daltonisme : simuler au lieu de supposer	29
1.3.4 Le contraste se calcule	30
1.3.5 Une page de design pour tous les tableaux de bord	31
1.4 ⊕ Erreurs courantes et graphiques trompeurs	33
1.4.1 L'axe tronqué	33
1.4.2 Les aires proportionnelles au mauvais carré, et la 3D	34
1.4.3 Des échelles différentes qui rendent comparable ce qui ne l'est pas	34
1.4.4 Le double axe	35
1.4.5 La période choisie	36
1.4.6 Les pourcentages sans effectifs	37
1.4.7 La corrélation suggérée	38
1.4.8 Le camembert à neuf parts	38
1.4.9 Le paradoxe de Simpson en image	39
1.4.10 L'échelle logarithmique non signalée	40
1.4.11 Une liste de contrôle avant de publier	41
Bilan du chapitre 1	42
Chapitre 2 : Tableaux de bord	43
Le chemin de ce chapitre	44
Les données du chapitre	44
2.1 Power BI : modèle, visuels, publication	45
2.1.1 Ce que fait un outil de tableaux de bord	45
2.1.2 Le modèle en étoile	46
2.1.3 Mesures et colonnes calculées	48
2.1.4 Les visuels	49
2.1.5 Publier, partager, actualiser	50
2.2 Tableau : feuilles, tableaux de bord, partage	51
2.2.1 Feuilles, marques et canaux visuels	52
2.2.2 Calculs de table et niveaux de détail	53
2.2.3 Du classeur au tableau de bord	55
2.2.4 Deux philosophies, un même chemin	56
2.3 Conception d'un tableau de bord selon les besoins des utilisateurs	57
2.3.1 Partir de la décision, pas des données	57
2.3.2 Trois niveaux : vue d'ensemble, analyse, détail	58
2.3.3 La page de la gérante	59
2.3.4 Disposition et lecture : le test de cinq secondes	61
2.3.5 Valider, documenter, faire adopter	63
2.4 ⊕ Pour aller plus loin : Power BI avancé, DAX, Power Query, sécurité par lignes, déploiement	64
2.4.1 DAX : le langage des mesures	65
2.4.2 Power Query : la préparation, étape par étape	67
2.4.3 La sécurité au niveau des lignes	68
2.4.4 Le déploiement	69
2.5 ⊕ Pour aller plus loin : Looker, Metabase, Superset, Qlik	70
2.5.1 Quatre familles d'outils	71
2.5.2 Choisir sans se laisser guider par la marque	71
2.5.3 Ce que les outils ne font pas à votre place	73
Bilan du chapitre 2	74
Chapitre 3 : Visualisation avec Python	76
Le chemin de ce chapitre	76

Les données du chapitre	77
3.1 matplotlib et seaborn	77
3.1.1 Trois couches, et l'anatomie d'une figure	77
3.1.2 Du graphique par défaut au graphique qui dit quelque chose	79
3.1.3 Lignes : les séries temporelles	80
3.1.4 Histogrammes et nuages de points	81
3.1.5 Petits multiples	83
3.1.6 Le thème maison du livre	84
3.1.7 seaborn : les graphiques statistiques	85
3.1.8 Reproductibilité : une figure est une fonction	87
3.1.9 La même figure en R avec ggplot2	88
3.1.10 ☉ Les autres bibliothèques de graphiques	89
3.2 plotly et graphiques interactifs	90
3.2.1 Deux niveaux : plotly express et graph_objects	90
3.2.2 Survol, zoom, légende : l'interactivité en trois gestes	90
3.2.3 Quand l'interactivité aide, et quand elle gêne	92
3.2.4 plotly ou matplotlib ?	93
3.3 ☉ Pour aller plus loin : tableaux de bord avec Dash, Streamlit et Shiny	94
3.3.1 Le cahier des charges commun	94
3.3.2 Streamlit : un script qui se rejoue	94
3.3.3 Dash : des fonctions de rappel	95
3.3.4 Shiny : un graphe réactif	96
3.3.5 Les trois applications, photographiées	97
3.3.6 Déploiement, secrets, et choix de l'outil	97
3.4 ☉ Pour aller plus loin : cartes et visualisation géospatiale	100
3.4.1 Les données : des villes dans un plan fictif	100
3.4.2 Les cercles proportionnels	101
3.4.3 Une carte colorée (choroplèthe) sur des territoires fictifs	102
3.4.4 Le biais des grandes zones	103
3.4.5 Une carte, ou un tableau ?	103
3.4.6 Projection : pourquoi une carte déforme	104
3.4.7 Les outils réels : à décrire, non exécutés ici	105
Bilan du chapitre 3	105
Chapitre 4 : Storytelling et rédaction de rapports	107
Le chemin de ce chapitre	107
Les données du chapitre	108
4.1 Structurer un récit de données	108
4.1.1 Le rapport qu'on a écrit, et celui qu'on devrait écrire	108
4.1.2 L'arc : contexte, tension, preuves, résolution	109
4.1.3 Commencer par la réponse : la pyramide	110
4.1.4 Un message par figure, un titre qui conclut	111
4.1.5 L'analyse des promotions racontée en quatre figures	112
4.1.6 Le storyboard : décider avant de dessiner	114
4.1.7 Quand un récit devient malhonnête	116
4.2 Rédiger des rapports d'analyse clairs	118
4.2.1 La structure d'un rapport d'analyse	118
4.2.2 Écrire pour un lecteur pressé	119
4.2.3 Les chiffres : arrondir, comparer, nommer	120
4.2.4 Tableaux, figures et légendes	121
4.2.5 Dire l'incertitude et les limites sans perdre le lecteur	122
4.2.6 La relecture : une liste et un outil	123

4.2.7 Un rapport reproductible	124
4.3 ⊕ Pour aller plus loin : synthèses de direction, rapports d'une page et présentations .	124
4.3.1 Le résumé de direction en cinq lignes	125
4.3.2 La page unique	126
4.3.3 La présentation de huit diapositives	126
4.3.4 Anticiper les questions	129
4.3.5 Choisir le format selon le lecteur et le moment	129
4.4 ⊕ Pour aller plus loin : rapports récurrents automatisés	130
4.4.1 Quand automatiser, et quand ne pas le faire	130
4.4.2 Le rapport de la semaine	130
4.4.3 Un texte qui sait se taire	131
4.4.4 Les contrôles avant envoi	133
4.4.5 Les notebooks paramétrés	134
4.4.6 Planifier l'exécution	134
4.4.7 Ce qu'un rapport automatique ne remplace pas	135
Bilan du chapitre 4	135

Chapitre 5 : Présenter à des interlocuteurs non techniques 138

Le chemin de ce chapitre	138
Les données du chapitre	139
5.1 Comprendre son public	139
5.1.1 Qui est dans la salle ?	139
5.1.2 Trois questions avant de préparer quoi que ce soit	140
5.1.3 Un résultat, trois publics	141
5.1.4 Le jargon, traduit	141
5.1.5 La culture des chiffres : points, pour cent, pour mille	142
5.1.6 Rendre le support lisible : daltonisme, contraste, projection	143
5.1.7 Présenter à distance	144
5.2 Présenter résultats et recommandations	144
5.2.1 La réponse d'abord	144
5.2.2 Dix minutes, trois preuves	145
5.2.3 Parler d'incertitude sans perdre la salle	146
5.2.4 Une recommandation qu'on peut exécuter	147
5.2.5 Les questions et les objections	148
5.2.6 Le chiffre qui déplaît	149
5.2.7 L'erreur découverte en séance	149
5.2.8 Supports, répétition, chronométrage, suivi	149
5.3 ⊕ Pour aller plus loin : recueil des besoins, entretiens avec les parties prenantes, formulation de la question métier	151
5.3.1 De « je veux un tableau de bord » à une question	151
5.3.2 L'entretien structuré	152
5.3.3 Cartographier les parties prenantes	152
5.3.4 Critères de succès, périmètre, délais, données	152
5.3.5 La fiche de cadrage	154
5.3.6 Trois pièges du recueil des besoins	155
5.4 ⊕ Pour aller plus loin : compétences transversales	155
5.4.1 Raconter	155
5.4.2 Négocier : priorités, délais, périmètre	156
5.4.3 Collaborer avec les équipes métier	157
5.4.4 Quand deux chiffres divergent	157
5.4.5 Éthique professionnelle	158
5.4.6 Donner et recevoir un retour	158

5.4.7 Votre développement, en une page	158
Bilan du chapitre 5	159
Points clés	160
Les points clés, chapitre par chapitre	161
Six idées qui traversent tout le volume	162
Et maintenant ?	162

Introduction : un enseignement que personne ne comprend n'a aucune valeur

« Une analyse juste que personne ne comprend est une analyse perdue. »

Un résultat juste, trois présentations

Les trois volumes précédents vous ont appris à **trouver** des réponses : lire des données, les préparer, les analyser. Il reste l'étape que beaucoup oublient et qui décide de tout : **faire comprendre**. Un résultat reste sans effet tant que la personne qui doit décider ne l'a pas compris, ne s'en souvient pas ou n'y croit pas.

Un vendredi, la gérante vous écrit :

« Lundi, je décide si nous refaisons la promotion de janvier telle quelle. Qu'est-ce que je dois savoir ? Je n'aurai que dix minutes. »

Vous avez fait l'analyse au volume III : sur les 153 jours de promotion de ces trois ans, la promotion a fait monter les commandes, mais baissé la marge. Reste à **montrer** ce résultat. Voici les quatre chiffres que vous avez en main.

```
tableau = pd.DataFrame({"jours ordinaires": normal[["nb_commandes", "chiffre_affaires",  
↪ "marge"]].mean(), "jours de promotion": promo[["nb_commandes", "chiffre_affaires",  
↪ "marge"]].mean()}).round(1)  
tableau.loc["marge par commande"] = [round(marge_ordre["normal"], 1), round(marge_ordre["promo"], 1)]  
print(tableau)
```

	jours ordinaires	jours de promotion
nb_commandes	32.8	35.4
chiffre_affaires	3338.5	3300.1
marge	1053.9	836.4
marge par commande	32.1	23.6

Ces quatre lignes disent tout, mais **ne disent rien à quelqu'un qui a dix minutes**. Il faut les comparer de tête, deviner lesquelles comptent, et deviner que l'écart de commandes (32,8 contre 35,4) est **sous-estimé** par la comparaison brute (la saison est creuse quand on fait des promotions : voir le volume III, chapitre 3). C'est la **première présentation** : le tableau brut. Le résultat est exact et ne sert à personne.

Deuxième présentation : vous faites un graphique, vite, dans le premier format venu.

Troisième présentation : vous partez de la **décision** (« refaire ou non ? ») et de ce que la gérante doit retenir (« plus de commandes, mais moins de marge »), et vous construisez le graphique et la phrase pour cela.

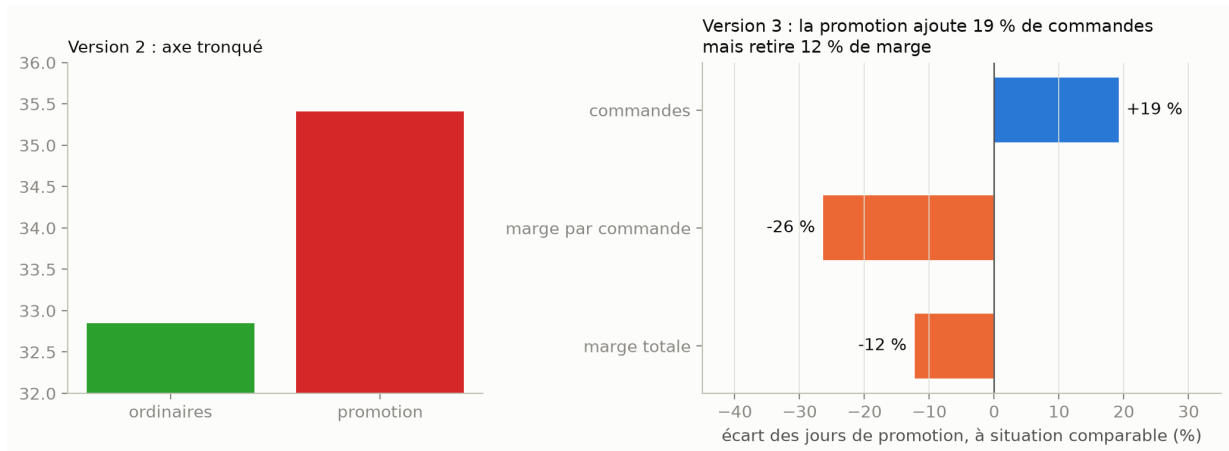


Figure 1: Deux présentations du même résultat. À gauche, des barres dont l'axe commence à 32 commandes : la promotion semble quadrupler les ventes. À droite, la même analyse présentée à partir de la décision, avec un titre qui énonce la conclusion.

Regardez ce que chaque présentation fait faire à la gérante.

Présentation	Ce qu'elle retient	Ce qu'elle décide
1. Le tableau brut	rien de précis : « les chiffres sont proches »	refaire comme chaque année, faute de signal
2. Le graphique à axe tronqué	la barre de la promotion est 4 fois plus haute que l'autre, alors que l'écart réel n'est que de 7,8 %	prolonger la promotion : elle « quadruple les ventes »
3. Le graphique construit pour décider	« plus de commandes, mais moins de marge »	refaire la promotion avec des remises moins fortes , ou la tester

Le résultat est le **même** dans les trois cas ; la décision, non. Le graphique de gauche n'est pas fautif par maladresse seule : un axe qui commence à 32 au lieu de 0 **fabrique** une impression (une promotion qui quadruple les ventes) que les données ne soutiennent pas. Celui de droite ne contient aucun artifice : un zéro au centre, des barres qui partent de ce zéro, des couleurs qui disent le signe de l'écart, un titre qui énonce la conclusion, des valeurs écrites. C'est tout le propos de ce volume : **la visualisation est un argument, et un argument se construit pour un lecteur.**

Explorer ou expliquer

On fait des graphiques pour deux raisons très différentes, et le même graphique ne sert presque jamais les deux.

	Explorer	Expliquer
Pour qui ?	vous	quelqu'un d'autre

	Explorer	Expliquer
Question	« Que contiennent ces données ? »	« Que dois-je retenir, et que décider ? »
Quantité	des dizaines de graphiques, vite faits	un ou deux, longuement soignés
Soin	aucun : des défauts, des axes automatiques	titre-conclusion, couleurs choisies, annotations
Durée de vie	quelques minutes	un rapport, une réunion, un tableau de bord

Les volumes précédents (surtout le volume III, chapitre 1) ont enseigné l'**exploration**. Ce volume enseigne l'**explication** : choisir, **simplifier**, **mettre en page**, puis **raconter**. Le geste le plus important de l'explication est de **retirer** : retirer les graphiques qui ne servent pas la décision, les éléments qui encombrant, les chiffres que personne ne lira.

Le lecteur et la décision d'abord

Avant de tracer quoi que ce soit, on répond à trois questions, dans cet ordre.

1. **Qui lit ?** La gérante n'a pas le même temps, la même culture statistique ni les mêmes inquiétudes que la responsable logistique ou qu'un comptable.
2. **Quelle décision cela doit-il éclairer ?** « Refaire la promotion ? », « changer de transporteur ? », « embaucher ? ». Un graphique sans décision en vue décore.
3. **Quelle est la seule chose à retenir ?** Si elle ne tient pas en une phrase, il y a **deux** graphiques, ou aucun.

La réponse à ces trois questions fixe presque tout le reste : le type de graphique (chapitre 1), l'outil, statique ou interactif (chapitres 2 et 3), la structure du récit (chapitre 4) et la manière de présenter (chapitre 5).

Intuition. Un bon graphique est une phrase dessinée. Si vous ne pouvez pas dire la phrase, vous ne savez pas encore ce que le graphique doit montrer.

De l'analyse à la décision : cinq maillons

Le travail de l'analyste est une chaîne. Un maillon faible suffit à perdre le résultat.

Les trois premiers maillons se préparent seul, devant un écran. Le quatrième, la **présentation**, se joue à plusieurs : on y découvre que la question posée n'était pas tout à fait celle qu'on croyait. D'où l'importance, en amont, du **recueil des besoins** (chapitre 5, section ⑤ 5.3) : une grande partie des analyses ratées ne le sont pas par erreur de calcul, mais parce qu'elles répondent à une question que personne n'avait posée.

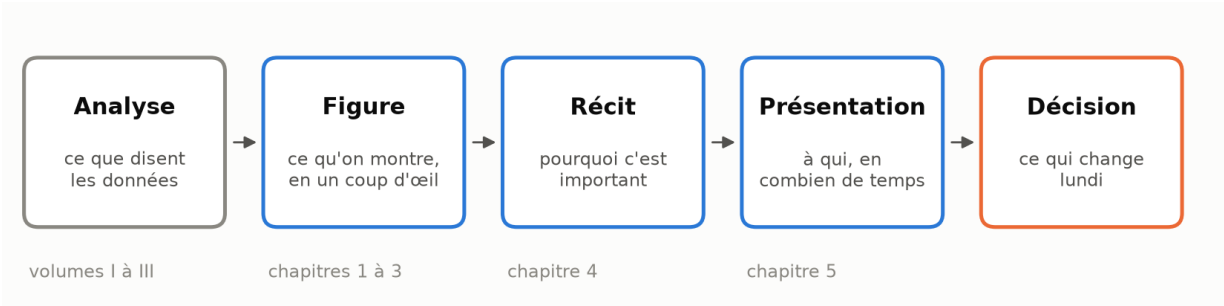


Figure 2: La chaîne de l'analyse à la décision : analyse, figure, récit, présentation, décision. Les volumes précédents couvrent le premier maillon ; ce volume couvre les quatre suivants.

Ne pas tromper

La visualisation a un pouvoir particulier : **un graphique se croit**. Un tableau de chiffres invite à lire ; un graphique donne une impression avant même qu'on le lise. Cette puissance impose trois règles que ce volume applique à chaque figure.

1. **Montrer ce que disent les données, pas ce que l'on souhaiterait.** Un axe tronqué, une échelle choisie pour faire monter une courbe, un titre qui affirme plus que la figure ne montre sont des **choix**, et des choix qui trompent (chapitre 1, section ⊕ 1.4).
2. **Montrer l'incertitude.** Un chiffre sans intervalle ni contexte laisse croire à une précision qui n'existe pas ; le volume III l'a répété, il faut le **dessiner** (barres d'erreur, bandes, fourchettes).
3. **Dire ce que l'on ne sait pas.** Une annotation « hors période de promotion » ou « données incomplètes avant mars » est plus utile qu'une belle courbe muette.

Ces règles ne sont pas morales seulement : elles sont **pratiques**. Un décideur trompé une fois ne fait plus confiance à l'analyste, et la confiance est le capital de ce métier.

Comment lire ce volume

Le volume compte cinq chapitres, tous autour de la même idée : **partir du lecteur et de la décision**.

Chapitre	Question de fond	Ce que vous saurez faire
1. Principes de visualisation	« Quel graphique, et comment le rendre lisible ? »	choisir un type de graphique, simplifier, mettre en page ; ⊕ couleurs et accessibilité, graphiques trompeurs
2. Tableaux de bord	« Comment suivre l'activité sans se noyer ? »	concevoir un tableau de bord pour ses utilisateurs ; situer Power BI, Tableau et leurs équivalents
3. Visualisation avec Python	« Comment produire ces figures de façon reproductible ? »	matplotlib, seaborn, plotly ; ⊕ tableaux de bord Dash, Streamlit, Shiny ; ⊕ cartes
4. Storytelling et rapports	« Comment raconter une analyse ? »	structurer un récit, rédiger un rapport ; ⊕ synthèse d'une page, rapports automatisés

Chapitre	Question de fond	Ce que vous saurez faire
5. Présenter à des non-techniciens	« Comment faire adopter une recommandation ? »	comprendre son public, présenter ; ⊕ recueil des besoins, compétences transversales

Les chapitres se lisent dans l'ordre, les sections ⊕ sont facultatives. Chaque chapitre s'ouvre sur une situation de la gérante et se termine sur un bilan ; les exercices et le projet du volume (un **tableau de bord** et une **courte présentation** pour un décideur) se trouvent dans le **cahier d'exercices**.

Une remarque sur les outils. Ce volume ne vous demande pas de maîtriser tous les outils cités. Les principes (chapitres 1, 4 et 5) valent pour n'importe lequel d'entre eux ; les outils du chapitre 3 sont ceux que le livre exécute. Les outils commerciaux de tableau de bord sont **décrits, non exécutés** (voir « Carte du volume, données et environnement »).

À retenir. Une analyse n'a de valeur que si elle est **comprise** et **utilisée**. On part du **lecteur** et de la **décision**, on construit **un** message, on choisit la forme qui le porte, et l'on ne trompe jamais : ni par l'échelle, ni par le titre, ni par le silence sur l'incertitude.

Carte du volume, données et environnement

Cette section ouvre le volume par cinq choses : la **carte des chapitres**, le **catalogue des jeux de données**, l'**environnement** nécessaire pour refaire les figures, la **politique de captures d'écran** et le **style visuel** du livre. Elle se lit en dix minutes et sert ensuite de référence.

Carte du volume

Chaque chapitre répond à une situation de la gérante. Les cinq chapitres forment le parcours essentiel et se lisent dans l'ordre ; les sections marquées ⊕ sont facultatives.

Chapitre	Situation de départ	Contenu
1. Principes de visualisation	« Ce graphique ne dit rien : refais-le. »	choisir le bon graphique, clarté et mise en page ; ⊕ couleurs, accessibilité, systèmes de design ; ⊕ erreurs courantes et graphiques trompeurs
2. Tableaux de bord	« Je veux voir l'essentiel de la semaine sur une seule page. »	Power BI, Tableau (décrits, non exécutés), conception selon les besoins des utilisateurs ; ⊕ Power BI avancé ; ⊕ Looker, Metabase, Superset, Qlik
3. Visualisation avec Python	« Chaque lundi, je veux les mêmes figures, sans les refaire à la main. »	matplotlib et seaborn, plotly et l'interactif ; ⊕ Dash, Streamlit, Shiny ; ⊕ cartes
4. Storytelling et rapports	« Raconte-moi l'année en deux pages. »	structurer un récit, rédiger un rapport ; ⊕ synthèse de direction, présentations ; ⊕ rapports automatisés
5. Présenter à des non-techniciens	« Tu présentes ça au comité jeudi. »	comprendre son public, présenter résultats et recommandations ; ⊕ recueil des besoins ; ⊕ compétences transversales
Projet du volume (cahier)	« Un tableau de bord et dix minutes pour convaincre »	un tableau de bord et une courte présentation pour un décideur

Les jeux de données

Tout est **simulé**, avec des graines fixes. Le volume reprend **sans les modifier** les jeux du volume III (build/donnees_a3.py, lui-même construit sur la base de la boutique des volumes I et II) et y ajoute un seul fichier, villes.csv, produit par build/donnees_a4.py. Les analyses sont donc **déjà faites** : ici, on les **montre**. La docstring de donnees_a3.py donne la vérité programmée de chaque jeu.

Fichier	Contenu	Lignes	Usage principal (indicatif)
clients.csv	clients (inscription, naissance, ville, canal d'acquisition, carte de fidélité)	6 000	chapitres 1 et 3 (cartes, répartitions)
produits.csv	catalogue (catégorie, prix, coût d'achat, fournisseur)	120	chapitres 1 et 3
commandes.csv	une ligne par commande (date, client, canal, livraison, code promo)	36 395	chapitres 1 à 4
lignes_commande.csv	une ligne par article de commande	83 905	chapitres 1 à 4
retours.csv	lignes retournées	5 002	chapitre 1
jours_exploitation.csv	une ligne par jour : commandes, chiffre d'affaires, météo, promotion, publicité	1 096	chapitres 1 à 4 (séries, promotion)
jours_incidents.csv	la même série avec des incidents injectés	1 097	chapitre 1 (annoter une anomalie)
ab_email.csv, ab_site.csv	deux tests A/B	12 000 ; 38 622	chapitres 1 et 4 (incertitude, résultat non significatif)
sessions_web.csv	sessions du site en 2025 : source, appareil, entonnoir	127 022	chapitres 2 et 3 (tableau de bord marketing)
campagnes.csv	dépenses publicitaires mensuelles par source	36	chapitres 2 et 4
budget_reel_2025.csv	budget et réalisé par mois, catégorie et canal	216	chapitres 2 et 4 (écarts)
benchmark_secteur.csv	médiane et quartiles fictifs d'un secteur	12	chapitres 1 et 5
compte_resultat_mensuel.csv, bilan_annuel.csv	comptes de la boutique	36 ; 3	chapitres 2 et 4
livraisons.csv	livraisons des commandes Site et Réseaux	19 420	chapitres 1, 2 et 5
reapro_fournisseur.csv, stock_quotidien.csv	achats et stocks	1 500 ; 7 300	chapitre 2 (tableau de bord d'exploitation)
employes.csv, employes_annees.csv, departs.csv	ressources humaines (petits effectifs)	64 ; 245 ; 20	chapitre 5 (sujets sensibles)

Fichier	Contenu	Lignes	Usage principal (indicatif)
villes.csv	nouveau : 20 villes fictives, coordonnées dans un plan fictif, région fictive, habitants	20	chapitre 3 (cartes)

Le fichier des villes

villes.csv est le seul jeu nouveau. Il sert aux cartes de la section 3.4, **sans aucune géographie réelle** : les villes sont posées dans un **plan fictif** de 120 km sur 90 km, mesuré par deux coordonnées x_km et y_km.

```
print(tables["villes"].head(4).to_string(index=False))
```

ville	x_km	y_km	region	habitants
Ville A	10.1	4.6	Région 1	43700
Ville B	32.5	20.3	Région 1	170000
Ville C	57.9	7.7	Région 3	11300
Ville D	84.5	7.3	Région 3	17000

Les habitants et les régions sont inventés ; les villes sont les mêmes que celles de la colonne ville des clients. Il n'y a **ni fond de carte du monde réel, ni tuiles téléchargées** : une carte est ici un nuage de points, de cercles proportionnels et de polygones calculés (section 3.4).

Les fichiers de vérité

Les jeux du volume III embarquent leur vérité programmée dans la docstring de donnees_a3.py et, pour les incidents, dans verite_incidents.csv. Ce volume en fait un usage particulier : **un graphique peut être jugé par rapport à ce qu'il aurait dû montrer**. Quand un exemple compare un bon et un mauvais graphique des mêmes données, la vérité sert à dire lequel est fidèle.

Régénérer les données

Les données se régénèrent en quelques secondes (python build/donnees_a4.py) et sont identiques à chaque fois (graines fixes). Il faut le lancer depuis le dossier du volume.

```
python build/donnees_a4.py          # réécrit donnees/*.csv (jeux du volume III + villes.csv)
```

Besoin	Outil	Chapitres
--------	-------	-----------

L'environnement

Le volume utilise les outils des volumes précédents, plus ceux de la visualisation. Voici ce qui sert à quoi.

Besoin	Outil	Chapitres
Manipuler les données	pandas , SQL (SQLite, DuckDB)	tous
Figures statiques	matplotlib (style du livre), seaborn	1, 3, 4
Figures interactives	plotly (HTML), photographié avec un navigateur sans interface	3
Tableaux de bord programmés	Dash , Streamlit (Python), Shiny (R)	3
Graphiques en R	ggplot2 (blocs r, quand utile)	1, 3
Tableur	Excel (non installé ici), LibreOffice Calc pour recalculer des formules	2
Outils de BI commerciaux	Power BI, Tableau, Looker, Qlik : décrits, non exécutés	2
Outils de BI libres	Metabase, Superset : décrits, non exécutés	2

```
from importlib.metadata import version
print({p: version(p) for p in ["pandas", "matplotlib", "seaborn", "plotly", "dash", "streamlit"]})
```

```
{'pandas': '3.0.6', 'matplotlib': '3.11.2', 'seaborn': '0.13.2', 'plotly': '7.1.0', 'dash': '4.4.1',
↵ 'streamlit': '1.65.0'}
```

```
cat(R.version.string, "| ggplot2", as.character(packageVersion("ggplot2")), "| shiny",
↵ as.character(packageVersion("shiny")), "\n")
```

```
R version 4.4.3 (2025-02-28) | ggplot2 3.5.1 | shiny 1.10.0
```

Les versions exactes importent peu : elles sont données pour que les chiffres et les figures du livre soient reproductibles.

Les captures d'écran : ce que ce livre montre, et ce qu'il ne montre pas

Ce livre parle d'outils, mais il ne **reproduit aucune interface de produit commercial**. Voici la règle suivie, parce qu'elle importe pour la confiance que vous pouvez accorder aux figures.

- **Les figures de graphiques** sont produites par le code du livre : ce sont de vrais résultats, reproductibles.
- **Les outils libres que l'on exécute ici** (Dash, Streamlit, Shiny, plotly) sont photographiés pour de vrai, avec un navigateur sans interface : ce sont de **vraies captures** de ce qu'ils affichent.
- **Les outils commerciaux** (Power BI, Tableau, Looker, Qlik) ne sont **pas installés** sur la ma-

chine qui produit le livre et leurs écrans sont protégés : on les **décrit**, on explique leurs notions (modèle de données, visuel, filtre, publication) avec la mention « non exécuté », et l'on montre des **maquettes dessinées** (rectangles génériques, sans logo ni identité de produit) pour situer les zones d'un tableau de bord. Leurs menus changent d'une version à l'autre : à vérifier dans la documentation de votre version.

- **Les images d'Internet** ne sont utilisées que si leur licence autorise la réutilisation, avec crédit et source dans la légende ; ce volume n'en utilise pas dans cette section.

Pourquoi ne pas simplement copier une capture ? Parce que l'on ne peut pas redistribuer librement les écrans d'un produit commercial, et parce qu'une capture vieillit vite. Une maquette honnête et une explication des **concepts** durent plus longtemps que l'aspect d'une interface.

Le style visuel du livre

Toutes les figures du livre partagent un style, défini dans `build/style.py`. Il obéit à quatre choix, qui sont les **principes du chapitre 1 mis en pratique**.

1. **Un fond clair et neutre**, des **grilles discrètes** : le contenu passe avant le décor.
2. **Une palette courte, dans un ordre fixe** (bleu, orange, aqua, violet, rouge) : la même couleur désigne la même chose d'un graphique à l'autre ; on n'en utilise **jamais plus qu'on ne peut nommer**.
3. **Des étiquettes écrites sur les données** plutôt qu'une légende à décoder, chaque fois que c'est possible.
4. **Une police sans empattement** (DejaVu Sans), lisible à toutes les tailles.

La question du contraste est mesurable : le **rapport de contraste** entre une couleur et son fond se calcule (formule de luminance relative des recommandations d'accessibilité du web) et les recommandations demandent au moins **4,5 : 1** pour un texte courant et **3 : 1** pour des éléments graphiques.

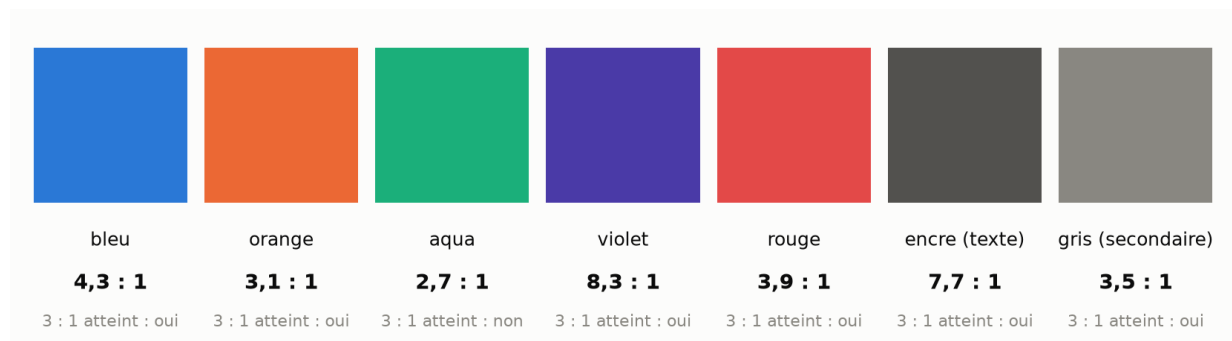


Figure 3: Les couleurs de la palette du livre avec leur rapport de contraste calculé sur le fond. Seul l'aqua n'atteint pas le seuil de 3 : 1 recommandé pour des éléments graphiques.

On lit un résultat **honnête** : le bleu (4,3 : 1), l'orange (3,1 : 1) et le rouge (3,9 : 1) dépassent le seuil de 3 : 1 pour des éléments graphiques, le violet et l'encre le dépassent largement, et le **gris secondaire** (3,5 : 1) convient à des éléments, **pas à un texte courant**. L'**aqua** (2,7 : 1) **n'atteint pas** le seuil : la palette l'utilise pour des surfaces et des courbes épaisses, **toujours doublées d'une étiquette ou d'une forme**, jamais seul. La règle dont dépend l'accessibilité (chapitre 1, section ① 1.3) est précisément celle-là : **ne jamais faire porter l'information par la couleur seule**.

Conventions

- **Monnaie et noms** : les montants sont en euros, les villes s'appellent « Ville A » à « Ville T », les régions « Région 1 » à « Région 4 », les noms de personnes n'existent pas (on parle de « la gérante », « la responsable logistique », par leur fonction) ; la TVA est fictive (20 %).
- **Dates** : on analyse l'année 2025 (et 2023–2024 pour les tendances), la photographie étant prise au 31 décembre 2025.
- **Graines** : toute simulation fixe sa graine, pour que les résultats se reproduisent.
- **Figures** : chaque figure du livre a une légende qui dit **ce qu'il faut y lire**, et un texte alternatif implicite : la phrase qui l'accompagne énonce son message.
- **Vérité programmée** : quand elle existe, elle est révélée **après** l'analyse.

À retenir. Les données sont fictives, propres, et leur vérité est connue : elles servent à apprendre à **montrer** une analyse, pas à la faire. Le livre ne montre que ce qu'il exécute ou qu'il dessine honnêtement, et il applique à ses propres figures les principes qu'il enseigne.

Chapitre 1 : Principes de visualisation

« Un graphique n'est pas une illustration de l'analyse : c'est l'analyse, telle que l'autre va la lire. »

Un lundi, la gérante vous transmet un message d'une ligne, accompagné d'une capture d'écran : « **Je reçois ton graphique et je ne comprends pas ce qu'il faut en conclure.** » Le graphique en question est celui que vous avez envoyé vendredi, avec fierté : la conversion du site selon la source de trafic, six barres de six couleurs, une grille noire, une légende, un titre qui dit « Graphique 1 ». Tous les chiffres sont justes. Et pourtant, une lectrice attentive, qui connaît son métier mieux que vous, n'en tire rien.

Ce n'est pas un problème de données : c'est un problème de **traduction**. Entre le tableau de chiffres que vous avez sous les yeux et la décision que la gérante doit prendre, il y a une personne, et cette personne dispose de quelques secondes. Le graphique est ce qui franchit cet espace. S'il est mal choisi, mal ordonné, trop chargé, trompeur sans le vouloir, le travail d'analyse des trois volumes précédents est perdu à la dernière étape.

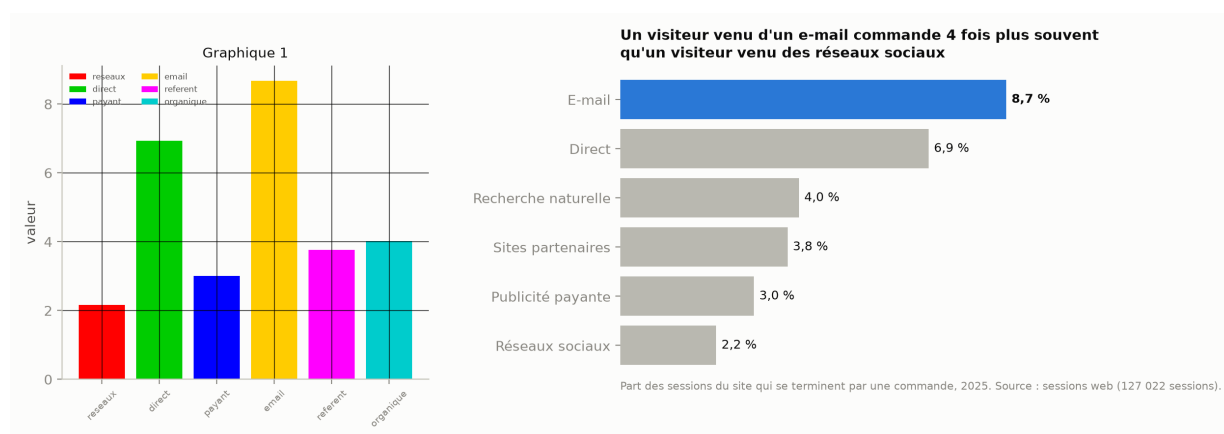


Figure 4: Avant et après : le même tableau de six conversions par source, à gauche tel que l'outil le produit par défaut, à droite après quelques gestes simples (tri, une couleur, étiquettes directes, titre qui énonce la conclusion). Figure construite avec matplotlib à partir des sessions du site 2025 (données simulées).

La figure ci-dessus est le programme de ce chapitre. À gauche, les réglages par défaut ; à droite, les mêmes données après une demi-heure de travail. Rien n'a été ajouté qui ne figure déjà dans le tableau : on a **trié**, **retiré**, **nommé** et **énoncé**. Le message, lui, ne change pas : l'e-mail convertit environ **quatre fois** mieux que les réseaux sociaux (8,7 % contre 2,2 %). Mais à droite, la gérante le **lit** en trois secondes.

Ce chapitre fixe les principes qui rendent cela possible. Il ne parle encore d'aucun outil : les outils de tableaux de bord arrivent au chapitre 2, la programmation des graphiques au chapitre 3. Les principes, eux, valent pour tous.

Le chemin de ce chapitre

- **1.1 Choisir le bon graphique.** On part de la **question** que pose la lectrice, pas du type de graphique que l'on sait faire. Un arbre de décision, la **hiérarchie des encodages visuels** (position, longueur, angle, aire, couleur), les mêmes données dites de quatre façons, le catalogue (barres, courbes, nuages, histogrammes, cascades, cartes thermiques) et ce qu'il vaut mieux éviter (camemberts chargés, doubles axes, radars, 3D).
- **1.2 Clarté, simplicité et mise en page.** Retirer ce qui ne dit rien, ordonner, étiqueter directement, écrire un **titre qui énonce la conclusion**, soigner axes et unités, comparer en **petits multiples**, penser à la salle de projection, et **redessiner** un graphique en cinq gestes, avant et après.
- **1.3 ☉ Théorie des couleurs, accessibilité, systèmes de design.** Trois familles de palettes, la couleur qui a un sens et un seul, le **daltonisme** simulé par le calcul, le **contraste** calculé selon une formule publique, et la **page de design** qui rend cohérents tous les tableaux de bord d'une entreprise.
- **1.4 ☉ Erreurs courantes et graphiques trompeurs.** Dix pièges, chacun avec le graphique qui trompe et sa version corrigée : axe tronqué, aires et 3D, échelles différentes, double axe, période choisie, pourcentages sans effectifs, corrélation suggérée, camembert à neuf parts, paradoxe de Simpson en image, échelle logarithmique non signalée. Pour chacun : **qui est trompé, par quoi, avec quelle conséquence.**

Intuition. Un graphique est un **argument**. Comme tout argument, il a une thèse (ce qu'il faut conclure), des preuves (les données) et une forme (ce qui le rend convaincant, ou trompeur). Choisir un graphique, c'est choisir la forme la plus **honnête** et la plus **rapide à lire** pour une thèse que l'on peut défendre.

Piège. « C'est mon outil qui l'a fait comme ça » n'est pas une justification. Les réglages par défaut d'un logiciel ne connaissent ni votre lectrice, ni votre question, ni la décision qui suit : ils produisent un graphique **possible**, pas un graphique **bon**.

Les données du chapitre

Les exemples viennent de la **boutique** simulée des volumes précédents : commandes et lignes de commande 2023–2025 (chiffre d'affaires de 1 139, 1 189 puis 1 325 k€), jours d'exploitation (publicité, promotions, météo), sessions du site en 2025 (conversion par source), compte de résultat, retours de marchandises, **villes** (vingt villes fictives, de « Ville A » à « Ville T », dans un plan inventé), et la série « avec incidents » (une panne du site, une fermeture exceptionnelle) du volume III. Tout est **simulé** ; la vérité programmée sert ici de juge : un graphique est « fidèle » s'il laisse voir ce qui s'est réellement passé dans les données.

Les personnes sont désignées **par leur fonction** : la gérante, la responsable logistique, le financeur. Les nombres de la prose sont calculés par des blocs exécutés ; les figures sont toutes dessinées avec matplotlib (le code est rangé dans `build/outils_ch01.py` pour ne pas encombrer le texte).

Aucune capture d'un produit commercial n'apparaît dans ce chapitre.

1.1 Choisir le bon graphique

Avant de dessiner quoi que ce soit, il faut savoir **ce que la lectrice cherche** : comparer, suivre une évolution, voir une répartition, repérer une relation, situer un écart. Cette section donne une méthode pour passer de la question au graphique, explique pourquoi certaines formes se lisent mieux que d'autres (c'est de la perception, pas du goût), montre les mêmes données dites de quatre façons, puis passe en revue les graphiques à connaître et ceux qu'il vaut mieux éviter.

1.1.1 On part de la question, pas du graphique

Un analyste débutant ouvre son outil, choisit « graphique » dans le menu, puis regarde ce que l'outil propose. Un analyste expérimenté fait l'inverse : il écrit d'abord, en une phrase, **la question à laquelle le graphique doit répondre**. Le type de graphique en découle presque toujours.

Quelle est ma question ?	Quel graphique ?
Comparer des quantités (catégories, canaux)	→ barres horizontales, triées
Voir une évolution dans le temps	→ courbe (ou barres si peu de points)
Montrer une composition (parts d'un total)	→ barres empilées à 100 %
Voir la distribution d'une variable	→ histogramme, boîte à moustaches
Voir une relation entre deux variables	→ nuage de points (+ courbe de tendance)
Comparer au budget, à une cible	→ barres d'écart ou barres + repère de cible
Expliquer le passage d'un total à un autre	→ cascade (waterfall)
Suivre une conversion étape par étape	→ barres d'entonnoir
Croiser deux catégories (jour x mois)	→ carte thermique (heatmap)
Situer des lieux	→ carte : cercles proportionnels, polygones

Figure 5: Arbre de décision : à chaque question que l'on se pose sur les données correspond une famille de graphiques. Schéma dessiné avec matplotlib.

Lisons l'arbre avec trois questions de la gérante :

- « Quelles catégories pèsent le plus dans le chiffre d'affaires ? » C'est une **comparaison de quantités** entre catégories : des barres horizontales, triées.
- « Est-ce que les ventes remontent ? » C'est une **évolution dans le temps** : une courbe.
- « Est-ce que les jours de forte publicité sont aussi les jours de forte vente ? » C'est une **relation entre deux variables** : un nuage de points.

Deux graphiques ne répondent pas à la même question, même avec les mêmes données. La **première règle** de ce chapitre tient en une ligne : *si vous ne savez pas écrire la question, vous ne savez pas encore quel graphique faire.*

Intuition. Le bon graphique est celui qui rend la **comparaison utile** immédiate. Si la question est « qui est le plus grand ? », le graphique doit placer les grandeurs côte à côte, sur une même ligne de base. Si la question est « comment cela évolue-t-il ? », il doit relier les points dans l'ordre du temps. Tout le reste est décoration.

Avant de continuer, une distinction qui évite beaucoup d'erreurs : **explorer** et **expliquer** ne demandent pas le même graphique. Pour explorer, vous en produisez vingt, rapides et laids, pour vous seul : c'est le travail du volume III. Pour expliquer, vous en choisissez **un**, que vous soignez, parce qu'une autre personne le lira sans vous. Ce chapitre traite de la seconde situation.

1.1.2 Ce que l'œil sait faire : la hiérarchie des encodages

Un graphique transforme des nombres en **marques visuelles** : des positions, des longueurs, des angles, des aires, des couleurs. Mais toutes les marques ne se valent pas : des expériences de psychologie de la perception, répétées depuis plusieurs décennies, montrent que nous comparons très **précisément** des positions le long d'une même échelle, assez bien des longueurs, moins bien des angles, mal des aires, et très mal des intensités de couleur.

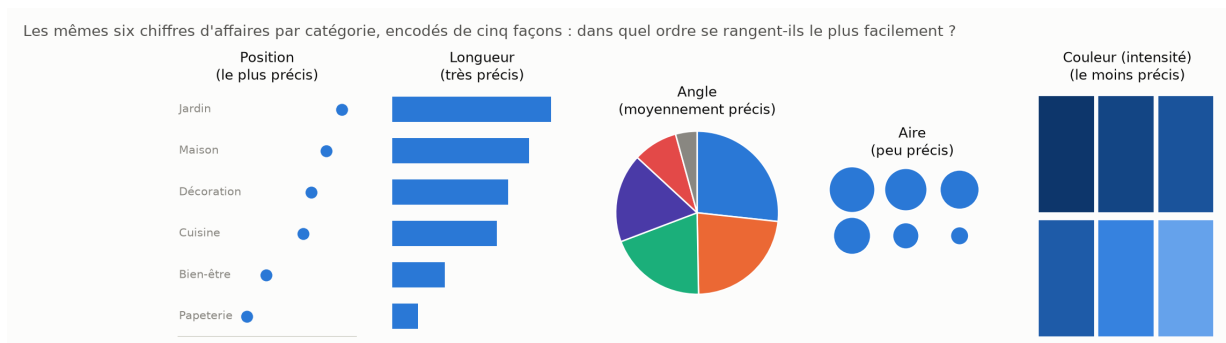


Figure 6: Les six chiffres d'affaires par catégorie encodés par la position, la longueur, l'angle, l'aire et l'intensité de la couleur : la même information devient de moins en moins facile à ranger. Schéma dessiné avec matplotlib à partir du chiffre d'affaires 2025 (données simulées).

Dans la figure, les six mêmes chiffres d'affaires par catégorie (de 354 k€ pour le jardin à 57 k€ pour la papeterie) sont montrés de cinq façons. Essayez de **ranger** les six catégories du plus grand au plus petit dans chaque panneau :

1. **Position** (points sur une échelle commune) : l'ordre se lit sans effort, et l'écart aussi.
2. **Longueur** (barres) : presque aussi précis, à condition que les barres partent du **même zéro**.
3. **Angle** (parts d'un camembert) : l'ordre des grandes parts se devine, celui des parts voisines devient incertain.
4. **Aire** (cercles) : on sait que « c'est plus grand », pas de combien.
5. **Couleur** (intensité) : on distingue clair et foncé, pas six niveaux.

Un petit calcul rend l'écart concret. Un camembert traduit 100 % en 360 degrés : la différence entre deux parts voisines se joue en quelques degrés.

```
angle = F["part_cat"] * 3.6 # 100 % = 360 degrés
print(angle.round(0).head(4).to_string())
print("écart Jardin - Maison :", round(angle.iloc[0] - angle.iloc[1]), "degrés")
```

```
categorie
Jardin      96.0
Maison      83.0
Décoration  70.0
```

Cuisine 63.0
écart Jardin - Maison : 13 degrés

Le jardin pèse 16 % de plus que la maison (354 contre 305 k€) ; sur le camembert, cela fait **13 degrés** d'écart entre deux angles de 96 et 83 degrés. Sur des barres, l'écart de 16 % est celui de deux longueurs, que l'œil mesure directement.

À retenir. Quand on a le choix, on encode l'information importante par la **position** ou la **longueur**, jamais par l'aire ou la couleur seule. La couleur sert à **distinguer** (des catégories) ou à **mettre en valeur** (un élément), pas à **mesurer**.

Cette hiérarchie n'interdit rien : un camembert, une carte avec des cercles ou une carte thermique ont leur place. Elle indique **le prix à payer** : chaque fois que l'on quitte la position ou la longueur, on perd en précision, donc on doit compenser par des **étiquettes chiffrées** ou par un message simple.

1.1.3 Le même jeu de données, dit de quatre façons

Prenons un seul tableau : le chiffre d'affaires mensuel 2025 de chacun des trois canaux, soit trente-six nombres. Quatre graphiques raisonnables existent, et **chacun répond à une question différente**.

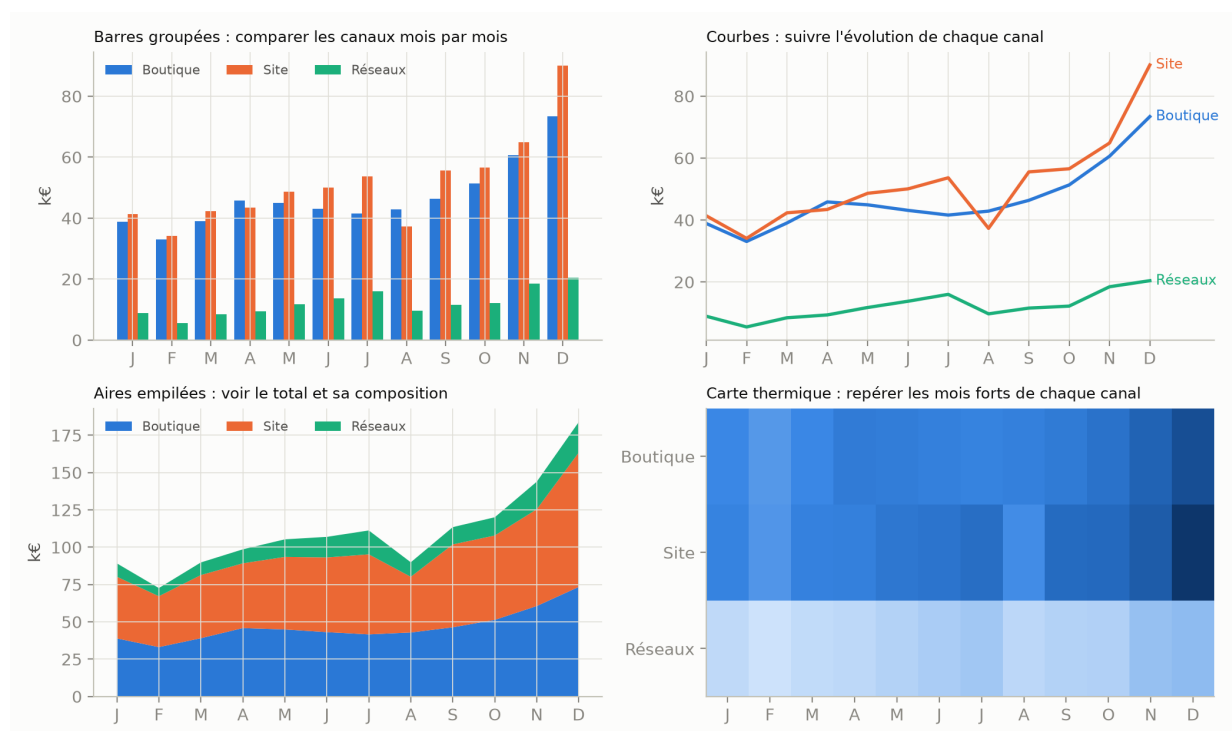


Figure 7: Le chiffre d'affaires mensuel 2025 par canal, dit de quatre façons : barres groupées, courbes, aires empilées, carte thermique. Figure construite avec matplotlib (données simulées).

- **Barres groupées** : « *quel canal est le plus fort, mois par mois ?* » On compare des hauteurs voisines. En décembre, le site vend 90 k€, la boutique 73 k€ et les réseaux 20 k€. Trente-six barres, c'est beaucoup : on lit bien un mois, mal une tendance.
- **Courbes** : « *comment chaque canal évolue-t-il ?* » On suit une forme dans le temps ; le site dépasse la boutique presque tous les mois (sauf en avril et en août), et sa baisse d'août

est plus marquée. Les courbes sont **étiquetées directement** (le nom au bout du trait) : pas besoin de légende.

- **Aires empilées** : « *quel est le total, et de quoi se compose-t-il ?* » Le total de décembre (184 k€) se lit d'un coup. En revanche, seule la couche du bas (la boutique) part du zéro : la forme de la couche du milieu mêle son évolution propre et celle de la couche du dessous. On ne s'en sert pas pour comparer les canaux.
- **Carte thermique** : « *où sont les mois forts de chaque canal ?* » Elle donne un repérage très rapide (décembre, plus foncé), mais **aucun chiffre précis**.

Sur l'année entière, le site représente 46,6 % du chiffre d'affaires (618 k€ sur 1 325 k€), la boutique 42,3 % et les réseaux 11,0 %. Aucun des quatre graphiques ne dit cela d'emblée : c'est une **cinquième question** (la composition de l'année), qui appelle encore une autre forme (une barre unique à 100 %, ou trois barres triées).

Piège. Faire **un seul** graphique qui réponde à toutes les questions. Il n'existe pas : un graphique est bon pour **une** question, et un rapport en contient plusieurs. La section 1.2 y revient avec l'exigence d'un message par figure.

1.1.4 Le catalogue : huit graphiques à connaître

Voici les graphiques de base, regroupés par question, tous dessinés avec les données de la boutique.

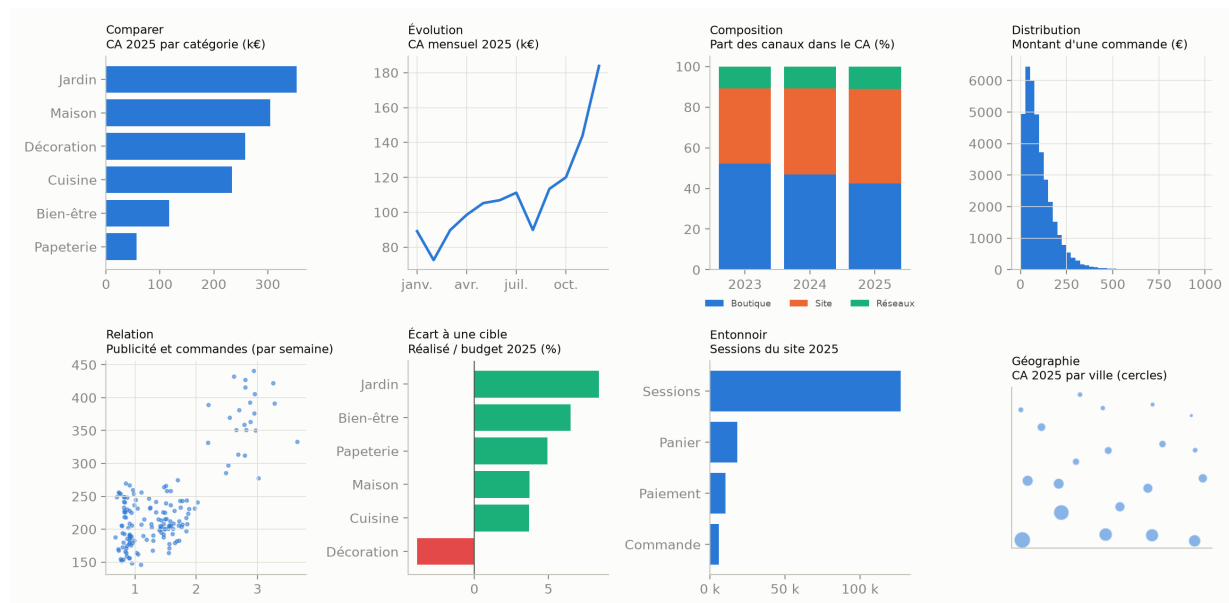


Figure 8: Huit graphiques à connaître, chacun avec sa question : comparer, suivre dans le temps, composer, voir une distribution, relier deux variables, comparer à une cible, suivre un entonnoir, situer des lieux. Figure construite avec matplotlib (données simulées).

Graphique	Il répond à...	À retenir	Piège
Barres (horizontales si les noms sont longs)	Comparer des quantités entre catégories	Toujours partir de zéro ; trier par valeur, sauf ordre naturel (mois, âges)	Axe tronqué (1.4.1) ; trop de barres

Graphique	Il répond à...	À retenir	Piège
Courbe	Suivre une évolution dans le temps	Le temps est en abscisse, régulier ; l'axe peut ne pas partir de zéro si on le dit	Relier des points qui ne se suivent pas ; peu de points
Barres empilées à 100 %	Comparer des compositions	Mettre en bas la part importante ; peu de segments	Plus de quatre segments : on ne compare plus que celui du bas
Histogramme	Voir la forme d'une variable (asymétrie, valeurs extrêmes)	Choisir la largeur des classes, la dire	Largeur arbitraire qui crée ou efface des pics
Boîte à moustaches	Comparer des distributions entre groupes	Médiane, quartiles, valeurs extrêmes visibles	Cache les formes (bimodalité) ; à expliquer à un public non technique
Nuage de points	Voir (et calculer) la relation entre deux variables	Un point par unité ; courbe de tendance en option	La corrélation n'est pas une cause (1.4.7)
Cascade	Expliquer le passage d'un total à un autre	Un seul total de départ, des étapes ordonnées, un total d'arrivée	Étapes trop nombreuses ; aucun repère du total
Carte thermique	Croiser deux catégories (jour × mois)	Palette séquentielle , valeurs écrites dans les cellules	Rampe de couleurs mal choisie (1.3.1)

Deux graphiques de ce catalogue méritent un exemple chiffré : la cascade et la carte thermique, qui sont moins connues des débutants.

La cascade (« waterfall ») explique **comment on passe d'un total à un autre**. Depuis la marge brute de 2025 (417 k€) jusqu'au résultat d'exploitation (40 k€), le trajet est une suite de charges.

On y lit sans calcul que le personnel (141 k€), le marketing (73 k€) et les loyers (65 k€) absorbent à eux trois **67 %** de la marge brute, et que le résultat d'exploitation n'en conserve que **9,6 %**. Un tableau donnerait les mêmes nombres ; la cascade donne en plus **leur poids relatif** et **l'ordre dans lequel ils s'enchaînent**. Son titre, lui, énonce la conclusion (« les charges font le trajet »), ce que nous apprendrons à faire en 1.2.3.

La carte thermique (« heatmap ») croise deux catégories, ici le jour de la semaine et le mois, et représente le nombre de commandes par une intensité de couleur.

Sur les 12 946 commandes de 2025, le samedi en concentre 2 607 (20,1 %) et le dimanche 1 212 (9,4 %). Deux motifs apparaissent d'un coup d'œil : une **colonne** (novembre et décembre, 26 % des commandes de l'année) et une **ligne** (le samedi). La case la plus foncée (le samedi de décembre, 341 commandes) vaut **5,4 fois** la plus claire (le dimanche de février, 63). Écrire les valeurs dans les cases compense la faible précision de la couleur (1.1.2).

En pratique. Pour une carte thermique, une rampe **séquentielle** (du clair au foncé, une seule teinte) convient aux comptages ; une rampe **divergente** (deux teintes de part et d'autre d'un centre neutre) convient aux écarts à une référence (budget, moyenne). La section 1.3.1 montre les deux.

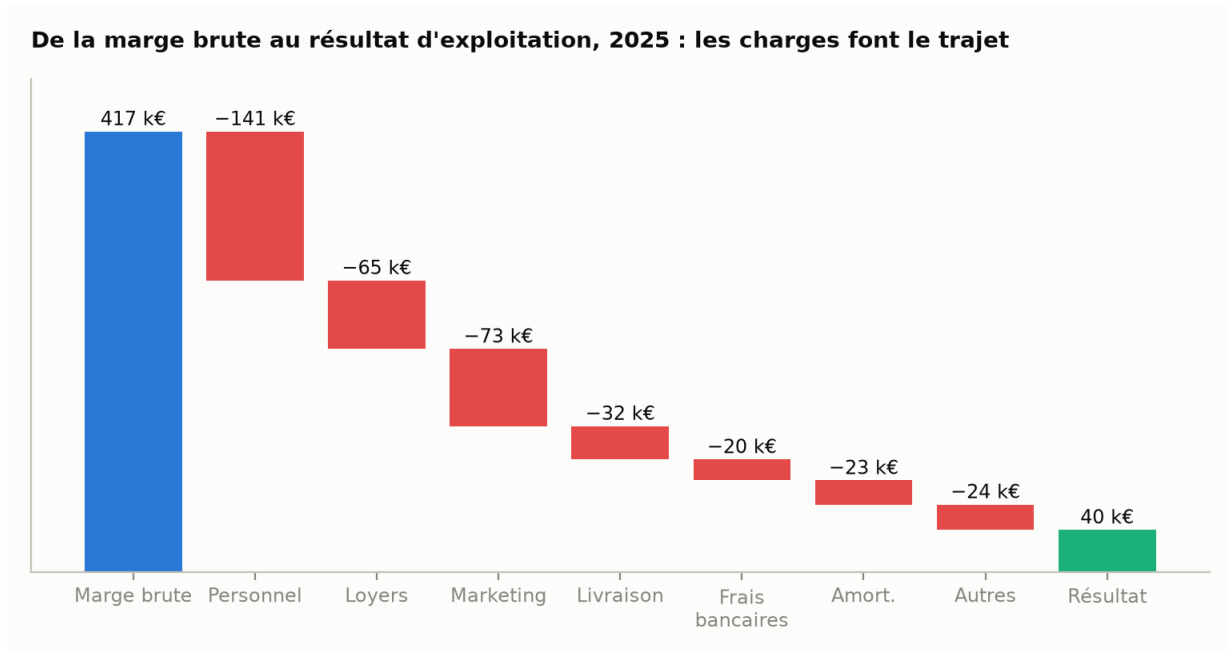


Figure 9: Cascade : de la marge brute de 2025 (417 k€) au résultat d'exploitation (40 k€), les charges retirées une à une. Figure construite avec matplotlib à partir du compte de résultat simulé.

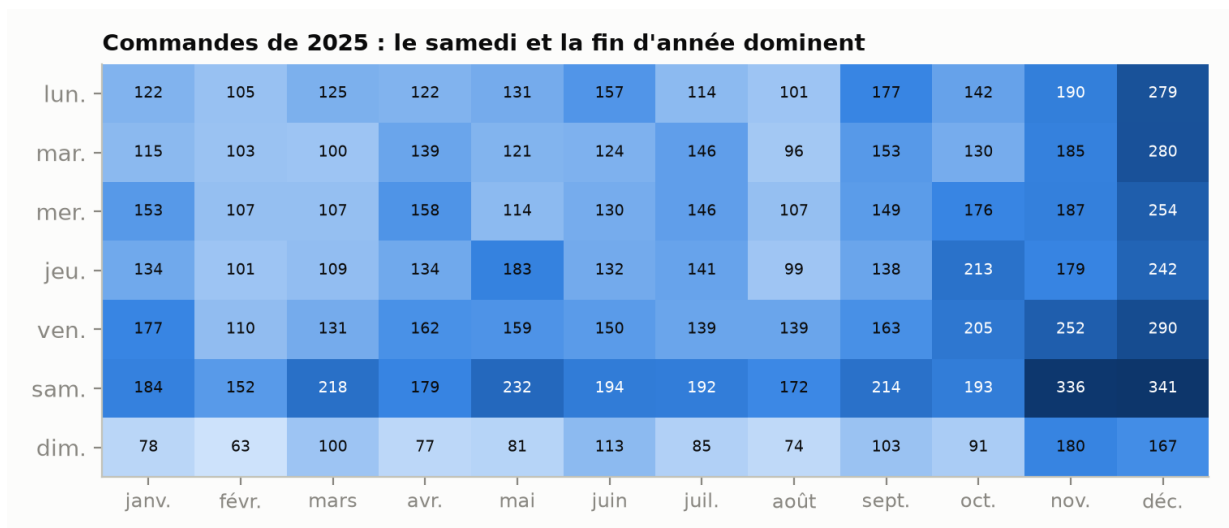


Figure 10: Carte thermique des commandes de 2025 : le jour de la semaine en lignes, le mois en colonnes, le nombre de commandes écrit dans chaque case. Figure construite avec matplotlib (données simulées).

1.1.5 Ce qu'il vaut mieux éviter, et pourquoi

Certains graphiques sont si répandus qu'on les croit neutres. Ils ont tous une alternative qui se lit mieux.

Le camembert. Il convient à **deux ou trois parts**, dont l'une s'approche d'un quart, d'un demi ou des trois quarts (des angles que l'œil repère), et dont la **somme est un tout**. Dès que l'on compare des parts voisines, la barre l'emporte.

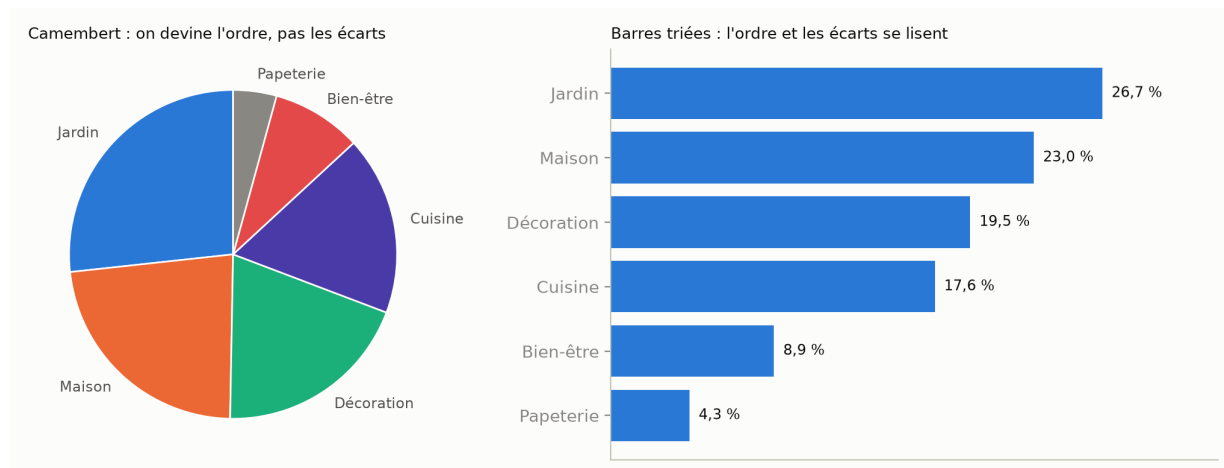


Figure 11: Camembert et barres triées pour les mêmes six parts du chiffre d'affaires 2025 par catégorie (jardin 26,7 %, maison 23,0 %, décoration 19,5 %, cuisine 17,6 %, bien-être 8,9 %, papeterie 4,3 %). Figure construite avec matplotlib (données simulées).

Sur le camembert, on devine que le jardin est la plus grande part et la papeterie la plus petite. On ne **voit pas** que la décoration (19,5 %) dépasse la cuisine (17,6 %) de moins de deux points, ni que le bien-être pèse environ deux fois la papeterie. Sur les barres triées, tout cela se lit sans effort, et les pourcentages écrits à droite évitent toute estimation.

Le double axe. Deux courbes, l'une à l'échelle de gauche, l'autre à l'échelle de droite : la lectrice croit voir que les courbes « se suivent », mais c'est vous qui avez choisi les deux échelles. La section 1.4.4 montre comment on peut faire coïncider presque n'importe quelles deux séries ; à la place, on trace deux graphiques l'un au-dessus de l'autre (même axe du temps), ou un nuage de points.

Le graphique en radar (en toile d'araignée). Il aligne des variables sur des axes rayonnants et relie les valeurs par un polygone. On compare mal des **aires** de polygones, et l'ordre des axes (arbitraire) change la forme. Une série de petites barres, une par variable, répond à la même question avec une précision bien meilleure.

La 3D et les effets d'ombre. Une barre en perspective cache celles qui sont derrière, déforme les hauteurs selon l'angle de vue, et ajoute une **troisième dimension qui ne porte aucune information**. Il n'y a pas de cas où la version 3D lit mieux que la version plane ; elle ajoute seulement du bruit et des occasions de tromper (1.4.2).

Le « donut » et la jauge de voiture. Ce sont des camemberts évidés ou des demi-camemberts : ils ajoutent à la perte de précision de l'angle celle de l'aire, pour économiser de la place. Pour **un seul** indicateur par rapport à une cible, un nombre en grand, accompagné de son écart à la cible, est plus lisible.

Piège. Choisir une forme « parce qu'elle fait moderne » ou « parce que le logiciel la propose en premier ». Si une forme demande une explication pour être lue, elle fait perdre à votre lectrice le temps que vous vouliez lui faire gagner.

1.1.6 Une méthode en cinq questions

Pour choisir, posez ces cinq questions dans l'ordre :

1. **Quelle est la question de la lectrice**, en une phrase ? (Comparer ? Suivre ? Composer ? Relier ? Écarter ?)
2. **Quelle décision suivra** ? Si aucune, il faut peut-être un chiffre, pas un graphique.
3. **Combien de valeurs** ? Deux à quinze : barres. Trente et plus dans le temps : courbe. Des milliers de points : nuage ou histogramme.
4. **Quelle précision faut-il** ? Une comparaison fine demande la position ou la longueur ; un repérage grossier supporte la couleur.
5. **Dans quel contexte sera-t-il lu** ? Un écran de bureau, une salle de projection, une feuille imprimée en noir et blanc ? Cela change la taille, les couleurs et la quantité d'information (1.2.6 et 1.3).

À retenir. La forme découle de la **question**, la précision de la **hiérarchie des encodages**, la sobriété de la **lectrice**. Quand deux formes se valent, prenez celle qui demande le moins d'explication.

Pour s'entraîner. Cahier, chapitre 1 : applications 1.1 et 1.2, exercices 1.1 à 1.3.

1.2 Clarté, simplicité et mise en page

Le bon type de graphique ne suffit pas : un graphique peut être juste et illisible. Cette section donne les gestes qui transforment un graphique « correct » en graphique **clair** : retirer, ordonner, nommer, titrer, aligner, comparer, adapter à la salle. Elle se termine par un redesign complet, étape par étape, de celui que la gérante n'a pas compris.

1.2.1 Retirer ce qui ne dit rien

Chaque élément d'un graphique demande un petit effort à l'œil : un trait, une couleur, une légende, une grille. Quand l'élément **porte de l'information**, l'effort vaut la peine. Quand il n'en porte pas, il fait seulement concurrence aux éléments qui comptent. On appelle parfois cela le **rapport données/encre** : la part de l'encre (des pixels) qui dessine des données, par rapport à celle qui dessine autre chose.

Regardons le graphique de départ, celui que votre outil a produit sans aucun réglage, sur la conversion du site par source de trafic :

Le premier panneau (« le brouillon ») cumule des défauts courants que l'on peut compter :

- **Six couleurs vives** pour six catégories qui n'ont aucune raison d'être distinguées par la couleur (l'axe les nomme déjà).
- Une **légende** qui répète ce que dit l'axe, et qui chevauche la grille.
- Une **grille noire** et épaisse, plus visible que les données.

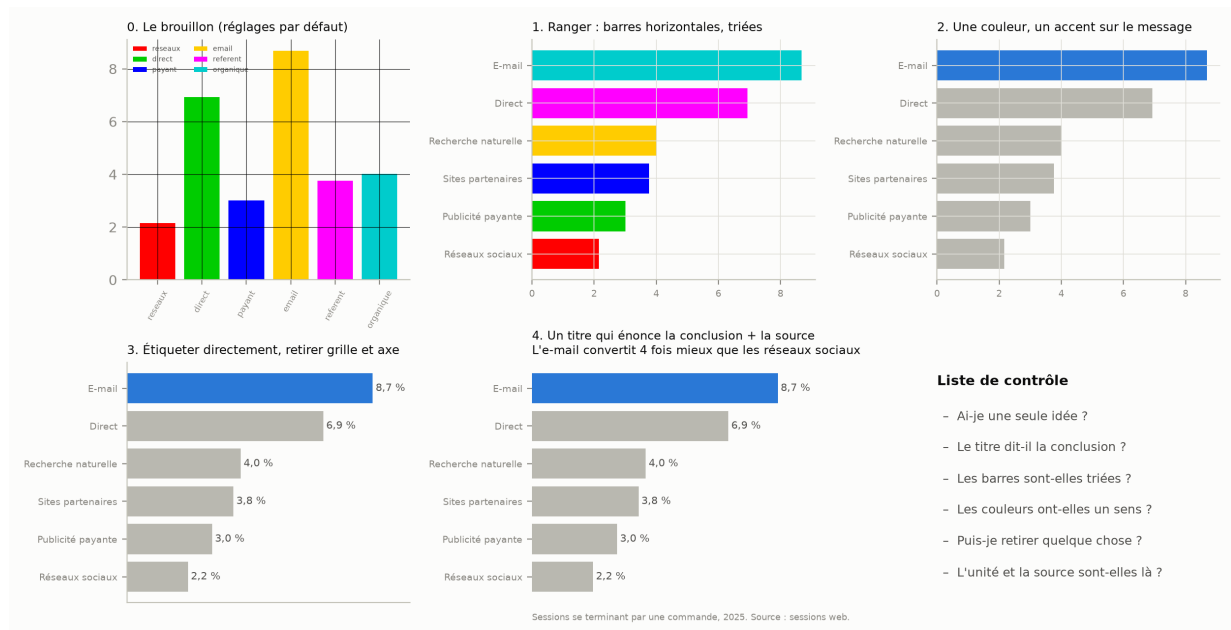


Figure 12: Cinq étapes du redessin d'un même graphique : le brouillon (réglages par défaut), puis le tri, une couleur d'accentuation, l'étiquetage direct sans grille ni axe, enfin le titre qui énonce la conclusion. Figure construite avec matplotlib à partir des sessions web 2025 (données simulées).

- Un axe vertical appelé « valeur », sans unité ; un titre, « Graphique 1 », qui ne dit rien.
- Des noms **inclinés** à 45 degrés, qu'il faut pencher la tête pour lire, et des noms bruts de la base (« referent », « reseaux ») au lieu d'un langage humain.

Pour décider quoi retirer, un test simple : **si je masque cet élément, la lectrice perd-elle quelque chose ?** Si non, on le retire. Le test paraît brutal ; il donne presque toujours un graphique meilleur. La seule exception est ce qui est nécessaire pour **ne pas se tromper** : l'unité, la période, la source, un zéro.

Intuition. Un graphique clair n'est pas un graphique **vide** : c'est un graphique où tout ce qui reste a une raison. La grille, par exemple, n'est pas interdite ; on la garde **très claire** (elle sert à lire une valeur) ou on la remplace par les valeurs écrites sur les barres.

1.2.2 Ordonner et étiqueter directement

Ordonner. Les catégories d'une base de données sont souvent dans l'ordre alphabétique, ou dans l'ordre où elles sont apparues : cet ordre n'a **aucun sens** pour la lectrice. Il faut ranger :

- par **valeur décroissante** (ou croissante) pour une comparaison de catégories : le plus grand en haut d'un graphique horizontal ;
- par **ordre naturel** quand il existe : les mois dans l'ordre du calendrier, les tranches d'âge de la plus jeune à la plus âgée, les étapes d'un entonnoir dans l'ordre du parcours ;
- et seulement en dernier recours, par ordre alphabétique (quand on cherche un nom précis dans une longue liste).

Dans le deuxième panneau de la figure précédente, le simple tri fait apparaître immédiatement le classement : l'e-mail (8,7 %), le direct (6,9 %), la recherche naturelle (4,0 %), les sites partenaires (3,8 %), la publicité payante (3,0 %), les réseaux (2,2 %). On a aussi mis les barres à l'**horizontale**, parce que les noms sont longs : un nom couché se lit sans effort.

Étiqueter directement. Une légende oblige l'œil à faire des allers-retours entre la barre et la clé de couleur. Quand on peut écrire le nom **sur** ou **à côté** de l'élément, on supprime la légende :

- pour des barres, le nom est déjà sur l'axe, et la **valeur** s'écrit au bout de la barre (panneau 3) ;
- pour des courbes, le nom s'écrit **au bout du trait** (nous l'avons fait pour les courbes de la section 1.1.3) ;
- pour un nuage, on étiquette les quelques points qui comptent et on laisse les autres anonymes.

Avec les valeurs écrites, l'axe horizontal et la grille deviennent inutiles : on les retire. Le tableau ci-dessous résume la logique de ces trois premiers gestes.

Geste	Ce que l'on retire	Ce que l'on gagne
Trier	l'ordre arbitraire de la base	le classement lu sans effort
Étiqueter directement	la légende et l'axe des valeurs	des allers-retours en moins
Renommer	les noms bruts (« referent »)	un langage que la lectrice comprend

1.2.3 Le titre qui dit la conclusion

Le titre est la **première chose lue**, et très souvent la seule. Il y a deux façons de s'en servir :

- le titre **descriptif** dit de quoi parle le graphique : « Chiffre d'affaires mensuel 2025 » ;
- le titre **informatif** dit **ce qu'il faut en conclure** : « Le chiffre d'affaires mensuel est multiplié par 2,5 entre février et décembre : novembre et décembre font 25 % de l'année. »

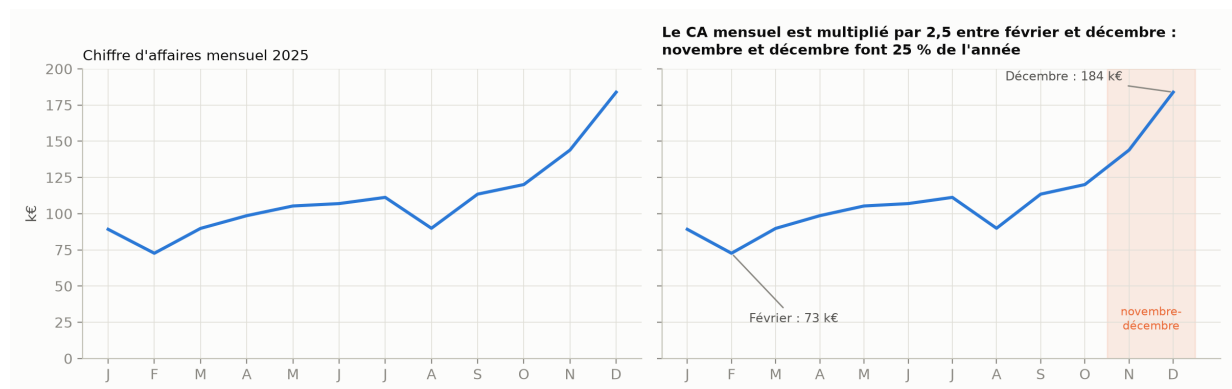


Figure 13: Même courbe, deux titres : à gauche le titre descriptif, à droite le titre informatif, avec deux repères annotés (février et décembre) et la zone novembre-décembre en surbrillance. Figure construite avec matplotlib à partir du chiffre d'affaires mensuel 2025 (données simulées).

Les deux courbes sont identiques. Mais à gauche, la lectrice doit chercher l'information : où est la bosse ? de combien ? pourquoi la montrer ? À droite, on lui dit : de 73 k€ en février à 184 k€ en décembre, soit **2,5 fois** plus, et les deux derniers mois font un quart de l'année (24,7 %). Elle peut être en désaccord, mais elle sait **ce que vous affirmez**.

Une recette pour un titre informatif : **un sujet, un verbe, un chiffre, une comparaison**. « Le chiffre d'affaires (sujet) est multiplié (verbe) par 2,5 (chiffre) entre février et décembre (comparaison). » On met l'information secondaire (période, unité, méthode) dans un **sous-titre** plus petit, en gris, et la **source** en pied de graphique.

Piège. Un titre informatif **s'engage** : il doit être vrai, et le graphique doit le montrer. « Les ventes s'effondrent » sous une baisse de deux pour cent est un mensonge ; « Les ventes baissent de 2 % » est une information. Le titre n'est pas un endroit pour exagérer.

Annoter ce que l'on sait. Un graphique peut aussi porter des **annotations** : une phrase courte, au bon endroit, qui explique un creux ou une bosse. Voici le chiffre d'affaires quotidien de la boutique sur dix semaines de 2025, sans puis avec annotation.

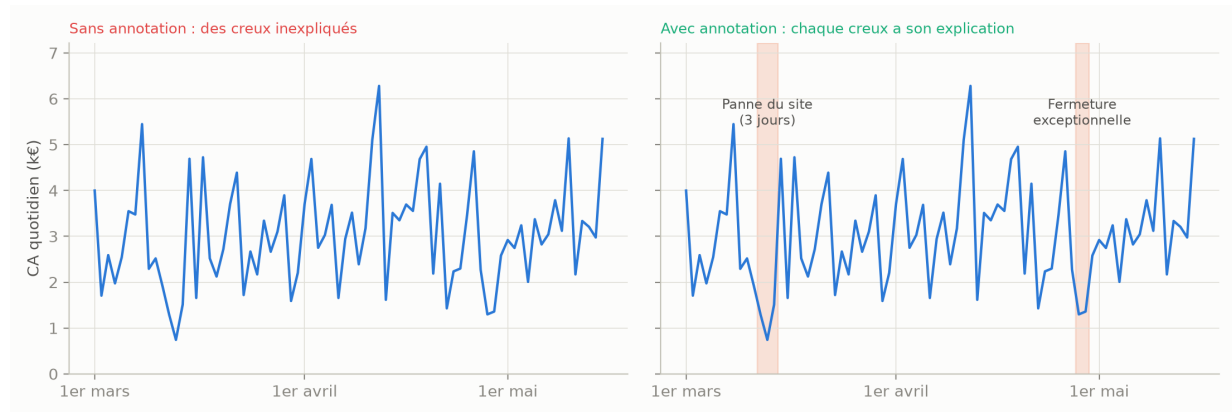


Figure 14: Chiffre d'affaires quotidien du 1er mars au 15 mai 2025 : à gauche sans annotation, avec des creux inexpliqués ; à droite avec les deux événements connus (une panne du site sur trois jours, une fermeture exceptionnelle sur deux jours). Figure construite avec matplotlib à partir de la série « avec incidents » (données simulées).

À gauche, les deux creux interrogent : une erreur de mesure, une vraie baisse ? À droite, un simple bandeau les explique : la **panne du site** de trois jours (1,18 k€ par jour en moyenne, soit 39 % d'un jour ordinaire, dont la médiane est de 3,04 k€) et la **fermeture exceptionnelle** de deux jours (1,33 k€, soit 44 %). La lectrice ne se demande plus ce qui s'est passé ; elle peut passer à la question suivante. Annoter, c'est faire à sa place le travail d'enquête que vous avez déjà fait.

1.2.4 Axes, unités, alignement et hiérarchie visuelle

Quelques règles courtes, qui évitent la plupart des erreurs de lecture.

Les axes.

- Une **barre** commence **toujours** à zéro (la longueur est la mesure). Une **courbe** peut ne pas commencer à zéro, à condition que l'axe soit **lisible** et que l'on ne cherche pas à faire croire à une catastrophe ou à un miracle (1.4.1).
- L'**unité** est écrite : « k€ », « % », « commandes par jour ». Un axe sans unité oblige à deviner.
- Les **graduations** sont peu nombreuses, rondes (0, 50, 100, 150) et à la française : espace pour les milliers, virgule pour les décimales.
- On évite de **couper** un axe au milieu (les « zigzags » qui sautent des valeurs) ; si une donnée dépasse les autres, on la sort dans un second graphique.

Les nombres. On arrondit à ce que la décision demande : « 1 325 k€ » et non « 1 324 763,72 € ». On garde le **même nombre de décimales** dans toute une série (8,7 %, 6,9 %, 4,0 % : pas 8,7 %, 6,9 % et 4 %). Un nombre sans unité ni période n'est pas un résultat.

L'alignement et l'espace. On aligne à **gauche** les titres, les sous-titres et les notes sur une même ligne verticale (celle du bord de l'axe) ; on laisse de l'espace **blanc** entre les blocs plutôt que des traits ; on regroupe ce qui va ensemble (un titre et son sous-titre) et on sépare ce qui est différent.

La hiérarchie visuelle. Dans un graphique, tout n'est pas aussi important. On le dit par la **taille** (le titre plus grand que les graduations), le **poids** (le titre en gras), la **couleur** (une seule teinte vive, le reste en gris) et la **position** (le message en haut à gauche, la source en bas). Une règle utile : on devrait pouvoir **flouter** la figure et distinguer encore le titre, l'élément mis en valeur et le reste.

En pratique. Gardez une **seule couleur d'accentuation** par graphique (le bleu du livre) et mettez tout le contexte en gris. L'œil va tout de suite à ce qui est coloré, donc ce qui est coloré doit être ce que vous voulez dire.

1.2.5 Les petits multiples

Quand on a **plusieurs séries à comparer dans le temps**, la tentation est de tout tracer sur un seul graphique. Avec trois courbes, cela fonctionne ; avec six, cela devient un plat de spaghettis. Les **petits multiples** (ou « petits graphiques juxtaposés ») offrent une alternative : **un petit graphique par série, tous à la même échelle, côte à côte**.

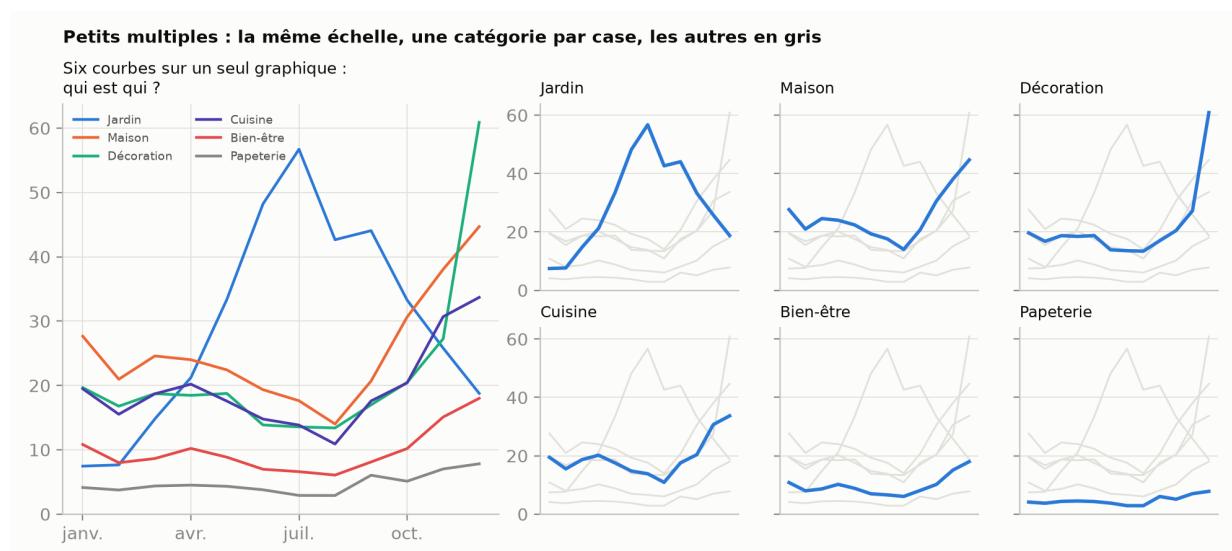


Figure 15: À gauche, six courbes de chiffre d'affaires mensuel par catégorie sur un seul graphique ; à droite, les mêmes courbes en petits multiples, une catégorie par case, avec les cinq autres en gris pour repère. Figure construite avec matplotlib à partir des ventes 2025 (données simulées).

À gauche, on peine à suivre quelle courbe est laquelle, et plusieurs courbes se confondent. À droite, chaque case raconte une histoire simple :

- le **jardin** culmine en juillet (56,7 k€) et redescend ;
- la **décoration** reste entre 13 et 28 k€ de janvier à novembre, puis **plus que double** entre novembre et décembre (60,9 k€, soit 2,2 fois novembre) ;
- la **maison**, la **cuisine**, le **bien-être** et la **papeterie** montent en fin d'année, la papeterie restant sous 8 k€ par mois.

Trois règles pour réussir des petits multiples : la **même échelle** dans toutes les cases (sinon on compare des pentes qui n'ont pas la même unité, voir 1.4.3) ; l'**ordre** des cases significatif (par valeur, par saison, pas par hasard) ; et les **autres séries en gris** dans chaque case, pour que l'on voie comment chacune se situe par rapport aux autres.

1.2.6 Penser à la salle : lisibilité en projection

Un graphique conçu sur un écran de bureau est souvent illisible une fois projeté dans une salle de réunion ou réduit pour entrer dans une diapositive. Deux raisons : la **distance** (on lit de loin) et la **réduction** (la taille des caractères diminue avec celle de la figure).

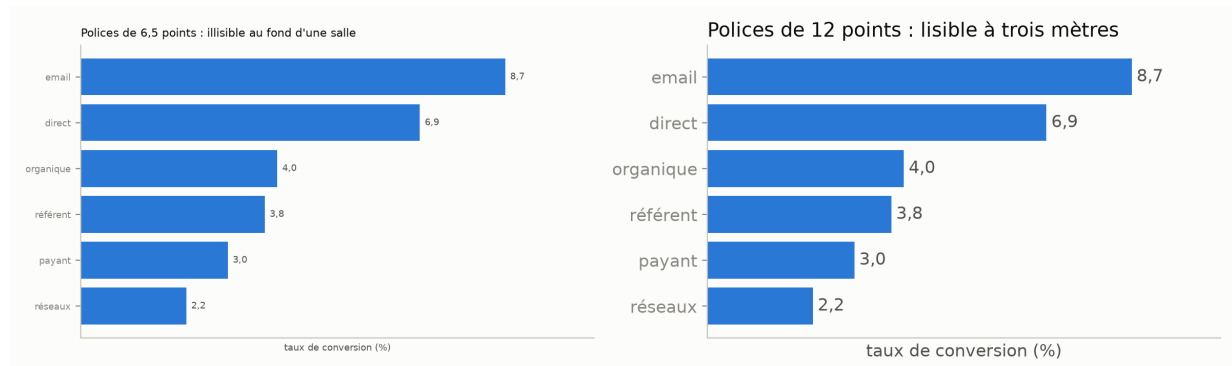


Figure 16: Le même graphique avec des polices de 6,5 points (à gauche) et de 12 points (à droite) : à trois mètres de l'écran, seule la version de droite se lit. Figure construite avec matplotlib à partir de la conversion du site par source (données simulées).

Les repères de ce livre sont les suivants :

- **18 points au minimum** pour tout texte projeté, **24 points ou plus** pour les titres ;
- une figure insérée dans une diapositive est **réduite** : une police de 10 points dans une figure de 11 pouces de large, ramenée à 6 pouces de large, devient une police de **5,5 points**. On dessine donc la figure **à la taille** où elle sera vue, ou l'on augmente les polices ;
- **peu d'éléments** : six barres se lisent à trois mètres, soixante non ;
- des **traits épais** (au moins 2 points) et des **marqueurs gros** ;
- un **fort contraste** avec le fond (section 1.3.4) ; un fond clair et uni se lit mieux qu'un fond dégradé ou photographique ;

Piège. Tester son graphique **à l'écran, de près**. Reculez de trois mètres, ou réduisez-le à la taille d'un timbre-poste : si vous n'y voyez plus que du gris, il faut simplifier.

1.2.7 Un seul message : redessiner en cinq gestes

Le chemin qui mène du brouillon de la section 1.2.1 au graphique final tient en cinq gestes, que l'on peut appliquer à presque tout graphique :

1. **Ranger** : trier, orienter à l'horizontale si les noms sont longs, renommer.
2. **Une couleur, un accent** : tout en gris sauf l'élément du message (ici l'e-mail).
3. **Étiqueter directement** : les valeurs au bout des barres ; retirer légende, grille et axe devenu inutile.
4. **Titrer par la conclusion** : un titre qui énonce le message, une source en pied.
5. **Relire** : le test du flou, le test du « rien à retirer », la lecture par quelqu'un d'autre.

Voici le cœur du dernier état, en quinze lignes : on voit que la qualité vient du **choix des éléments**, pas de la quantité de code.

```
import matplotlib.pyplot as plt
s = F["conv"].sort_values()                                # conversion du site par source, en %
fig, ax = plt.subplots(figsize=(6, 3))
ax.barh(s.index, s.values, color=["#b9b8b0"] * 5 + ["#2a78d6"]) # gris, sauf l'e-mail
```



```

for i, v in enumerate(s.values):
    ax.text(v + 0.1, i, f"{v:.1f} %".replace(".", ","), va="center")
ax.set_title("L'e-mail convertit 4 fois mieux que les réseaux sociaux", loc="left", fontweight="bold")
ax.set_xticks([]); ax.grid(False)
for bord in ("top", "right", "bottom", "left"):
    ax.spines[bord].set_visible(False)
print(len(ax.patches), "barres ; titre :", ax.get_title(loc="left"))
plt.close(fig)

```

6 barres ; titre : L'e-mail convertit 4 fois mieux que les réseaux sociaux

Le sixième geste, **la liste de contrôle**, vaut d'être épinglée au-dessus du bureau :

Question avant d'envoyer	Réponse attendue
Ai-je une seule idée ?	Oui, et je peux la dire en une phrase.
Le titre dit-il la conclusion ?	Oui, avec un chiffre.
Les barres sont-elles triées ?	Oui, ou l'ordre est naturel.
Les couleurs ont-elles un sens ?	Oui, une couleur d'accent, le reste en gris.
Puis-je retirer quelque chose ?	Non : tout ce qui reste sert.
L' unité , la période et la source sont-elles là ?	Oui, en petit, mais lisibles.

À retenir. La clarté n'est pas un talent artistique : c'est une **procédure**. Ranger, accentuer, nommer, titrer, relire. Un graphique clair est un graphique où la lectrice n'a rien à deviner.

Pour s'entraîner. Cahier, chapitre 1 : applications 1.3 à 1.5, exercices 1.4 à 1.6.

1.3 ⊕ Théorie des couleurs, accessibilité, systèmes de design pour tableaux de bord

La couleur est l'outil le plus puissant et le plus mal employé de la visualisation : elle attire l'œil avant tout le reste, et elle ne se lit pas de la même façon pour tout le monde. Cette section pose trois familles de palettes, la règle « une couleur, un sens », la **simulation du daltonisme** et le **calcul du contraste**, puis montre comment réunir ces décisions dans une **page de design** que l'on applique à tous les tableaux de bord de l'entreprise. C'est une section facultative pour qui veut seulement faire un graphique correct, mais indispensable pour qui construit des tableaux de bord que d'autres liront chaque jour.

1.3.1 Trois familles de palettes

On n'utilise pas la même palette pour des **catégories**, pour des **quantités ordonnées** et pour des **écarts autour d'un centre**.

- **Qualitative** (panneau de gauche) : des teintes **bien distinctes**, sans ordre entre elles, pour des **catégories**. Six teintes au plus : au-delà, l'œil ne les distingue plus et la lectrice doit consulter la légende (on regroupe alors, ou on passe en petits multiples, section 1.2.5).
- **Séquentielle** (au milieu) : **une teinte**, du clair au foncé, pour une **quantité ordonnée** (du faible au fort). Les valeurs vont ici de 2,9 k€ (papeterie en juillet) à 60,9 k€ (décoration en décembre). Le foncé signifie « beaucoup », sans exception.

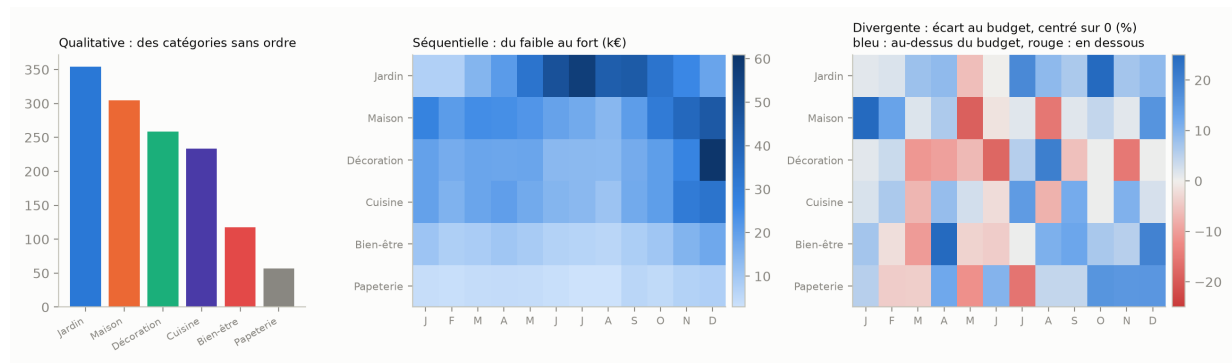


Figure 17: Trois familles de palettes : qualitative (des catégories sans ordre), séquentielle (du faible au fort, ici le chiffre d'affaires 2025 par catégorie et par mois en k€) et divergente (un écart à un budget, centré sur zéro : le bleu est au-dessus du budget, le rouge en dessous). Figure construite avec matplotlib (données simulées).

- **Divergente** (à droite) : deux teintes de part et d'autre d'un **centre neutre**, pour un **écart à une référence**. La référence est ici le budget : sur les soixante-douze cases (six catégories, douze mois), vingt-deux sont **en dessous** du budget (en rouge), les autres au-dessus (en bleu). L'écart extrême (+32,4 %, jardin en octobre) dépasse la limite de ± 25 % de l'échelle : au-delà, toutes les cases ont la même teinte, ce qu'une note devrait signaler.

Trois erreurs classiques à éviter :

- Utiliser une palette **qualitative** (arc-en-ciel) pour une quantité : l'œil lit des **catégories** là où il y a un continuum, et croit voir des frontières nettes là où il n'y en a pas.
- Utiliser une palette **séquentielle** pour un écart : le centre (« conforme au budget ») devient une couleur arbitraire au milieu de l'échelle.
- Placer le **centre** d'une palette divergente ailleurs que sur la valeur neutre : le zéro doit être gris clair, pas bleu pâle.

Intuition. Choisir une palette, c'est répondre à la question : « *qu'est-ce que la couleur doit dire ?* » Des catégories distinctes ? Une intensité ? Un côté du seuil ? La réponse décide de la famille.

1.3.2 Une couleur, un sens (et un seul)

Dans un ensemble de graphiques, une couleur doit toujours **signifier la même chose** : si le bleu désigne le canal « Boutique » dans un graphique, il ne désigne pas « budget » dans le suivant. Sans cette discipline, la lectrice doit relire la légende de chaque figure, ce qui est exactement ce que les couleurs devaient lui épargner.

Quelques conventions simples :

- une **couleur principale** (ici le bleu) pour l'élément du message ; le **gris** pour le contexte ;
- une **couleur d'alerte** (rouge) réservée à ce qui est défavorable, et une **couleur de réussite** (aqua) à ce qui est favorable. On les emploie **avec parcimonie** : un tableau de bord où tout est rouge ou vert ne signale plus rien ;
- jamais de **signification culturelle** supposée universelle : le rouge n'est « mauvais » que dans les contextes où cela a été convenu, et le vert « bon » de même ;
- la **même couleur** pour la même catégorie **dans tout le document** (la boutique est toujours bleue, le site toujours orange).

Piège. « Rouge pour les pertes, vert pour les gains » semble évident. Ce n'est pas lisible par tout le monde (1.3.3), ni toujours exact : une baisse des retours est une **bonne** nouvelle. Dire en mots ce que la couleur signifie (« en dessous du budget », « au-dessus ») vaut mieux que de supposer la lecture.

1.3.3 Le daltonisme : simuler au lieu de supposer

On désigne par « daltonisme » une famille de particularités de la vision des couleurs. La forme la plus fréquente est la difficulté à distinguer le **rouge** et le **vert** (elle touche environ un homme sur douze, et beaucoup moins de femmes). Les deux grands types sont la **protanopie** (absence de la sensibilité au rouge) et la **deutéranopie** (absence de la sensibilité au vert) ; plus rare, la **tritanopie** concerne le bleu et le jaune.

On n'a pas besoin de deviner ce que voient ces personnes : on peut le **calculer**. Chaque type de vision correspond à une **matrice 3 × 3** qui transforme les couleurs d'origine en couleurs perçues. Les matrices utilisées ici ont été publiées en 2009 par Machado et ses collègues ; on convertit d'abord la couleur sRGB en intensités lumineuses linéaires, on multiplie par la matrice, puis on revient à l'sRGB.

```
import numpy as np
from matplotlib.colors import to_rgb
M_DEUT = np.array([[0.367322, 0.860646, -0.227968],      # deutéranopie (sévérité 1,0)
                  [0.280085, 0.672501, 0.047413],
                  [-0.011820, 0.042940, 0.968881]])
def vue_deut(couleur):
    v = np.array(to_rgb(couleur))
    lin = np.where(v <= 0.04045, v / 12.92, ((v + 0.055) / 1.055) ** 2.4)  # sRGB -> linéaire
    o = np.clip(M_DEUT @ lin, 0, 1)
    return np.where(o <= 0.0031308, 12.92 * o, 1.055 * o ** (1 / 2.4) - 0.055)
rouge, vert = "#d62728", "#2ca02c"
dist = lambda a, b: round(float(np.linalg.norm((np.array(a) - np.array(b)) * 255)))
print("distance rouge-vert, vision typique :", dist(to_rgb(rouge), to_rgb(vert)))
print("distance rouge-vert, deutéranopie   :", dist(vue_deut(rouge), vue_deut(vert)))
```

```
distance rouge-vert, vision typique : 209
distance rouge-vert, deutéranopie   : 30
```

La « distance » est ici la longueur du segment entre deux couleurs dans l'espace rouge-vert-bleu (de 0 à 441) : elle n'est qu'un repère grossier, mais l'ordre de grandeur parle. Le rouge et le vert classiques, qui sont à 209 l'un de l'autre pour une vision typique, tombent à 30 en deutéranopie : ils deviennent deux bruns presque indiscernables.

La ligne du haut montre le résultat : en protanopie et en deutéranopie, les cinq barres « rouge-vert » se réduisent à des nuances d'olive et de brun. En niveaux de gris, ce qui arrive aussi à une page imprimée en noir et blanc, le rouge et le vert ont presque la même **luminosité** (rapport de 1,48 seulement) : on ne les distingue plus non plus. La palette du livre, en bas, fait mieux sur les teintes : le bleu reste bleu et l'orange devient un ocre bien distinct en protanopie et en deutéranopie. Elle n'est pas pour autant parfaite : l'orange et le rouge, déjà proches pour une vision typique (distance 38), le deviennent encore davantage en deutéranopie (31), et en niveaux de gris le bleu et le rouge (rapport de luminosité de 1,12), comme l'orange et l'aqua (1,14), sont presque identiques. **C'est pourquoi la palette ne suffit jamais** : les barres portent leur nom, et les séries leur étiquette.

Les règles qui en découlent sont simples :

1. **Ne jamais faire porter l'information par la couleur seule.** Doubler la couleur par une **étiquette** (le nom du canal au bout de la courbe), un **symbole** (▲ ▼), une **forme** de marqueur ou un **motif** (hachures, traits pleins et pointillés).

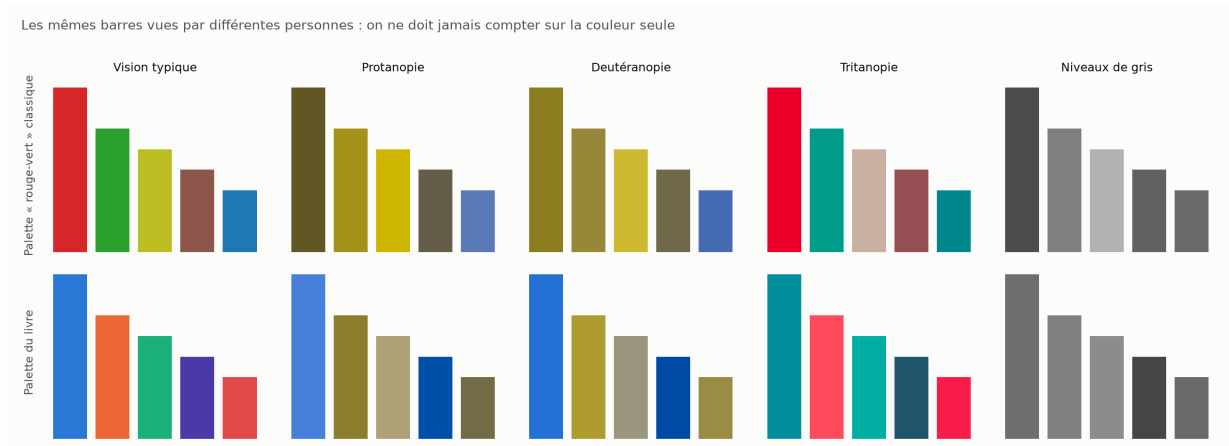


Figure 18: Les mêmes barres vues par différentes personnes. Ligne du haut : la palette « rouge-vert » classique ; ligne du bas : la palette du livre. Colonnes : vision typique, protanopie, deutéranopie, tritanopie, niveaux de gris (impression en noir et blanc). Simulation calculée avec matplotlib.

2. **Varier la luminosité**, pas seulement la teinte : un clair et un foncé se distinguent dans toutes les formes de vision.
3. **Éviter l'association rouge-vert** pour des catégories qui s'opposent. Si la convention du métier l'impose (favorable/défavorable), ajouter le signe (+ et –) ou un mot.
4. **Tester** : simuler les trois formes avant de publier un graphique ou un tableau de bord, comme on relit l'orthographe.

À retenir. Un graphique doit rester lisible **sans la couleur** (en noir et blanc, ou pour une personne qui confond rouge et vert). La couleur renforce l'information, elle n'en est pas le seul porteur.

1.3.4 Le contraste se calcule

Un texte gris clair sur fond blanc fatigue l'œil et devient illisible à la projection. Plutôt que de juger « à l'œil », on mesure le **rapport de contraste** entre deux couleurs, avec la formule publiée dans les recommandations d'accessibilité du web (les « WCAG »). Elle part de la **luminance relative** d'une couleur, qui mesure combien de lumière elle émet, avec un poids différent pour chaque primaire (l'œil est plus sensible au vert qu'au bleu) :

$$L = 0,2126 R + 0,7152 G + 0,0722 B \quad (\text{après conversion en intensités linéaires})$$

Le rapport de contraste entre deux couleurs de luminances $L_1 \geq L_2$ vaut alors $(L_1 + 0,05) / (L_2 + 0,05)$. Il va de **1** (deux couleurs identiques) à **21** (noir sur blanc). Les recommandations demandent au moins **4,5 : 1** pour un texte courant, et **3 : 1** pour un grand texte (18 points ou 14 points en gras) et pour les éléments graphiques utiles à la compréhension (barres, traits, icônes).

```
def lum_relative(c):
    lin = [v / 12.92 if v <= 0.04045 else ((v + 0.055) / 1.055) ** 2.4 for v in c]
    return 0.2126 * lin[0] + 0.7152 * lin[1] + 0.0722 * lin[2]
def rapport_contraste(c1, c2):
    a, b = sorted([lum_relative(c1), lum_relative(c2)], reverse=True)
    return (a + 0.05) / (b + 0.05)
print("noir sur blanc :", round(rapport_contraste((0, 0, 0), (1, 1, 1)), 1))
print("bleu du livre sur blanc :", round(rapport_contraste(to_rgb("#2a78d6"), (1, 1, 1)), 1))
```

noir sur blanc : 21.0
bleu du livre sur blanc : 4.4

Le noir sur blanc atteint le maximum (21 : 1), comme prévu. Le bleu du livre sur blanc donne 4,4 : 1, **juste sous** le seuil de 4,5 pour un texte courant. On peut s'en servir pour des titres, des traits et des barres, pas pour un paragraphe en petits caractères.

Rapport de contraste (seuil de lisibilité usuel : 4,5 : 1 pour un texte courant, 3 : 1 pour un grand texte)

Livraison en retard : 12 commandes	Texte principal	19,2 : 1	AA
Livraison en retard : 12 commandes	Texte secondaire	7,7 : 1	AA
Livraison en retard : 12 commandes	Texte discret (graduations)	3,5 : 1	grand texte
Livraison en retard : 12 commandes	Bleu sur fond clair	4,3 : 1	grand texte
Livraison en retard : 12 commandes	Orange sur fond clair	3,1 : 1	grand texte
Livraison en retard : 12 commandes	Aqua sur fond clair	2,7 : 1	insuffisant
Livraison en retard : 12 commandes	Blanc sur bleu	4,4 : 1	grand texte
Livraison en retard : 12 commandes	Gris clair sur blanc	1,7 : 1	insuffisant

Figure 19: Rapport de contraste de huit combinaisons de couleurs, calculé avec la formule des recommandations d'accessibilité, sur le fond clair du livre : seuls les textes sombres atteignent 4,5 : 1 ; l'aqua (2,7 : 1) et un gris trop clair (1,7 : 1) sont insuffisants. Figure construite avec matplotlib.

La figure applique la formule aux couleurs du livre, sur son fond clair. Le texte principal (19,2 : 1) et le texte secondaire (7,7 : 1) passent largement. Le gris des graduations (3,5 : 1) ne convient qu'à de grands éléments : c'est un choix délibéré, car on veut qu'une graduation se **voie à peine**. Le bleu (4,3 : 1) et l'orange (3,1 : 1) conviennent aux traits et aux grands titres, pas à un texte courant. L'aqua (2,7 : 1) est **insuffisant** pour écrire : on le réserve aux surfaces (une barre, une aire), toujours **doublées d'une étiquette**. Enfin, un gris très clair sur blanc (1,7 : 1) est un classique du « texte discret » que personne ne lit.

En pratique. Les chiffres importants s'écrivent en **encre** (noir ou gris très foncé), jamais en couleur claire. La couleur sert à désigner un élément, pas à écrire sur lui. Et pour un texte écrit **sur** une couleur (un chiffre dans une barre), on calcule le contraste dans les deux sens : blanc sur le bleu du livre donne 4,4 : 1, juste acceptable pour un grand texte.

1.3.5 Une page de design pour tous les tableaux de bord

Quand une entreprise produit des dizaines de tableaux de bord, il arrive que chacun ait ses couleurs, ses polices, sa façon de nommer « chiffre d'affaires » (hors taxe ? toutes taxes comprises ?). La lectrice qui passe de l'un à l'autre doit **réapprendre** à lire à chaque fois. La réponse tient en une page : le **système de design** (ou « charte graphique des données »), qui fixe une fois pour toutes les choix de cette section.

La page de la figure tient en cinq rubriques :

1. **Palette** : cinq couleurs plus un gris, chacune avec son **rôle** (principal, à regarder, favorable, secondaire, défavorable, contexte).



Figure 20: Une page de règles pour les tableaux de bord de la boutique : palette (avec le sens de chaque couleur), typographie, grille, composants (une carte d'indicateur, une carte de graphique) et conventions. Maquette dessinée avec matplotlib ; elle ne reproduit l'interface d'aucun outil.

- 2. Typographie** : trois ou quatre tailles, avec leur usage (titre de page, titre de graphique qui énonce la conclusion, texte, source).
- 3. Grille et espacements** : une marge, une gouttière, un nombre de colonnes, un plafond de visuels par page (six, ici).
- 4. Composants** : la forme d'une carte d'indicateur (ci-dessus : « Chiffre d'affaires, 2025 : 1 325 k€, +11,4 % sur 2024 ») et celle d'une carte de graphique.
- 5. Conventions** : « un titre = une conclusion », « toujours la période, l'unité, la source », « jamais la couleur seule », des **noms** sans ambiguïté (« CA hors taxe », jamais « CA » seul), le format des nombres.

Trois bénéfices, du plus évident au plus sous-estimé : la **cohérence** (la lectrice reconnaît tout de suite ce qu'elle voit), la **vitesse** (on ne rediscute pas des couleurs à chaque nouveau tableau de bord), et la **qualité** (les décisions d'accessibilité sont prises **une fois**, par quelqu'un qui a le temps de les tester, plutôt que dix fois, par quelqu'un qui est pressé).

En pratique. Conservez la page de design à côté du code ou du fichier du tableau de bord, avec un numéro de version, et relisez-la avant chaque nouveau tableau de bord. Les outils de tableaux de bord (chapitre 2) permettent d'enregistrer une palette et un thème pour les réutiliser ; les menus changent d'une version à l'autre : à vérifier dans la documentation de votre outil.

À retenir. Trois familles de palettes pour trois usages ; une couleur, un sens ; la couleur n'est **jamais** seule ; le contraste se **calcule** ; et tout cela se range dans une **page de design** que l'on applique partout.

Pour s'entraîner. Cahier, chapitre 1 : applications 1.6 et 1.7, exercices 1.7 et 1.8.

1.4 ⊕ Erreurs courantes et graphiques trompeurs

Un graphique peut tromper sans qu'aucun chiffre soit faux. Il suffit d'un axe qui ne part pas de zéro, d'une période bien choisie ou de deux échelles ajustées pour que la lectrice tire une conclusion que les données ne soutiennent pas. Cette section présente **dix pièges**. Chacun est montré par un graphique qui trompe, puis par sa version corrigée ; et pour chacun, nous répondons à trois questions, parce que c'est ainsi que l'on mesure la gravité d'une erreur : **qui est trompé, par quoi, avec quelle conséquence**.

Une précision avant de commencer : la plupart de ces pièges sont **involontaires**. On les commet parce que l'outil le propose par défaut, parce que le graphique « avait l'air mieux » ainsi, ou parce que l'on n'a pas vu ce que la lectrice verrait. Mais l'effet est le même, et la responsabilité de la personne qui publie aussi.

Règle d'honnêteté. Si un graphique vous plaît **parce qu'il appuie votre message**, relisez-le comme si vous cherchiez à le contredire. Les trois questions de cette section (qui, quoi, quelle conséquence) servent à cela.

1.4.1 L'axe tronqué

Le piège. Une barre représente une valeur par sa **longueur**. Si l'axe ne part pas de zéro, la longueur ne représente plus la valeur : elle représente l'**écart** à la valeur où l'on a coupé. Les chiffres d'affaires de 2023, 2024 et 2025 (1 139, 1 189 et 1 325 k€) en donnent un exemple.

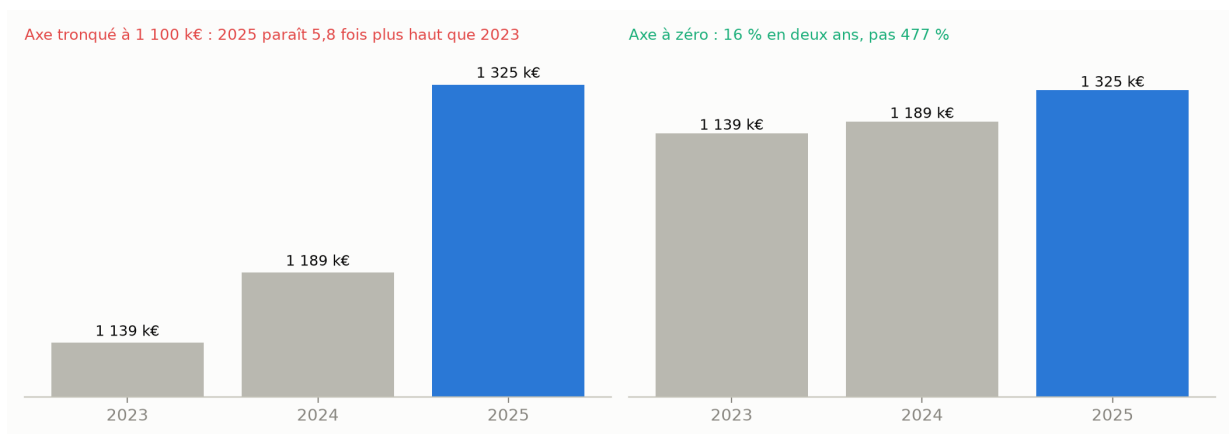


Figure 21: À gauche, les trois chiffres d'affaires annuels sur des barres dont l'axe est coupé à 1 100 k€ : 2025 paraît presque six fois plus haut que 2023. À droite, les mêmes barres avec un axe à zéro : +16 % en deux ans. Figure construite avec matplotlib (données simulées).

```
t = F["t"]
visuel = (t[2025] - 1100) / (t[2023] - 1100)      # hauteurs des barres si l'axe part de 1 100 k€
reel = t[2025] / t[2023]
print(f"hauteur apparente : x{visuel:.1f} ; rapport réel : x{reel:.2f} ({(reel - 1) * 100:.0f} %)")
```

```
hauteur apparente : x5.8 ; rapport réel : x1.16 (+16 %)
```


Avec un axe coupé à 1 100 k€, la barre de 2025 paraît **5,8 fois** plus haute que celle de 2023 ; en réalité, elle ne l'est que de 16 %. Le graphique de droite, à zéro, dit la vérité.

Qui est trompé, par quoi, avec quelle conséquence ? La gérante, ou le financeur, par la **longueur des barres**, qui laisse croire à un quasi-sextuplement de l'activité. *Conséquence* : un investissement ou un recrutement décidé sur une croissance imaginaire ; et, quand la réalité rattrape l'enthousiasme, une confiance durablement entamée dans tous les graphiques de l'analyste.

Le correctif. Pour des **barres**, l'axe part de zéro, sans exception. Pour une **courbe**, on peut ne pas partir de zéro, à condition que l'axe soit visible et que l'on ne cherche pas à dramatiser (si l'écart est petit, on le **dit** dans le titre : « +16 % en deux ans »).

1.4.2 Les aires proportionnelles au mauvais carré, et la 3D

Le piège. Quand on représente une valeur par un cercle, un carré ou une icône, il faut décider ce qui est proportionnel à la valeur : la **dimension** (rayon, côté) ou l'**aire**. Si l'on prend le rayon, une valeur trois fois plus grande donne un cercle d'une aire **neuf** fois plus grande, et c'est l'aire que l'œil perçoit.

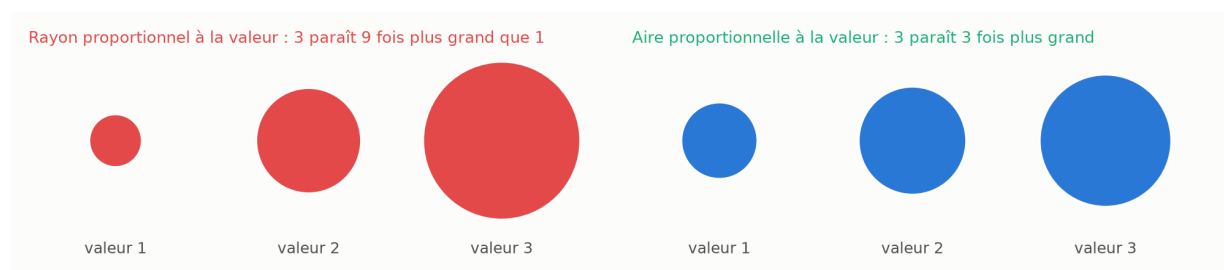


Figure 22: À gauche, des cercles dont le rayon est proportionnel à la valeur (1, 2, 3) : celui de la valeur 3 paraît neuf fois plus grand que celui de la valeur 1. À droite, des cercles dont l'aire est proportionnelle à la valeur : l'impression de 1 à 3 est respectée. Schéma dessiné avec matplotlib.

Le même effet vaut pour les **graphiques en 3D** : une barre en perspective ne se lit plus contre une grille, sa hauteur dépend de l'angle de vue, et celles du fond sont partiellement cachées. Il n'y a **aucun cas** où l'effet 3D aide à lire un chiffre.

Qui est trompé, par quoi, avec quelle conséquence ? Tout lecteur, par l'**aire** et la **perspective**, qui amplifient les grandes valeurs. *Conséquence* : des écarts surestimés ou mal classés (la barre de devant paraît plus grande que celle de derrière, même quand elle est plus petite).

Le correctif. Représenter les valeurs par la **longueur** (barres planes) ; si l'on doit absolument utiliser des cercles (une carte), faire l'**aire** proportionnelle à la valeur, et ajouter des **étiquettes chiffrées**. Pas de 3D.

1.4.3 Des échelles différentes qui rendent comparable ce qui ne l'est pas

Le piège. Quand on place deux graphiques côte à côte avec des **axes propres** à chacun, ils ont l'air aussi grands l'un que l'autre, même si l'un représente sept fois plus que l'autre. Les chiffres d'affaires mensuels de la boutique et des réseaux sociaux en 2025 le montrent.

En haut, la courbe de la boutique et celle des réseaux ont toutes les deux une allure ascendante spectaculaire, et on peut les croire de poids comparable. En bas, avec un axe commun, la boutique atteint 73,4 k€ en décembre et les réseaux 20,4 k€ : les réseaux ne représentent que **11,0 %** du chiffre d'affaires de l'année. On voit aussi que, relativement, les réseaux progressent **plus** de février à décembre ($\times 3,7$) que la boutique ($\times 2,2$), mais à partir d'un niveau bien plus bas : les deux lectures sont vraies, et c'est l'axe commun qui permet de les tenir ensemble.

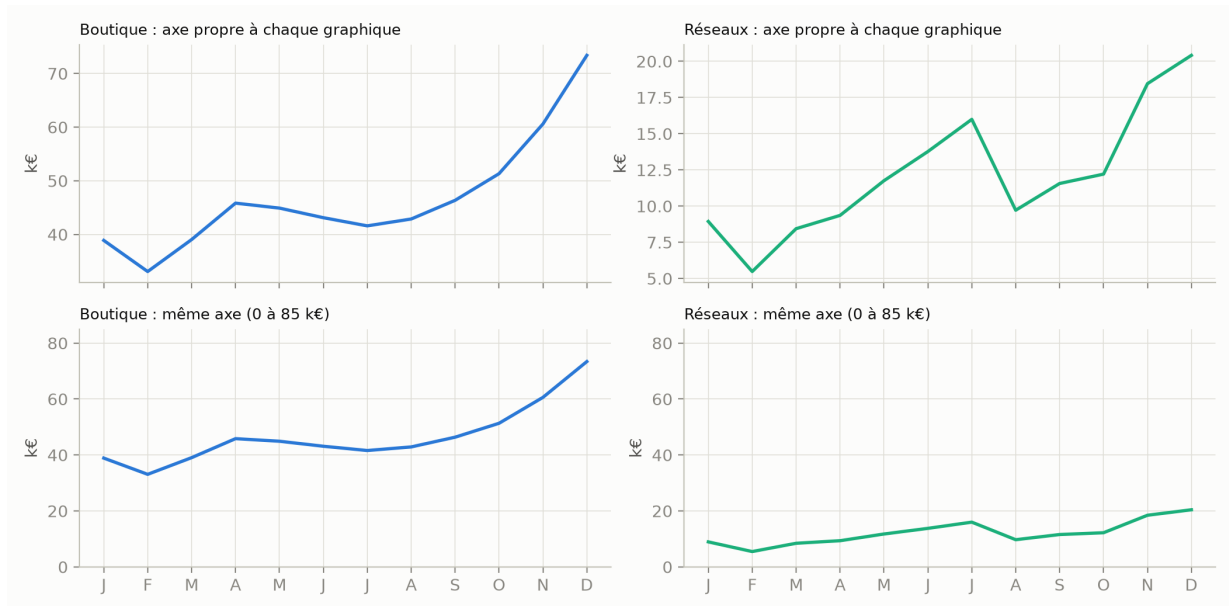


Figure 23: En haut, deux courbes (boutique, réseaux sociaux) avec chacune son propre axe : les deux variations semblent de même ampleur. En bas, les mêmes courbes sur le même axe de 0 à 85 k€ : l'échelle réelle apparaît. Figure construite avec matplotlib (données simulées).

Qui est trompé, par quoi, avec quelle conséquence ? Un responsable qui compare des canaux, par l'apparente égalité de graphiques à axes libres. Conséquence : un budget publicitaire réparti comme si les deux canaux avaient le même poids.

Le correctif. Quand on compare, on met les séries sur le même axe (petits multiples à échelle commune, section 1.2.5). Quand on veut montrer la forme d'une série à petite échelle, on peut laisser un axe libre, mais on le dit (« axe propre à chaque graphique ») et l'on évite de les juxtaposer comme s'ils étaient comparables.

1.4.4 Le double axe

Le piège. Deux courbes sur un seul graphique, l'une lue à gauche (en k€), l'autre à droite (en nombre de commandes) : on peut choisir les deux échelles pour que les courbes coïncident, ou au contraire qu'elles divergent. Avec la publicité et les commandes mensuelles de 2025, voici ce que donne un bon choix d'échelles.

La corrélation entre les deux séries, sur ces douze mois, vaut 0,80 : elle existe réellement. Mais le graphique de gauche **ne le prouve pas** : l'accord apparent des courbes est le résultat de l'échelle choisie à droite (0 à 2 000) et à gauche (0 à 14). Avec 0 à 3 000 à droite, la courbe des commandes serait bien moins pentue et ne ressemblerait plus à celle de la publicité. Le nuage de droite, lui, **ne dépend d'aucun choix** : un point par mois, la publicité en abscisse, les commandes en ordonnée.

Qui est trompé, par quoi, avec quelle conséquence ? Une direction qui lit « la pub fait vendre », par la coïncidence fabriquée de deux échelles. Conséquence : un budget de publicité augmenté sur la foi d'une ressemblance de courbes qui aurait pu être obtenue avec n'importe quelles deux séries.

Le correctif. Deux graphiques superposés sur le même axe du temps (un par série), ou un nuage de points. Si l'on emploie malgré tout un double axe, on colore chaque axe comme sa courbe et on écrit les unités ; on évite d'ajuster les échelles pour que les courbes se touchent. Mais la corrélation, même réelle, n'est pas une cause : voir 1.4.7.

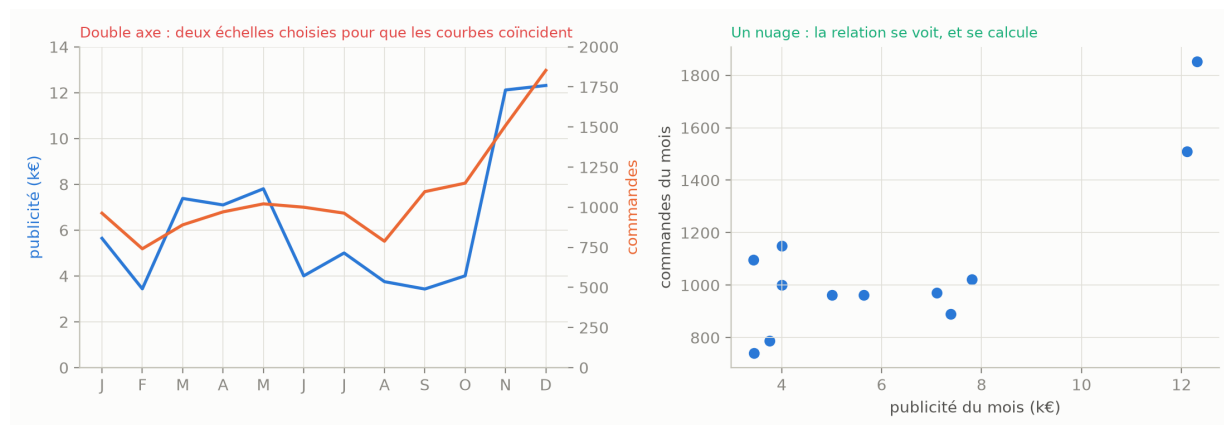


Figure 24: À gauche, la publicité mensuelle (axe de gauche, de 0 à 14 k€) et le nombre de commandes (axe de droite, de 0 à 2 000) en 2025 : les deux courbes semblent se suivre, parce que les deux échelles ont été choisies pour cela. À droite, le nuage de points des mêmes douze mois : la relation se voit sans choix d'échelle. Figure construite avec matplotlib (données simulées).

1.4.5 La période choisie

Le piège. « Les ventes ont progressé de 53 % en trois mois. » La phrase est **vraie** : le chiffre d'affaires mensuel passe de 120 k€ en octobre à 184 k€ en décembre 2025. Mais présentée seule, elle laisse croire à une accélération.

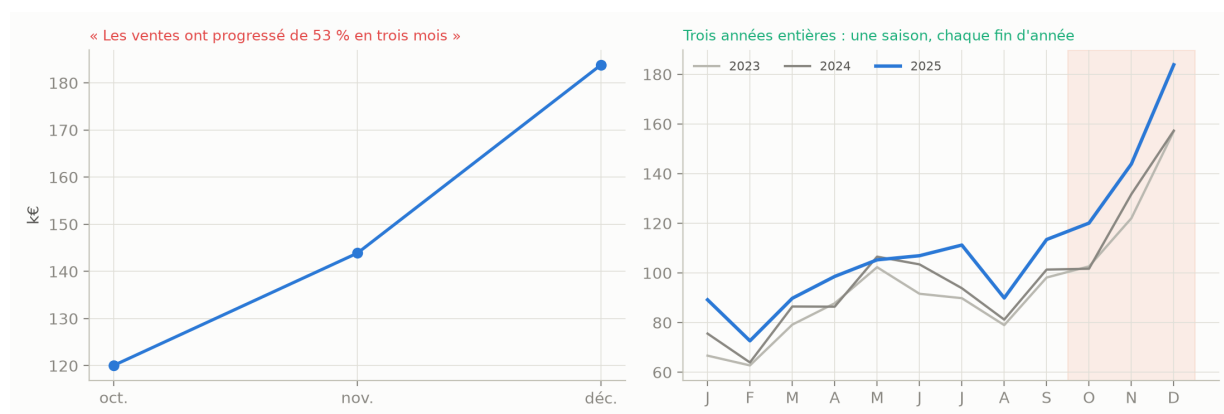


Figure 25: À gauche, une courbe sur trois mois seulement (octobre à décembre 2025), titrée « les ventes ont progressé de 53 % en trois mois ». À droite, trois années entières : la même hausse se reproduit chaque fin d'année (zone orangée), c'est une saison. Figure construite avec matplotlib (données simulées).

Sur trois années entières, la même hausse d'octobre à décembre se retrouve chaque fois : +53,0 % en 2023, +54,7 % en 2024, +53,1 % en 2025. Ce n'est pas une accélération, c'est la **saison**. Et l'on peut aussi, en choisissant d'autres bornes, raconter l'inverse : de janvier à février 2025, le chiffre d'affaires baisse de 19 %, et personne ne dira pourtant que la boutique s'effondre.

Qui est trompé, par quoi, avec quelle conséquence ? Le *financier* ou la *gérante*, par le **choix des bornes**. Conséquence : une prévision extrapolée d'une pente saisonnière (on commande trop de stock en janvier, on recrute en décembre pour une demande qui ne durera pas).

Le correctif. Montrer **au moins un cycle complet** (un an de plus que ce que l'on veut dire), comparer **au même mois de l'an dernier**, et dire pourquoi on a choisi la période. Une phrase honnête vaut mieux qu'une période flatteuse : « Le chiffre d'affaires de décembre est supérieur

de 17 % à celui de décembre 2024. »

1.4.6 Les pourcentages sans effectifs

Le piège. Un classement de **taux** (« les produits les plus retournés ») met en haut les produits qui ont **peu de ventes**, parce qu'un petit effectif fait varier un taux par à-coups. Prenons les retours de marchandises sur les lignes vendues par les réseaux sociaux : 120 produits, un taux moyen de retour de 6,7 %.

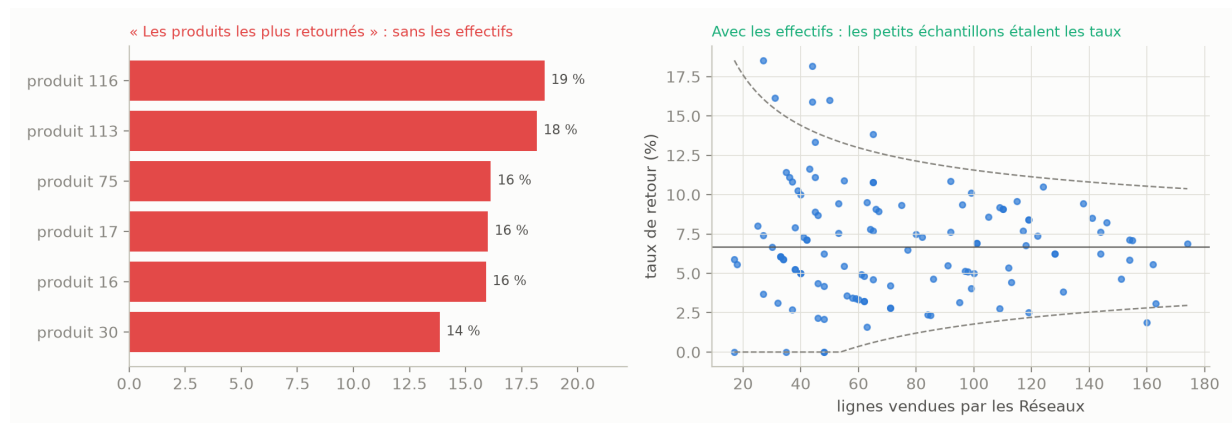


Figure 26: À gauche, le classement des six produits au taux de retour le plus élevé (de 19 % à 14 %), sans leurs effectifs. À droite, le taux de chaque produit en fonction du nombre de lignes vendues, avec la moyenne (trait plein) et les bornes à 95 % (pointillés) : les petits effectifs s'étalent en entonnoir. Figure construite avec matplotlib (données simulées).

À gauche, le classement semble alarmant : le produit 116 retourne **18,5 %** de ses ventes, trois fois la moyenne. Mais le produit 116 n'a été vendu que **27 fois** par les réseaux, et **5** de ces lignes ont été retournées. Un retour de plus ou de moins déplace son taux de **3,7 points**. Le produit « typique » a 64,5 lignes vendues ; les six du palmarès en ont entre 27 et 65.

Pour savoir si ces taux sont **trop** élevés, on compare chaque taux à ce que le seul hasard donnerait pour un effectif de cette taille : une borne autour de la moyenne, qui se resserre quand l'effectif grandit (le « diagramme en entonnoir » de droite).

```
p0 = F["ret_moy"] / 100 # taux de retour moyen des réseaux
g = F["ret_g"]
ecart = np.sqrt(p0 * (1 - p0) / g["n"])
for nom, z in (("95 %", 1.96), ("99,8 %", 3.09)):
    haut = int((g["taux"] / 100 > p0 + z * ecart).sum())
    print(f"produits au-dessus de la borne à {nom} : {haut}")
print("attendu par hasard à 95 % :", round(0.025 * len(g)))
```

produits au-dessus de la borne à 95 % : 6
 produits au-dessus de la borne à 99,8 % : 0
 attendu par hasard à 95 % : 3

Six produits dépassent la borne à 95 %, pour **trois** attendus par pur hasard sur 120 produits ; aucun ne dépasse la borne à 99,8 %. Le bon message n'est donc ni « ces produits sont mauvais » ni « tout va bien » : c'est « **à surveiller**, avec des effectifs trop petits pour conclure ».

Qui est trompé, par quoi, avec quelle conséquence ? La responsable des achats, par un **classement de taux sans effectifs**. Conséquence : un produit retiré du catalogue, ou un fournisseur mis en cause, sur la foi de cinq retours.

Le correctif. Toujours **donner l'effectif** à côté du taux (« 18,5 % de 27 lignes »), ne classer que les produits ayant un effectif minimal, ou tracer un **diagramme en entonnoir** qui montre où le hasard suffit à expliquer l'écart (les intervalles et les petits effectifs sont traités au volume III, chapitres 2 et 4).

1.4.7 La corrélation suggérée

Le piège. Deux courbes qui montent ensemble ne se **causent** pas forcément. Ici, la publicité et les commandes de la boutique, mois par mois, sur trois ans.

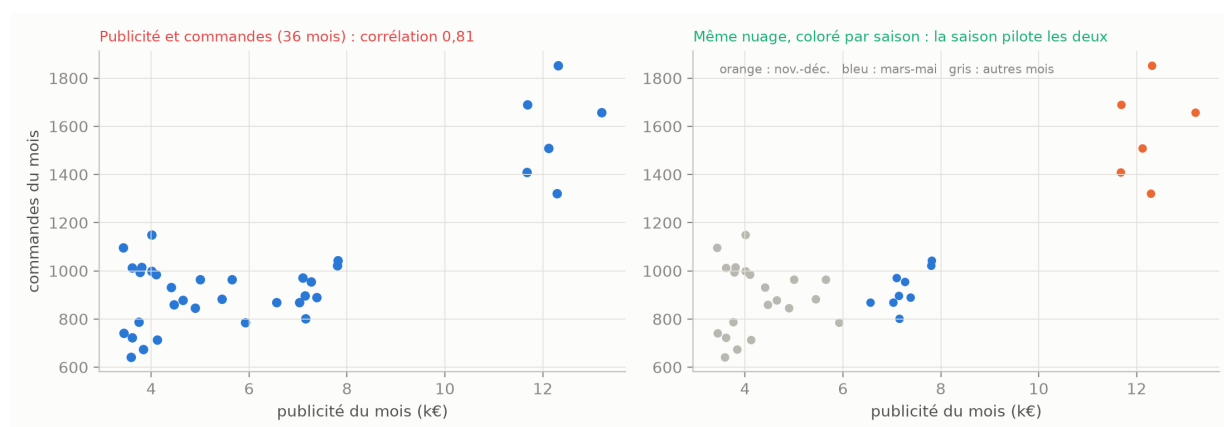


Figure 27: À gauche, le nuage de points de la publicité mensuelle et des commandes mensuelles sur 36 mois, titré « corrélation 0,81 ». À droite, le même nuage où novembre-décembre sont en orange, mars-mai en bleu et les autres mois en gris : la saison commande à la fois la publicité et les ventes. Figure construite avec matplotlib (données simulées).

Sur 36 mois, la corrélation est de **0,81** : forte. La figure de droite explique pourquoi elle ne prouve rien : en novembre et décembre, la boutique dépense en moyenne **12 214 €** en publicité, contre **5 158 €** les autres mois (2,4 fois plus), et enregistre **1 574** commandes contre **898** (1,8 fois plus). La saison fait monter les deux. Si l'on retire novembre et décembre, la corrélation tombe à **0,16** : presque rien. (Ce piège et la façon de le démêler sont traités au volume III, chapitres 2 et 3 : ici, on retient que le **graphique** ne doit pas suggérer une cause que l'analyse n'a pas établie.)

Qui est trompé, par quoi, avec quelle conséquence ? La gérante, par un **nuage ou un titre qui parle de corrélation sans la mettre en perspective**. *Conséquence* : doubler le budget publicitaire en comptant sur un effet que la saison seule expliquait.

Le correctif. Colorer ou séparer par la variable cachée (la saison), **comparer à saison égale**, et écrire le titre avec prudence : « Publicité et commandes montent ensemble en fin d'année, mais la saison suffit à l'expliquer. »

1.4.8 Le camembert à neuf parts

Le piège. Au-delà de quatre ou cinq parts, un camembert ne se lit plus : les couleurs se confondent, la légende oblige à des allers-retours et les parts voisines sont indiscernables. Le chiffre d'affaires 2025 par ville (20 villes fictives) en fait un cas d'école.

Deux parts, la ville F (6,2 %) et la ville G (6,0 %), ne diffèrent que de 0,2 point, soit **0,8 degré** d'angle : personne ne peut les départager sur un camembert. Sur les barres, l'ordre et les valeurs sont immédiats. Et la plus grande « part » est celle des **autres villes** (30,4 %), qui est un fourre-tout : on la met en gris, en bas, pour qu'elle ne passe pas pour une ville.

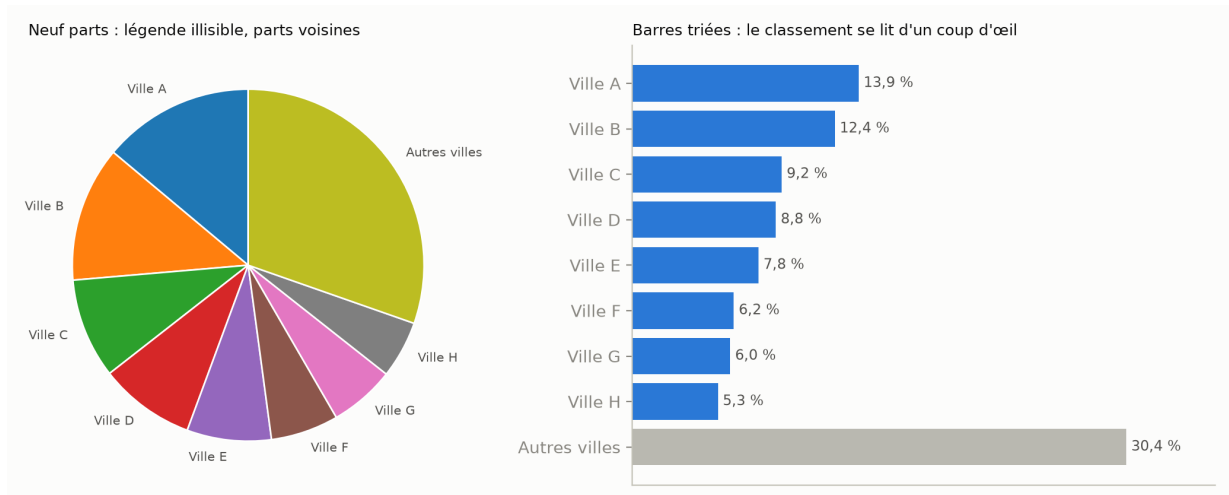


Figure 28: À gauche, un camembert à neuf parts (les huit premières villes et « autres villes ») : couleurs et angles voisins se confondent. À droite, les mêmes parts en barres triées, avec « autres villes » (30,4 %) en gris. Figure construite avec matplotlib (données simulées).

Qui est trompé, par quoi, avec quelle conséquence ? Un responsable de zone, par une légende illisible et des angles voisins. *Conséquence* : un classement des villes faux, donc une priorité commerciale mal placée.

Le correctif. Barres triées, étiquettes chiffrées, regroupement des petites catégories en un « autres » discret, ou limitation du graphique aux huit premières.

1.4.9 Le paradoxe de Simpson en image

Le piège. Une moyenne globale peut dire l'inverse de chaque sous-groupe (c'est le paradoxe de Simpson, vu au volume III, chapitre 1). Un graphique qui ne montre que la moyenne globale trompe d'autant plus qu'il est « simple ». Le chiffre d'affaires moyen par jour, avec et sans promotion, en est un exemple.

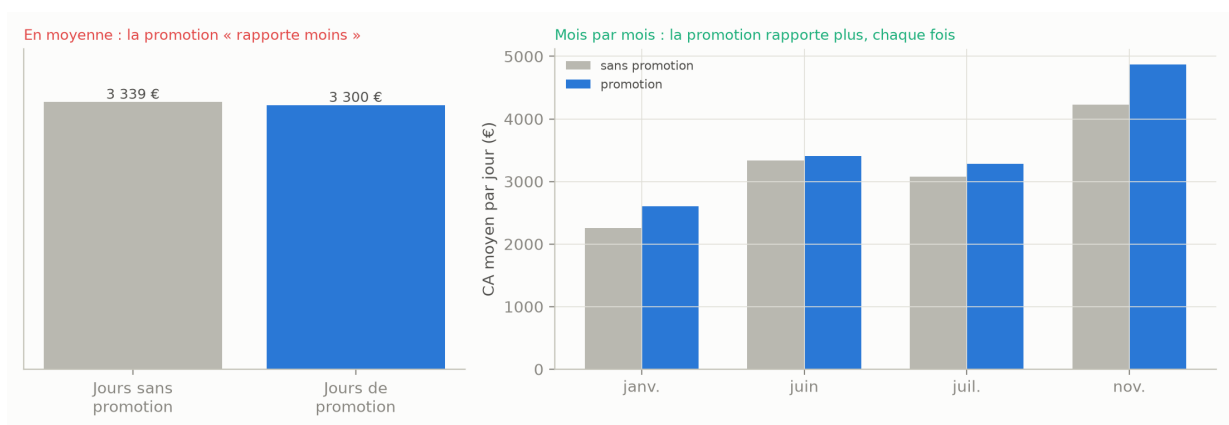


Figure 29: À gauche, le chiffre d'affaires moyen par jour des jours sans promotion (3 339 €) et des jours de promotion (3 300 €) : la promotion « rapporte moins ». À droite, la même comparaison mois par mois (janvier, juin, juillet, novembre, les seuls mois avec des promotions) : la promotion rapporte plus, chaque fois. Figure construite avec matplotlib (données simulées).

```
promo = F["promo_glob"] # CA moyen par jour : sans (0) et avec (1)
↳ promotion
```

```
mois = F["promo_mois"] # idem, mois par mois (les mois qui ont des
↳ promotions)
print(promo.round(0).to_string())
print(((mois[1] / mois[0] - 1) * 100).round(1).to_string())
```

```
promo_active
0    3339.0
1    3300.0
mois
1     15.5
6      2.2
7      6.7
11    15.3
```

À gauche, un jour de promotion rapporte en moyenne 3 300 €, soit **1,2 % de moins** qu'un jour sans promotion (3 339 €). À droite, mois par mois, la promotion rapporte **plus** : +15,5 % en janvier, +2,2 % en juin, +6,7 % en juillet, +15,3 % en novembre. L'explication est une question de **composition** : les 153 jours de promotion tombent en janvier (63 jours), juin (21), juillet (42) et novembre (27), jamais en décembre, alors que les jours **sans** promotion incluent tout décembre, le meilleur mois de l'année (5 357 € par jour en moyenne). La moyenne globale compare des jours de saisons différentes.

Qui est trompé, par quoi, avec quelle conséquence ? La gérante, par une **moyenne globale** présentée sans la saison. *Conséquence* : suppression d'une promotion qui rapporte réellement, pour une raison qui tient au calendrier.

Le correctif. Comparer à **situation égale** (même mois, même jour de la semaine) et le montrer : le deuxième graphique est le bon.

1.4.10 L'échelle logarithmique non signalée

Le piège. Une échelle logarithmique transforme les **rapports** en **distances égales** : de 1 à 10 occupe autant de place que de 10 à 100. Elle est précieuse pour des valeurs qui s'étalent sur plusieurs ordres de grandeur (ou pour des taux de croissance), mais elle **comprime** les écarts, et, utilisée sans le dire, elle trompe. Le chiffre d'affaires 2025 des 120 produits, du plus au moins vendu, en donne l'exemple.

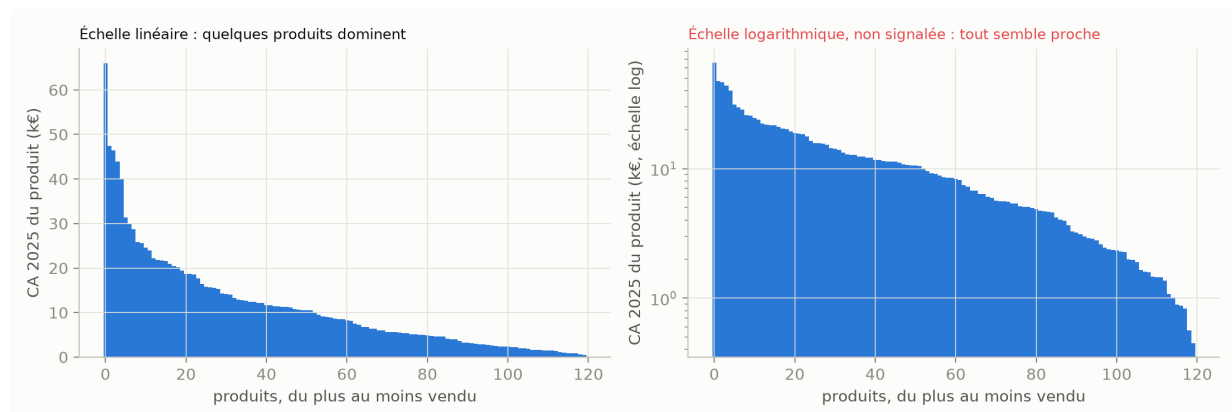


Figure 30: À gauche, le chiffre d'affaires 2025 de chacun des 120 produits en barres sur une échelle linéaire : quelques produits dominant. À droite, le même graphique en échelle logarithmique non signalée : la décroissance paraît douce et régulière. Figure construite avec matplotlib (données simulées).

À gauche, on voit ce qu'il y a de vrai : quelques produits dominant (le premier réalise 66,0 k€,

soit **7,9 fois** le produit médian), et dix produits font à eux seuls **29,1 %** du chiffre d'affaires. À droite, la même série en échelle logarithmique, sans mention, semble une pente régulière : les écarts entre produits paraissent modestes. De plus, une **barre** sur une échelle logarithmique ne mesure plus rien : sa longueur dépend de l'endroit, arbitraire, où l'axe est coupé.

Qui est trompé, par quoi, avec quelle conséquence ? Une lectrice non technique, par une **échelle qu'elle n'a pas reconnue**. Conséquence : sous-estimer la concentration du chiffre d'affaires, donc le risque de dépendre de quelques produits.

Le correctif. On utilise l'échelle logarithmique **seulement** pour des courbes (jamais des barres), on **l'écrit** (« échelle logarithmique ») dans l'axe ou le titre, et on met en évidence les graduations (1, 10, 100) qui font comprendre qu'un pas est un facteur 10.

1.4.11 Une liste de contrôle avant de publier

Pour terminer, voici la liste à parcourir **avant d'envoyer** un graphique. Chaque ligne renvoie à un piège de cette section.

Piège	La question à se poser
Axe tronqué (1.4.1)	Mes barres partent-elles de zéro ?
Aires et 3D (1.4.2)	La grandeur est-elle représentée par une longueur, ou par une aire proportionnelle à la valeur ?
Échelles différentes (1.4.3)	Si je compare, les axes sont-ils communs ? Sinon, est-ce écrit ?
Double axe (1.4.4)	Les deux échelles sont-elles choisies sans arrière-pensée ? Un nuage ne serait-il pas plus honnête ?
Période choisie (1.4.5)	Mon graphique montre-t-il au moins un cycle complet ?
Pourcentages sans effectifs (1.4.6)	L'effectif est-il visible à côté du taux ?
Corrélation suggérée (1.4.7)	Une variable cachée (saison, taille, prix) explique-t-elle les deux courbes ?
Camembert à neuf parts (1.4.8)	Plus de quatre parts ? Alors des barres.
Simpson (1.4.9)	Ma moyenne mélange-t-elle des sous-groupes qui ne sont pas comparables ?
Échelle logarithmique (1.4.10)	Est-elle écrite ? Mes barres sont-elles à échelle linéaire ?

À retenir. Un graphique trompeur n'est pas un graphique faux : c'est un graphique **vrai qui suggère une conclusion fausse**. Pour chaque graphique, demandez : *qui pourrait être trompé, par quoi, avec quelle conséquence ?* Et préférez toujours la version qui montre les effectifs, l'axe complet, la période entière et la comparaison à situation égale.

Pour s'entraîner. Cahier, chapitre 1 : applications 1.8 et 1.9, exercices 1.9 à 1.14.

Bilan du chapitre 1

Vous savez maintenant :

- **partir de la question** de la lectrice pour choisir le graphique (comparer, suivre, composer, distribuer, relier, expliquer un passage, croiser) plutôt que de partir de l'outil ;
- **ranger les encodages** par précision de lecture (position, longueur, angle, aire, couleur) et compenser par des étiquettes chiffrées chaque fois que l'on quitte la position ou la longueur ;
- **dire les mêmes données de quatre façons** et savoir à quelle question répond chacune ;
- **éviter** les camemberts chargés, les doubles axes, les radars, la 3D et les jauges, et leur substituer des barres, deux graphiques superposés ou un nuage ;
- **retirer, ordonner, nommer, titrer, relire** : redessiner un graphique en cinq gestes, avec un **titre qui énonce la conclusion**, une seule couleur d'accentuation, des étiquettes directes ;
- **comparer en petits multiples** à échelle commune, et **annoter** ce que l'on sait des creux et des bosses ;
- **adapter** la figure à la salle (18 points au minimum, peu d'éléments, traits épais) ;
- (en option) **choisir une palette** selon ce que la couleur doit dire (catégories, intensité, écart), ne jamais faire porter l'information par la couleur seule, **simuler le daltonisme** et **calculer le contraste** ;
- (en option) **fixer le tout dans une page de design** applicable à tous les tableaux de bord ;
- (en option) **reconnaître dix pièges** (axe tronqué, aires et 3D, échelles différentes, double axe, période choisie, pourcentages sans effectifs, corrélation suggérée, camembert à neuf parts, Simpson en image, échelle logarithmique non signalée) et poser pour chacun la question : *qui est trompé, par quoi, avec quelle conséquence ?*

Le tableau suivant résume le chapitre en six questions, à parcourir **avant d'envoyer un graphique**.

Question	Ce qu'on attend comme réponse
Quelle question le graphique pose-t-il ?	Une phrase, avec la décision qui suit.
Quelle forme ?	Celle qui répond à cette question par une position ou une longueur (barres, courbe, nuage).
Que peut-on retirer ?	Tout ce qui ne dit rien : légende redondante, grille lourde, couleurs sans sens.
Que dit le titre ?	La conclusion, avec un chiffre ; l'unité, la période et la source en pied.
Qui ne le verra pas bien ?	Une personne daltonienne, un écran de projection, une impression en noir et blanc : le graphique reste lisible.
Qui pourrait être trompé ?	Personne : axe à zéro, période complète, effectifs visibles, comparaison à situation égale.

Le fil conducteur du chapitre tient en une phrase : **un graphique est un argument, et la lectrice doit pouvoir le lire, le croire et le contester**. Il vous reste, dans le volume, à voir **où fabriquer** ces graphiques : les outils de tableaux de bord (chapitre 2), la programmation en Python (chapitre 3), puis la façon de les ranger dans un récit (chapitre 4) et de les présenter (chapitre 5).

Pour s'entraîner. Cahier, chapitre 1 : applications 1.1 à 1.9 (choisir une forme, camembert contre barres, redessin en cinq gestes, titres et annotations, petits multiples, simulation du daltonisme, contrastes d'une palette, axe tronqué et période choisie, entonnoir et paradoxe de Simpson) et exercices 1.1 à 1.14.

Chapitre 2 : Tableaux de bord

« Un tableau de bord ne sert pas à tout montrer : il sert à savoir quoi faire lundi matin.
»

Un lundi matin, la gérante s'arrête devant votre bureau. « Chaque semaine, tu m'envoies un fichier. Je l'ouvre, je cherche ce qui m'intéresse, je ne trouve pas, je te rappelle. **Je veux voir chaque lundi où nous en sommes, sans te demander un fichier.** Et que ça me dise tout seul quand quelque chose ne va pas. »

La demande semble technique (« fais-moi un tableau de bord »), mais elle contient trois exigences très différentes. La première est **technique** : il faut des données qui se mettent à jour seules, organisées pour que les chiffres soient toujours calculés de la même façon. La deuxième est **visuelle** : il faut que la page se lise en quelques secondes, sans mode d'emploi. La troisième est **humaine** : il faut que la gérante s'en serve vraiment, que le jour où le tableau de bord signale un problème, elle sache ce qu'elle décidera. Un tableau de bord réussi est un **outil de décision** ; un tableau de bord raté est un joli tableau que l'on ouvre deux semaines, puis plus jamais.

Le chapitre précédent vous a donné les principes : choisir le bon graphique, soigner la mise en page, ne pas tromper. Ce chapitre les met en œuvre dans des **outils de tableaux de bord**, ceux que l'on rencontre le plus en entreprise, et il montre surtout ce qu'ils ont en commun, parce que les menus changent d'une version à l'autre alors que les idées durent.

Ce que ce chapitre ne fait pas, et pourquoi. Power BI, Tableau, Looker, Metabase, Superset et Qlik sont des produits que **nous n'avons pas exécutés** pour écrire ce livre. Nous ne montrons donc **aucune capture de leurs écrans** et nous ne prétendons pas décrire leurs menus : ils changent d'une version à l'autre, et ce que nous en disons est à **vérifier dans la documentation de votre version**. Les figures de ce chapitre sont des **maquettes dessinées** avec matplotlib (rectangles génériques, aucune identité d'un produit), ou des graphiques calculés sur les données de la boutique. Ce qui peut être **vérifié** l'est : le modèle de données, les mesures, les filtres de sécurité et les chiffres de la page finale sont recalculés ici avec pandas et SQL, et chaque nombre du texte vient d'un calcul exécuté.

Le chemin de ce chapitre

Le chapitre suit la demande de la gérante, de la donnée à l'écran, puis du premier outil au choix d'un outil.

- **2.1 Power BI : modèle, visuels, publication.** Avant de dessiner quoi que ce soit, on organise les données en **étoile** (une table de faits, des dimensions), on définit des **mesures** qui se recalculent selon les filtres, on choisit des **visuels**, puis on **publie** et on planifie l'actualisation. Power BI sert d'exemple de l'approche « modèle d'abord ».
- **2.2 Tableau : feuilles, tableaux de bord, partage.** Tableau illustre l'approche inverse, plus **exploratoire** : on compose des *feuilles* en glissant des champs sur des **marques** (position, couleur, taille), on ajoute des **calculs de table** et des **niveaux de détail**, puis on assemble les feuilles en tableau de bord. Les deux approches se rejoignent.
- **2.3 Conception d'un tableau de bord selon les besoins des utilisateurs.** La section la plus importante : **partir de la décision**, choisir peu d'indicateurs, les ranger en trois niveaux (vue d'ensemble, analyse, détail), tester en cinq secondes, **valider les chiffres** et prévoir l'**adoption**. Nous construisons la page complète de la gérante.
- **2.4 ☉ Power BI avancé.** Le langage des mesures (DAX), la préparation des données (Power Query), la **sécurité au niveau des lignes** et le déploiement, avec des équivalents pandas qui permettent de **vérifier** chaque résultat.
- **2.5 ☉ Looker, Metabase, Superset, Qlik.** Quatre autres familles d'outils, ce qui les distingue, et une méthode pour **choisir** sans se laisser guider par la marque.

Le chapitre se termine par un bilan, et le **cahier** propose huit applications (modèle en étoile, mesures, calculs de table, critique de tableaux de bord, sécurité, choix d'outil...) et douze exercices.

Intuition. Un outil de tableaux de bord est une **chaîne de montage** : les données brutes entrent à gauche, un chiffre juste et lisible sort à droite, et à chaque étape on peut se tromper. La qualité d'un tableau de bord se joue **en amont** (le modèle, les définitions) autant qu'en aval (les couleurs et la disposition).

Les données du chapitre

Nous reprenons la boutique des volumes précédents : les commandes de 2023 à 2025, les clients, les produits, les livraisons, les sessions du site, les stocks. Toutes les données sont **simulées**. Nous utilisons un fichier supplémentaire, *villes.csv*, qui associe chaque ville fictive à une **région fictive** (quatre régions, dans un plan inventé) : elle sert à illustrer la sécurité par région, sans aucune géographie réelle.

Les données. *commandes.csv*, *lignes_commande.csv* (83 905 lignes, 2023 à 2025), *produits.csv*, *clients.csv*, *villes.csv*, *livraisons.csv*, *sessions_web.csv*, *stock_quotidien.csv*, *jours_exploitation.csv*. Le script *build/outils_ch02.py* contient le chargement, le modèle en étoile et le dessin des maquettes ; les définitions des indicateurs sont celles du volume III, chapitre 6 (KPI).

Deux rappels des volumes précédents serviront souvent. D'abord, **les 120 produits portent seulement 60 noms** : on relie toujours les tables par l'**identifiant** du produit, jamais par son nom (volume II, section 2.2.5). Ensuite, les chiffres d'affaires de la boutique sont en **euros TTC** pour les ventes et la marge se calcule **hors taxes** avec une TVA fixée à 20 % **pour l'illustration** (volume III, chapitre 6).

Pour s'entraîner. Cahier, chapitre 2 : applications 2.1 à 2.8 et exercices 2.1 à 2.12 ; chacun renvoie à la section du livre qui l'éclaire.

2.1 Power BI : modèle, visuels, publication

Avant d'ouvrir un outil de tableaux de bord, il faut comprendre **ce qu'il fait entre vos données et l'écran**. Cette section décrit la chaîne complète, avec Power BI comme exemple d'une approche « **modèle d'abord** » : on organise les données, on définit les calculs une fois, puis on dessine. Nous reproduisons chaque étape en pandas, pour que vous puissiez **vérifier** ce que l'outil affiche.

Une précaution. Power BI est un produit commercial que nous n'avons **pas exécuté**. Les notions (modèle, mesures, visuels, publication) sont stables ; les noms de menus, les limites de la version gratuite, les licences de partage et les fonctions précises **changent** : vérifiez-les dans la documentation de votre version. Les figures sont des maquettes dessinées, pas des captures.

2.1.1 Ce que fait un outil de tableaux de bord

Un outil de tableaux de bord reçoit des tables en entrée et produit une page interactive en sortie. Entre les deux, le même flux se retrouve partout, quel que soit le produit.

1. **Se connecter aux sources** : fichiers, bases de données, services en ligne.
2. **Préparer** les données : filtrer, corriger les types, fusionner, regrouper (c'est le rôle de Power Query dans Power BI).
3. **Modéliser** : relier les tables entre elles, dire laquelle contient les événements (les faits) et lesquelles les décrivent (les dimensions).
4. **Définir des mesures** : les calculs (chiffre d'affaires, taux de marge, panier moyen) écrits **une fois** et réutilisés partout.
5. **Dessiner** : poser des visuels sur une page, régler les filtres et les interactions.
6. **Publier et partager** : envoyer le résultat sur un serveur, choisir qui a le droit de le voir.
7. **Actualiser** : mettre à jour les données automatiquement, à heure fixe.

La figure met en regard chaque étape et son équivalent à **la main** en Python. Ce n'est pas un détour pédagogique : **c'est la méthode de contrôle de l'analyste**. Un tableau de bord affiche un chiffre ; si l'on est incapable de le recalculer autrement, on ne peut pas répondre à la gérante le jour où elle demande « pourquoi ce chiffre n'est pas le même que celui de la comptabilité ? ».

Intuition. Un outil de tableaux de bord **n'invente aucun calcul** : il automatise des jointures, des filtres, des regroupements et des sommes. Tout ce qu'il affiche peut se reproduire avec un tableur, du SQL ou pandas. C'est ce qui permet de **vérifier croisé** (voir 2.3.5).

Deux modes de connexion existent presque partout. En mode **import**, l'outil copie les données dans sa mémoire : les visuels réagissent instantanément, mais les données ne sont à jour qu'à la dernière actualisation. En mode **requête directe**, il interroge la base à chaque clic : les données sont fraîches, mais chaque visuel dépend de la vitesse de la base. Pour un tableau de bord hebdomadaire de la boutique, l'import suffit largement ; la requête directe se justifie pour un suivi presque en temps réel, avec une base conçue pour cela.

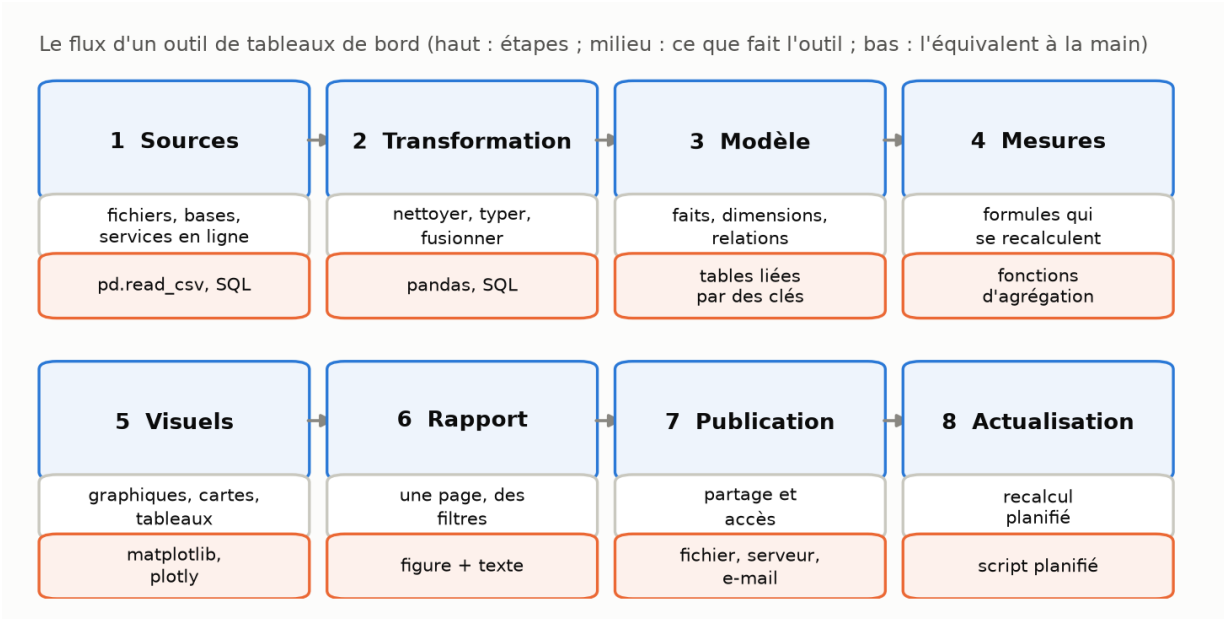


Figure 31: Le flux d'un outil de tableaux de bord, étape par étape : ce que fait l'outil (milieu) et son équivalent à la main (bas). Maquette dessinée, pas une capture.

2.1.2 Le modèle en étoile

Les lignes de commande de la boutique (qui a acheté quoi, quand, par quel canal) sont des **événements**. Les produits, les clients, les jours et les canaux sont des **descriptions**. Séparer les deux donne le **modèle en étoile** (*star schema*) : au centre, une **table de faits** (une ligne par événement, avec des chiffres : quantité, montant, marge) ; autour, des **tables de dimensions** (une ligne par produit, par client, par jour, par canal, avec des attributs : catégorie, région, mois, type de canal).

Chaque relation est **de un à plusieurs** : un produit apparaît dans beaucoup de lignes de faits, mais chaque ligne de faits concerne un seul produit. La clé d'une dimension doit donc être **unique**, et chaque ligne de faits doit avoir une correspondance. C'est exactement le contrôle d'effectifs du volume II (section 2.2.2), appliqué au modèle entier.

```
faits = star["fait_ventes"]
controle = 0.controle_etoile(star)
print(controle.to_string(index=False))
print("lignes de faits :", len(faits), "| CA 2023-2025 :", round(faits["montant"].sum()))
```

```
dimension  lignes  cle_unique  faits_orphelins
dim_date    1096      True          0
dim_produit   120      True          0
dim_client   6000      True          0
dim_canal     3       True          0
lignes de faits : 83905 | CA 2023-2025 : 3653157
```

Les quatre dimensions ont des clés uniques (1 096 jours, 120 produits, 6 000 clients, 3 canaux) et **aucune ligne de faits n'est orpheline** : chacune des 83 905 lignes trouve son jour, son produit, son client et son canal. Le modèle est sain, et le chiffre d'affaires total de la période, 3 653 157 €, est celui qu'on obtient en sommant les lignes sans aucune jointure.

La dimension des **dates** mérite une remarque : on la construit **explicitement**, avec un jour par ligne sur toute la période (même les jours sans vente), et des colonnes utiles (année, trimestre, mois, début de semaine, jour de la semaine, jour de promotion). Les calculs de comparaison dans le temps (« même période de l'an dernier », « cumul depuis le début de l'année », voir 2.4.1) ont

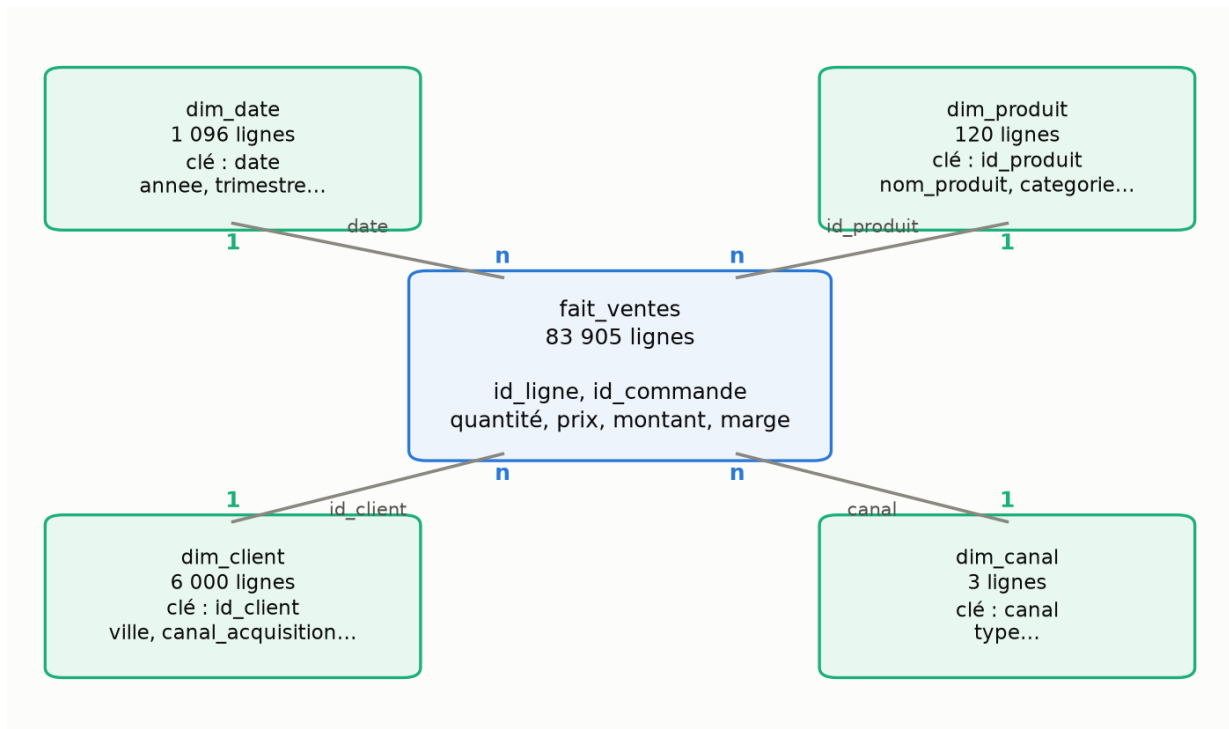


Figure 32: Le modèle en étoile de la boutique, calculé par pandas : une table de faits au centre, quatre dimensions autour. Chaque relation relie une clé unique (1) à plusieurs lignes de faits (n). Maquette dessinée avec les effectifs réels.

besoin d'un calendrier complet et continu : les dates que l'on déduirait des seules ventes auraient des trous.

Le piège de la clé qui multiplie

Relier les tables par une clé qui n'est pas unique est l'erreur la plus fréquente, et la plus discrète : l'outil ne signale rien, les chiffres sont simplement faux. Un exemple minuscule suffit. Dans cette boutique-jouet, deux produits différents, P1 et P3, portent le même nom « Vase » :

id_ligne	nom_produit	montant
1	Vase	20
2	Bougie	30
3	Vase	20
4	Bougie	30
5	Vase	10
6	Plaid	50

Les six lignes totalisent 160 €. Si l'on relie à la dimension des produits **par le nom**, chaque ligne « Vase » trouve **deux** correspondances (P1 et P3) : elle est comptée deux fois. On obtient neuf lignes et 210 € au lieu de 160 €, soit les 50 € des trois vases comptés en double. Dans la boutique, **chaque nom de produit est porté par deux produits** : c'est le piège du volume II. Essayons.

```

dim_prod = star["dim_produit"][["id_produit", "nom_produit"]]
par_nom = faits.merge(dim_prod,
    ↪ on="id_produit").drop(columns="id_produit").merge(dim_prod.drop(columns="id_produit"),
    ↪ on="nom_produit")
print(len(faits), "->", len(par_nom), "lignes | CA :", round(faits["montant"].sum()), "->",
    ↪ round(par_nom["montant"].sum()))
  
```

83905 -> 167810 lignes | CA : 3653157 -> 7306315

Le chiffre d'affaires est **doublé** (7,31 M€ au lieu de 3,65 M€) sans qu'aucune erreur n'apparaisse. Dans un outil de tableaux de bord, ce sont les **relations** du modèle qui jouent ce rôle de jointure : si l'on en déclare une sur une colonne non unique, l'outil refuse parfois (il demande une relation « plusieurs à plusieurs »), parfois accepte et calcule faux. Règle : **relier par des identifiants uniques, jamais par des libellés**, et vérifier les effectifs après chaque relation.

Piège. Une relation « plusieurs à plusieurs » est presque toujours le **symptôme** d'une dimension mal construite, pas une fonctionnalité à utiliser. Avant de l'accepter, cherchez la colonne qui rend la clé unique (ici, `id_produit`).

2.1.3 Mesures et colonnes calculées

Un outil de tableaux de bord propose deux manières de calculer.

Une **colonne calculée** ajoute une colonne à une table : pour **chaque ligne**, on calcule une valeur (la marge de la ligne, par exemple). Elle est calculée à l'**actualisation** et **stockée** : elle occupe de la mémoire, et sa valeur ne dépend d'aucun filtre.

Une **mesure** n'est pas stockée. C'est une **recette** : « somme des montants », « somme des marges divisée par somme du chiffre d'affaires hors taxes ». Elle se calcule **au moment de l'affichage, pour le contexte de filtres courant** : si l'on filtre sur la catégorie Jardin, la mesure se recalcule pour les seules lignes de cette catégorie. C'est pourquoi **la même mesure** donne des valeurs différentes dans chaque case d'un tableau croisé.

La mesure « chiffre d'affaires » vaut 1 324 764 € sur l'ensemble de 2025, 353 955 € dans le contexte « catégorie Jardin », et 166 201 € dans le contexte « catégorie Jardin et canal Site » : une seule recette, trois contextes.

Cette distinction a une conséquence pratique, qu'illustrent trois règles.

Règle 1 : un ratio est une mesure, jamais une colonne que l'on moyenne. Le taux de marge d'un ensemble de lignes est la **somme des marges divisée par la somme du chiffre d'affaires**, pas la moyenne des taux. Si l'on calcule un taux par catégorie puis qu'on en fait la moyenne, on donne autant de poids à la petite catégorie Papeterie qu'à Jardin :

```
par_cat = v25.groupby("categorie").agg(ca=("montant", "sum"), marge=("marge_ht", "sum"))
par_cat["taux"] = par_cat["marge"] / (par_cat["ca"] / 1.2)
global_ = v25["marge_ht"].sum() / (v25["montant"].sum() / 1.2)
print(f"taux global {global_:.1%} | moyenne des taux par catégorie {par_cat['taux'].mean():.1%}")
```

taux global 38.0% | moyenne des taux par catégorie 37.5%

L'écart est ici modeste (38,0 % contre 37,5 %), parce que les taux de marge des catégories sont proches ; il serait considérable si les tailles et les taux différaient davantage. Le principe, lui, ne se discute pas : **le bon taux est celui de la recette « somme sur somme »**, définie une fois dans une mesure.

Règle 2 : certaines mesures ne s'additionnent pas. Le nombre de **commandes** est un comptage de valeurs **distinctes** : une commande qui contient un vase et une bougie compte pour une commande, mais elle apparaît dans deux catégories.

```
n_total = v25["id_commande"].nunique()
n_cat = v25.groupby("categorie")["id_commande"].nunique()
```

```
print("commandes 2025 :", n_total, "| somme des catégories :", n_cat.sum())
print(n_cat.to_string())
```

```
commandes 2025 : 12946 | somme des catégories : 25163
categorie
Bien-être      3224
Cuisine        4381
Décoration     4912
Jardin         4315
Maison         4487
Papeterie      3844
```

Les six lignes d'un tableau croisé par catégorie totalisent **25 163 commandes**, alors que la boutique n'en a reçu que **12 946** : le total d'un tableau de bord ne doit **jamais** être la somme des lignes d'une mesure non additive. Les outils sérieux calculent le total **en refaisant la mesure sur l'ensemble** (c'est ce qui se passe dans une mesure bien écrite) ; mais une colonne calculée ou un tableau déjà agrégé fige l'erreur.

Règle 3 : la mesure se définit dans le modèle, pas dans chaque visuel. Si deux visuels recalculent chacun le « panier moyen » à leur façon, on aura un jour deux chiffres différents sur la même page. Les définitions du volume III (chapitre 6, section 6.1.2) se traduisent donc en **mesures nommées**, écrites une fois, documentées, et utilisées par tout le monde.

Pour qui veut la formule. Pour une table de faits T et un contexte de filtres C (l'ensemble des lignes retenues), une mesure est une fonction $m(T_C)$ de l'ensemble filtré. La somme et le comptage de lignes sont additifs ($m(A \cup B) = m(A) + m(B)$ si A et B sont disjoints) ; le comptage de valeurs distinctes, le ratio et la médiane ne le sont pas. Une **colonne calculée** est une fonction de **la ligne seule** : elle ne peut donc ni recalculer un ratio sur un groupe, ni compter des valeurs distinctes.

2.1.4 Les visuels

Le choix d'un visuel obéit aux principes du chapitre 1 (section 1.1) : la **question** décide du graphique. Les outils de tableaux de bord proposent quelques familles, et il est utile de les connaître par **ce qu'elles permettent de lire**.

- **La carte** (un chiffre) : à lire d'un coup, avec un repère de comparaison (l'an dernier, l'objectif).
- **Les barres** : comparer des catégories ; le tri décroissant est le réglage le plus utile.
- **La courbe** : suivre une évolution ; on y superpose la période de référence.
- **Le tableau** : retrouver une valeur exacte, ou un **tableau croisé** (matrice) pour croiser deux dimensions avec une mise en forme conditionnelle.
- **Le nuage** : voir une relation entre deux mesures, à utiliser avec parcimonie dans un tableau de bord de direction.
- **Les barres à 100 %** : comparer des parts, avec peu de catégories.
- **Les segments** (*slicers*, filtres) : laisser le lecteur choisir une période, un canal, une catégorie.

Deux fonctionnalités changent la nature d'un visuel. Les **interactions croisées** : cliquer sur une barre filtre (ou met en évidence) tous les autres visuels de la page. C'est puissant, et c'est aussi la source d'erreurs de lecture : un lecteur qui a cliqué sans s'en souvenir lit des chiffres filtrés comme s'ils étaient globaux. Il faut donc **indiquer clairement** quels filtres sont actifs, prévoir un bouton de remise à zéro, et décider **visuel par visuel** s'il réagit ou non (les cartes de chiffre global, par exemple, ne devraient pas réagir). Les **infobulles** (*tooltips*) ajoutent du détail au survol sans encombrer la page ; elles ne fonctionnent pas sur téléphone ou sur papier : **n'y cachez jamais une**

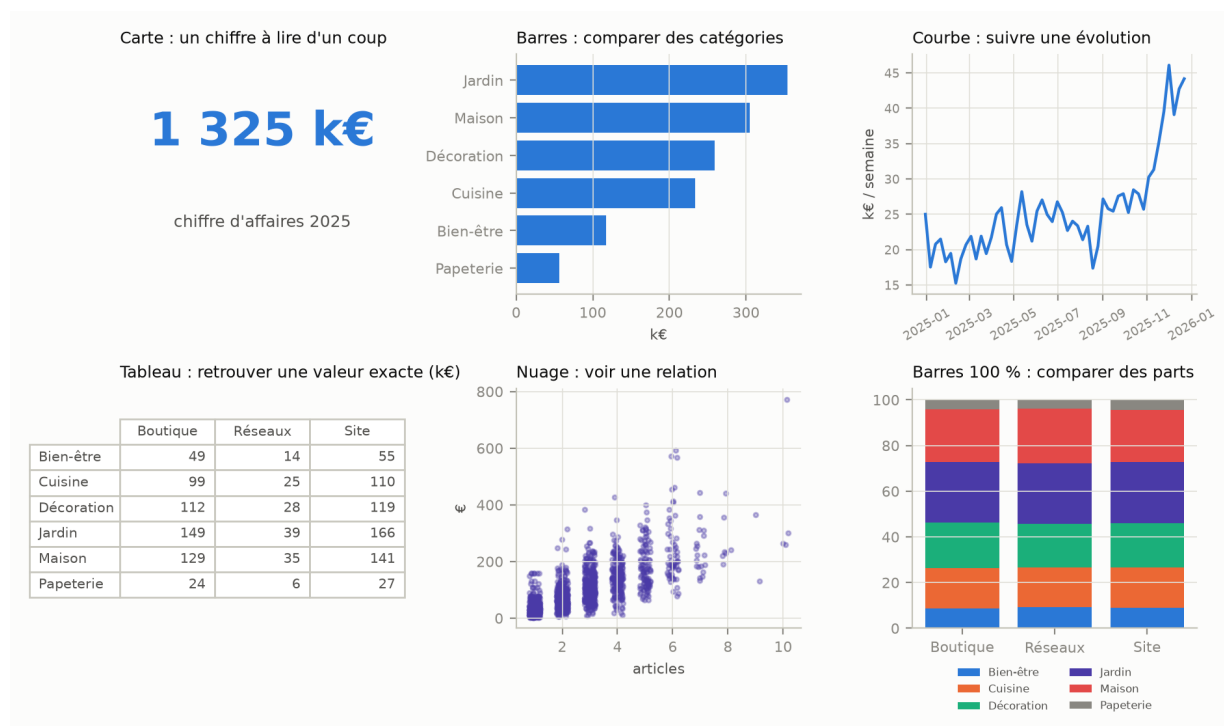


Figure 33: Les familles de visuels d'un outil de tableaux de bord, redessinées avec les données de la boutique : chaque visuel répond à une question différente. Maquette dessinée, pas une capture.

information indispensable.

Piège. Un visuel qui n'existe que par défaut : un graphique en secteurs avec dix catégories, un double axe, un 3D. Les outils les proposent, la page les accepte, mais les principes du chapitre 1 (sections 1.1, 1.2 et 1.4) s'appliquent **comme ailleurs**.

2.1.5 Publier, partager, actualiser

Un rapport sur l'ordinateur de l'analyste n'est pas un tableau de bord : c'est un brouillon. La **publication** envoie le rapport (et son modèle de données) dans un **espace de travail** en ligne, d'où d'autres personnes peuvent le consulter dans un navigateur. Quatre décisions accompagnent toujours la publication.

Qui voit quoi ? On donne les droits à des **groupes** (« direction », « responsables de région »), pas à des personnes, et on choisit entre partager le **rapport** (lecture seule) et le **jeu de données** (qui permet à d'autres de construire leurs propres rapports sur le même modèle). Selon les offres, la consultation par d'autres personnes peut exiger des **licences** payantes : renseignez-vous **avant** de promettre un partage à toute l'équipe.

Piège de sécurité. Certains outils offrent une option de **publication sur le web**, qui rend le rapport lisible **par n'importe qui sur Internet, sans authentification**. Elle ne convient qu'à des données **déjà publiques**. Ne l'utilisez jamais pour les chiffres de la boutique, et vérifiez dans la documentation ce que fait chaque option de partage **avant** de cliquer.

Quand les données sont-elles à jour ? On planifie une **actualisation** (par exemple chaque nuit). Si les données viennent d'un fichier ou d'une base **sur un poste ou un réseau local**, il faut en général une **passerelle** (un petit programme qui relie le serveur en ligne à vos données). Une

actualisation peut **échouer** sans bruit : on active les notifications d'échec, et on **affiche sur la page la date des données** (« données actualisées le 31/12/2025 »), comme la page de la gérante le fait en 2.3.

Quelle taille ? Le temps de réponse d'un tableau de bord dépend du volume de données chargées. Une technique classique est l'**agrégation** : on garde la table fine pour le détail et une **table agrégée** pour les vues d'ensemble. Nous la simulons :

```
sem = ventes["date_commande"].dt.to_period("W-SUN").dt.start_time
agreg = ventes.assign(semaine=sem).groupby(["semaine", "canal", "categorie"],
↳ as_index=False).agg(ca=("montant", "sum"), commandes=("id_commande", "nunique"))
print(len(ventes), "->", len(agreg), "lignes | CA identique :", np.isclose(ventes["montant"].sum(),
↳ agreg["ca"].sum()))
print("commandes distinctes :", ventes["id_commande"].nunique(), "| somme de la table agrégée :",
↳ agreg["commandes"].sum())
```

```
83905 -> 2830 lignes | CA identique : True
commandes distinctes : 36395 | somme de la table agrégée : 70758
```

La table agrégée est **trente fois plus petite** (2 830 lignes au lieu de 83 905), et le chiffre d'affaires est identique. Mais la **somme des commandes** y est de 70 758 au lieu de 36 395 : on a retrouvé le piège de la règle 2. **On n'agrège que des mesures additives**, et les mesures non additives restent calculées sur la table fine. Un bon outil sait choisir la table agrégée quand la question le permet et la table fine sinon ; vous devez vérifier qu'il ne fait pas d'erreur.

Qui est responsable ? Un tableau de bord publié a un **propriétaire** (le nom de la personne à contacter), une **version** et une **documentation** (le dictionnaire des indicateurs, 2.3.5). Sans cela, personne ne le corrige, et il continue d'afficher un chiffre faux pendant des mois.

À retenir.

- Un outil de tableaux de bord enchaîne **connexion, préparation, modèle, mesures, visuels, publication, actualisation** ; chaque étape se reproduit à la main et se **vérifie**.
- Le **modèle en étoile** (une table de faits, des dimensions à clé unique) est la base ; on contrôle les effectifs et les lignes orphelines ; on relie par des **identifiants**, jamais par des libellés.
- Une **mesure** est une recette recalculée dans chaque contexte de filtres ; un **ratio** est une somme sur une somme ; une mesure comme le nombre de commandes **ne s'additionne pas**.
- On définit les mesures **une fois**, dans le modèle ; on **planifie** l'actualisation, on **affiche la date des données**, on partage par **groupes** et on ne publie jamais sur le web des données internes.

Pour s'entraîner. Cahier, chapitre 2 : applications 2.1 et 2.2, exercices 2.1 à 2.3.

2.2 Tableau : feuilles, tableaux de bord, partage

Tableau est un outil de visualisation né de l'**exploration** : on glisse des champs sur une feuille, le graphique se dessine, on le modifie, on en essaie un autre. L'approche est symétrique de celle de la section 2.1 : au lieu de **construire le modèle puis dessiner**, on **dessine pour comprendre**, et le modèle suit. Cette section présente les idées de Tableau (les **marques** et leurs canaux, les **calculs de table**, les **niveaux de détail**, les tableaux de bord et leur partage), avec le même réflexe qu'avant : reproduire chaque résultat en pandas pour le vérifier.

Même précaution. Tableau est un produit commercial **non exécuté** ici : nous décrivons ses **notions**, pas ses menus, qui changent d'une version à l'autre et sont à vérifier dans la documentation de la vôtre. Les figures sont des maquettes calculées ou dessinées avec matplotlib, jamais des captures.

2.2.1 Feuilles, marques et canaux visuels

Une **feuille** est un graphique. Elle se construit en déposant des **champs** sur des **étagères**. Deux familles de champs existent :

- les **dimensions**, qui **découpent** (catégorie, canal, mois, région) ;
- les **mesures**, qui **comptent** (chiffre d'affaires, quantité, marge).

Les champs déposés sur les étagères *Colonnes* et *Lignes* fixent la **structure** de la feuille ; ceux déposés sur la carte des **marques** fixent l'**apparence** de chaque mark (un point, une barre, une ligne, un carré). Les canaux de la carte des marques sont ceux que vous connaissez : la **position**, la **couleur**, la **taille**, une **étiquette**, le **détail** (qui multiplie les marques sans les colorier) et l'**infobulle**. C'est exactement la grammaire des graphiques du chapitre 1 (section 1.1) : on **encode** une variable de la table par un canal visuel, et l'on choisit le canal selon la précision avec laquelle l'œil le lit (la position est lue le plus précisément, la taille et la couleur beaucoup moins).

Champ déposé...	...sur	Effet dans la feuille
categorie (dimension)	Lignes	une ligne de marques par catégorie
canal (dimension)	Colonnes	une colonne de marques par canal
montant (mesure, somme)	Couleur	l'intensité de chaque case dépend du chiffre d'affaires
montant (mesure, somme)	Taille	l'aire de chaque cercle dépend du chiffre d'affaires
id_commande (dimension)	Détail	une marque par commande, sans couleur supplémentaire

La « feuille » de la première ligne de ce tableau est, en pandas, un **tableau croisé** : une dimension en lignes, une en colonnes, une mesure agrégée dans les cases.

```
feuille = v25.pivot_table(index="categorie", columns="canal", values="montant", aggfunc="sum")
print((feuille / 1000).round(0).to_string())
```

canal	Boutique	Réseaux	Site
categorie			
Bien-être	49.0	14.0	55.0
Cuisine	99.0	25.0	110.0
Décoration	112.0	28.0	119.0
Jardin	149.0	39.0	166.0
Maison	129.0	35.0	141.0
Papeterie	24.0	6.0	27.0

La même feuille, projetée sur trois canaux visuels différents, donne trois lectures.

La position (barres) permet de **comparer précisément** ; la couleur (carte de chaleur) montre le **motif d'ensemble** mais ne se lit pas au chiffre près, d'où les valeurs écrites dans les cases ; la taille (cercles proportionnels) est la plus approximative. L'exercice de l'analyste est de choisir **le canal qui correspond à la question**, pas celui que l'outil propose par défaut. Ici, la question de la

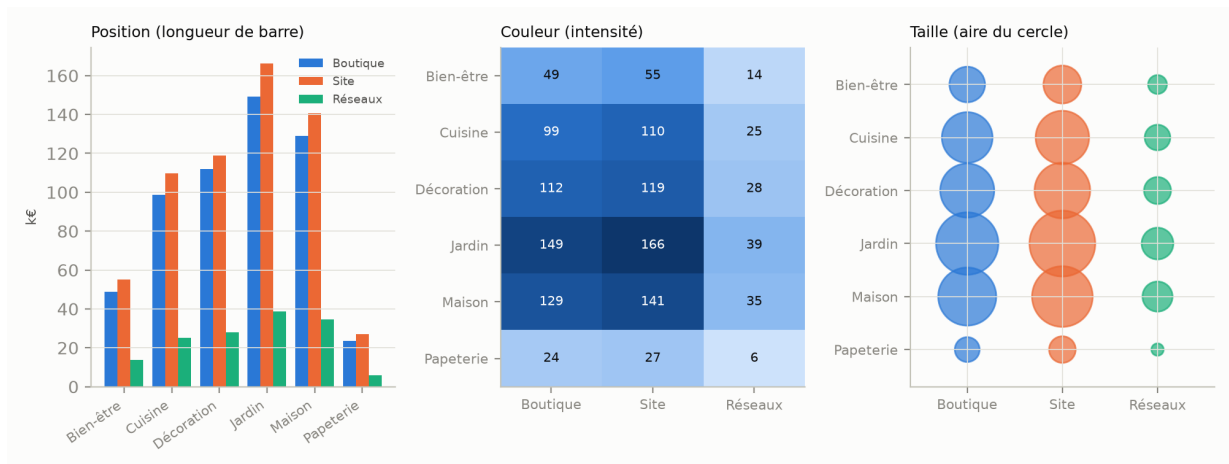


Figure 34: Une même feuille (chiffre d'affaires 2025 par catégorie et par canal) encodée de trois façons : la position, la couleur et la taille. Maquette dessinée avec matplotlib, pas une capture de Tableau.

gérante (« qu'est-ce qui vend le plus ? ») est une question de comparaison : on prend la position, on trie, et l'on garde la couleur pour une seule chose (par exemple, mettre en valeur la catégorie qui pose problème).

Intuition. Dans un outil comme Tableau, **construire une feuille revient à répondre à trois questions** : qu'est-ce que je découpe (dimensions), qu'est-ce que je compte (mesures), et par quel canal visuel je montre chaque variable. Le glisser-déposer ne dispense pas de les poser.

Les champs d'une feuille ont aussi un **type de granularité** : une dimension « discrète » donne des catégories séparées (un en-tête par valeur) ; une dimension « continue » (une date, un nombre) donne un axe continu. Une date peut s'utiliser de **deux façons** : discrète (« chaque mois, tous les ans confondus ») ou continue (une ligne du temps). Choisir la mauvaise donne soit une courbe sans sens, soit un axe sans repères ; vérifiez toujours quelle est la granularité attendue.

2.2.2 Calculs de table et niveaux de détail

Les champs de base ne suffisent pas : on veut des parts, des cumuls, des moyennes mobiles, des rangs. Tableau offre deux mécanismes, que l'on confond souvent parce qu'ils donnent des résultats d'apparence voisine, alors que **leur logique est différente**.

Les calculs de table : sur le résultat affiché

Un **calcul de table** s'applique à **ce qui est déjà affiché**, au tableau agrégé que la feuille vient de calculer : « part du total », « cumul », « moyenne mobile », « rang », « différence avec la ligne précédente ». Il dépend donc de la **structure** de la feuille : si l'on change les dimensions, il se recalcule autrement. Voici les quatre calculs sur les catégories de la boutique.

```
t = v25.groupby("categorie")["montant"].sum().sort_values(ascending=False).to_frame("ca")
t["pct_total"] = t["ca"] / t["ca"].sum() * 100
t["cumul_pct"] = t["pct_total"].cumsum()
t["rang"] = t["ca"].rank(ascending=False).astype(int)
print(t.round(1).to_string())
```

categorie	ca	pct_total	cumul_pct	rang
Jardin	353954.6	26.7	26.7	1
Maison	304614.0	23.0	49.7	2

Décoration	258742.3	19.5	69.2	3
Cuisine	233312.6	17.6	86.9	4
Bien-être	117510.8	8.9	95.7	5
Papeterie	56629.3	4.3	100.0	6

On lit tout de suite que **Jardin pèse 26,7 %** du chiffre d'affaires 2025, que **les trois premières catégories en font 69,2 %**, et que Papeterie, avec 4,3 %, ferme la marche. Chaque colonne est un calcul de table différent ; remarquez que le **cumul** n'a de sens que si l'ordre est celui du tri (sinon, il additionne dans un ordre arbitraire), et que le **rang** dépend du sens du tri choisi. Ce sont ces dépendances qu'il faut vérifier quand on pose un calcul de table : *sur quelle dimension se calcule-t-il, dans quel ordre ?*

La **moyenne mobile** est le calcul de table le plus utile pour une courbe hebdomadaire bruitée : la moyenne des quatre dernières semaines.

```
mm4 = hebdo["ca"].rolling(4).mean()
print(f"dernière semaine : {hebdo['ca'].iloc[-1]:,.0f} € | moyenne mobile sur 4 semaines :
↳ {mm4.iloc[-1]:,.0f} €".replace(", ", " "))
```

```
dernière semaine : 44 168 € | moyenne mobile sur 4 semaines : 43 018 €
```

La semaine du 22 décembre a rapporté 44 168 €, mais la moyenne des quatre dernières semaines est de 43 018 € : la moyenne mobile **lisse** le bruit, au prix d'un **retard** (elle met quelques semaines à réagir à un changement). Montrer les **deux** courbes est souvent la bonne réponse.

Les niveaux de détail : à une granularité choisie

Une expression de **niveau de détail** (en anglais *level of detail*, LOD) calcule une valeur à **une granularité que vous choisissez, indépendamment de la structure de la feuille**. Trois variantes existent : **FIXED** (on fixe la granularité : « le total par client »), **INCLUDE** (on ajoute une dimension à la feuille pour le calcul) et **EXCLUDE** (on en retire une). Le cas d'usage type est le **calcul en deux temps** : agréger à un niveau fin, puis agréger de nouveau à un niveau plus grossier.

Un exemple. « Quel est le chiffre d'affaires moyen par client ? » est une question à deux temps : on calcule d'abord le **total de chaque client**, puis on **moyenne ces totaux**. Moyenner directement les lignes de commande répondrait à une autre question (le montant moyen d'une ligne).

```
ca_client = v25.groupby("id_client")["montant"].sum()
print(len(ca_client), "clients actifs en 2025 | moyenne", round(ca_client.mean(), 2), "€ | médiane",
↳ round(ca_client.median(), 2), "€")
print("montant moyen d'une ligne :", round(v25["montant"].mean(), 2), "€")
```

```
3875 clients actifs en 2025 | moyenne 341.87 € | médiane 233.21 €
montant moyen d'une ligne : 44.41 €
```

Les **3 875 clients actifs** ont dépensé en moyenne **342 €** en 2025, la médiane est de **233 €** (quelques gros clients tirent la moyenne vers le haut, comme au volume III, section 1.1) ; une ligne de commande, elle, vaut en moyenne 44 €. Le **panier moyen** de 102 € est encore autre chose : le total d'une **commande**. Trois « **moyennes** » **légitimes, trois granularités**, et la boutique en a besoin des trois. C'est exactement ce que les niveaux de détail permettent de nommer proprement dans l'outil.

Intuition. Un calcul de table répond à « **que dit le tableau que j'ai sous les yeux ?** » (part, cumul, rang) ; un niveau de détail répond à « **que dit la table au niveau X, quoi que j'affiche ?** ». Quand un chiffre change parce que l'on a retiré une colonne de la feuille, c'est un calcul de table ; quand il ne bouge pas, c'est un niveau de détail.

Un point de méthode, enfin : les filtres n'agissent pas tous **au même moment**. Tableau applique ses opérations dans un **ordre fixe** (la documentation le résume sous le nom d'*ordre des opérations*) : par exemple, les filtres de dimension d'une feuille s'appliquent **après** un niveau de détail FIXED, ce qui peut surprendre (un filtre sur la catégorie ne change pas le « total par client » calculé en FIXED, à moins d'être défini comme filtre de contexte). L'équivalent pandas est de se demander si l'on **filtre avant ou après** le groupby :

```
tous = v25.groupby("id_client")["montant"].sum()
jardin = v25[v25["categorie"] == "Jardin"].groupby("id_client")["montant"].sum()
print(len(jardin), "clients du Jardin")
print("dépende dans le Jardin :", round(jardin.mean(), 2), "€ | dépense totale :",
      round(tous[jardin.index].mean(), 2), "€")
```

```
2402 clients du Jardin
dépende dans le Jardin : 147.36 € | dépense totale : 457.93 €
```

Selon que l'on filtre la catégorie **avant** ou **après** le premier regroupement, on ne répond pas à la même question : « combien dépense un client du Jardin **en tout** ? » ou « combien dépense-t-il **dans** la catégorie Jardin ? ». Les 2 402 clients qui ont acheté dans le Jardin ont dépensé **457,93 €** en moyenne sur l'ensemble de la boutique en 2025, mais **147,36 €** dans la seule catégorie Jardin : un facteur trois. Le premier chiffre est celui d'une expression FIXED (le total du client, quoi que la feuille filtre) ; le second, celui d'un calcul sur la feuille filtrée. **Les deux sont justes, mais l'un d'eux seulement est celui que l'on attendait** ; vérifiez dans l'outil lequel s'affiche.

Piège. Un calcul de table ou un niveau de détail qui **donne un résultat plausible** n'est pas forcément le résultat voulu. Après chaque calcul, posez-vous trois questions : sur quelle granularité ? avec quels filtres ? dans quel ordre ? Et reproduisez le chiffre d'une case en pandas.

2.2.3 Du classeur au tableau de bord

Les feuilles sont des briques ; le **tableau de bord** les assemble sur une page. Les outils de la famille de Tableau l'organisent avec des **conteneurs** (horizontaux ou verticaux) qui répartissent l'espace, et offrent le choix entre une **taille fixe** (la page a les dimensions voulues, elle est identique sur tous les écrans, avec des barres de défilement si l'écran est trop petit) et une taille **adaptée** (elle s'étire, mais la disposition peut s'abîmer). Pour un tableau de bord lu **sur un écran de bureau**, la taille fixe et un ordre de lecture soigné (section 2.3.4) donnent les résultats les plus prévisibles ; pour un affichage **sur téléphone**, on prépare une **mise en page dédiée**, plus étroite et plus simple, plutôt que de réduire la version de bureau.

Ce qui distingue un tableau de bord d'un ensemble de graphiques, ce sont les **actions** : cliquer sur une barre pour **filtrer** les autres feuilles, **surligner** les points apparentés, ouvrir une page de détail ou un document externe, changer un **paramètre** (par exemple, le seuil de l'alerte). Chaque action est un petit contrat avec le lecteur : « si vous cliquez ici, il se passe ceci ». Il faut le **dire** (une phrase d'instruction, un curseur visible, un bouton « effacer les filtres ») ; une interaction cachée n'existe pas pour la plupart des lecteurs.

La **connexion** aux données reprend le choix de la section 2.1.1 : une connexion **directe** interroge la source à chaque action ; un **extrait** (*extract*) est une copie compressée, plus rapide, actualisée à des heures fixes. Pour un tableau de bord hebdomadaire, l'extrait est le choix raisonnable.

Le **partage** se fait selon trois grandes formules. Un **serveur** ou un **service en ligne** de l'éditeur, avec des comptes, des groupes et des droits : c'est la voie normale en entreprise. Une **publication publique** (il existe une offre gratuite qui publie sur un site ouvert à tous) : **tout y est public**, données comprises, et l'on n'y met jamais de données de la boutique. Enfin, le **fichier** exporté (image, PDF, classeur) : pratique pour une réunion, mais figé, sans actualisation, et facile à transmettre

hors de tout contrôle. On vérifie dans la documentation de sa version ce que chaque formule permet exactement, et à quelles conditions de licence.

Les **histoires** (*stories*) enchaînent plusieurs feuilles ou tableaux de bord en une séquence commentée : c'est un outil de présentation, étudié au chapitre 4 (section 4.3).

2.2.4 Deux philosophies, un même chemin

Les approches de Power BI et de Tableau diffèrent par leur **point de départ**, et se rejoignent très vite. L'analyste qui connaît l'une apprend l'autre en quelques semaines **à condition d'avoir compris les notions de fond**.

Notion	Approche « modèle d'abord » (Power BI, 2.1)	Approche « exploration d'abord » (Tableau, 2.2)
Point de départ	le modèle de données, relié en étoile	une feuille, puis les liens entre les sources
Calcul réutilisable	mesure écrite dans le modèle	champ calculé , calcul de table ou niveau de détail
Granularité	contexte de filtres de chaque visuel	dimensions de la feuille, niveaux de détail
Force	cohérence de définitions entre rapports	rapidité d'exploration, souplesse visuelle
Risque	modèle lourd à faire évoluer	calculs dispersés dans chaque classeur

Cette comparaison est volontairement **schématique** : les deux produits ont tellement évolué que chacun a emprunté à l'autre. Retenez-en l'essentiel : **dans les deux cas, le travail difficile est le même** : un modèle de données juste, des définitions claires, des chiffres vérifiés. Le reste (choix de couleur, disposition, interaction) relève de la conception, que nous abordons en 2.3.

À retenir.

- Une **feuille** encode des champs par des **canaux** (position, couleur, taille, détail) ; on choisit le canal selon la **question** et selon la précision de lecture.
- Un **calcul de table** opère sur le **tableau affiché** (part, cumul, moyenne mobile, rang) et dépend de la structure de la feuille ; un **niveau de détail** opère à une **granularité choisie**, indépendamment d'elle.
- Un chiffre plausible n'est pas forcément le **chiffre voulu** : on se demande sur quelle granularité, avec quels filtres, dans quel ordre, et on **recalcule en pandas** une case de la feuille.
- Un tableau de bord assemble des feuilles avec des **actions** qu'il faut **annoncer** ; on partage par groupes, et une publication **publique** ne convient jamais aux données internes.

Pour s'entraîner. Cahier, chapitre 2 : application 2.3, exercices 2.4 et 2.5.

2.3 Conception d'un tableau de bord selon les besoins des utilisateurs

Les sections précédentes ont montré **comment** un outil fabrique un tableau de bord. Celle-ci répond à la question qui décide de sa réussite : **pour qui et pour quoi** ? Un tableau de bord conçu à partir des données disponibles (« montrons tout ce que nous avons ») est presque toujours abandonné ; un tableau de bord conçu à partir des **décisions** de la personne qui le lit devient un rituel du lundi matin. Nous suivons la méthode de bout en bout et construisons la page de la gérante, dont tous les chiffres sont recalculés ici.

2.3.1 Partir de la décision, pas des données

La première erreur est de commencer par la donnée : « nous avons des ventes, des stocks, des livraisons, faisons un graphique de chaque ». La méthode inverse tient en une chaîne de questions, que l'on pose **à la gérante avant d'ouvrir l'outil** :

1. Quelles **décisions** prenez-vous régulièrement ? (commander, relancer un transporteur, lancer ou arrêter une promotion...)
2. Quelle **question** chacune de ces décisions pose-t-elle ? (« Allons-nous manquer de produits ? »)
3. Quel **indicateur** y répond, avec quelle définition, et par rapport à quelle référence ?
4. À partir de quelle valeur **agit-on** ?
5. À quelle **fréquence** regarde-t-on, et sur quel écran ?

Pour la boutique, l'entretien d'une demi-heure avec la gérante donne le tableau suivant.

Décision	Question	Indicateur	On agit si...
Relancer le transporteur	Les clients reçoivent-ils à temps ?	Livraisons à l'heure (%)	sous la limite basse de l'habituel
Commander du stock	Allons-nous manquer de produits ?	Rupture sur les 20 produits suivis (%)	au-dessus de la limite haute de l'habituel
Ajuster l'effort commercial	Les ventes suivent-elles ?	Chiffre d'affaires, commandes	durablement sous l'an dernier
Corriger l'offre ou les prix	Gagnons-nous toujours de l'argent ?	Taux de marge brute, panier moyen	en recul marqué
Travailler le site	Le site convertit-il ?	Taux de conversion	hors de la fourchette habituelle

Remarquez ce que le tableau ne contient pas : aucun indicateur « parce qu'on peut le calculer ». Chaque ligne est **reliée à une décision**, ce qui est la règle du volume III (section 6.1.1) ; chaque indicateur a une définition écrite (6.1.2) et, pour les deux alertes, un seuil fondé sur la variabilité observée (6.3.5). Ce que l'on ajoute ici, c'est un **écran** : les alertes ne sont utiles que si la gérante les voit au moment où elle décide.

Le processus est **itératif** : on interroge, on **croque** la page sur papier (cinq minutes, aucun outil), on construit un **prototype** avec des données réelles, on le fait **tester** (on montre la page cinq secondes à quelqu'un qui ne la connaît pas, puis on lui demande ce qu'il en a retenu), on publie, puis on **mesure l'usage** et l'on corrige. Le test de cinq secondes est volontairement cruel : si la personne ne dit pas « les livraisons ne vont pas bien », la page n'a pas rempli son rôle, quel que soit son aspect.

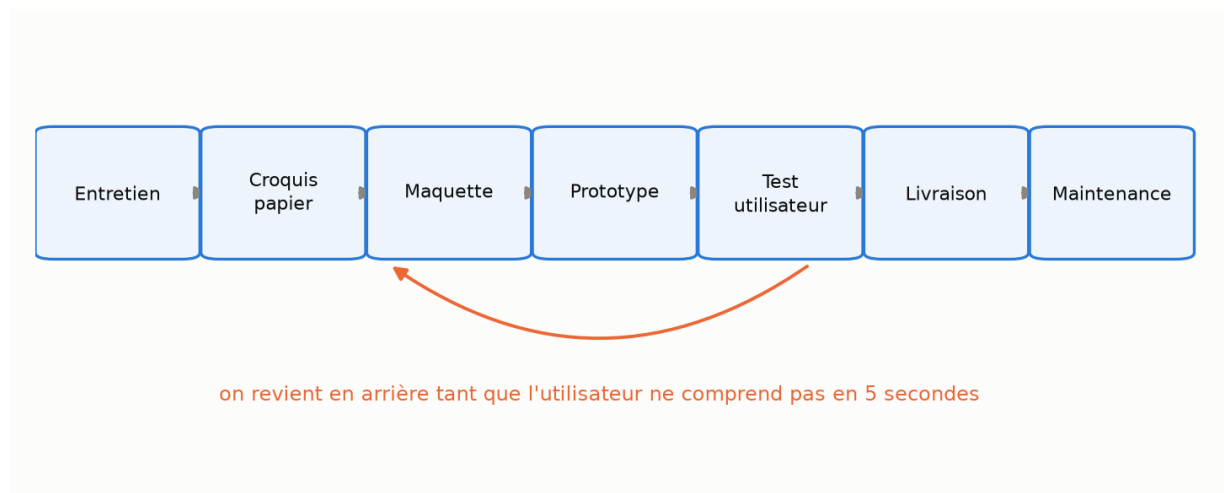


Figure 35: Le processus de conception d'un tableau de bord : interroger, croquer, prototyper, tester en cinq secondes, publier, mesurer l'usage ; on revient en arrière tant que l'utilisateur ne comprend pas. Maquette dessinée.

Intuition. Pour savoir ce qu'un tableau de bord doit contenir, demandez à son lecteur **ce qu'il fera lundi matin** selon ce qu'il verra. Un chiffre qui ne change aucune décision est du décor.

2.3.2 Trois niveaux : vue d'ensemble, analyse, détail

Un utilisateur n'a pas toujours le même besoin : tantôt **savoir si tout va bien**, tantôt **comprendre pourquoi** quelque chose ne va pas, tantôt **retrouver une ligne précise**. Un tableau de bord qui tente de tout faire sur une page est illisible ; la structure classique répartit ces trois besoins sur **trois niveaux**, reliés par des liens de navigation.

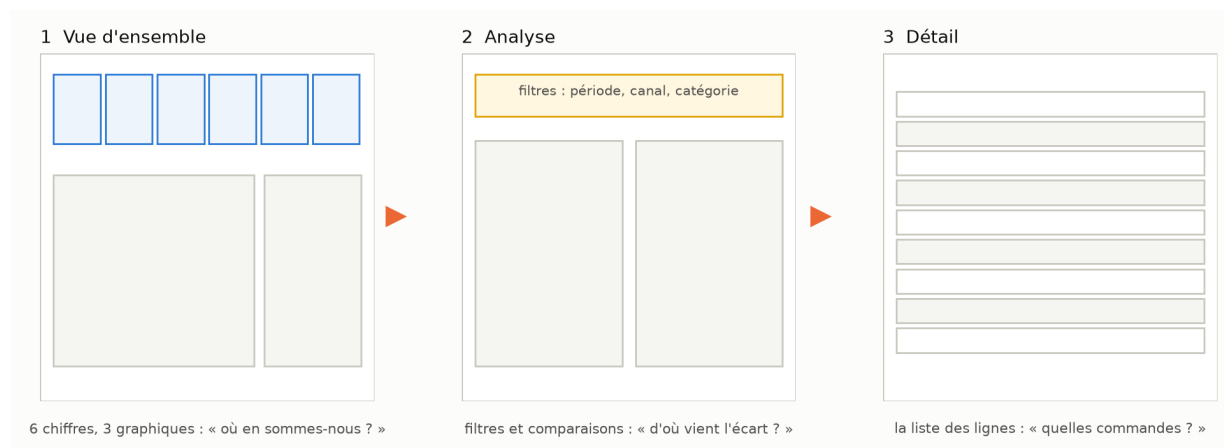


Figure 36: Les trois niveaux d'un tableau de bord : une vue d'ensemble (quelques chiffres et alertes), des pages d'analyse (comparer, ventiler, filtrer), un niveau de détail (tableau de lignes). Chaque niveau répond à une question plus fine que le précédent. Maquette dessinée.

- **Niveau 1, la vue d'ensemble** : cinq à huit chiffres, une courbe, une zone d'alertes. Elle se lit en **cinq secondes** et ne demande aucun clic. C'est la page de la gérante.
- **Niveau 2, l'analyse** : des pages par thème (ventes, livraisons, stock), avec des filtres (période, canal, catégorie) et des comparaisons. Elle répond à « **où et pourquoi ?** ». Les arbres

d'indicateurs du volume III (section 6.2) en sont le plan : si le chiffre d'affaires baisse, on descend vers le trafic, la conversion ou le panier.

- **Niveau 3, le détail : le tableau de lignes** (les commandes en retard, les produits en rupture), exportable. Il répond à « **lesquelles ?** » et sert à agir (appeler le transporteur au sujet de ces douze colis).

La règle de navigation est de **descendre en cliquant** (de la carte vers l'analyse, de l'analyse vers le détail) et de pouvoir **remonter** en un clic. Chaque page porte un titre qui dit **ce qu'elle montre** et la date des données ; les filtres actifs sont écrits en toutes lettres, pour qu'un lecteur ne prenne pas un chiffre filtré pour un chiffre global (voir 2.1.4).

2.3.3 La page de la gérante

Construisons maintenant la page du niveau 1. Elle présente six chiffres, un graphique de tendance, une répartition par canal, une courbe de contrôle et une zone d'alertes. D'abord les chiffres : la semaine du 22 au 28 décembre 2025, comparée à la **même semaine un an plus tôt** (364 jours avant, pour comparer un lundi à un lundi).

```
k, _ = 0.kpi_semaine(d, "2025-12-22")
cs, ad = k["cette_semaine"], k["an_dernier"]
print(f"CA {cs['ca'] / 1000:.1f} k€ ({cs['ca'] / ad['ca'] - 1:+.1%}) | commandes
↳ {cs['commandes']:.0f} ({cs['commandes'] / ad['commandes'] - 1:+.1%})")
print(f"panier {cs['panier']:.1f} € ({cs['panier'] / ad['panier'] - 1:+.1%}) | marge
↳ {cs['taux_marge']:.1%} ({(cs['taux_marge'] - ad['taux_marge']) * 100:+.1f} pts)")
```

```
CA 44.2 k€ (+34.9%) | commandes 441 (+19.2%)
panier 100.2 € (+13.2%) | marge 39.4% (+2.2 pts)
```

Sur ces quatre chiffres, la semaine est **excellente** : 44,2 k€ de chiffre d'affaires (+34,9 % sur l'an dernier), 441 commandes (+19,2 %), un panier moyen de 100,2 € (+13,2 %) et un taux de marge brute de 39,4 % (+2,2 points). C'est la **deuxième meilleure semaine de 2025**. Mais la gérante a demandé qu'on lui dise **quand quelque chose ne va pas**, et deux indicateurs ne vont pas bien du tout. Les alertes utilisent la règle de 6.3.5 : on compare la semaine à la **moyenne des 26 semaines précédentes**, avec une **limite à trois écarts-types**.

```
t0 = pd.Timestamp("2025-12-22")
moy, bas, _ = 0.limite(hebdo["a_l_heure"], t0)
moy_r, _, haut_r = 0.limite(hebdo["rupture"], t0)
print(f"livraisons à l'heure : {cs['a_l_heure']:.1%} | habituel {moy:.1%} | limite basse {bas:.1%}")
print(f"rupture : {cs['rupture']:.1%} | habituel {moy_r:.1%} | limite haute {haut_r:.1%}")
```

```
livraisons à l'heure : 44.5% | habituel 76.3% | limite basse 48.5%
rupture : 35.0% | habituel 8.4% | limite haute 28.3%
```

Les **livraisons à l'heure** sont à **44,5 %** alors que l'habituel est de **76,3 %** (la limite basse est à 48,5 %) : la semaine est **sous la limite**. La **rupture** touche **35,0 %** des produits suivis, contre 8,4 % d'habitude (limite haute : 28,3 %) : elle est **au-dessus de la limite**. Les deux alertes se confirment l'une l'autre, ce qui est cohérent avec l'histoire du volume III : en décembre, la demande monte, les stocks se vident, et le transporteur le plus lent s'engorge.

Un détail vaut d'être souligné. Le taux de livraisons à l'heure de **la même semaine, il y a un an**, était de 40,2 % : l'indicateur est donc **en hausse de 4,3 points** sur l'an dernier, alors qu'il est à **32 points de son niveau habituel**. Si la page ne montrait que la comparaison à l'an dernier, elle afficherait un petit +4,3 en vert : **la bonne référence n'est pas toujours l'an dernier**. Pour des livraisons, c'est le niveau **habituel** ; pour des ventes saisonnières, c'est la même période de l'an dernier. La page affiche donc, pour chaque carte, **la référence qui convient à sa décision**, et l'écrit.

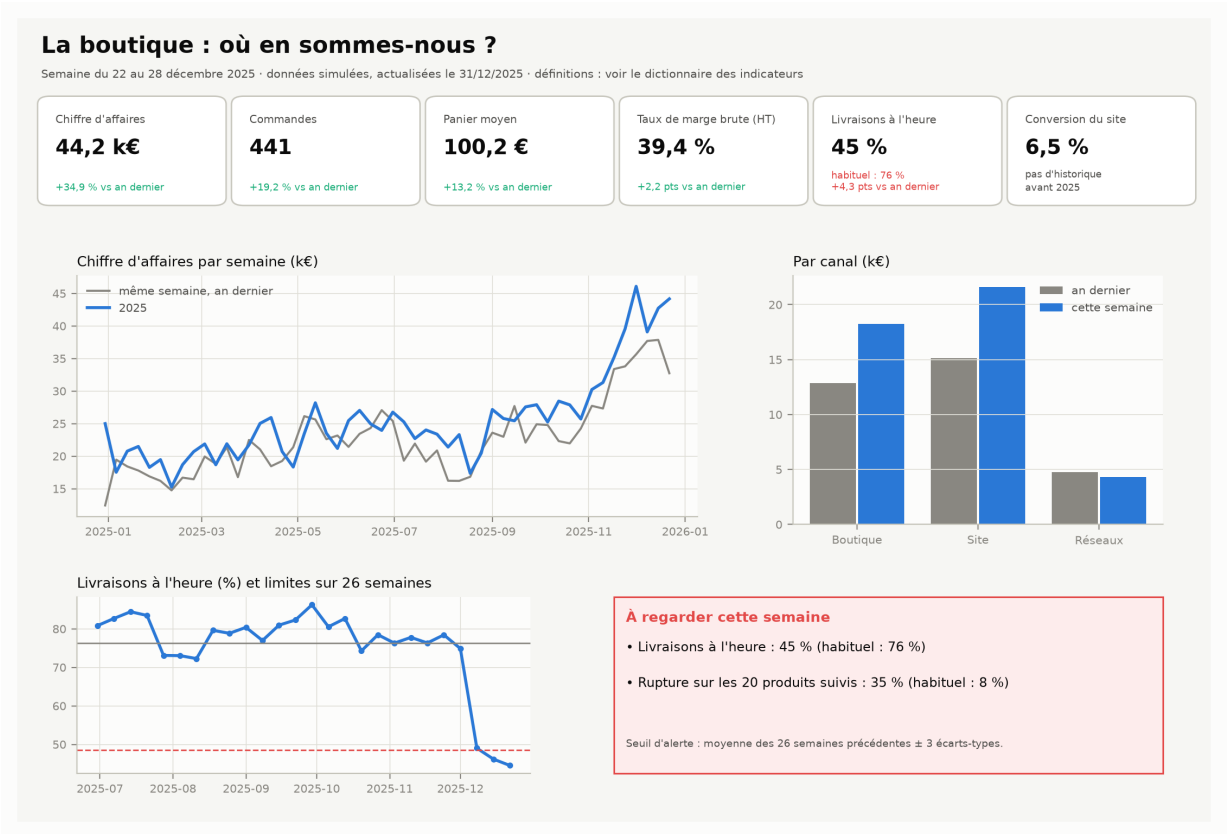


Figure 37: La page d'une semaine pour la gérante : six chiffres avec leur référence, la tendance du chiffre d'affaires contre l'an dernier, la répartition par canal, la courbe de contrôle des livraisons et la zone d'alertes. Données simulées ; maquette calculée avec matplotlib, pas une capture d'un outil de tableaux de bord.

La page respecte les quatre règles du chapitre 1 : **un message par graphique** (le titre dit ce que l'on regarde), **une seule couleur d'alerte** (le rouge ne sert qu'aux deux alertes), **la même couleur pour la même chose** (le gris est toujours « l'an dernier », le bleu toujours « cette année ») et **les références visibles** (« habituel : 76 % », « vs an dernier »). Les chiffres des cartes viennent des fonctions de ce chapitre, et ceux du graphique de tendance sont les mêmes que ceux de la section 2.1 : c'est le **même modèle** qui les alimente.

2.3.4 Disposition et lecture : le test de cinq secondes

Un lecteur ne **lit** pas une page : il la **balaie**. Pour une page en langue française, le regard part du coin supérieur gauche, parcourt la première ligne, redescend en diagonale vers la gauche, puis balaie la ligne suivante : c'est le **parcours en Z**. On y place donc, dans l'ordre, ce qui compte le plus : le titre et le contexte, les **chiffres clés**, la **tendance**, puis le **détail** ou les alertes.

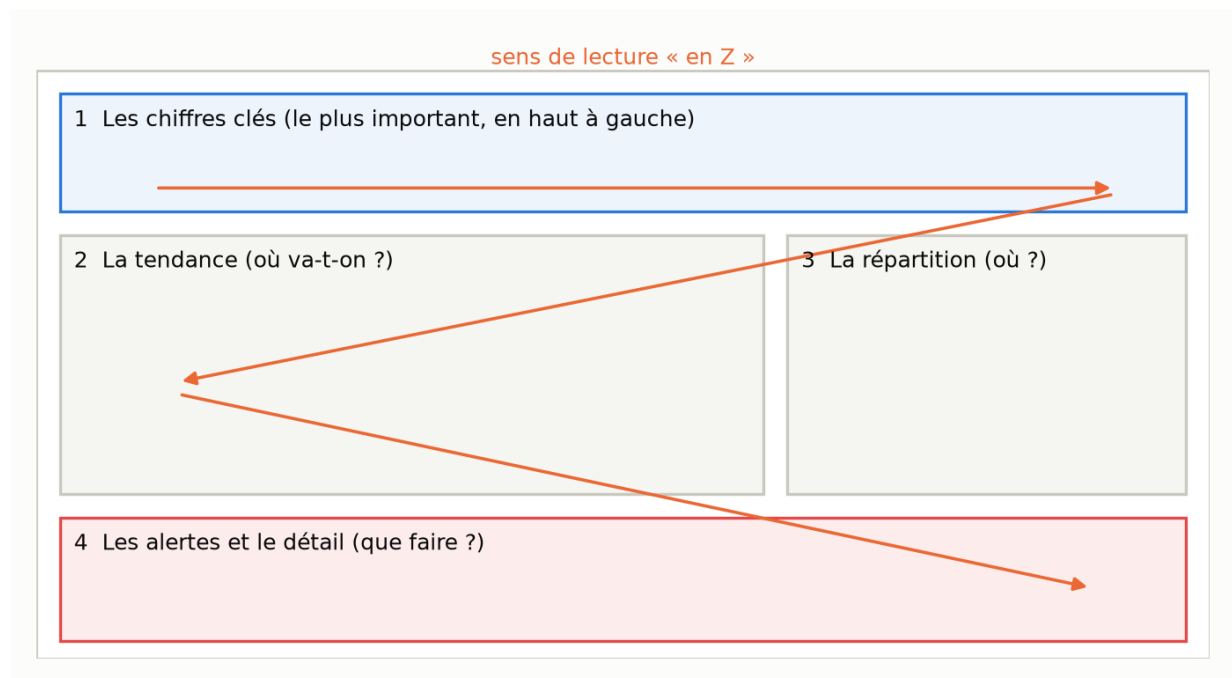


Figure 38: Le parcours de lecture en « Z » : titre et contexte en haut à gauche, chiffres clés sur la première ligne, tendance et répartition au milieu, alertes et notes en bas. Maquette dessinée.

La **densité** compte autant que l'ordre. Les maquettes suivantes présentent le **même contenu** : à gauche, quatorze éléments et neuf couleurs ; à droite, six chiffres, trois graphiques et une seule couleur d'alerte.

Une règle pratique : **entre cinq et huit chiffres** sur la page principale, et pas plus de trois graphiques. Au-delà, on a un document, pas un tableau de bord. Si la gérante a besoin de plus, c'est qu'il faut une deuxième page (niveau 2), pas une page plus chargée.

La liste de contrôle, appliquée à notre propre page

Une liste de contrôle est un outil de relecture, pas une récitation. Passons notre page au crible, avec cinq questions.

1. **Le test de cinq secondes.** Que lit-on en cinq secondes ? Les chiffres en gros, puis la zone rouge « À regarder cette semaine ». C'est le but.
2. **Chaque chiffre a-t-il sa référence ?** Oui, sauf la conversion du site : aucune comparaison à l'an dernier, car les sessions n'existent que pour 2025 (volume III, section 6.1.4). La page le **dit** (« pas d'historique avant 2025 ») au lieu de laisser un blanc.

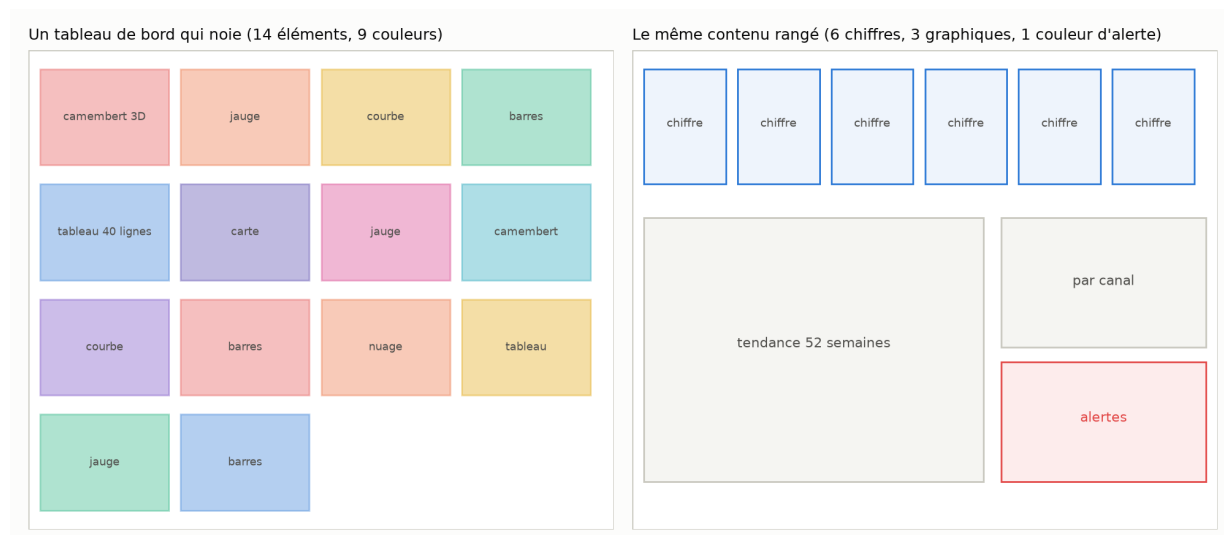


Figure 39: Le même contenu, noyé puis ordonné : à gauche un tableau de bord qui montre tout, à droite le même rangé selon l'importance. Maquettes dessinées pour comparer la lecture.

3. **Les couleurs portent-elles seules le message ?** Non : le sens de chaque variation est écrit (« +34,9 % », « habituel : 76 % »), le rouge n'est jamais la seule information. Un lecteur daltonien comprend la même chose.
4. **Les textes sont-ils lisibles ?** C'est le point le plus faible. Le contraste d'un texte se mesure : le rapport entre la luminance du texte et celle du fond doit atteindre **4,5 : 1** pour du petit texte (3 : 1 pour du grand texte ou des éléments graphiques), selon les recommandations d'accessibilité usuelles. Calculons-le pour les couleurs de la page sur fond blanc.

```
def contraste(a, b="#ffffff"):
    def lum(h):
        c = [int(h[i:i + 2], 16) / 255 for i in (1, 3, 5)]
        c = [x / 12.92 if x <= 0.03928 else ((x + 0.055) / 1.055) ** 2.4 for x in c]
        return 0.2126 * c[0] + 0.7152 * c[1] + 0.0722 * c[2]
    hi, lo = sorted([lum(a), lum(b)], reverse=True)
    return (hi + 0.05) / (lo + 0.05)
for nom, c in {"bleu": "#2a78d6", "rouge": "#e34948", "gris moyen": "#898781", "gris texte": "#52514e"}.items():
    print(f"{nom:<11} {contraste(c):.2f} : 1")
```

```
bleu      4.42 : 1
rouge     3.95 : 1
gris moyen 3.59 : 1
gris texte 7.94 : 1
```

Le **bleu** (4,42 : 1) sert à des lignes et des barres : c'est suffisant pour des éléments graphiques (3 : 1). Le **gris texte** (7,94 : 1) convient à tous les textes. Mais le **rouge** des petits textes d'alerte (3,95 : 1) et le **gris moyen** des axes (3,59 : 1) sont **sous le seuil de 4,5 : 1** pour du petit texte. Dans notre maquette, on les corrigerait en production en **fonçant** le rouge des textes (en gardant le rouge actuel pour les aplats et les traits) et en écrivant les axes en gris texte. Un contrôle de cette sorte prend une minute et évite que la page soit illisible sur un écran de salle de réunion.

5. **Le tableau de bord résiste-t-il à la semaine suivante ?** C'est-à-dire : la page se **recalcule-t-elle seule**, avec une date paramétrée, et reste-t-elle lisible quand les chiffres changent (une valeur à sept chiffres, une cible à zéro, une semaine sans donnée) ? C'est la question de l'automatisation, que traite le chapitre 4 (section 4.4).

Pièges de lecture à repérer. Les cartes vertes ou rouges **sans** texte ; un axe qui ne part pas de zéro pour des barres ; des couleurs qui changent de sens d'un graphique à l'autre ; une période de comparaison non précisée ; un filtre actif qu'on ne voit pas. Chaque défaut figure dans la liste du chapitre 1 (section 1.4) : on les cherche **sur sa propre page**.

2.3.5 Valider, documenter, faire adopter

Une page jolie et fausse est plus dangereuse qu'une page laide et juste. Trois gestes protègent la gérante.

La recette : le même chiffre par deux chemins. Avant la mise en service, on recalcule les chiffres clés **indépendamment de l'outil** et on compare. Pour la boutique, deux contrôles suffisent : le total d'une année calculé en SQL et en pandas, et la semaine de la page recalculée directement sur la table de faits.

```
import sqlite3
con = sqlite3.connect(":memory:")
faits.to_sql("fait_ventes", con, index=False)
sql = "SELECT ROUND(SUM(montant), 2) FROM fait_ventes WHERE date >= '2025-01-01'"
print("CA 2025 : SQL", con.execute(sql).fetchone()[0], "| pandas", round(v25["montant"].sum(), 2))
sem = faits[(faits["date"] >= "2025-12-22") & (faits["date"] <= "2025-12-28")]
print("semaine du 22/12 : table de faits", round(sem["montant"].sum(), 2), "| page",
      round(hebdo.loc["2025-12-22", "ca"], 2))
```

```
CA 2025 : SQL 1324763.72 | pandas 1324763.72
semaine du 22/12 : table de faits 44167.99 | page 44167.99
```

Les deux calculs du chiffre d'affaires 2025 donnent **1 324 763,72 €** (SQL et pandas) et la semaine du 22 décembre **44 167,99 €** de deux manières : la page affiche bien ce que la table contient. On ajoute le **rapprochement avec une source indépendante** quand il en existe une (la comptabilité, les relevés de caisse), et l'on **explique** l'écart jusqu'au centime, comme au volume II (section 3.3).

En pratique. Gardez un petit **classeur de recette** : une page par indicateur, avec la valeur affichée par le tableau de bord, la valeur recalculée à part, l'écart, la date et la personne qui a vérifié. Il sert à **chaque** modification du modèle ou d'une mesure.

Le dictionnaire des indicateurs. Chaque chiffre de la page a une fiche : nom, définition en une phrase, formule, source, fréquence d'actualisation, propriétaire (désigné par sa **fonction**). Voici celle de la page de la gérante.

Indicateur	Définition	Source	Actualisation
Chiffre d'affaires	somme des montants TTC des lignes de commande de la semaine (lundi à dimanche)	table de faits	chaque nuit
Commandes	nombre de commandes distinctes de la semaine	table de faits	chaque nuit
Panier moyen	chiffre d'affaires ÷ commandes	mesure	chaque nuit
Taux de marge brute (HT)	marge hors taxes ÷ chiffre d'affaires hors taxes (TVA 20 % pour l'illustration)	mesure	chaque nuit

Indicateur	Définition	Source	Actualisation
Livraisons à l'heure	part des colis livrés dans la semaine qui n'ont pas de retard	livraisons	chaque nuit
Rupture	part des jours-produits en rupture, sur 20 produits suivis	stock quotidien	chaque nuit
Conversion du site	part des sessions du site qui donnent une commande	sessions web	chaque nuit

Les définitions importantes se discutent **avant** : « livraisons à l'heure » est calculé sur les colis **livrés** cette semaine (on ne connaît le retard d'un colis qu'à sa livraison) ; une autre définition (colis **commandés** cette semaine) donnerait un autre chiffre, et le dictionnaire dit laquelle est la bonne.

L'adoption. Un tableau de bord s'impose par l'usage, pas par la décision de le déployer. Quatre habitudes y aident : un **rendez-vous** (cinq minutes le lundi, avec la page à l'écran), un **propriétaire** nommé qui répond aux questions, une **mesure de l'usage** (les outils disent qui ouvre quoi : un visuel que personne ne regarde se retire) et un **journal des changements** (« la définition des livraisons à l'heure a changé le... »). Cinq raisons font mourir un tableau de bord : trop de chiffres, des chiffres qui contredisent ceux d'un autre document, des données qui ne sont plus à jour, aucune action qui en découle, et une page lente.

À retenir.

- On part des **décisions** de l'utilisateur : décision, question, indicateur, seuil d'action, fréquence. Un chiffre qui ne change aucune décision est du décor.
- **Trois niveaux** : vue d'ensemble (cinq à huit chiffres, lisible en cinq secondes), analyse (où, pourquoi), détail (lesquels). On **descend en cliquant** et on peut remonter.
- Chaque chiffre porte la **référence qui convient à sa décision** (l'habituel pour les livraisons, l'an dernier pour les ventes) ; les alertes viennent de **limites fondées sur la variabilité**.
- Le test de cinq secondes, le parcours en Z et la liste de contrôle (références, couleurs, contraste, lisibilité) s'appliquent à **sa propre page** ; le contraste se **calcule**.
- On **valide** par deux chemins indépendants, on **documente** dans un dictionnaire, on **suit l'usage** et l'on tient un journal des changements.

Pour s'entraîner. Cahier, chapitre 2 : applications 2.4 et 2.5, exercices 2.6 à 2.9.

2.4 ⊕ Pour aller plus loin : Power BI avancé, DAX, Power Query, sécurité par lignes, déploiement

Section optionnelle. Elle s'adresse à celles et ceux qui construiront des modèles de données pour d'autres personnes. Elle suppose les notions de 2.1 (étoile, mesures, contexte de filtres). Comme avant, **Power BI n'est pas exécuté** : le code DAX et M qui apparaît dans cette section est donné à **titre indicatif**, dans des blocs marqués « non exécuté » ; **ce qui est vérifié, c'est l'équivalent pandas**, dont les chiffres sont recalculés ici. La syntaxe exacte et les fonctions

disponibles sont à vérifier dans la documentation de votre version.

Quatre savoirs distinguent un tableau de bord de démonstration d'un modèle de production : le langage des mesures (**DAX**), la préparation des données (**Power Query**), la **sécurité par lignes** et le **déploiement**. Chacun a un équivalent que vous connaissez déjà, ce qui donne une méthode : on écrit l'équivalent pandas, on **le fait parler**, puis on compare au chiffre de l'outil.

2.4.1 DAX : le langage des mesures

DAX est le langage des mesures de Power BI. Il ressemble à une formule de tableur, mais il se comporte comme un **langage de requête sur un modèle** : toute expression s'évalue **dans un contexte de filtres**. Deux notions font l'essentiel.

Le **contexte de ligne** : « pour cette ligne de la table », utilisé dans une colonne calculée ou dans une fonction d'itération (SUMX). Le **contexte de filtres** : « sur ces lignes-là », construit par la page, les segments et les cellules du visuel. La fonction centrale, CALCULATE, **modifie** le contexte de filtres avant d'évaluer une expression : elle ajoute un filtre, en remplace un, ou en retire un (ALL).

Quelques mesures de la boutique, écrites en DAX :

```
-- DAX : indicatif, non exécuté
CA = SUM ( fait_ventes[montant] )
Marge HT = SUMX ( fait_ventes,
    fait_ventes[montant] / 1,2
    - fait_ventes[quantite] * RELATED ( dim_produit[cout_achat] ) )
CA an dernier = CALCULATE ( [CA], SAMEPERIODYEAR ( dim_date[date] ) )
Évolution = DIVIDE ( [CA] - [CA an dernier], [CA an dernier] )
CA cumul annuel = TOTALYTD ( [CA], dim_date[date] )
Part du canal = DIVIDE ( [CA], CALCULATE ( [CA], ALL ( dim_canal ) ) )
```

Lisons-les. Marge HT itère sur chaque ligne de faits (contexte de ligne) et va chercher le coût d'achat du produit grâce à la relation (RELATED). CA an dernier demande à CALCULATE de **décaler le contexte de date d'un an** : c'est la fonction de comparaison dans le temps, qui **exige une dimension de dates continue** (section 2.1.2). DIVIDE est une division qui renvoie un résultat vide, au lieu d'une erreur, quand le dénominateur est nul. Part du canal enlève le filtre de canal au dénominateur (ALL) pour obtenir le total de référence.

Chacune a un équivalent en pandas, que nous exécutons pour vérifier les chiffres que l'outil devrait afficher. D'abord les comparaisons dans le temps.

```
mois = 0.ca_par_mois(d)
dec25, dec24 = mois.loc[12, 2025], mois.loc[12, 2024]
print(f"décembre : {dec25:,.0f} € contre {dec24:,.0f} € ({dec25 / dec24 - 1:+.1%})".replace(","," "))
cumul = mois.cumsum()
c25, c24 = cumul.loc[9, 2025], cumul.loc[9, 2024]
print(f"cumul à fin septembre : {c25:,.0f} € contre {c24:,.0f} € ({c25 / c24 - 1:+.1%})".replace(","," ",
↵ " "))
print(f"... comparé à toute l'année 2024 : {c25 / cumul.loc[12, 2024] - 1:+.1%}")
```

```
décembre : 183 845 € contre 157 304 € (+16.9%)
cumul à fin septembre : 876 963 € contre 798 738 € (+9.8%)
... comparé à toute l'année 2024 : -26.3%
```

Décembre 2025 a rapporté 183 845 € contre 157 304 € en décembre 2024 (+16,9 %), et le cumul depuis janvier est de 876 963 € à fin septembre contre 798 738 € un an plus tôt (+9,8 %). La dernière ligne rappelle le **piège classique** : comparer neuf mois de 2025 aux **douze** mois de 2024 donne -26,3 %, un résultat absurde parce qu'on compare des périodes de longueurs différentes. La fonction « même période de l'an dernier » existe pour l'éviter : **on compare à période égale**.

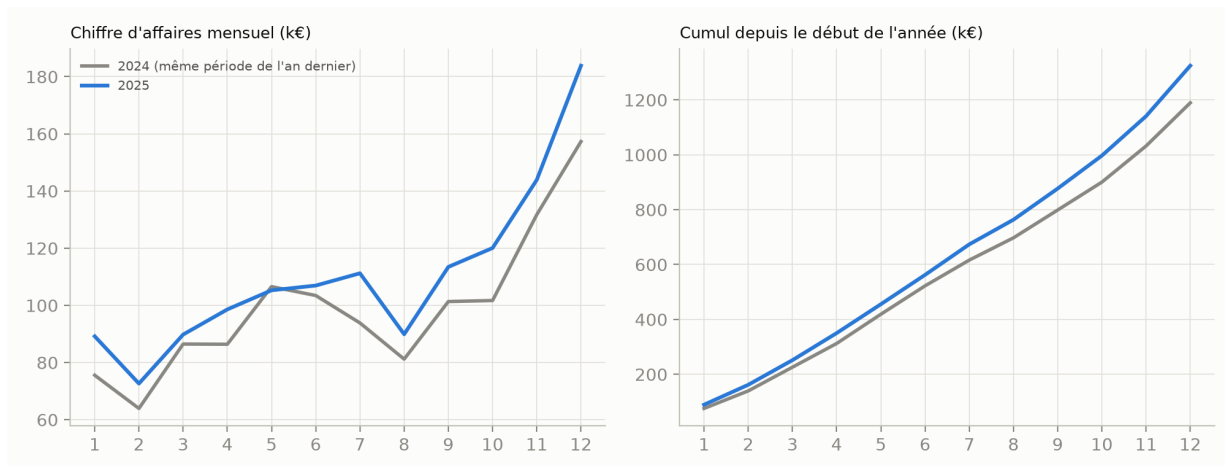


Figure 40: Les deux comparaisons dans le temps : le chiffre d'affaires mensuel de 2025 contre celui de 2024 (même période de l'an dernier), et le cumul depuis le début de l'année.

Ensuite, le changement de contexte et le ratio protégé.

```
ca_site = v25.loc[v25["canal"] == "Site", "montant"].sum()
jardin_25 = v25[v25["categorie"] == "Jardin"]
part_jardin = jardin_25.loc[jardin_25["canal"] == "Site", "montant"].sum() /
↳ jardin_25["montant"].sum()
print(f"part du Site : {ca_site / v25['montant'].sum():.1%} (toutes catégories) | {part_jardin:.1%}
↳ (dans Jardin)")
```

part du Site : 46.6% (toutes catégories) | 47.0% (dans Jardin)

La même mesure « part du canal » donne **46,6 %** pour toute la boutique et **47,0 %** dans le contexte « catégorie Jardin » : la recette ne change pas, le **contexte** si. C'est ce que CALCULATE et ALL contrôlent : ALL(dim_canal) retire le filtre de canal au dénominateur, mais **conserve** le filtre de catégorie, ce qui fait de la mesure une part **dans la catégorie**. Si l'on voulait la part dans **tout** le chiffre d'affaires, il faudrait retirer aussi le filtre de catégorie : **chaque ALL est une décision de définition**. Enfin, DIVIDE protège la division : en pandas, une division par zéro donne l'infini, que l'on remplace par une valeur manquante. L'important est de **décider** ce que la page affiche (un blanc, un zéro, un tiret) quand le dénominateur est nul.

Piège. Les fonctions de comparaison dans le temps supposent une **table de dates marquée comme telle** et couvrant des **années entières** ; un trou (une année incomplète à la fin) ou une date en double suffit à produire un résultat faux sans message d'erreur. Contrôlez la dimension (effectif, unicité) comme en 2.1.2, et comparez toujours un résultat à son équivalent pandas.

2.4.2 Power Query : la préparation, étape par étape

La préparation des données, dans Power BI, se fait dans un éditeur dédié, **Power Query**, dont chaque transformation est une **étape nommée** : lire la source, promouvoir les en-têtes, changer les types, filtrer, fusionner deux tables, regrouper. La liste des étapes (appelées « étapes appliquées ») est **rejouée à chaque actualisation** ; elle est écrite en coulisses dans un langage de script, **M**. C'est exactement la **chaîne de nettoyage reproductible** du volume II : un script depuis le brut, rejouable.

```
-- M : indicatif, non exécuté
let
    Source = Csv.Document ( File.Contents ( "lignes_commande.csv" ), [Delimiter = ","] ),
    Entetes = Table.PromoteHeaders ( Source ),
    Types = Table.TransformColumnTypes ( Entetes, {{"montant", type number}} ),
    Fusion = Table.NestedJoin ( Types, {"id_produit"}, produits, {"id_produit"}, "produit",
    ↪ JoinKind.LeftOuter ),
    Resultat = Table.ExpandTableColumn ( Fusion, "produit", {"categorie"} )
in
    Resultat
```

Le chemin équivalent en pandas, avec le **compte de lignes à chaque étape** (la règle d'or du volume II) :

```
t = d["lig"].merge(d["cmd"][["id_commande", "date_commande"]], on="id_commande")
etapes = [("lecture + dates", len(t))]
t = t[t["date_commande"] >= "2025-01-01"]
etapes.append(("filtre 2025", len(t)))
t = t.merge(d["prod"][["id_produit", "categorie"]], on="id_produit", how="left", validate="m:1")
etapes.append(("fusion produits", len(t)))
etapes.append(("regroupement par catégorie", t["categorie"].nunique()))
print(pd.DataFrame(etapes, columns=["étape", "lignes"]).to_string(index=False))
```

	étape	lignes
	lecture + dates	83905
	filtre 2025	29827
	fusion produits	29827
	regroupement par catégorie	6

L'effectif passe de 83 905 lignes à 29 827 au filtre de 2025, **reste à 29 827** après la fusion (aucune ligne multipliée, aucune perdue) et le regroupement donne les six catégories. Le paramètre `validate="m:1"` demande à pandas de **vérifier que la clé du côté droit est unique** : si ce n'était pas le cas, la fusion échouerait au lieu de multiplier silencieusement les lignes (section 2.1.2). Dans Power Query, le même réflexe consiste à **regarder l'effectif après chaque fusion** et à vérifier la cardinalité de la relation.

Un mot sur la performance : quand la source est une base de données, Power Query essaie de **déléguer** les étapes à la base (on parle de *query folding*) : le filtre devient une clause `WHERE` du SQL envoyé, et seules les lignes utiles remontent. Un filtre placé **tôt** dans la liste d'étapes, avant une transformation que la base ne sait pas faire, est donc beaucoup plus efficace que le même filtre placé tard. La règle de bonne pratique rejoint celle du SQL : **filtrer et agréger au plus près de la source**.

2.4.3 La sécurité au niveau des lignes

Un même tableau de bord est souvent lu par des personnes qui **n'ont pas le droit de voir les mêmes données** : la direction voit toutes les régions, un responsable de région voit la sienne. Plutôt que de publier un rapport par personne, on applique une **sécurité au niveau des lignes** (*row-level security*, RLS) : un **filtre**, rattaché à un rôle, retire à l'utilisateur les lignes qu'il n'a pas le droit de voir, **avant** qu'une mesure ne soit calculée. Le filtre est posé sur une dimension (ici, la région du client) et se **propage** à la table de faits par la relation de l'étoile.

L'équivalent pandas est un simple filtre, appliqué à partir d'une **table de droits** : qui a le droit de voir quelle région ? Notre fonction vue renvoie les lignes de faits autorisées ; un utilisateur absent de la table n'en voit **aucune** (refus par défaut).

```
droits = pd.DataFrame({"utilisateur": ["direction"] * 4 + ["resp_1", "resp_3", "resp_12", "resp_12"],
                      "region": ["Région 1", "Région 2", "Région 3", "Région 4", "Région 1", "Région 3", "Région 1", "Région 2"]})
f25 = faits[faits["date"] >= "2025-01-01"].merge(star["dim_client"])[["id_client", "region"]],
      on="id_client")
vue = lambda u: f25[f25["region"].isin(droits.loc[droits["utilisateur"] == u, "region"])]
for u in ["direction", "resp_1", "resp_3", "resp_12", "stagiaire"]:
    print(f"{u:<10} {vue(u)['montant'].sum():>11,.0f} €".replace(","," "))
```

```
direction    1 324 764 €
resp_1       511 532 €
resp_3       537 175 €
resp_12      641 872 €
stagiaire          0 €
```

Le responsable de la Région 1 voit 511 532 € (sur 1 324 764 € pour toute la boutique), celui de la Région 3 voit 537 175 €, la personne qui a droit aux Régions 1 et 2 voit 641 872 €, et un stagiaire absent de la table des droits voit **zéro**. Les quatre régions fictives, additionnées, redonnent exactement le total de la direction : **aucune ligne n'est perdue ni comptée deux fois**. C'est le **contrôle de la sécurité** : la somme des vues autorisées doit être égale à la vue complète, et le nombre de lignes que **personne** ne voit doit être nul.

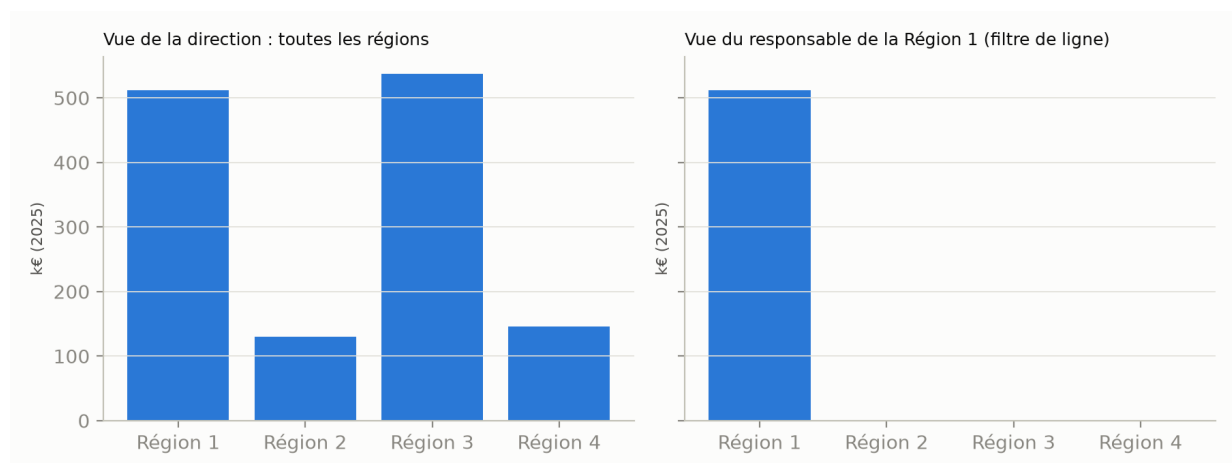


Figure 41: La même mesure (chiffre d'affaires 2025 par région fictive) vue par la direction et par le responsable de la Région 1 : le filtre de ligne retire les autres régions avant le calcul.

Deux modes de mise en œuvre existent dans la plupart des outils. Les rôles **statiques** : un rôle par région, avec un filtre fixe (région = Région 1), et l'on place les personnes dans les rôles. Simples, mais lourds à gérer quand les règles changent. Les rôles **dynamiques** : un seul rôle, dont le filtre consulte une **table de droits** avec l'**identité de l'utilisateur connecté** ; on gère alors les droits par une table (c'est le cas de notre droits), pas dans l'outil.

Ce que la sécurité par lignes ne fait pas.

- Elle **ne protège pas** contre ceux qui ont le droit de **modifier** le modèle ou de lire la source : les administrateurs et les auteurs voient tout, et l'on ne confond pas rôle de lecture et rôle de modification (à vérifier dans la documentation de votre outil).
- Elle ne règle ni les **exports** ni les **extractions** : un utilisateur qui peut télécharger les données autorisées les partage ensuite comme il veut.
- Elle ne rend pas une donnée **anonyme** (voir le volume II, chapitre 5) : un petit effectif dans une région reste identifiable.
- Elle se **teste** : on se connecte « en tant que » chaque rôle et l'on compare aux chiffres attendus, comme ci-dessus, **avant** chaque publication.

2.4.4 Le déploiement

Un tableau de bord qui compte dans l'entreprise ne se modifie pas directement en production. Les équipes qui en gèrent plusieurs adoptent les pratiques du logiciel, adaptées à la BI.

- **Plusieurs environnements** : développement, test, production, avec des données et des droits distincts. On modifie dans l'un, on teste dans l'autre, on publie dans le dernier. Certaines offres proposent des **pipelines de déploiement** qui promeuvent un contenu d'un environnement à l'autre en conservant les paramètres propres à chacun (à vérifier dans votre version).
- **Le suivi des versions** : un fichier de rapport est souvent un **fichier binaire**, mal adapté aux outils de versions comme Git ; certaines offres proposent un **format de projet en fichiers texte** qui s'y prête mieux (à vérifier). À défaut, on **nomme** les versions et on tient un journal des changements.
- **Les jeux de données certifiés** : un seul modèle de référence, porté par une équipe, que tous les rapports réutilisent, plutôt qu'un modèle différent dans chaque rapport. C'est la façon la plus sûre d'avoir **un seul chiffre d'affaires** dans toute l'entreprise.
- **L'actualisation incrémentale** : au lieu de recharger trois ans de commandes chaque nuit, on ne recharge que **les derniers jours** et l'on garde le reste. On gagne en temps et en charge ; en contrepartie, une correction faite sur une donnée ancienne **n'apparaît pas** tant qu'on ne force pas un rechargement complet.
- **La surveillance** : alertes sur les échecs d'actualisation, suivi des temps de chargement, revue des droits à intervalles réguliers (les personnes changent de poste).

Cette liste se résume en un **contrôle de mise en production** que l'on coche avant chaque publication : effectifs des tables et clés vérifiés (2.1.2) ; chiffres clés recalculés par un autre chemin (2.3.5) ; définitions à jour dans le dictionnaire ; sécurité par lignes testée avec chaque rôle ; actualisation planifiée, notifications d'échec activées ; date des données affichée ; propriétaire et journal des changements renseignés.

À retenir.

- Une mesure DAX s'évalue dans un **contexte de filtres** ; CALCULATE le modifie, ALL en retire une partie : **chaque ALL est une décision de définition**. Les comparaisons dans le temps exigent une dimension de dates continue et se font **à période égale**.
- Power Query enchaîne des **étapes nommées** rejouées à chaque actualisation : on **compte les lignes** après chaque fusion et l'on vérifie la cardinalité.
- La **sécurité par lignes** filtre avant le calcul ; on la **teste** (la somme des vues autorisées égale la vue complète, un inconnu ne voit rien) et l'on connaît ses limites (administrateurs, exports, petits effectifs).
- Le déploiement suit les pratiques du logiciel : environnements, versions, **jeu de don-**

nées certifié, actualisation incrémentale, surveillance, et un contrôle de mise en production.

Pour s'entraîner. Cahier, chapitre 2 : applications 2.6 et 2.7, exercices 2.10 et 2.11.

2.5 ⊕ Pour aller plus loin : Looker, Metabase, Superset, Qlik

Section optionnelle. Elle replace Power BI et Tableau dans un paysage plus large et propose une méthode pour **choisir un outil**. Aucun des produits cités ici n'a été exécuté pour ce livre : les descriptions viennent de la **documentation publique** telle que nous la connaissons, elles évoluent vite (fonctions, licences, offres), et **tout est à vérifier** dans la documentation à jour avant de décider quoi que ce soit. Les figures sont des maquettes dessinées, sans l'identité d'aucun produit.

Les outils de tableaux de bord se ressemblent plus qu'ils ne le prétendent : tous assemblent les **mêmes quatre briques**, en mettant l'accent sur l'une ou sur l'autre. Comprendre ces briques permet de **lire** n'importe quelle plaquette commerciale et de poser les bonnes questions.

Les quatre briques que l'on retrouve, dans des proportions différentes, dans tous les outils de tableaux de bord

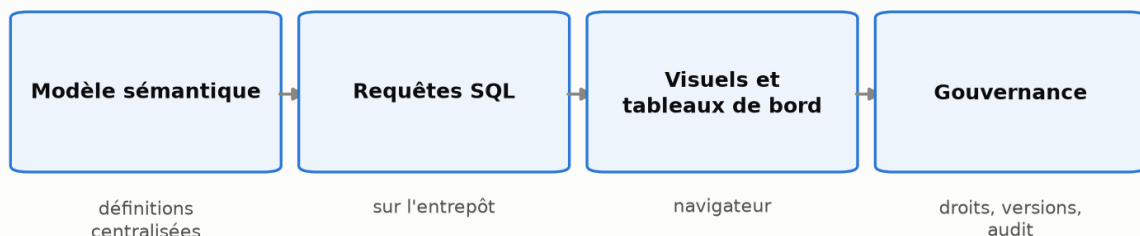


Figure 42: Les quatre briques que l'on retrouve, dans des proportions différentes, dans tous les outils de tableaux de bord : un modèle sémantique, des requêtes sur la source, des visuels, une gouvernance.

- Le **modèle sémantique** : les définitions (mesures, relations, dimensions) écrites **une fois** pour toute l'entreprise. C'est la brique que nous avons construite en 2.1.
- Les **requêtes** : la manière de **recupérer** les données de la source (SQL envoyé à une base, copie en mémoire).
- Les **visuels et tableaux de bord** : la partie visible, la plus démonstrative en vente, et la moins déterminante à long terme.
- La **gouvernance** : droits, versions, audit, certification, actualisation, surveillance. C'est elle qui sépare un outil d'équipe d'un outil d'entreprise.

2.5.1 Quatre familles d'outils

Looker (selon la documentation publique) est centré sur le **modèle sémantique écrit en code** : les mesures et les relations sont décrites dans un langage de modélisation, versionné comme du code, et **chaque visuel génère une requête SQL** envoyée à l'entrepôt de données. Il n'y a pas de copie en mémoire : la donnée reste chez soi. Force : des définitions **uniques et revues** comme du code, et une gouvernance forte. Vigilance : il faut des compétences de modélisation et de SQL, et l'entrepôt doit répondre vite.

Metabase est un outil **libre** (il existe aussi une offre hébergée) pensé pour que **des personnes sans SQL** posent des questions à une base : on construit une « question » par menus (filtrer, regrouper, résumer), ou on écrit du SQL, puis on place les questions dans un tableau de bord. Force : une mise en route très rapide et une prise en main simple. Vigilance : une modélisation sémantique plus légère ; sur de grandes organisations, il faut organiser les définitions pour éviter que chacun crée sa propre version de « chiffre d'affaires ».

Superset est un projet **libre** de la fondation Apache : un éditeur SQL (SQL Lab), une grande variété de graphiques, des « jeux de données » qui portent colonnes et métriques, des tableaux de bord et un système de rôles. Force : souplesse, absence de licence, intégration à de nombreuses bases. Vigilance : il faut **l'installer, le configurer et l'administrer** (ou payer une offre hébergée), ce qui demande un savoir-faire technique.

Qlik s'appuie sur un moteur **associatif** en mémoire : quand on sélectionne une valeur (une catégorie), l'outil montre non seulement les données qui lui sont liées, mais aussi celles qui **ne le sont pas** (leur couleur dit la différence). Les données se chargent par un **script** de chargement. Force : exploration très libre, sans parcours imposé. Vigilance : un moteur et un langage de script propres à apprendre, et une gouvernance à organiser.

Ces quatre descriptions sont des **raccourcis**, utiles pour s'orienter, et non des mesures : les produits changent, empruntent les uns aux autres, et la même fonction peut exister sous un autre nom. La figure suivante place les outils les uns par rapport aux autres **à titre indicatif**.

Piège. Une carte de positionnement, même honnête, **cache les compromis** : un outil « libre » n'est pas gratuit (on paie l'administration), un outil « cadré » n'est pas lourd si l'organisation est prête, et un outil « simple » devient complexe dès qu'il faut gouverner cent tableaux de bord. Elle aide à poser des questions ; elle ne répond pas à votre situation.

2.5.2 Choisir sans se laisser guider par la marque

On choisit un outil comme on choisit un fournisseur : à partir de **critères liés à la situation**, pas d'une impression de démonstration. Sept questions suffisent presque toujours.

1. **Où sont les données ?** Dans une base d'entreprise accessible par SQL, dans des fichiers, dans des services en ligne ? Un outil qui se branche mal à la source coûte plus cher que la licence.
2. **Qui construit, qui consulte ?** Des analystes qui écrivent du SQL, des métiers qui veulent cliquer, quelques lecteurs (la gérante) ou des centaines ?
3. **Quel besoin de gouvernance ?** Un chiffre d'affaires unique pour toute l'entreprise exige un modèle partagé et revu ; un suivi d'équipe se contente de moins.
4. **Quel coût réel ?** Licences par personne, hébergement, **temps d'administration**, formation, et coût de sortie si l'on change d'outil.
5. **Quelles compétences existent déjà ?** Un outil bien adapté mais que personne ne sait utiliser ne sert à rien.

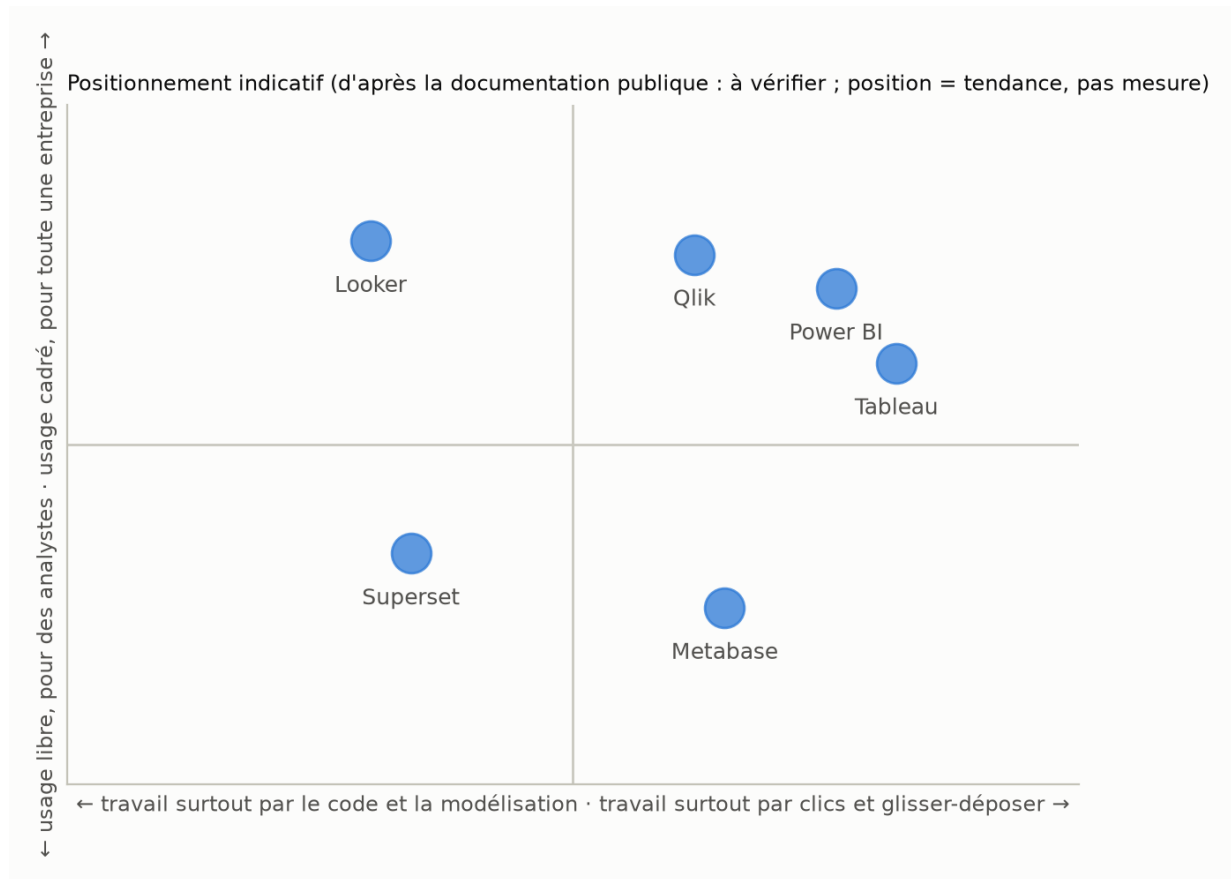


Figure 43: Positionnement indicatif de six outils selon deux axes : mode de travail (par code et modélisation, ou par clics et glisser-déposer) et cadre d'usage (usage libre d'analystes ou cadre d'entreprise). Il s'agit d'une tendance d'après la documentation publique, à vérifier, pas d'une mesure.

6. **Quelles intégrations ?** Envoi par courriel, intégration dans une application, sécurité par lignes, **sécurité des données** (où elles sont stockées, qui y accède).
7. **Peut-on partir ?** Les définitions (mesures, relations) sont-elles **exportables**, ou prisonnières de l'outil ?

Pour décider, on **pondère** les critères et l'on **note** chaque option. Cette méthode, simple, a une propriété importante : elle rend les **préférences visibles** et discutables. Prenons trois options fictives, A, B et C, notées de 1 à 5 sur quatre critères, avec un premier jeu de poids, puis un second où la gouvernance pèse davantage.

```
notes = pd.DataFrame({"coût": [5, 2, 4], "facilité": [4, 3, 2], "gouvernance": [2, 5, 3],
↳ "flexibilité": [2, 4, 5]}, index=["A", "B", "C"])
poids_1 = pd.Series({"coût": 3, "facilité": 3, "gouvernance": 2, "flexibilité": 2})
poids_2 = poids_1.mask(poids_1.index == "gouvernance", 4)
print(pd.DataFrame({"poids 1": notes @ poids_1, "poids 2": notes @ poids_2}))
```

	poids 1	poids 2
A	35	39
B	33	43
C	34	40

Avec le premier jeu de poids, **A** arrive en tête (35 points, devant C à 34 et B à 33) ; quand la gouvernance double de poids (de 2 à 4), c'est **B** qui gagne (43 points, devant C à 40 et A à 39). Aucun classement n'est « le vrai » : **il dépend de ce qui compte pour vous**, et la discussion sur les poids est précisément la discussion utile. Ces notes sont fictives ; les vôtres viendraient d'un essai.

L'**essai** est la meilleure preuve : prenez **la même page** (trois questions de la gérante, une semaine de données) et construisez-la dans deux outils, en notant le **temps** passé, l'**exactitude** des chiffres (recette, 2.3.5), le résultat du **test de cinq secondes** et l'**effort de maintenance** attendu. Deux jours d'essai valent mieux que deux semaines de plaquettes.

Intuition. L'outil est le **dernier** choix à faire : avant, il y a les décisions, les indicateurs, les définitions et le modèle. Un modèle propre et un dictionnaire clair se **transportent** d'un outil à l'autre ; un tableau de bord dessiné sans eux s'abîme dès qu'on change d'outil.

2.5.3 Ce que les outils ne font pas à votre place

Aucun outil ne **choisit les indicateurs**, ne **définit** « client actif », ne **vérifie** qu'une jointure ne multiplie pas les lignes, ne **décide** de la bonne période de comparaison, ne **sait** si un chiffre est du bruit ou un signal. Il fait de beaux graphiques de tout ce qu'on lui donne, y compris des chiffres faux. Ce qui reste à l'analyste, c'est le **travail de fond** de ce chapitre : un modèle juste (2.1), des calculs compris (2.2 et 2.4), une conception partie de la décision (2.3), et une recette qui compare deux chemins. Quand l'outil change, ce travail reste.

À retenir.

- Tous les outils assemblent **quatre briques** : un modèle sémantique, des requêtes, des visuels, une gouvernance ; ils diffèrent par l'accent mis sur chacune.
- **Looker** (modèle en code, SQL sur l'entrepôt), **Metabase** (libre, questions par menus), **Superset** (libre, SQL et administration) et **Qlik** (moteur associatif en mémoire) : des raccourcis à **vérifier** dans la documentation.
- On choisit selon **sept questions** (données, personnes, gouvernance, coût réel, compétences, intégrations, sortie), avec des **critères pondérés** dont les poids sont discutés, puis un **essai** sur la même page dans deux outils.

- L'outil est le dernier choix : un modèle propre, des définitions écrites et une recette **se transportent** d'un outil à l'autre.

Pour s'entraîner. Cahier, chapitre 2 : application 2.8, exercice 2.12.

Bilan du chapitre 2

La gérante voulait voir chaque lundi où en est la boutique, sans demander un fichier. Elle a désormais une **page**, six chiffres avec leur référence, une courbe de contrôle et deux alertes (livraisons à l'heure à 44,5 % pour un habituel de 76,3 %, ruptures à 35,0 % pour un habituel de 8,4 %) ; et vous avez un **modèle** (une table de faits de 83 905 lignes, quatre dimensions aux clés uniques, aucune ligne orpheline) dont chaque chiffre se recalcule par un autre chemin.

Section	Ce que vous devez emporter
2.1 Power BI	Un outil de tableaux de bord enchaîne connexion, préparation, modèle, mesures, visuels, publication, actualisation , et chaque étape se reproduit à la main. Le modèle en étoile relie par des identifiants uniques (relier par le nom double le chiffre d'affaires). Une mesure se recalcule dans chaque contexte de filtres ; un ratio est une somme sur une somme ; le nombre de commandes ne s'additionne pas (25 163 contre 12 946). On planifie l'actualisation, on affiche la date des données, on ne publie jamais sur le web des données internes.
2.2 Tableau	Une feuille encode des champs par des canaux (position, couleur, taille) ; on choisit le canal selon la question. Un calcul de table opère sur le tableau affiché ; un niveau de détail opère à une granularité choisie. Un chiffre plausible n'est pas forcément le chiffre voulu : on recalcule une case en pandas.
2.3 Conception	On part des décisions , pas des données ; trois niveaux (vue d'ensemble, analyse, détail) ; chaque chiffre a la référence qui convient (l'habituel pour les livraisons, l'an dernier pour les ventes) ; test de cinq secondes , parcours en Z , contraste calculé (4,5 : 1 pour du petit texte). On valide par deux chemins, on documente dans un dictionnaire, on mesure l' usage .
⊗ 2.4 Power BI avancé	DAX : contexte de filtres, CALCULATE, comparaisons à période égale (neuf mois contre douze : -26,3 %, absurde). Power Query : étapes nommées, effectifs après chaque fusion. Sécurité par lignes : la somme des vues autorisées égale la vue complète, un inconnu ne voit rien ; limites (administrateurs, exports, petits effectifs). Déploiement : environnements, jeu de données certifié, actualisation incrémentale.

Section	Ce que vous devez emporter
⊕ 2.5 Autres outils	Quatre briques communes (modèle, requêtes, visuels, gouvernance) ; Looker, Metabase, Superset, Qlik comme familles à vérifier dans la documentation ; sept questions de choix, critères pondérés (le classement change avec les poids), puis un essai sur la même page. L'outil est le dernier choix.

Trois idées à retenir de tout le chapitre.

1. **Un tableau de bord est un outil de décision.** On part de ce que la personne décidera, pas de ce que l'on sait calculer.
2. **Le travail se fait avant l'écran.** Un modèle juste, des définitions écrites et une recette qui compare deux chemins comptent plus que le choix des couleurs ou de l'outil.
3. **Un chiffre sans référence, sans date et sans définition est un décor.** Chaque carte dit à quoi on la compare, quand elle a été mise à jour et comment elle est calculée.

Vous savez maintenant **concevoir, modéliser et valider** un tableau de bord. Le chapitre 3 montre comment **construire des visualisations avec du code** (matplotlib, seaborn, plotly), ce qui donne un contrôle total sur chaque détail et permet d'**automatiser** ; le chapitre 4 explique comment **raconter** ce que le tableau de bord montre, et comment faire tourner un rapport **chaque semaine sans y toucher**.

À retenir, tout simplement. Quand la gérante regarde la page le lundi, elle doit savoir en cinq secondes **si tout va bien**, et si ce n'est pas le cas, **ce qu'elle va faire**. Le reste est de la technique au service de cette phrase.

Pour s'entraîner. Cahier, chapitre 2 : applications 2.1 à 2.8 et exercices 2.1 à 2.12, avec leurs corrigés.

Chapitre 3 : Visualisation avec Python

« Un graphique que l'on ne peut pas refaire n'est qu'une image ; un graphique que l'on peut refaire est une analyse. »

La gérante vous écrit un lundi matin : « *Peux-tu me faire le graphique des ventes par canal pour la réunion de jeudi ? En couleurs lisibles, avec les chiffres sur les barres. Et si tu peux, que je puisse cliquer dessus pour voir le détail.* » Trois demandes en une phrase : un graphique **juste**, un graphique **lisible**, un graphique **interactif**. Les chapitres 1 et 2 ont posé les principes (quel type de graphique, quelle mise en page, quel tableau de bord) ; celui-ci apprend à les **fabriquer avec du code**.

Pourquoi du code plutôt qu'un outil à cliquer ? Pour une raison que l'on a déjà rencontrée dans toute la série : la **reproductibilité**. Un graphique construit à la souris se refait à la souris, avec les mêmes oublis ; un graphique construit par un script se refait en une commande, le mois suivant, sur les nouvelles données, et l'on peut **tester** qu'il montre bien les chiffres qu'il prétend montrer. C'est aussi pour cela que, dans ce chapitre, **le code est le sujet** : les blocs restent courts, mais vous les verrez presque tous.

Le chemin de ce chapitre

- **3.1 matplotlib et seaborn** : l'anatomie d'une figure, les graphiques de base (barres, lignes, nuages, histogrammes), les étiquettes directes, les petits multiples, le thème maison du livre, puis seaborn pour les graphiques statistiques ; la même figure en R avec ggplot2 pour comparer les syntaxes.
- **3.2 plotly et graphiques interactifs** : survol, zoom, filtre par légende ; de vraies captures de figures interactives ; quand l'interactivité aide, quand elle gêne.
- ⊕ **3.3 Tableaux de bord avec Dash, Streamlit et Shiny** : le même mini-tableau de bord écrit dans trois outils libres, testé, photographié.
- ⊕ **3.4 Cartes et visualisation géospatiale** : cercles proportionnels et polygones sur un plan fictif, normaliser par habitant, savoir quand une carte ne sert à rien.

Les données du chapitre

Les mêmes données de la boutique que dans les volumes précédents (ventes par ligne de commande, clients, jours d'exploitation, sessions web, livraisons) et un fichier de **villes fictives** avec des coordonnées dans un plan imaginaire (`villes.csv`) : aucune carte du monde réel n'est utilisée. Toutes les données sont **simulées**.

En pratique. Les figures de ce chapitre sont produites par le code que vous lisez. Les captures d'outils interactifs (plotly, Dash, Streamlit, Shiny) sont de **vraies captures** de ces outils **libres**, lancés sur la machine qui a écrit le livre ; aucune interface d'un logiciel commercial n'est reproduite.

```
import sys
sys.path.insert(0, "build")
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
import style, outils_ch03 as o
style.setup()
x = o.ventes()
↪ catégorie
print(len(x), "lignes de commande |", x["date_commande"].min().date(), "->",
↪ x["date_commande"].max().date())
```

83905 lignes de commande | 2023-01-01 -> 2025-12-31

3.1 matplotlib et seaborn

matplotlib est la bibliothèque de dessin de base de Python : presque toutes les autres (seaborn, pandas, scikit-learn) l'utilisent en coulisse. Elle est un peu verbeuse, mais elle donne le contrôle de **chaque** élément d'une figure, et c'est ce contrôle qui sépare un graphique par défaut d'un graphique qui fait passer un message.

3.1.1 Trois couches, et l'anatomie d'une figure

Tout graphique statistique, quel que soit l'outil, se décrit en **trois couches**. Les **données** (un tableau), les **esthétiques** (quelle colonne va sur quel attribut visuel : position horizontale, hauteur, couleur, taille) et la **géométrie** (la forme dessinée : barres, lignes, points). Un graphique à barres du chiffre d'affaires par canal : le tableau est « un canal et un chiffre d'affaires par ligne », l'esthétique est « canal en abscisse, chiffre d'affaires en hauteur, couleur selon le canal », la géométrie est « une barre par ligne ».

Cette grammaire est explicite dans ggplot2 (voir 3.1.9) ; elle est implicite dans matplotlib, où l'on **dessine** étape par étape. Il faut donc connaître le vocabulaire de ce que l'on dessine.

Une **Figure** est l'image entière ; elle contient un ou plusieurs **Axes**, c'est-à-dire des zones de dessin avec leur titre et leurs deux **Axis** (l'axe horizontal et l'axe vertical, avec graduations et étiquettes). Tout ce qui s'y dessine (une ligne, une barre, un texte, la légende) est un **Artist**. Presque tous les gestes de ce chapitre sont « prendre l'Axe et lui dire quoi dessiner », et la bonne habitude est l'**interface orientée objet** : on crée `fig`, `ax = plt.subplots()` puis l'on écrit `ax.bar(...)`, `ax.set_title(...)`. (L'interface plus ancienne `plt.bar(...)` agit sur un « Axes courant » implicite : elle suffit pour un essai, elle devient confuse dès qu'il y a deux graphiques.)

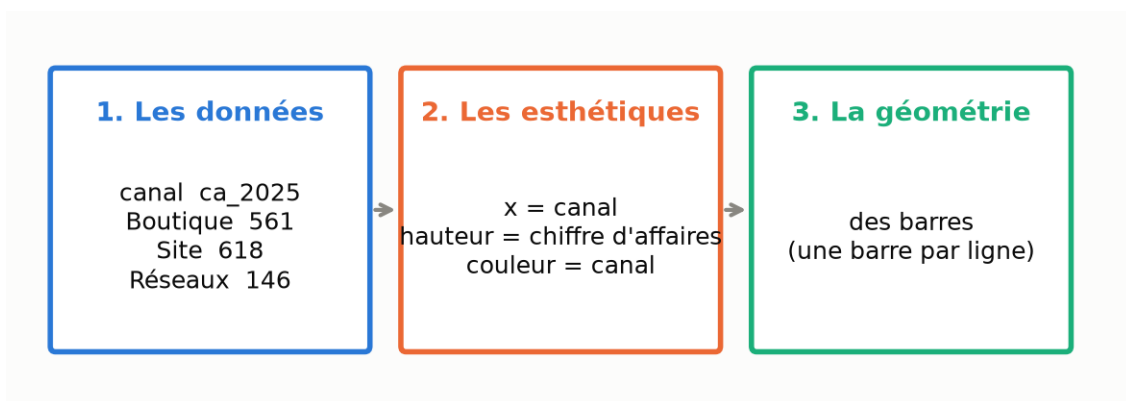


Figure 44: Les trois couches d'un graphique. Schéma dessiné avec matplotlib.

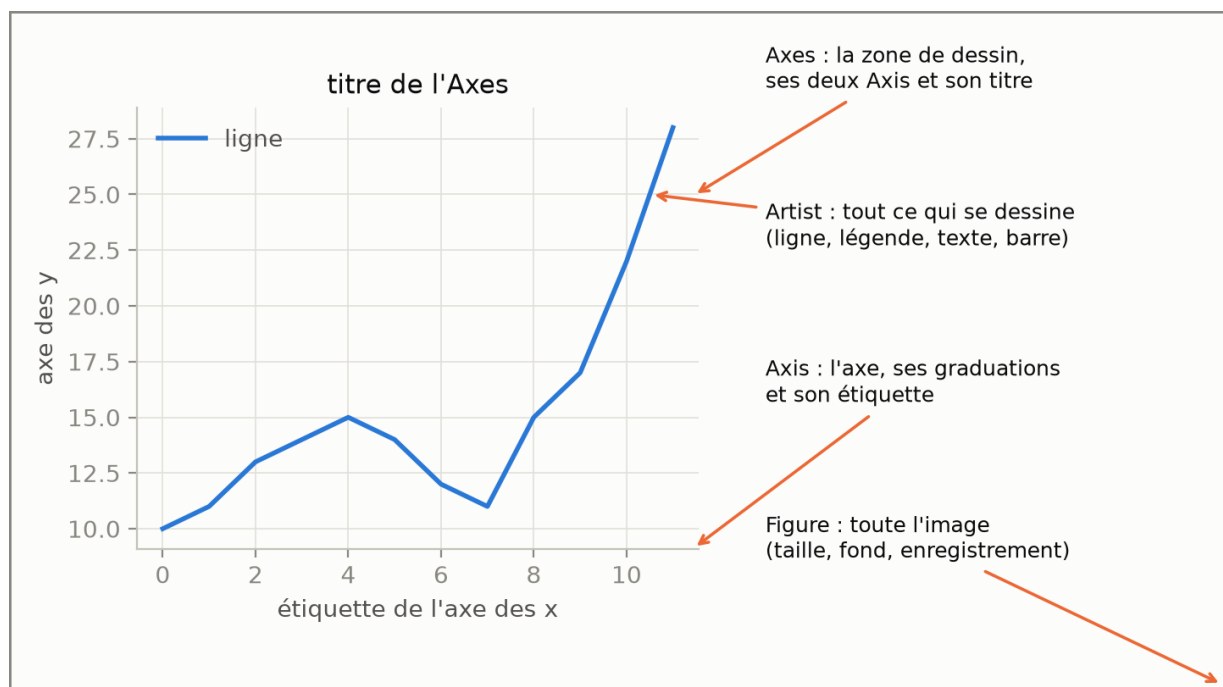


Figure 45: Anatomie d'une figure matplotlib. Schéma dessiné avec matplotlib.

Voici le graphique demandé par la gérante, **sans aucun réglage** : le chiffre d'affaires 2025 par canal.

```
ca = x[x["annee"] == 2025].groupby("canal")["montant"].sum() / 1000
fig, ax = plt.subplots()
ax.bar(ca.index, ca.values)
0.sauver(fig, "ch03-barres-default.png")
print(ca.round(1).to_dict())
```

```
figure : ch03-barres-default.png
{'Boutique': 561.0, 'Réseaux': 146.1, 'Site': 617.7}
```

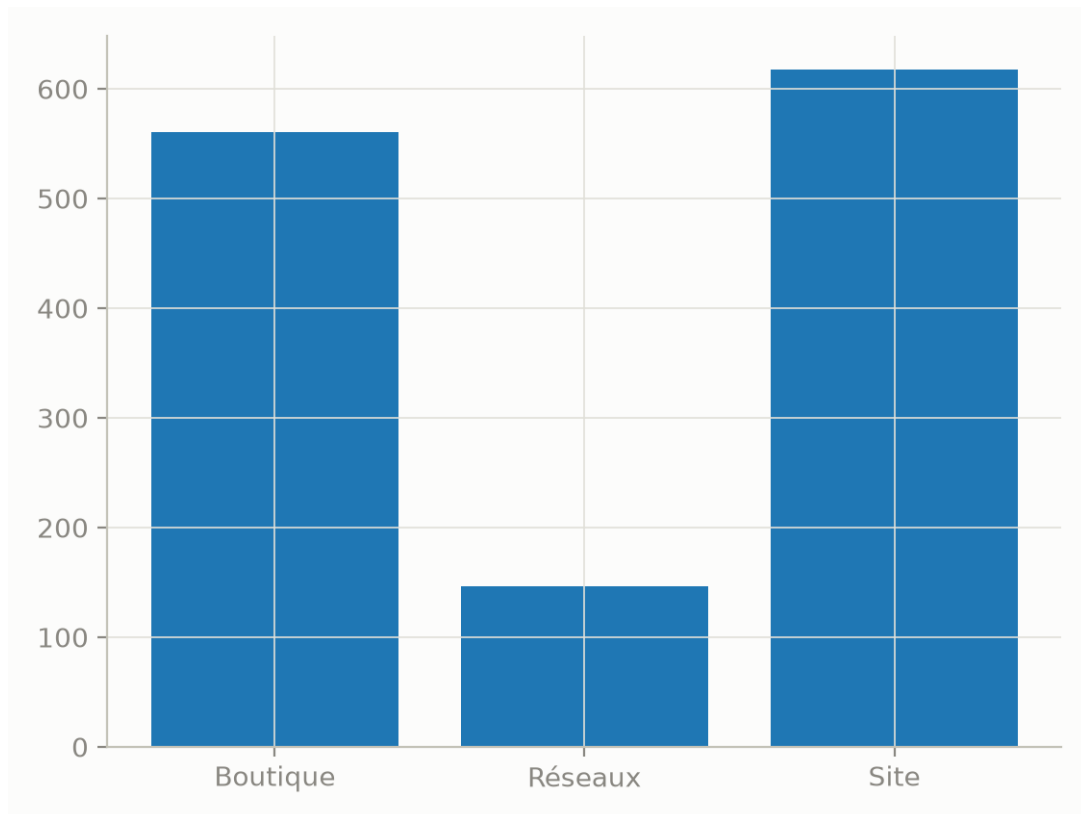


Figure 46: Le graphique par défaut : correct, mais il ne dit rien.

Ce graphique est **exact**, et c'est tout ce qu'on peut en dire : pas de titre, pas de chiffres, des barres de la même couleur dans l'ordre alphabétique, des graduations en milliers d'euros sans unité, un cadre et un quadrillage qui n'apportent rien. Le lecteur doit chercher le message ; nous allons le lui donner.

3.1.2 Du graphique par défaut au graphique qui dit quelque chose

On applique cinq retouches, chacune justifiée par le chapitre 1 : **ordonner** les barres (du plus grand au plus petit, sauf s'il existe un ordre naturel), **étiqueter directement** les barres plutôt que de forcer à lire l'axe, **supprimer** ce qui ne sert pas (axe vertical, quadrillage, cadre), **colorer avec intention** (une couleur par canal, la même dans tout le document), et **titrer par le message**, pas par la description.

```
ca = x[x["annee"] == 2025].groupby("canal")["montant"].sum().sort_values(ascending=False) / 1000
fig, ax = plt.subplots(figsize=(5.6, 3.2))
barres = ax.bar(ca.index, ca.values, color=[0.COUL[c] for c in ca.index], width=0.62)
ax.bar_label(barres, labels=[f"{0.fr(v)} k€" for v in ca.values], padding=3, color=style.ENCRE)
```

```

ax.set(ylim=(0, ca.max() * 1.15), yticks=[])
ax.grid(False)
ax.spines["left"].set_visible(False)
part = ca[["Site", "Boutique"]].sum() / ca.sum() * 100
ax.set_title(f"Le Site et la Boutique font {part:.0f} % du chiffre d'affaires 2025", loc="left",
            fontweight="bold")
O.sauver(fig, "ch03-barres-final.png")
print({c: 0.fr(v) for c, v in ca.items()}, "|", round(part, 1), "%")

```

```

figure : ch03-barres-final.png
{'Site': '618', 'Boutique': '561', 'Réseaux': '146'} | 89.0 %

```

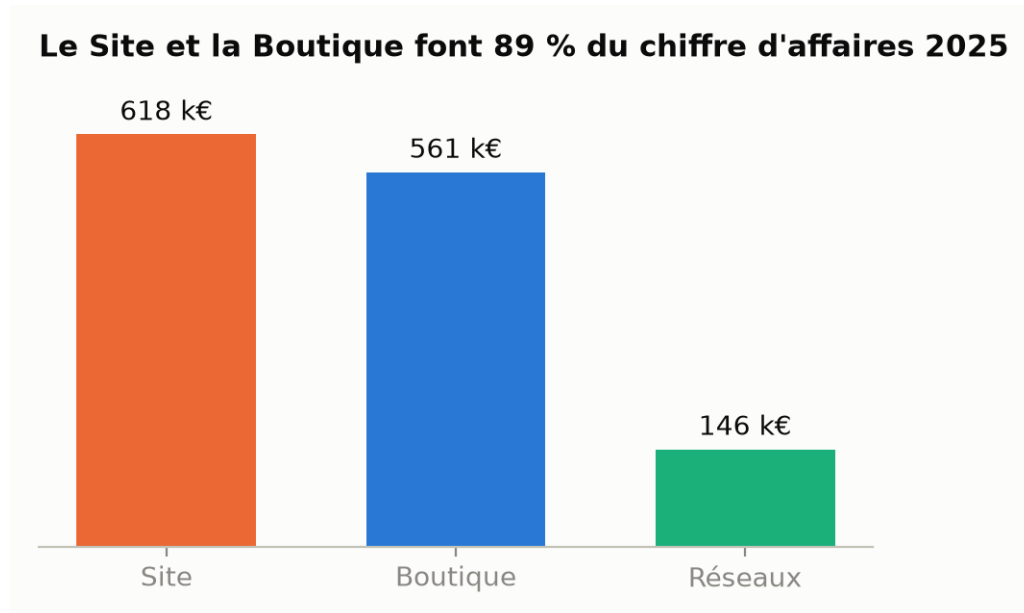


Figure 47: Le même graphique après les cinq retouches : le message est dans le titre, les chiffres sur les barres.

Le titre n'est pas écrit à la main : il est **calculé** (part). Ainsi, quand on rejoue le script sur les ventes du mois suivant, le titre reste vrai ; c'est l'avantage du code, et c'est aussi un piège (un titre automatique qui dit « fait 89 % » ne doit jamais dire une chose fausse si les données changent : on le teste, voir 3.1.8). L'axe vertical a disparu parce que chaque barre porte son chiffre ; **on ne supprime pas** un axe quand les chiffres ne sont pas écrits ailleurs.

Piège. Les barres doivent **partir de zéro** : leur longueur représente la valeur. Raccourcir l'axe vertical (par exemple le faire commencer à 500) fait paraître trois fois plus grand un écart de 10 % ; la section 1.4 montre ce piège en détail. Pour une courbe, en revanche, on peut resserrer l'axe : on lit une position, pas une longueur.

3.1.3 Lignes : les séries temporelles

Pour une évolution, on passe à la **ligne**. La bonne pratique est de **supprimer la légende** et d'écrire le nom de chaque série **au bout de sa ligne** (étiquette directe) : l'œil n'a plus à faire la navette entre la courbe et une boîte de couleurs.

```

m = x.groupby(["mois", "canal"])["montant"].sum().unstack()[0.CANAU] / 1000
t = m.index.to_timestamp()
fig, ax = plt.subplots(figsize=(7.0, 3.4))
for canal in 0.CANAU:

```

```

ax.plot(t, m[canal], color=0.COUL[canal])
ax.text(t[-1] + pd.Timedelta(days=12), m[canal].iloc[-1], canal, color=0.COUL[canal],
        va="center", fontweight="bold")
ax.set_xlim(t[0], t[-1] + pd.Timedelta(days=95)); ax.set_ylabel("k€ par mois")
ax.xaxis.set_major_locator(mdates.MonthLocator(bymonth=[1, 7]));
ax.xaxis.set_major_formatter(mdates.DateFormatter("%m/%Y"))
ax.annotate("décembre 2025 : le pic de trois ans", xy=(t[-1], m["Site"].iloc[-1]), xytext=(t[-3],
        84), ha="right", arrowprops=dict(arrowstyle="->", color=style.MUET))
ax.set_title("Chiffre d'affaires mensuel par canal, 2023-2025", loc="left", fontweight="bold")
0.sauver(fig, "ch03-serie-mensuelle.png")
print("pic :", m.sum(axis=1).idxmax(), "|", 0.fr(m.sum(axis=1).max()), "k€")

```

figure : ch03-serie-mensuelle.png
pic : 2025-12 | 184 k€

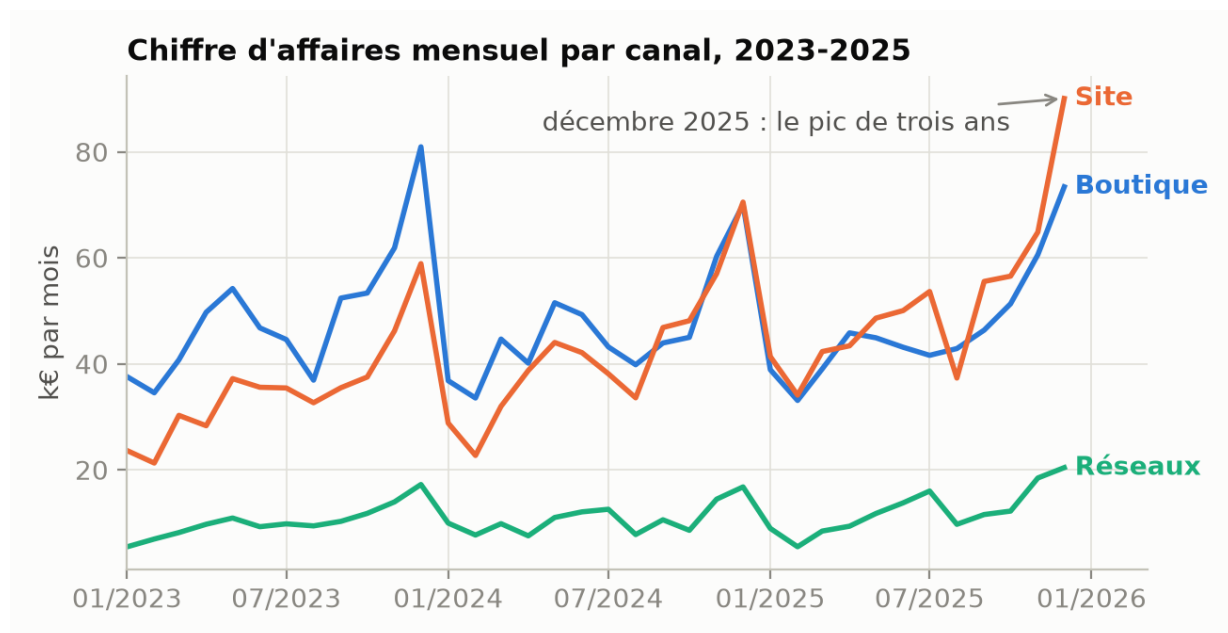


Figure 48: Chiffre d'affaires mensuel par canal : étiquettes directes, une annotation sur le fait marquant.

On voit d'un coup d'œil la **saisonnalité** (un pic chaque fin d'année), la croissance du Site (de 422 à 618 k€ par an en trois ans), et le fait que le Site dépasse la Boutique en décembre 2025. L'annotation (`ax.annotate`) désigne l'unique fait sur lequel on veut attirer le regard : **une annotation par graphique suffit** ; dix annotations noient le message.

3.1.4 Histogrammes et nuages de points

L'**histogramme** montre la forme d'une distribution (volume I, section 1.1) ; le **nuage de points** montre la liaison entre deux variables. Les paniers de 2025 sont **asymétriques** : on les montre d'abord sur une échelle linéaire, puis sur une échelle **logarithmique**, qui étale les petites valeurs et rend la forme plus lisible.

```

panier = x[x["annee"] == 2025].groupby("id_commande")["montant"].sum()
fig, (g, d) = plt.subplots(1, 2, figsize=(7.6, 3))
g.hist(panier, bins=np.arange(0, 700, 20), color=style.BLEU)
d.hist(panier, bins=np.logspace(np.log10(panier.min()), np.log10(panier.max()), 30),
        color=style.BLEU)
d.set_xscale("log")
for ax_, titre in ((g, "échelle linéaire"), (d, "échelle logarithmique")):

```

```

    ax.axvline(panier.median(), color=style.ORANGE); ax.set_title(titre); ax.set_xlabel("panier
    (€)")
    g.text(panier.median() + 10, g.get_ylim()[1] * 0.9, f"médiane {panier.median():.0f} €",
    color=style.ORANGE)
    0.sauver(fig, "ch03-histogrammes.png")
    print(len(panier), "commandes | moyenne", round(panier.mean(), 1), "| médiane",
    round(panier.median(), 1), "| maximum", round(panier.max()))

```

figure : ch03-histogrammes.png
 12946 commandes | moyenne 102.3 | médiane 81.6 | maximum 988

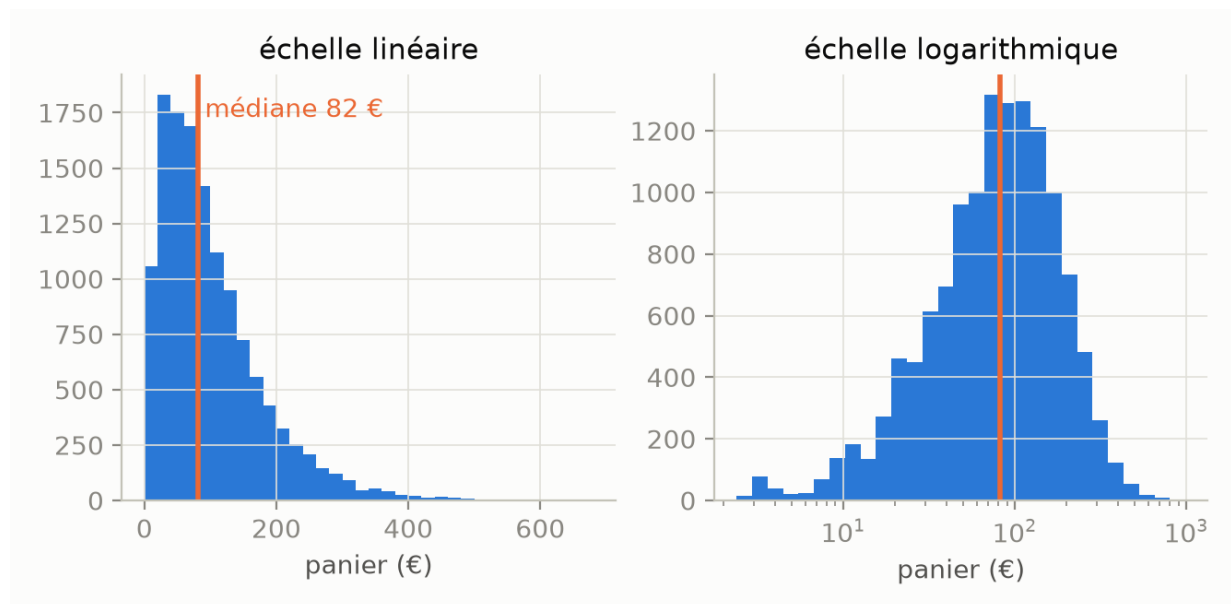


Figure 49: Les paniers de 2025 : une queue vers la droite à gauche, une forme presque symétrique à droite, sur échelle logarithmique.

La médiane (82 €) est nettement sous la moyenne (102 €) : la distribution est étirée vers les gros paniers. Sur l'échelle logarithmique, la forme devient presque symétrique, signe que les paniers se comportent comme des **produits** de facteurs plutôt que comme des sommes. Le choix de l'échelle est une décision d'analyste, et l'on **dit** laquelle on a prise.

Le nuage de points, lui, sert à voir une relation sans la supposer. Voici la dépense publicitaire quotidienne contre le nombre de commandes, en distinguant les jours de promotion.

```

j = 0.jours(); j25 = j[j["annee"] == 2025]
fig, ax = plt.subplots(figsize=(5.6, 3.4))
for promo, couleur, nom in ((0, style.MUET, "jour ordinaire"), (1, style.ORANGE, "jour de
    promotion")):
    d_ = j25[j25["promo_active"] == promo]
    ax.scatter(d_["depense_pub"], d_["nb_commandes"], s=14, alpha=0.6, color=couleur, label=nom)
ax.set(xlabel="dépense publicitaire du jour (€)", ylabel="commandes du jour")
ax.legend(loc="upper left")
0.sauver(fig, "ch03-nuage.png")
print("corrélation :", round(j25["depense_pub"].corr(j25["nb_commandes"]), 2))

```

figure : ch03-nuage.png
 corrélation : 0.54

La corrélation vaut 0,54 : la dépense et les commandes montent ensemble. Un graphique **montre**, il ne **prouve** pas : nous savons (volume III, section 2.3) que la saison fait monter les deux. Une transparence (alpha) de 0,6 évite que les points se masquent les uns les autres quand ils sont

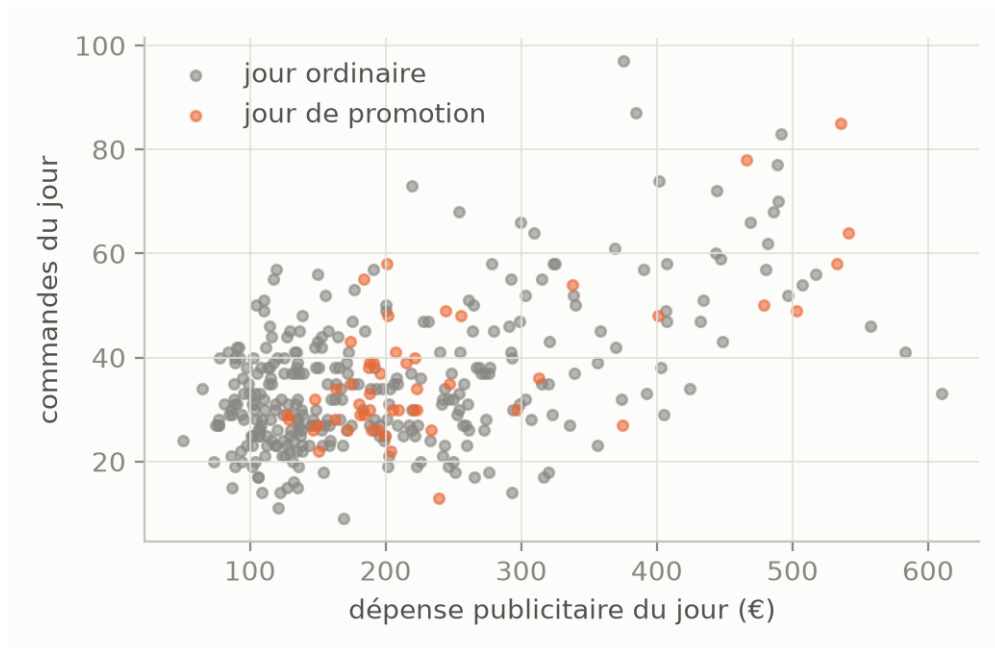


Figure 50: Dépense publicitaire et commandes, jour par jour en 2025 : une liaison nette, qui n'est pas une cause (volume III, chapitre 2).

nombreux.

3.1.5 Petits multiples

Quand on veut comparer plusieurs groupes, **plusieurs petits graphiques identiques** valent mieux qu'un seul graphique chargé : c'est le principe des **petits multiples**. Avec matplotlib, c'est `plt.subplots(1, 3)`, et l'argument important est `sharey=True` : tous les graphiques partagent la **même échelle verticale**, donc leurs hauteurs se comparent.

```
fig, axes = plt.subplots(1, 3, figsize=(8, 2.7), sharey=True)
for ax, canal in zip(axes, 0.CANAUX):
    ax.plot(t, m[canal], color=0.COUL[canal])
    ax.set_title(canal, loc="left", color=0.COUL[canal], fontweight="bold")
    ax.xaxis.set_major_locator(mdates.YearLocator())
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))
axes[0].set_ylabel("k€ par mois")
0.sauver(fig, "ch03-petits-multiples.png")
print("maximum mensuel (k€) :", {c: round(m[c].max()) for c in 0.CANAUX})
```

```
figure : ch03-petits-multiples.png
maximum mensuel (k€) : {'Boutique': 81, 'Site': 90, 'Réseaux': 20}
```

Avec `sharey=True`, le canal Réseaux apparaît **petit**, ce qu'il est (un maximum de 20 k€ par mois, contre 90 pour le Site). Sans cet argument, chaque graphique serait étiré sur toute la hauteur et le canal Réseaux paraîtrait aussi important que les autres. Choisir d'**uniformiser ou non** l'échelle est un choix de sens : on uniformise pour comparer des ordres de grandeur, on libère pour comparer des **formes**.

Pour s'entraîner. Cahier, chapitre 3 : applications 3.1 et 3.2, exercices 3.1 à 3.4.

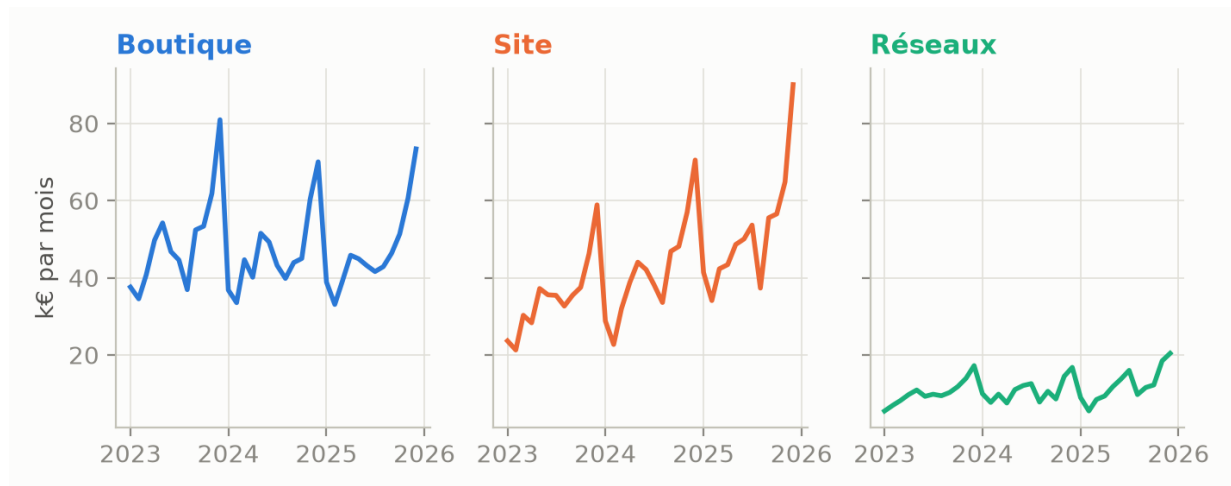


Figure 51: Un petit graphique par canal, sur la même échelle verticale : le canal Réseaux est petit, et on le voit.

3.1.6 Le thème maison du livre

Les retouches précédentes seraient fastidieuses à répéter dans chaque script. On les range dans un **thème** : un jeu de réglages appliqué une fois (`style.setup()`), qui fixe les polices, les tailles, le fond, la grille discrète, la suppression des cadres inutiles et, surtout, une **palette** fixe. Voici l'essentiel du fichier `style.py` utilisé dans tout le livre.

```
plt.rcParams.update({
    "axes.spines.top": False, "axes.spines.right": False,      # pas de cadre inutile
    "axes.grid": True, "grid.color": style.GRILLE,             # grille discrète
    "axes.titlesize": 11, "font.size": 10,                    # tailles lisibles
    "legend.frameon": False,
    "figure.facecolor": style.SURFACE, "savefig.dpi": 200,
})
print({n: getattr(style, n) for n in ("BLEU", "ORANGE", "AQUA", "VIOLET", "ROUGE")})

{'BLEU': '#2a78d6', 'ORANGE': '#eb6834', 'AQUA': '#1baf7a', 'VIOLET': '#4a3aa7', 'ROUGE': '#e34948'}
```

Une palette est un **choix de conception** : cinq couleurs distinctes, dans un ordre fixe, et la **même couleur pour la même chose** dans tout le document (le Site est toujours orange). Il faut aussi la **mesurer** : le rapport de contraste entre une couleur et le fond (formule des règles d'accessibilité du web, qui demandent au moins 3 pour un élément graphique).

```
def luminance(h):
    c = [int(h[i:i + 2], 16) / 255 for i in (1, 3, 5)]
    c = [v / 12.92 if v <= 0.03928 else ((v + 0.055) / 1.055) ** 2.4 for v in c]
    return 0.2126 * c[0] + 0.7152 * c[1] + 0.0722 * c[2]

def contraste(a, b):
    la, lb = luminance(a), luminance(b)
    return (max(la, lb) + 0.05) / (min(la, lb) + 0.05)

print({n: round(contraste(getattr(style, n), style.SURFACE), 1) for n in ("BLEU", "ORANGE", "AQUA",
    "ROUGE", "VIOLET", "GRILLE")})

{'BLEU': 4.3, 'ORANGE': 3.1, 'AQUA': 2.7, 'ROUGE': 3.9, 'VIOLET': 8.3, 'GRILLE': 1.3}
```

Le bleu (4,3), l'orange (3,1) et le rouge (3,9) passent le seuil de 3 ; l'aqua (2,7) reste en dessous : on ne doit **jamais** s'y fier seul. Dans nos graphiques, la couleur de l'aqua est toujours doublée d'un **nom** (étiquette directe) : la couleur n'est jamais la seule information. Cette règle d'accessibilité

est développée au chapitre 1 (section 1.3). La grille, volontairement, a un contraste de 1,3 : elle doit se **voir à peine**.

Enregistrer une figure demande de choisir le **format** et la **résolution**. Un graphique pour un document imprimé ou projeté s'enregistre en **image** (PNG) à 200 points par pouce, ou en **vectériel** (SVG, PDF), qui reste net à toute taille et pèse peu pour un graphique simple.

```
import os, tempfile
dossier = 0.dossier_temp()
tailles = {}
for ext in ("png", "svg", "pdf"):
    chemin = os.path.join(dossier, "default." + ext)
    0.barres_default(x).savefig(chemin, dpi=200)
    tailles[ext] = os.path.getsize(chemin) // 1024
print("taille en Ko :", tailles)
0.supprimer_dossier(dossier)
```

```
taille en Ko : {'png': 23, 'svg': 19, 'pdf': 10}
```

Pour ce graphique simple, le vectériel pèse moins que l'image. Il pèse **plus** pour un nuage de 100 000 points (chaque point est un objet) : on rasterise alors (`rasterized=True`). La règle pratique : **vectériel pour les graphiques à peu d'éléments, PNG pour les nuages denses et pour le web**. La taille de la figure (`figsize`, en pouces) doit correspondre à l'emplacement visé : une figure de 5,6 pouces de large tient dans une colonne de rapport sans être réduite, et le texte reste à la taille voulue.

3.1.7 seaborn : les graphiques statistiques

seaborn est construit sur matplotlib. Il fournit en une ligne des graphiques **statistiques** (distributions par groupe, cartes thermiques, nuages avec droite de tendance) et il s'occupe des agrégations. Il retourne des objets matplotlib : on peut ensuite les retoucher avec `ax.set_title` comme avant.

Les paniers par canal, avec une **boîte à moustaches** (médiane, quartiles, valeurs extrêmes) :

```
p25 = x[x["annee"] == 2025].groupby(["id_commande", "canal"], as_index=False)["montant"].sum()
g = sns.catplot(data=p25, x="canal", y="montant", order=0.CANAU, kind="box", hue="canal",
    ↪ palette=0.COUL, height=3.2, aspect=1.5, fliersize=2)
g.set_axis_labels("", "panier (€)")
g.figure.subplots_adjust(top=0.86)
g.figure.suptitle("Les paniers se ressemblent d'un canal à l'autre", x=0.02, ha="left",
    ↪ fontweight="bold")
0.sauver(g.figure, "ch03-seaborn-paniers.png")
print(p25.groupby("canal")["montant"].median().round(1).reindex(0.CANAU).to_dict())
```

```
figure : ch03-seaborn-paniers.png
{'Boutique': 82.0, 'Site': 81.2, 'Réseaux': 81.2}
```

Les trois boîtes sont presque identiques : les médianes vont de 81 à 82 €. Sans le graphique, on aurait pu croire à un panier plus élevé sur le Site ; avec lui, on voit qu'il n'y a **pas de différence** de panier entre les canaux (la différence est dans le **nombre** de commandes). La **carte thermique** (heatmap) montre deux variables catégorielles et une valeur : ici, le nombre moyen de commandes par jour selon le mois et le jour de la semaine.

```
piv = j25.pivot_table(index="mois", columns="jour_semaine", values="nb_commandes", aggfunc="mean")
fig, ax = plt.subplots(figsize=(6.2, 4))
sns.heatmap(piv, annot=True, fmt=".0f", cmap=style.SEQ, cbar=False, linewidths=0.5, ax=ax)
ax.set_xticklabels(["lun", "mar", "mer", "jeu", "ven", "sam", "dim"], rotation=0); ax.set(xlabel="",
    ↪ ylabel="mois"); ax.grid(False)
ax.set_title("Commandes par jour en moyenne, 2025 : le samedi et décembre", loc="left",
    ↪ fontweight="bold")
```

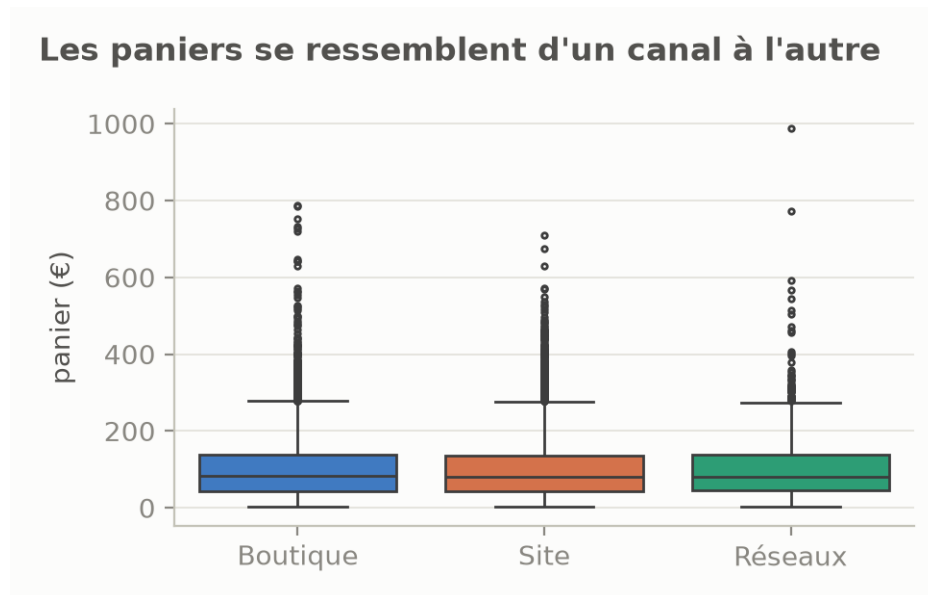


Figure 52: Distribution des paniers par canal (seaborn, boîtes à moustaches).

```
0.sauver(fig, "ch03-seaborn-carte-thermique.png")
print("samedi de décembre :", round(piv.loc[12, 6]), "| dimanche de janvier :", round(piv.loc[1, 7]))
```

```
figure : ch03-seaborn-carte-thermique.png
samedi de décembre : 85 | dimanche de janvier : 20
```

Le samedi de décembre compte 85 commandes en moyenne, le dimanche de janvier 20 : un rapport de plus de quatre. Une carte thermique est un bon outil pour les **motifs croisés** (saison × semaine), à condition de **noter les valeurs** dans les cases (annot=True) et d'utiliser une échelle de couleur **séquentielle** (du clair au foncé), pas un arc-en-ciel.

Reste une subtilité sur laquelle seaborn est discret : ses graphiques **ajoutent des intervalles de confiance par défaut** (une bande autour d'une courbe ou une barre d'erreur sur une barre). Cet intervalle décrit l'incertitude sur la **moyenne**, calculée par rééchantillonnage des observations ; il **suppose** que les observations sont **indépendantes**, et il ne décrit pas la **dispersion** des valeurs.

```
dec = j[j["mois"] == 12]["nb_commandes"]
se = dec.std() / np.sqrt(len(dec))
print("décembre :", len(dec), "jours | moyenne", round(dec.mean(), 1), "| écart-type des jours",
      ↪ round(dec.std(), 1), "| erreur type de la moyenne", round(se, 2))
print("intervalle de confiance de la moyenne : %.1f à %.1f | jours observés : %d à %d" % (dec.mean() -
      ↪ 1.96 * se, dec.mean() + 1.96 * se, dec.min(), dec.max()))
```

```
décembre : 93 jours | moyenne 55.9 | écart-type des jours 13.9 | erreur type de la moyenne 1.44
intervalle de confiance de la moyenne : 53.1 à 58.8 | jours observés : 31 à 97
```

La moyenne de décembre est connue à environ **±3 commandes** près (de 53 à 59), alors que les jours individuels vont de 31 à 97. Un lecteur qui prend la bande pour la **plage de variation des jours** se trompe d'un facteur dix environ. Quand le graphique doit montrer la variabilité, on montre les **jours** (boîte à moustaches, nuage) ; quand il doit montrer la précision d'une moyenne, on montre l'intervalle **et on le dit dans la légende**.

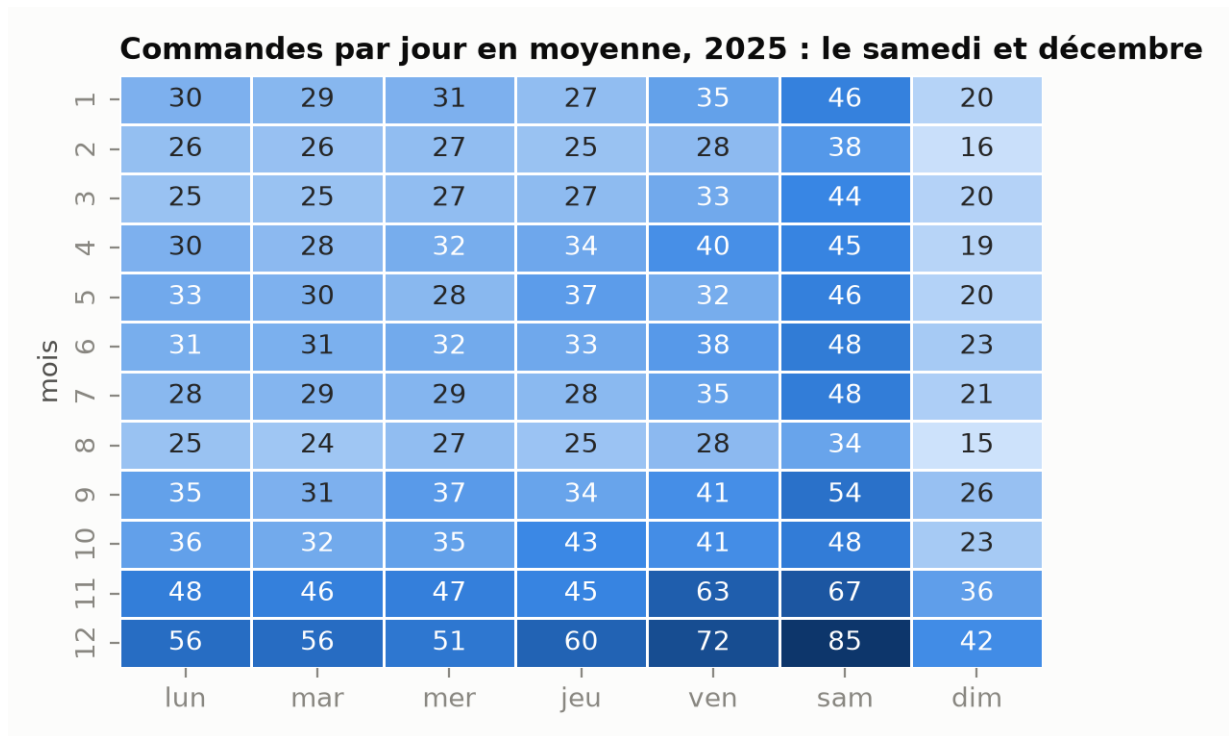


Figure 53: Commandes moyennes par jour selon le mois et le jour de la semaine (carte thermique).

3.1.8 Reproductibilité : une figure est une fonction

Un script de graphique devient fiable quand on l'enferme dans une **fonction qui reçoit les données et retourne la figure**, sans lire de fichier ni modifier d'état global. On peut alors la rejouer sur d'autres données et la **tester**.

```
def figure_ventes_canal(donnees, annee):
    ca = donnees[donnees["annee"] ==
    annee].groupby("canal")["montant"].sum().sort_values(ascending=False) / 1000
    fig, ax = plt.subplots(figsize=(5.6, 3.2))
    barres = ax.bar(ca.index, ca.values, color=[0.COUL[c] for c in ca.index], width=0.62)
    ax.bar_label(barres, labels=[f"{0.fr(v)} k€" for v in ca.values], padding=3)
    ax.set(ylim=(0, ca.max() * 1.15), yticks=[]); ax.grid(False)
    return fig

for annee in (2024, 2025):
    fig = figure_ventes_canal(x, annee)
    hauteurs = [b.get_height() for b in fig.axes[0].patches]
    attendu = x[x["annee"] == annee].groupby("canal")["montant"].sum().sort_values(ascending=False) /
    1000
    print(annee, "| hauteurs des barres :", [round(float(h), 1) for h in hauteurs], "| égales au
    tableau :", np.allclose(hauteurs, attendu.values))
    plt.close(fig)
```

```
2024 | hauteurs des barres : [558.1, 502.5, 128.8] | égales au tableau : True
2025 | hauteurs des barres : [617.7, 561.0, 146.1] | égales au tableau : True
```

On ne teste pas les **pixels** (ils changent d'une version à l'autre) mais les **données dessinées** : les hauteurs des barres, les valeurs des lignes (`ax.lines[0].get_ydata()`), le texte du titre. Ce test attrape l'erreur la plus grave d'un graphique, celle qui ne se voit pas : une barre qui montre un autre chiffre que celui du tableau. Deux autres règles de reproductibilité : **fixer les graines** quand le graphique contient de l'aléatoire (un nuage décalé aléatoirement pour ne pas empiler

les points, `rng = np.random.default_rng(0)`) et **enregistrer la figure par le script**, jamais par une capture d'écran.

À retenir. Une figure matplotlib est une Figure qui contient des Axes ; on la construit en cinq retouches (ordre, étiquettes directes, suppression du superflu, couleur avec intention, titre qui dit le message), on range les réglages dans un thème, et l'on enferme chaque graphique dans une fonction que l'on peut tester.

3.1.9 La même figure en R avec ggplot2

R et ggplot2 expriment les trois couches de 3.1.1 **explicitement**. Voici le même graphique : les données (`ca`), les esthétiques (`aes`), la géométrie (`geom_col`), puis les retouches. La figure enregistrée est celle du R, pas celle du Python.

```
library(ggplot2); library(dplyr, warn.conflicts = FALSE)
dossier <- Sys.getenv("DONNEES")
l <- read.csv(file.path(dossier, "lignes_commande.csv")); cm <- read.csv(file.path(dossier,
  ↪ "commandes.csv"))
ca <- inner_join(l, cm, by = "id_commande") |> filter(substr(date_commande, 1, 4) == "2025") |>
  group_by(canal) |> summarise(ca = sum(montant) / 1000)
p <- ggplot(ca, aes(reorder(canal, -ca), ca, fill = canal)) + geom_col(width = 0.62) +
  geom_text(aes(label = paste(format(round(ca), big.mark = " "), "k€")), vjust = -0.5) +
  scale_fill_manual(values = c(Boutique = "#2a78d6", Site = "#eb6834", Réseaux = "#1baf7a")) +
  scale_y_continuous(expand = expansion(mult = c(0, 0.12))) +
  labs(x = NULL, y = NULL, title = "Le Site et la Boutique font 89 % du chiffre d'affaires 2025") +
  theme_minimal() + theme(legend.position = "none", axis.text.y = element_blank(), panel.grid =
    ↪ element_blank(),
                        plot.title = element_text(face = "bold", size = 10), plot.title.position =
                        ↪ "plot")
ggsave(file.path(dirname(dossier), "figures", "ch03-ggplot-barres.png"), p, width = 6.2, height =
  ↪ 3.2, dpi = 200)
print(as.data.frame(ca |> mutate(ca = round(ca, 1))))
```

```
   canal   ca
1 Boutique 561.0
2 Réseaux 146.1
3 Site 617.7
```

Les deux figures disent la même chose. La différence est dans la **manière de penser** : ggplot2 **décrit** le graphique (on additionne des couches avec +), matplotlib le **dessine** (on donne des ordres à un Axes). Le tableau suivant met en regard les gestes équivalents.

Geste	matplotlib / seaborn	ggplot2
Barres	<code>ax.bar(x, y)</code>	<code>geom_col()</code>
Nuage de points	<code>ax.scatter(x, y)</code>	<code>geom_point()</code>
Courbe	<code>ax.plot(x, y)</code>	<code>geom_line()</code>
Histogramme	<code>ax.hist(v, bins=...)</code>	<code>geom_histogram(bins = ...)</code>
Couleur selon une colonne	<code>color=[...], hue= (seaborn)</code>	<code>aes(fill = canal)</code>
Petits multiples	<code>plt.subplots(1, 3);</code> <code>sns.relplot(col=...)</code>	<code>facet_wrap(~ canal)</code>
Titre, étiquettes	<code>ax.set_title,</code> <code>ax.set(xlabel=...)</code>	<code>labs(title = ..., x = ...)</code>
Thème	<code>plt.rcParams, style.setup()</code>	<code>theme_minimal(), theme(...)</code>
Enregistrer	<code>fig.savefig(..., dpi=...)</code>	<code>ggsave(..., dpi = ...)</code>

Le Site et la Boutique font 89 % du chiffre d'affaires 2025

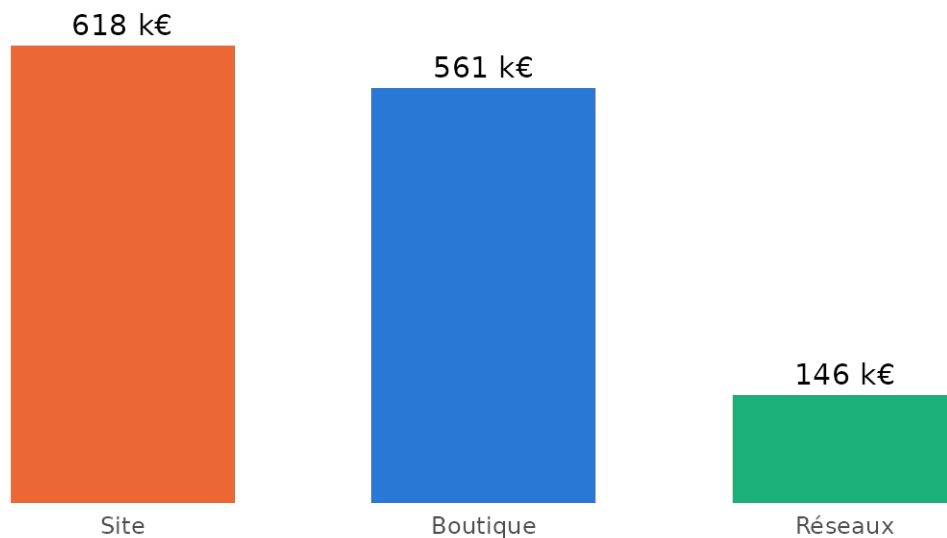


Figure 54: Le même graphique avec ggplot2 : mêmes données, mêmes couleurs, syntaxe en couches.

3.1.10 ☉ Les autres bibliothèques de graphiques

matplotlib et seaborn ne sont pas les seules options, et il est utile d'en connaître d'autres, ne serait-ce que pour reconnaître leur code. Nous ne les exécutons pas ici (elles ne font pas partie de l'environnement du livre) ; leurs fonctionnalités sont à **vérifier dans la documentation de votre version**.

- **plotnine** reprend la grammaire des graphiques de ggplot2 en Python : on écrit `ggplot(donnees, aes(...)) + geom_col()`, comme en 3.1.9. C'est le choix naturel pour qui vient de R.
- **Altair** est **déclaratif** : on décrit le lien entre colonnes et attributs visuels, et la bibliothèque produit une spécification (au format JSON, de la famille Vega-Lite) que le navigateur dessine. Les graphiques sont interactifs par construction, et la spécification est un texte que l'on peut versionner.
- **Bokeh** produit, comme plotly, des pages web interactives, avec un accent sur les grands volumes de données et sur les applications serveur.

Le choix se règle sur trois questions : **le support** (image fixe ou page web), **l'équipe** (quelle syntaxe connaît-elle ?) et **le contrôle** dont on a besoin (matplotlib reste le plus fin). Quelle que soit la bibliothèque, les règles des chapitres 1 et 2 ne changent pas : un outil ne choisit ni le message, ni le bon type de graphique.

Piège : les figures s'accumulent. matplotlib garde en mémoire chaque figure ouverte jusqu'à ce qu'on la ferme (`plt.close(fig)`). Un script qui en produit des centaines (un graphique par client, par exemple) sans les fermer finit par saturer la mémoire et par déclencher un avertissement. Les fonctions de ce chapitre qui enregistrent une figure la ferment ensuite ; prenez la même habitude.

Pour s'entraîner. Cahier, chapitre 3 : applications 3.3 à 3.5, exercices 3.5 à 3.9.

3.2 plotly et graphiques interactifs

Un graphique **interactif** réagit à la souris : une infobulle apparaît au survol, on peut zoomer, masquer une série en cliquant sur sa légende. Pour la gérante qui voulait « cliquer pour voir le détail », c'est la réponse naturelle, et **plotly** est la bibliothèque libre la plus répandue pour cela. Ses figures sont des pages web : elles s'ouvrent dans un navigateur, s'insèrent dans un tableau de bord (section 3.3) et s'envoient en pièce jointe. Dans ce livre papier, on ne peut pas cliquer : les figures de cette section sont donc de **vraies captures** (navigateur sans interface) de pages plotly **réelles**, avec le geste simulé (survol, clic) indiqué dans la légende.

3.2.1 Deux niveaux : plotly express et graph_objects

plotly s'utilise à deux niveaux. **plotly express** (px) fabrique une figure complète en une ligne à partir d'un tableau, un peu comme seaborn. **graph_objects** (go) construit la figure pièce par pièce, comme matplotlib, et permet tout régler. On commence par px, on descend à go quand il faut une personnalisation précise.

```
import os
import plotly.express as px
import plotly.graph_objects as go
ca = x[x["annee"] == 2025].groupby("canal", as_index=False)["montant"].sum().sort_values("montant",
↳ ascending=False)
f1 = px.bar(ca, x="canal", y="montant", color="canal", color_discrete_map=0.COUL)
f2 = go.Figure(go.Bar(x=ca["canal"], y=ca["montant"], marker_color=[0.COUL[c] for c in ca["canal"]]))
print(type(f1).__name__, "|", len(f1.data), "série(s) pour px,", len(f2.data), "pour go |",
↳ sorted(f2.to_dict()))
print("mêmes hauteurs :", sorted(float(v) for s in f1.data for v in s.y) == sorted(float(v) for v in
↳ f2.data[0].y))
```

Figure | 3 série(s) pour px, 1 pour go | ['data', 'layout']
mêmes hauteurs : True

Une figure plotly est une simple **structure de données** : des séries (data) et une mise en page (layout), que l'on peut afficher, modifier ou convertir en texte JSON. C'est ce qui permet à la page web de la redessiner à chaque geste. Notez que px crée **une série par valeur de la colonne de couleur** (ici trois séries d'une barre), alors que notre figure go n'a qu'une série de trois barres : le résultat visible est le même, la structure interne ne l'est pas.

3.2.2 Survol, zoom, légende : l'interactivité en trois gestes

Voici le graphique des ventes par canal, rendu interactif et habillé d'un **gabarit sobre** (simple_white). Le point important est le texte de l'infobulle (hovertemplate) : il dit **exactement** ce que l'on veut lire, avec son unité, au lieu du texte technique par défaut.

```
dossier = 0.dossier_temp()
fig = px.bar(ca, x="canal", y="montant", color="canal", color_discrete_map=0.COUL,
↳ template="simple_white")
fig.update_traces(hovertemplate="<b>{x}</b><br>{y:,.0f} € en 2025<extra></extra>")
fig.update_layout(separators=" ", showlegend=False, xaxis_title=None, yaxis_title="€",
↳ title="Chiffre d'affaires 2025 par canal : survolez une barre")
barres = os.path.join(dossier, "barres.html")
fig.write_html(barres, include_plotlyjs=True)
print("page HTML autonome :", round(os.path.getsize(barres) / 1e6, 1), "Mo")
0.capturer_html(barres, os.path.join(0.FIG, "ch03-plotly-barres.png"), 900, 460, survol=".bars .point
↳ >> nth=1")
```


page HTML autonome : 4.8 Mo
capture existante : ch03-plotly-barres.png

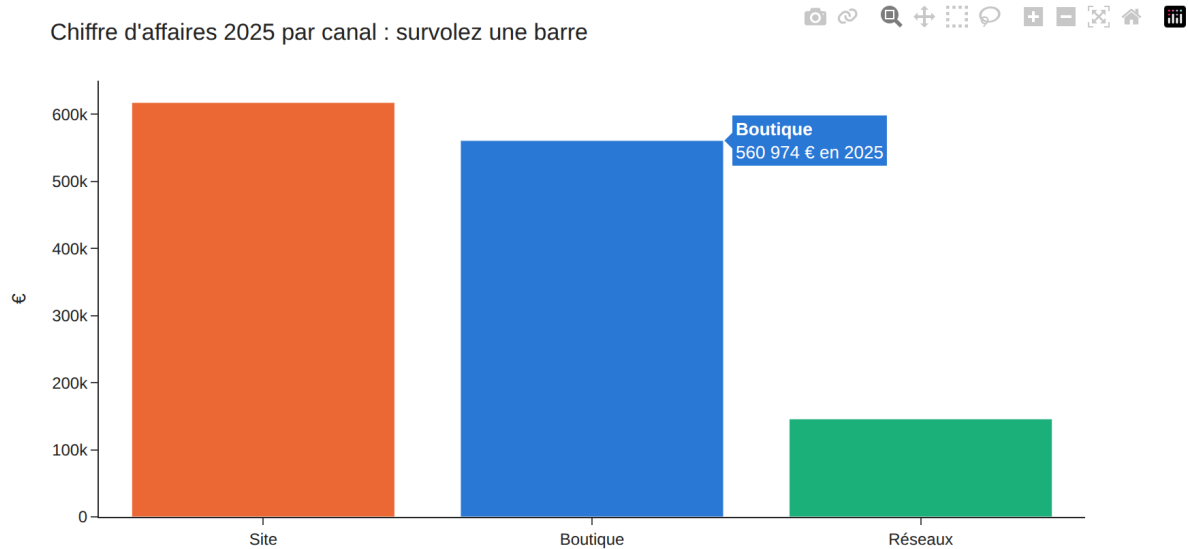


Figure 55: Capture réelle d'une figure plotly : le survol de la barre du Site affiche une infobulle ; la barre d'outils (appareil photo, zoom, déplacement, sélection, réinitialisation) est en haut à droite.

Le fichier HTML pèse plusieurs mégaoctets parce qu'il **embarque la bibliothèque** (`include_plotlyjs=True`) : il fonctionne **hors ligne**, ce qui est ce qu'on veut pour un envoi par courriel ; avec `include_plotlyjs="cdn"`, il ne fait que quelques dizaines de kilo-octets mais exige une connexion pour charger la bibliothèque. Pour une série temporelle, deux réglages changent tout : le **mode de survol unifié** (`hovermode="x unified"`), qui affiche toutes les séries d'un même mois dans une seule infobulle, et le **curseur de plage** (`rangeslider`), qui permet de zoomer sur une période.

```
mois = x.groupby([x["mois"].dt.to_timestamp().rename("mois"), "canal"],
↳ as_index=False)["montant"].sum()
fig = px.line(mois, x="mois", y="montant", color="canal", color_discrete_map=0.COUL,
↳ template="simple_white")
fig.update_traces(hovertemplate="%{y:,.0f} €")
fig.update_xaxes(rangeslider_visible=True, tickformat="%m/%Y")
fig.update_layout(separators=", ", hovermode="x unified", yaxis_title="€ par mois", xaxis_title=None,
↳ legend_title=None, title="Chiffre d'affaires mensuel par canal")
serie = os.path.join(dossier, "serie.html")
fig.write_html(serie, include_plotlyjs=True)
0.capturer_html(serie, os.path.join(0.FIG, "ch03-plotly-survol.png"), 1000, 520, survol=(0.78, 0.3))
print("séries :", [t.name for t in fig.data])
```

capture existante : ch03-plotly-survol.png
séries : ['Boutique', 'Réseaux', 'Site']

Les deux autres gestes se simulent de même : un **clic sur un nom de la légende** masque la série correspondante (ici on retire le Site pour regarder de près la Boutique et les Réseaux), et un **glissé** sur le graphique sélectionne une zone à agrandir.

```
0.capturer_html(serie, os.path.join(0.FIG, "ch03-plotly-zoom.png"), 1000, 520,
↳ clic=['.legend .traces:has-text("Site")'], glisser=[(0.62, 0.5, 0.9, 0.5)])
print("captures de la section :", sorted(f for f in os.listdir(0.FIG) if f.startswith("ch03-plotly")))
```

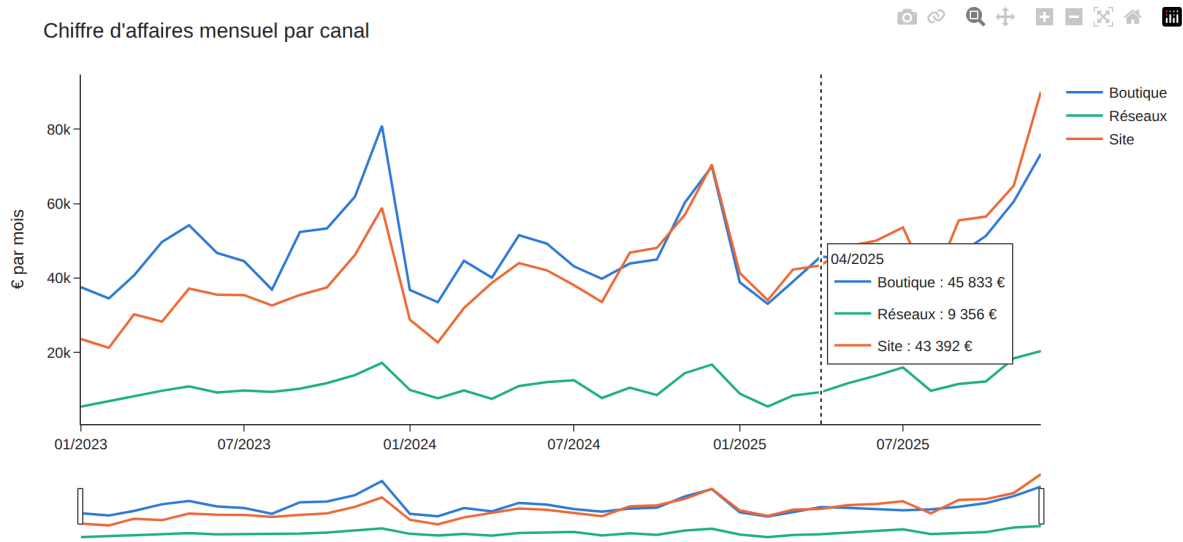


Figure 56: Capture réelle : survol unifié d'un mois (les trois canaux dans une infobulle) et curseur de plage sous le graphique.

capture existante : ch03-plotly-zoom.png

captures de la section : ['ch03-plotly-barres.png', 'ch03-plotly-survol.png', 'ch03-plotly-zoom.png']

3.2.3 Quand l'interactivité aide, et quand elle gêne

L'interactivité **aide** quand le lecteur doit **explorer** : beaucoup de points ou de séries, des détails à la demande (le montant exact d'un mois), des filtres qu'il choisit lui-même. Elle sert bien un tableau de bord consulté chaque semaine (chapitre 2). Elle **gêne** dans quatre cas.

1. **Le support est fixe.** Un rapport imprimé, un PDF, une diapositive : le survol n'existe pas. Une figure destinée à ces supports doit **se suffire à elle-même** : étiquettes directes, annotation du fait marquant, titre qui dit le message, comme en 3.1.
2. **Le message ne doit pas se chercher.** Si le fait important n'apparaît qu'au survol, la plupart des lecteurs ne le verront pas. L'information essentielle est **visible sans geste** ; l'interactivité n'apporte que le **détail**.
3. **L'accessibilité.** Une infobulle ne s'obtient pas au clavier, ni avec un lecteur d'écran, ni sur un téléphone sans souris sans geste précis. On double l'interactivité d'un **tableau** ou d'un **texte alternatif** qui donne le message.
4. **La reproductibilité.** Une page interactive est difficile à archiver et à citer ; on **garde une image** de ce qui a été montré à la réunion.

Piège. Une figure interactive **séduit** : elle donne l'impression d'une analyse approfondie. Mais l'ajout de survol et de zoom n'améliore ni la question, ni la justesse du chiffre. Un bon graphique statique vaut mieux qu'un mauvais graphique interactif.

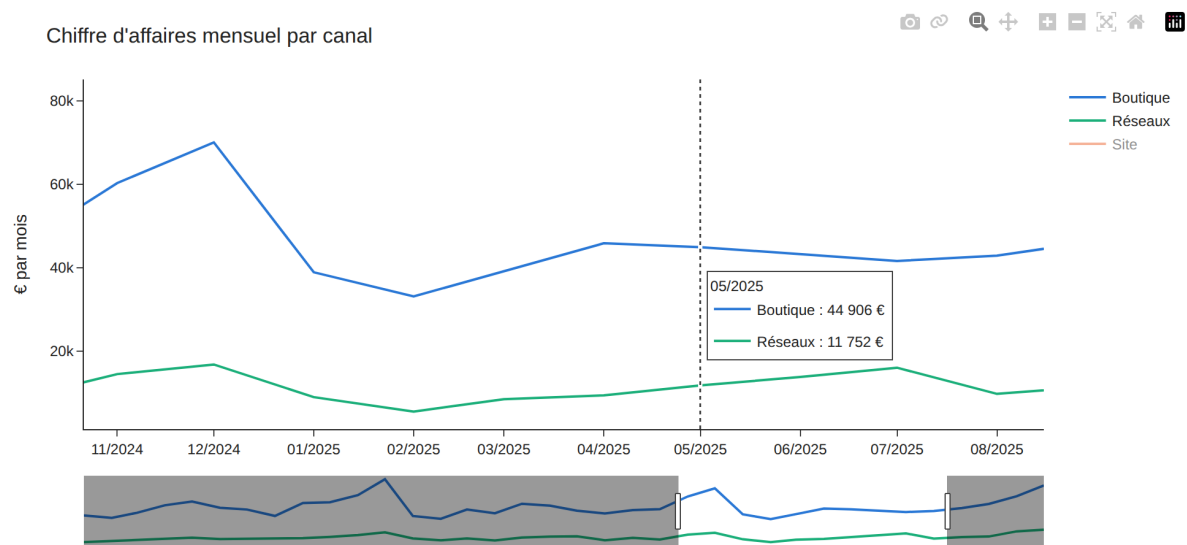


Figure 57: Capture réelle : légende cliquée (série masquée) et zone agrandie par glissé.

3.2.4 plotly ou matplotlib ?

Le choix ne tient pas à la qualité mais à l'usage. Le tableau résume ce que nous avons vu.

	matplotlib / seaborn	plotly
Sortie principale	image fixe (PNG, SVG, PDF)	page web interactive (HTML)
Rapport imprimé, PDF, diapositive	oui	non (il faut une capture)
Exploration, tableau de bord	possible, peu confortable	oui
Contrôle des détails	très fin	bon, via <code>update_layout</code>
Taille du fichier	quelques dizaines de Ko	plusieurs Mo (bibliothèque embarquée)
Accessibilité clavier, lecteur d'écran	image + texte alternatif	à soigner (tableau associé)
Reproductibilité d'une réunion	très bonne (une image)	plus fragile (page à archiver)

Une pratique courante : **explorer** avec plotly, **livrer** une image matplotlib ou une capture. Dans les deux cas, la discipline est la même : un titre qui dit le message, des unités, des couleurs avec intention, une annotation unique.

À retenir. plotly produit des pages web interactives (survol, zoom, filtre par légende) à partir d'une structure de données ; on l'emploie pour **explorer** et pour les **tableaux de bord**, jamais comme seul support d'un message : l'information essentielle doit rester visible **sans geste**, et une image doit en rester la trace.

Pour s'entraîner. Cahier, chapitre 3 : application 3.6, exercices 3.10 à 3.12.

3.3 ⊕ Pour aller plus loin : tableaux de bord avec Dash, Streamlit et Shiny

Les outils de tableau de bord à interface graphique (chapitre 2) ne conviennent pas toujours : on peut vouloir un tableau de bord **versionné**, **testé**, branché sur n'importe quelle source, déployable sans licence. Trois outils **libres** répondent à ce besoin en quelques dizaines de lignes de code : **Dash** (Python, par les auteurs de plotly), **Streamlit** (Python) et **Shiny** (R, et depuis peu Python). Cette section écrit le **même** mini-tableau de bord dans les trois, le **teste** sans navigateur, puis le **photographie** pour de vrai.

3.3.1 Le cahier des charges commun

La gérante demande : « un écran avec le chiffre d'affaires et le nombre de commandes, la répartition par catégorie, l'évolution par mois, et deux réglages : les canaux à inclure et l'année. » Soit deux **entrées** (canaux, année), un **calcul** commun (filtrer les ventes) et trois **sorties** (deux chiffres, un diagramme en barres, une courbe). Les trois outils diffèrent par la façon dont ils relient les entrées aux sorties : c'est leur **modèle d'exécution**.

Outil	Modèle d'exécution	Ce que fait l'outil à chaque changement d'une entrée
Streamlit	le script se rejoue de haut en bas	relance tout le script (les données chargées sont gardées en cache)
Dash	des fonctions de rappel (callbacks) déclarées	appelle la fonction reliée à l'entrée modifiée, avec ses entrées et sorties déclarées
Shiny	un graphe réactif	recalcule seulement les expressions qui dépendent de l'entrée modifiée

Les trois applications ci-dessous sont **écrites dans ce livre** : le code que vous lisez est extrait par le script de construction du chapitre, enregistré dans un dossier temporaire, puis **testé et lancé**. Il n'y a donc pas de copie de l'application qui pourrait diverger du texte.

3.3.2 Streamlit : un script qui se rejoue

```
# app_streamlit.py (1/2)
import os, pandas as pd, plotly.express as px, streamlit as st
@st.cache_data
def charger():
    d = os.environ["DONNEES"]
    c = pd.read_csv(f"{d}/commandes.csv", parse_dates=["date_commande"])
    p = pd.read_csv(f"{d}/produits.csv")[["id_produit", "categorie"]]
    l = pd.read_csv(f"{d}/lignes_commande.csv").merge(p, on="id_produit")
    return l.merge(c[["id_commande", "date_commande", "canal"]], on="id_commande")
ventes = charger()
st.title("Ventes de la boutique")

# app_streamlit.py (2/2)
tous = sorted(ventes["canal"].unique())
canaux = st.sidebar.multiselect("Canaux", tous, default=tous)
annee = st.sidebar.slider("Année", 2023, 2025, 2025)
v = ventes[ventes["canal"].isin(canaux) & (ventes["date_commande"].dt.year == annee)]
```

```

st.metric("Chiffre d'affaires (€)", f"{v['montant'].sum():,.0f}".replace(",", " "))
st.metric("Commandes", v["id_commande"].nunique())
st.plotly_chart(px.bar(v.groupby("categorie", as_index=False)["montant"].sum(), x="categorie",
    ↪ y="montant"))
mois = v.groupby(v["date_commande"].dt.to_period("M").astype(str), as_index=False)["montant"].sum()
st.plotly_chart(px.line(mois, x="date_commande", y="montant"))

```

Le script se lit comme une page : un titre, des réglages dans la barre latérale, des chiffres, deux graphiques. Les données sont chargées **une fois** (@st.cache_data), sinon elles seraient relues à chaque clic. **Tester** une application Streamlit sans navigateur est possible : AppTest exécute le script, permet de **manipuler les widgets** et de **lire les éléments produits**.

```

import os, sys
from streamlit.testing.v1 import AppTest
apps = 0.dossier_apps()
print("applications extraites du livre :", 0.extraire_apps("sections/03-3-dash-streamlit-shiny.md",
    ↪ apps))
at = AppTest.from_file(os.path.join(apps, "app_streamlit.py"), default_timeout=120).run()
ca25 = x[x["annee"] == 2025]["montant"].sum()
print("2025 :", [m.value for m in at.metric], "| attendu :", 0.fr(ca25), x[x["annee"] ==
    ↪ 2025]["id_commande"].nunique())
at.sidebar.multiselect[0].unselect("Réseaux").run()
at.sidebar.slider[0].set_value(2024).run()
ca24 = x[(x["annee"] == 2024) & (x["canal"] != "Réseaux")]
print("2024 hors Réseaux :", [m.value for m in at.metric], "| attendu :",
    ↪ 0.fr(ca24["montant"].sum()), ca24["id_commande"].nunique(), "| exceptions :", len(at.exception))

```

```

applications extraites du livre : ['app_dash.py', 'app_shiny.R', 'app_streamlit.py']
2025 : ['1 324 764', '12946'] | attendu : 1 324 764 12946
2024 hors Réseaux : ['1 060 674', '10730'] | attendu : 1 060 674 10730 | exceptions : 0

```

On vérifie ce qui compte : **les chiffres affichés sont ceux du tableau pandas**, après avoir retiré un canal et changé d'année. Le test ne voit pas l'aspect de la page (couleurs, alignement) : il valide le **calcul**, pas le **rendu**. C'est exactement la division du travail qu'on souhaite : la machine vérifie les chiffres, l'œil vérifie la page.

3.3.3 Dash : des fonctions de rappel

```

# app_dash.py (1/2)
import os, pandas as pd, plotly.express as px
from dash import Dash, dcc, html, Input, Output
d = os.environ["DONNEES"]
c = pd.read_csv(f"{d}/commandes.csv", parse_dates=["date_commande"])
p = pd.read_csv(f"{d}/produits.csv")[["id_produit", "categorie"]]
v = pd.read_csv(f"{d}/lignes_commande.csv").merge(p, on="id_produit").merge(c[["id_commande",
    ↪ "date_commande", "canal"]], on="id_commande")
CANAUUX = sorted(v["canal"].unique())
app = Dash(__name__)
app.layout = html.Div([html.H3("Ventes de la boutique"), dcc.Checklist(id="canaux", options=CANAUUX,
    ↪ value=CANAUUX, inline=True),
    dcc.Slider(2023, 2025, 1, value=2025, id="annee", marks={a: str(a) for a in (2023, 2024, 2025)}),
    ↪ html.Div(id="kpi",
    dcc.Graph(id="cat", style={"height": "230px"}), dcc.Graph(id="serie", style={"height":
    ↪ "230px"})), style={"width": "760px", "margin": "auto"})

```

```

# app_dash.py (2/2)
@app.callback(Output("kpi", "children"), Output("cat", "figure"), Output("serie", "figure"),
    ↪ Input("canaux", "value"), Input("annee", "value"))
def maj(canaux, annee):
    s = v[v["canal"].isin(canaux) & (v["date_commande"].dt.year == annee)]

```

```

kpi = f"Chiffre d'affaires : {s['montant'].sum():,.0f} € | commandes :
↳ {s['id_commande'].nunique()}.replace(", ", " ")
cat = px.bar(s.groupby("categorie", as_index=False)["montant"].sum(), x="categorie", y="montant")
mois = s.groupby(s["date_commande"].dt.to_period("M").astype(str),
↳ as_index=False)["montant"].sum()
return kpi, cat, px.line(mois, x="date_commande", y="montant")
if __name__ == "__main__":
    app.run(port=int(os.environ.get("PORT", 8050)))

```

Dash sépare nettement la **mise en page** (app. layout, une arborescence de composants) du **comportement** (la fonction maj, reliée par Input et Output). Chaque composant a un identifiant ; le décorateur déclare « quand canaux ou annee change, appelle maj et mets le résultat dans kpi, cat et serie ». C'est plus verbeux que Streamlit, mais le **graphe des dépendances est explicite**, ce qui aide pour les grosses applications. Le test le plus simple appelle la **fonction de rappel directement**, sans serveur.

```

import importlib.util
os.environ["PORT"] = "0"
spec = importlib.util.spec_from_file_location("app_dash", os.path.join(apps, "app_dash.py"))
mod = importlib.util.module_from_spec(spec); spec.loader.exec_module(mod)
kpi, fig_cat, fig_serie = mod.maj(["Boutique", "Site"], 2025)
ca = x[(x["annee"] == 2025) & (x["canal"] != "Réseaux")]
print(kpi, "| attendu :", 0.fr(ca["montant"].sum()), ca["id_commande"].nunique())
print("catégories dans le graphique :", sorted(fig_cat.data[0].x), "| mois :",
↳ len(fig_serie.data[0].x))

```

```

Chiffre d'affaires : 1 178 689 € | commandes : 11520 | attendu : 1 178 689 11520
catégories dans le graphique : ['Bien-être', 'Cuisine', 'Décoration', 'Jardin', 'Maison',
↳ 'Papeterie'] | mois : 12

```

3.3.4 Shiny : un graphe réactif

```

# app_shiny.R (1/2)
library(shiny); library(dplyr, warn.conflicts = FALSE); library(ggplot2)
d <- Sys.getenv("DONNEES")
v <- inner_join(read.csv(file.path(d, "lignes_commande.csv")), read.csv(file.path(d,
↳ "commandes.csv")), by = "id_commande") |>
  inner_join(read.csv(file.path(d, "produits.csv")), c("id_produit", "categorie"), by =
↳ "id_produit") |>
  mutate(annee = as.integer(substr(date_commande, 1, 4)), mois = substr(date_commande, 1, 7))
ui <- fluidPage(titlePanel("Ventes de la boutique"), sidebarLayout(
  sidebarPanel(checkboxGroupInput("canaux", "Canaux", sort(unique(v$canal))), selected =
↳ sort(unique(v$canal))),
  sliderInput("annee", "Année", 2023, 2025, 2025, sep = ""),
  mainPanel(textOutput("kpi"), plotOutput("cat", height = 220), plotOutput("serie", height = 220)))

# app_shiny.R (2/2)
server <- function(input, output, session) {
  sel <- reactive(filter(v, canal %in% input$canaux, annee == input$annee))
  output$kpi <- renderText(paste0("Chiffre d'affaires : ", format(round(sum(sel()$montant)), big.mark
↳ = " "), " € | commandes : ", n_distinct(sel()$id_commande)))
  output$cat <- renderPlot(sel() |> group_by(categorie) |> summarise(ca = sum(montant)) |>
    ggplot(aes(reorder(categorie, -ca), ca)) + geom_col(fill = "#2a78d6") + scale_y_continuous(labels
↳ = scales::label_number(big.mark = " ")) + labs(x = NULL, y = "€") + theme_minimal())
  output$serie <- renderPlot(sel() |> group_by(mois) |> summarise(ca = sum(montant)) |>
    ggplot(aes(mois, ca, group = 1)) + geom_line(color = "#eb6834") + labs(x = NULL, y = "€") +
↳ theme_minimal())
}
shinyApp(ui, server)

```

Le cœur de Shiny est réactive : l'expression `sel()` n'est calculée que si l'une de ses entrées a changé, et les trois sorties la réutilisent (le filtrage n'est fait qu'une fois, pas trois). C'est ce que Streamlit obtient avec le cache et Dash en le recalculant dans la fonction de rappel. `testServer` exécute le serveur sans navigateur.

```
library(shiny)
app <- shinyAppFile(file.path(Sys.getenv("TMPDIR"), "ch03-apps", "app_shiny.R"))
testServer(app, {
  session$setInputs(canaux = c("Boutique", "Site"), annee = 2025)
  cat("kpi :", output$kpi, "\n")
  session$setInputs(annee = 2024)
  cat("kpi :", output$kpi, "\n")
})
```

```
kpi : Chiffre d'affaires : 1 178 689 € | commandes : 11520
kpi : Chiffre d'affaires : 1 060 674 € | commandes : 10730
```

Les trois outils donnent, pour les mêmes réglages, le **même chiffre** : c'est la preuve que les trois applications calculent la même chose (la comparaison à pandas dans les deux premiers tests, la lecture du KPI dans le troisième).

3.3.5 Les trois applications, photographiées

On lance chaque application dans un sous-processus local sur un port libre, on la photographie avec un navigateur sans interface, puis on **arrête** le processus : aucune application ne reste active. Ce sont de vraies captures de ces outils libres.

```
env = {"DONNEES": os.environ["DONNEES"], "PORT": "{port}"}
fig_ = lambda nom: os.path.join(0.FIG, nom)
0.photographier_appli([sys.executable, "-m", "streamlit", "run", os.path.join(apps,
  ↪ "app_streamlit.py"), "--server.headless", "true", "--server.address", "127.0.0.1",
  ↪ "--server.port", "{port}", "--browser.gatherUsageStats", "false"],
  ↪ fig_("ch03-app-streamlit.png"), "Chiffre d'affaires", env=env,
  ↪ hauteur=1450)
0.photographier_appli([sys.executable, os.path.join(apps, "app_dash.py")], fig_("ch03-app-dash.png"),
  ↪ "Chiffre d'affaires", env=env, hauteur=560)
0.photographier_appli(["Rscript", "-e", "shiny::runApp(file.path(Sys.getenv('TMPDIR'), 'ch03-apps',
  ↪ 'app_shiny.R'), port = {port}, launch.browser = FALSE)"],
  ↪ fig_("ch03-app-shiny.png"), "Chiffre d'affaires", env=env, hauteur=520)
print(sorted(f for f in os.listdir(0.FIG) if f.startswith("ch03-app-")))
```

```
capture existante : ch03-app-streamlit.png
capture existante : ch03-app-dash.png
capture existante : ch03-app-shiny.png
['ch03-app-dash.png', 'ch03-app-shiny.png', 'ch03-app-streamlit.png']
```

Le **style par défaut** diffère (Streamlit soigne l'apparence sans effort, Dash et Shiny demandent un peu de mise en forme), mais le **contenu est le même**. On voit aussi la limite d'un tableau de bord produit en peu de lignes : pas de titres qui disent un message, pas de hiérarchie visuelle ; la section 2.3 (conception d'un tableau de bord) explique ce que l'on ajouterait avant de le livrer.

3.3.6 Déploiement, secrets, et choix de l'outil

Mettre ces applications à disposition d'autres personnes soulève des questions que le notebook ne posait pas. **Où tourne-t-elle ?** Sur un serveur (une machine de l'entreprise, un service d'hébergement), dont il faut assurer la disponibilité. **Qui a le droit de la voir ?** Une authentification, ou un accès restreint au réseau interne. **Où sont les secrets ?** Mot de passe de la base, clés d'API : **jamais dans le code** ni dans le dépôt, mais dans des **variables d'environnement**

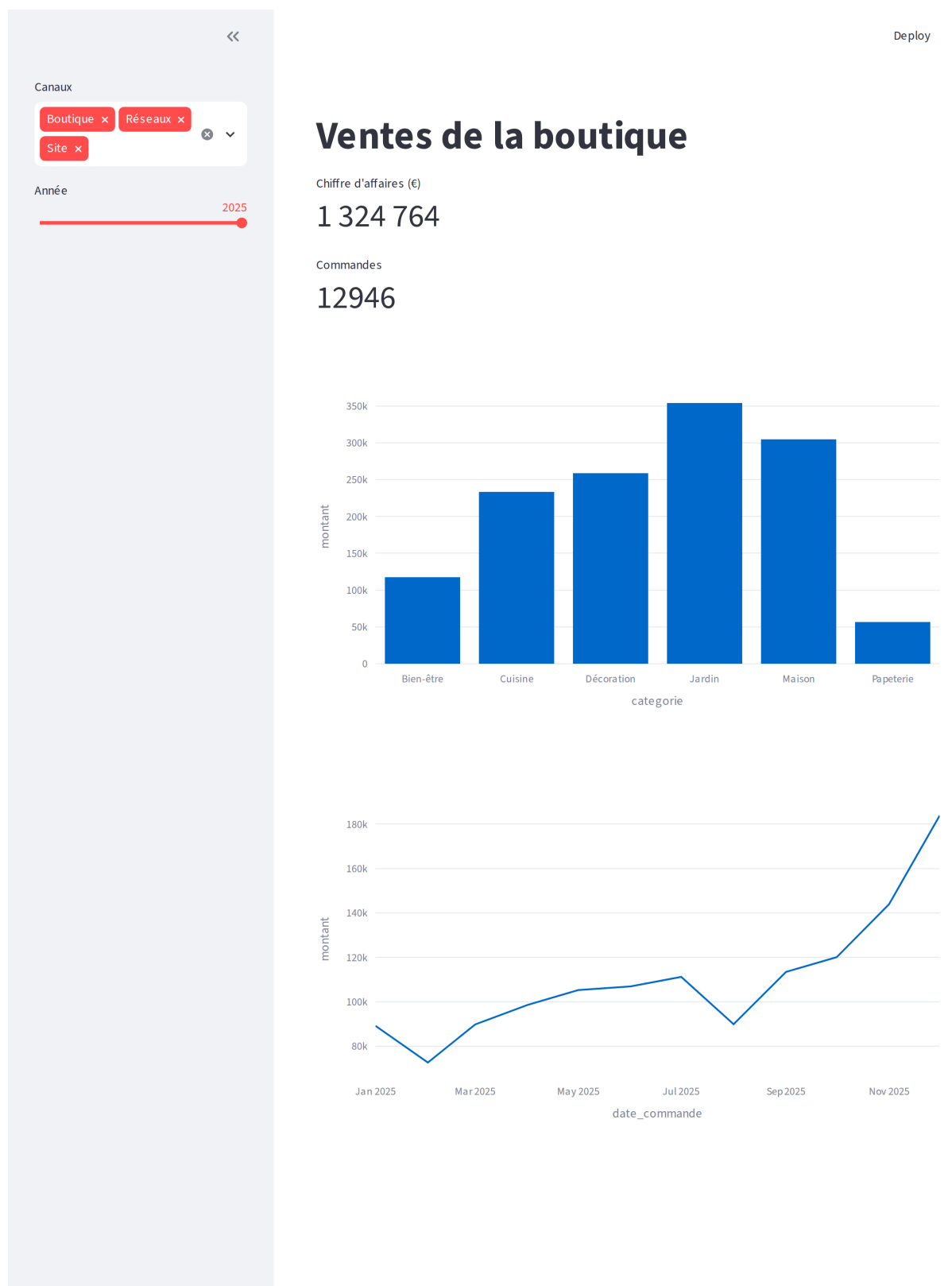


Figure 58: Capture réelle de l'application Streamlit : réglages à gauche, deux chiffres, deux graphiques.

Ventes de la boutique

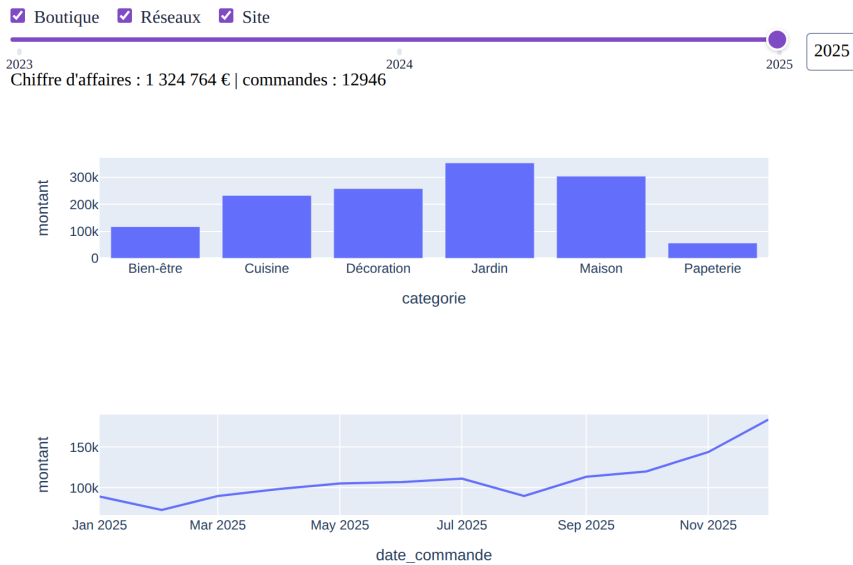


Figure 59: Capture réelle de l'application Dash : le même contenu, la mise en page est celle que l'on a écrite.

Ventes de la boutique

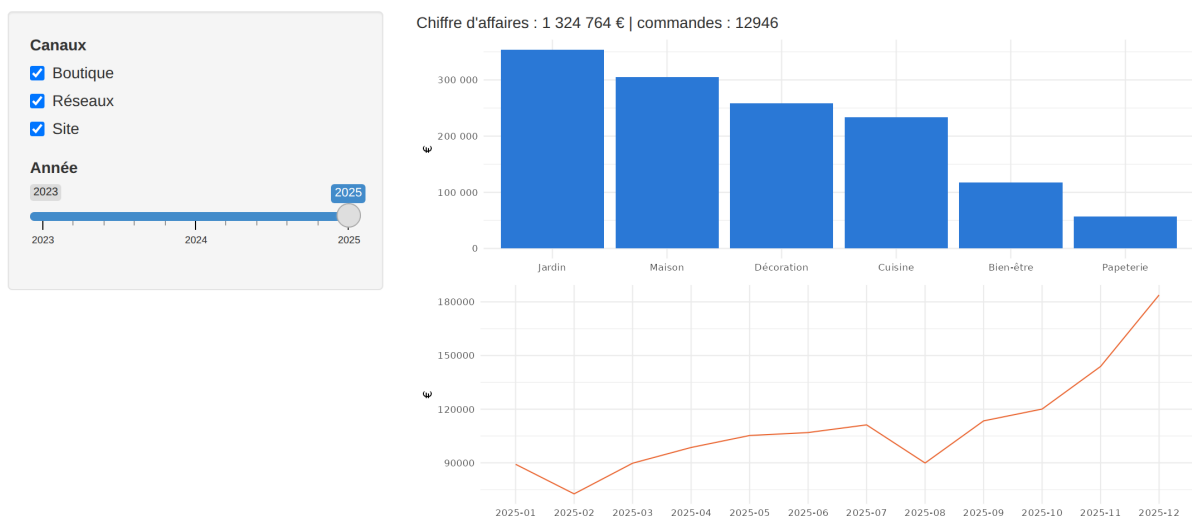


Figure 60: Capture réelle de l'application Shiny (R) : mêmes réglages, mêmes chiffres.

ou un gestionnaire de secrets (ici, le seul réglage externe est le dossier des données, `DONNEES`). **Qui la maintient ?** Une application est un petit **produit** : elle a des versions, des bogues, des utilisateurs. L'automatisation des traitements est traitée au volume V, chapitre 2.

Quand choisir quoi ? **Streamlit** pour aller vite et obtenir un joli résultat : prototypes, outils internes simples ; son modèle (le script se rejoue) est facile à comprendre, moins adapté aux interfaces très interactives. **Dash** pour les tableaux de bord construits autour de graphiques `plotly`, avec un contrôle fin des interactions, et quand on a besoin d'un composant précis. **Shiny** quand l'équipe travaille en **R**, ou pour les graphes réactifs complexes. Dans tous les cas, ces outils s'adressent à quelqu'un qui **sait programmer** : pour un décideur qui veut construire son propre tableau de bord sans code, les outils du chapitre 2 restent le bon choix.

À retenir. Dash, Streamlit et Shiny fabriquent un tableau de bord avec quelques dizaines de lignes de code : script rejoué, fonctions de rappel ou graphe réactif. On teste le calcul sans navigateur (`AppTest`, appel direct de la fonction de rappel, `testServer`) et l'on regarde la page soi-même ; secrets hors du code, déploiement et maintenance sont de vraies charges.

Pour s'entraîner. Cahier, chapitre 3 : applications 3.7 et 3.8, exercices 3.13 et 3.14.

3.4 ⊕ Pour aller plus loin : cartes et visualisation géospatiale

La gérante demande : « *peux-tu me montrer d'où viennent nos ventes ? Une carte, ce serait parlant.* » Une carte est le bon graphique quand **la position compte** (où se trouvent les clients, où sont les entrepôts) et le mauvais quand la question porte sur un **classement** ou sur une **évolution**. Cette section construit deux types de cartes, **sans aucune donnée géographique réelle** : les 20 villes de la boutique sont des points d'un **plan fictif** (en kilomètres), ce qui suffit à toucher tous les pièges du genre sans télécharger un fond de carte, ni dépendre d'un service tiers.

3.4.1 Les données : des villes dans un plan fictif

Le fichier `villes.csv` donne, pour chaque ville, deux coordonnées (`x_km`, `y_km`) dans un rectangle de 120 par 90 kilomètres, une **région** fictive et un **nombre d'habitants** fictif. On y ajoute le chiffre d'affaires 2025 de chaque ville (les clients ont une ville ; les commandes, un client).

```
v = 0.ventes_villes(x)
print(v.sort_values("ca", ascending=False)[["region", "habitants", "ca",
↪ "par_habitant"]].head(5).round({"ca": 0, "par_habitant": 1}))
print("chiffre d'affaires total des 20 villes :", 0.fr(v["ca"].sum()), "€ | habitants :",
↪ 0.fr(v["habitants"].sum()))
```

ville	region	habitants	ca	par_habitant
Ville A	Région 1	43700	184324.0	4.2
Ville B	Région 1	170000	164880.0	1.0
Ville C	Région 3	11300	121696.0	10.8
Ville D	Région 3	17000	116870.0	6.9
Ville E	Région 3	47800	102757.0	2.1
chiffre d'affaires total des 20 villes : 1 324 764 € habitants : 1 307 700				

Le **chiffre d'affaires** et le **chiffre d'affaires par habitant** ne racontent pas la même histoire : une grande ville peut vendre beaucoup sans que sa population achète beaucoup. On va le voir de deux façons.

3.4.2 Les cercles proportionnels

Quand on a un **nombre par lieu**, la carte la plus fidèle est souvent celle des **cercles proportionnels** : un cercle par ville, centré sur sa position, dont la **surface** est proportionnelle à la valeur. On proportionne la **surface** et non le rayon : doubler la valeur doit doubler l'aire ; si l'on doublait le rayon, l'aire quadruplerait et l'œil surestimerait l'écart (c'est l'argument `s` de `scatter`, qui est une surface).

```
fig, ax = plt.subplots(figsize=(6.4, 4.6))
ax.scatter(v["x_km"], v["y_km"], s=v["ca"] / v["ca"].max() * 1500, color=style.BLEU, alpha=0.55,
          ↪ edgecolor="white")
for ville, r in v.nlargest(5, "ca").iterrows():
    ax.annotate(f"{ville}\n{r['ca'] / 1000:.0f} k€", (r["x_km"], r["y_km"]), ha="center",
              ↪ va="center", fontsize=8, color=style.ENCRE)
ax.set(xlabel="km (plan fictif)", ylabel="km", aspect="equal", xlim=(-5, 125), ylim=(-5, 95));
    ↪ ax.grid(False)
ax.set_title("Chiffre d'affaires 2025 par ville (surface du cercle proportionnelle)", loc="left",
          ↪ fontweight="bold", fontsize=10)
O.sauver(fig, "ch03-carte-cercles.png")
print("les 5 premières villes pèsent", round(v.nlargest(5, "ca")["ca"].sum() / v["ca"].sum() * 100),
      ↪ "% du chiffre d'affaires")
```

figure : ch03-carte-cercles.png
les 5 premières villes pèsent 52 % du chiffre d'affaires

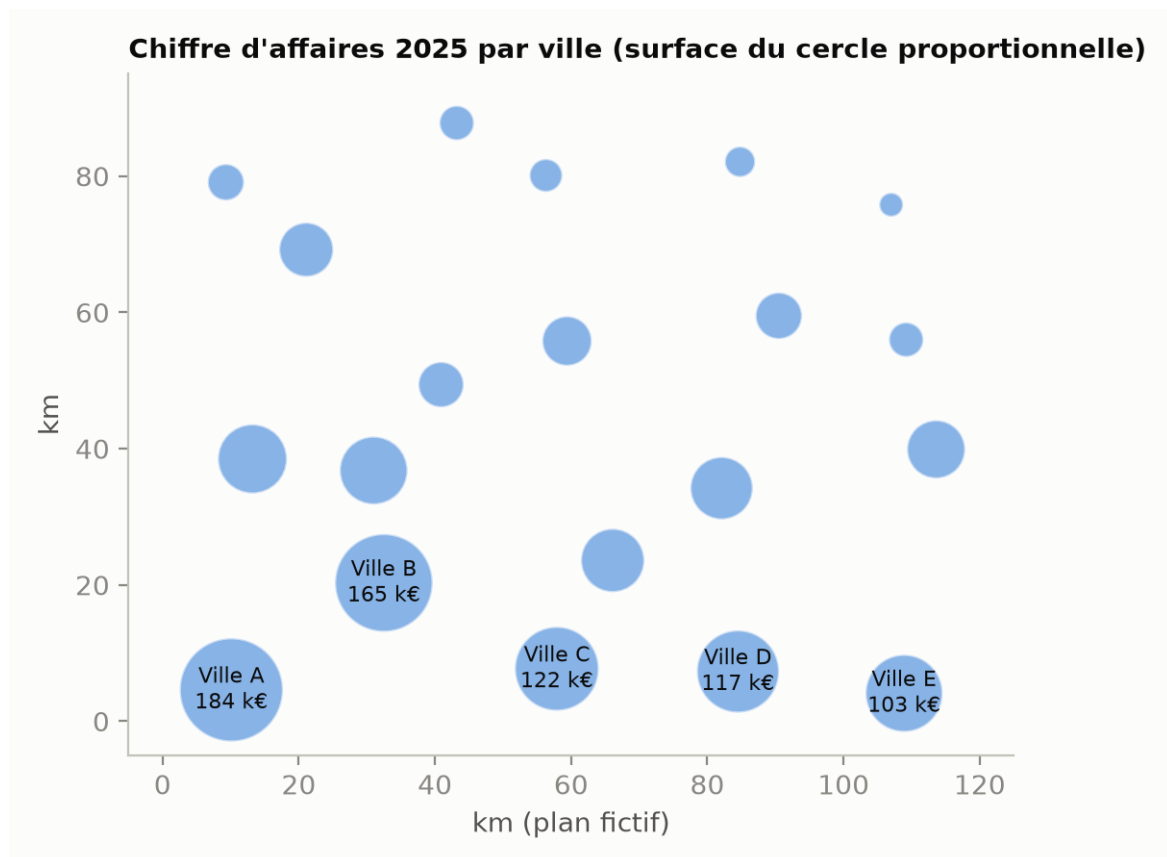


Figure 61: Carte en cercles proportionnels dans un plan fictif : les 5 premières villes sont annotées.

La carte montre **où** se concentre l'activité, et qu'elle n'est pas répartie comme le territoire. On a laissé une échelle en kilomètres et un repère orthonormé (`aspect="equal"`) : sans cela, les distances seraient déformées. Elle ne permet toutefois pas de **lire** les valeurs : pour cela, il faut un tableau ou des barres.

3.4.3 Une carte colorée (choroplèthe) sur des territoires fictifs

Une carte **choroplèthe** colore des **zones** selon une valeur. Sans fonds de carte administratif, on découpe le plan en **zones d'influence** : à chaque ville son **polygone de Voronoï**, l'ensemble des points du plan plus proches d'elle que de toute autre. Le calcul tient en quelques lignes (la fonction `voronoi_polygones` de `outils_ch03.py` coupe un rectangle par la médiatrice de chaque paire de villes). On colore ensuite par une échelle **séquentielle** (du clair au foncé), jamais par un arc-en-ciel.

```
from matplotlib.patches import Polygon
pts = v[["x_km", "y_km"]].to_numpy()
zones = 0.voronoi_polygones(pts, borne=(0, 120, 0, 90))
fig, axes = plt.subplots(1, 2, figsize=(9, 3.8))
for ax, col, titre in zip(axes, ("ca", "par_habitant"), ("chiffre d'affaires (k€)", "chiffre
↳ d'affaires par habitant (€)")):
    val = v[col] / (1000 if col == "ca" else 1)
    norm = plt.Normalize(val.min(), val.max())
    for poly, valeur in zip(zones, val):
        ax.add_patch(Polygon(poly, facecolor=style.SEQ(norm(valeur)), edgecolor="white", linewidth=1))
    ax.scatter(pts[:, 0], pts[:, 1], s=6, color=style.ENCORE); ax.set(xlim=(0, 120), ylim=(0, 90),
↳ aspect="equal", xticks=[], yticks=[]); ax.grid(False)
    ax.set_title(titre, loc="left", fontsize=10); fig.colorbar(plt.cm.ScalarMappable(norm,
↳ style.SEQ), ax=ax, shrink=0.7)
0.saver(fig, "ch03-carte-choroplethe.png")
print("zones :", len(zones), "| ville la plus foncée, par habitant :", v["par_habitant"].idxmax(), "|
↳ en chiffre d'affaires :", v["ca"].idxmax())
```

figure : ch03-carte-choroplethe.png

zones : 20 | ville la plus foncée, par habitant : Ville C | en chiffre d'affaires : Ville A

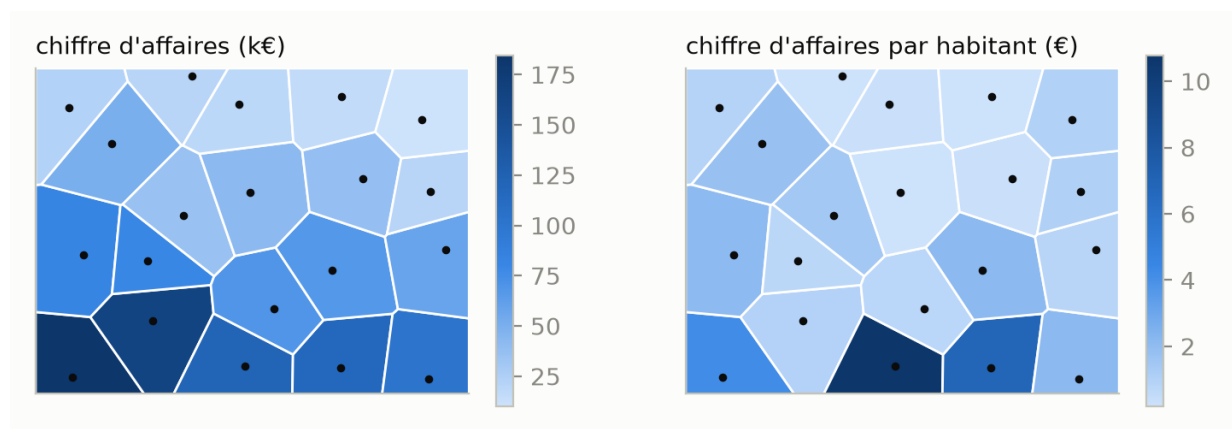


Figure 62: Deux cartes colorées du même plan fictif : à gauche le chiffre d'affaires, à droite le chiffre d'affaires par habitant.

Les deux cartes se ressemblent par endroits (les villes A, C, D et E sont foncées des deux côtés) et divergent pour les **grandes villes**. **Ville B**, deuxième en chiffre d'affaires, est pâle sur la carte de droite : 170 000 habitants qui rapportent environ 1 € chacun. À l'inverse, **Ville C**, avec 11 300 habitants, rapporte près de 11 € par habitant. Le lecteur de la carte de gauche voit en Ville B un marché majeur ; celui de la carte de droite, un marché à peine exploité. **Aucun n'a tort** : ils répondent à deux questions différentes (« où est le chiffre d'affaires ? » et « où la clientèle est-elle la plus dense ? »). La règle d'or : **une valeur absolue sur une carte reflète d'abord la population** ; pour mesurer une intensité, on **normalise** (par habitant, par client, par magasin).

3.4.4 Le biais des grandes zones

Il existe un second piège, **visuel** celui-là : l'œil juge l'importance d'une zone à sa **surface**, pas à sa valeur. Or la surface d'un territoire n'a presque aucun lien avec le nombre de personnes qui y vivent : sur notre plan, les villes les plus peuplées sont serrées, les moins peuplées ont de la place.

```
def aire(p):
    return 0.5 * abs(np.dot(p[:, 0], np.roll(p[:, 1], -1)) - np.dot(p[:, 1], np.roll(p[:, 0], -1)))
v["surface_zone"] = [aire(p) for p in zones]
gros = v.nlargest(5, "habitants")
print("corrélacion surface de la zone / habitants :", round(v["surface_zone"].corr(v["habitants"]),
    ↪ 2))
print("les 5 villes les plus peuplées :", round(gros["habitants"].sum() / v["habitants"].sum() *
    ↪ 100), "% des habitants,", round(gros["surface_zone"].sum() / v["surface_zone"].sum() * 100), "%
    ↪ de la carte")
```

```
corrélacion surface de la zone / habitants : 0.17
les 5 villes les plus peuplées : 54 % des habitants, 24 % de la carte
```

La corrélation est faible : la taille des zones ne suit pas la population. Les cinq villes les plus peuplées regroupent plus de la moitié des habitants, mais n'occupent qu'un quart de la carte : une carte colorée **sous-représente visuellement** les endroits où vit le plus de monde, et donne de l'importance à des territoires peu peuplés. Les remèdes sont connus : des **cercles proportionnels** (3.4.2), qui donnent à chaque lieu une taille liée à sa valeur ; des **cartes en carreaux** (une case de même taille par entité) ; et surtout **joindre un tableau ou des barres** à toute carte qui sert à comparer.

3.4.5 Une carte, ou un tableau ?

Si la question est « quelles sont les cinq premières villes ? », une carte est le **mauvais** outil : on ne classe pas des surfaces à l'œil. Un diagramme en barres horizontales, trié, répond mieux, en montrant en plus les valeurs.

```
tri = v.sort_values("ca")
top5 = list(tri.index[-5:])
fig, ax = plt.subplots(figsize=(6.2, 4.6))
ax.barh(tri.index, tri["ca"] / 1000, color=[style.BLEU if ville in top5 else style.MUET for ville in
    ↪ tri.index])
ax.set_xlabel="k€, 2025"; ax.grid(axis="y", visible=False); ax.set_axisbelow(True)
ax.set_title("Le classement est plus lisible en barres qu'en carte", loc="left", fontweight="bold",
    ↪ fontsize=10)
O.sauver(fig, "ch03-carte-ou-barres.png")
rang = v["ca"].rank(ascending=False); rang_h = v["par_habitant"].rank(ascending=False)
print("corrélacion de rang (chiffre d'affaires, par habitant) :", round(rang.corr(rang_h), 2), "|
    ↪ villes dans les cinq premières des deux classements :", sorted(set(rang[rang <= 5].index) &
    ↪ set(rang_h[rang_h <= 5].index)))
```

```
figure : ch03-carte-ou-barres.png
corrélacion de rang (chiffre d'affaires, par habitant) : 0.62 | villes dans les cinq premières des
    ↪ deux classements : ['Ville A', 'Ville C', 'Ville D', 'Ville E']
```

Les barres rendent le classement immédiat, avec les cinq premières villes en bleu. Elles ne disent pas **où** sont les villes : c'est le rôle de la carte. La combinaison gagnante est donc **la carte pour la position, les barres pour la valeur** : deux graphiques complémentaires plutôt qu'un graphique qui fait mal les deux.

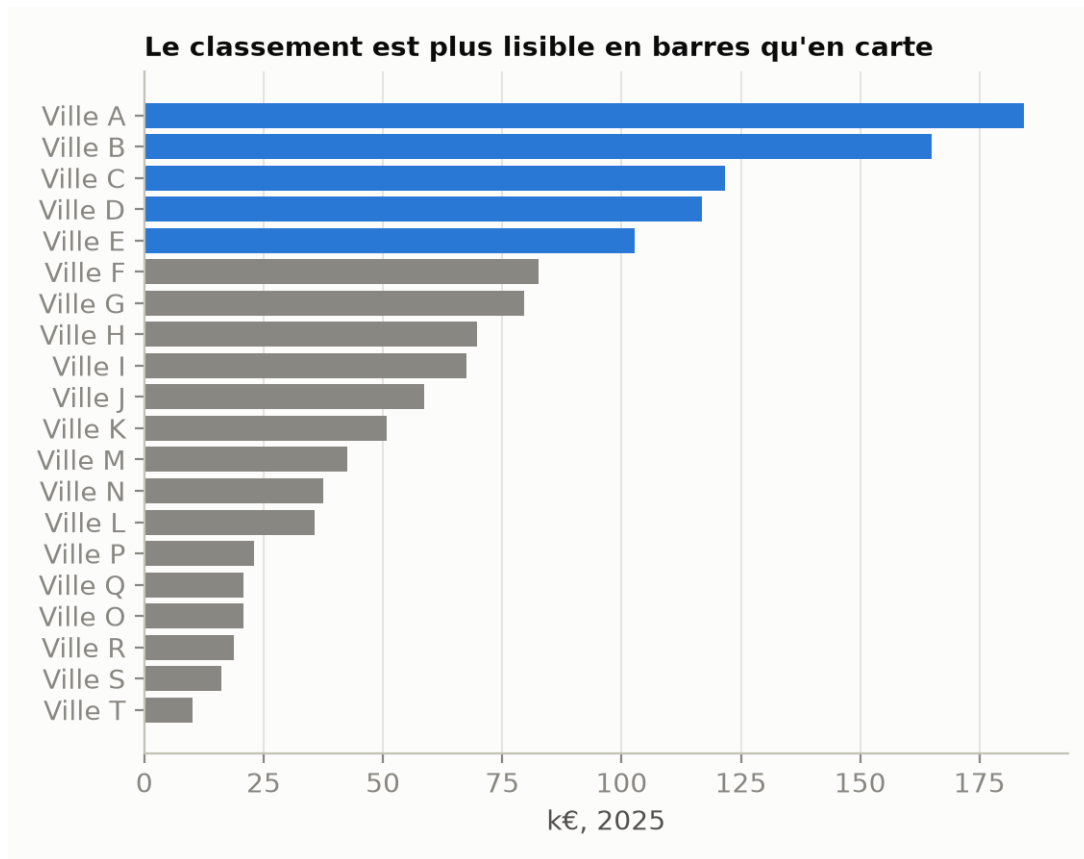


Figure 63: Le classement des villes en barres horizontales triées.

3.4.6 Projection : pourquoi une carte déforme

Représenter une surface courbe (la Terre) sur une feuille plate impose de **déformer** : aucune projection ne conserve à la fois les surfaces, les distances et les angles. Notre plan fictif est plat, donc exempt de ce problème ; avec de vraies coordonnées (latitude, longitude, exprimées en degrés), il faut le connaître. Un degré de longitude, par exemple, ne représente pas la même distance partout.

```
for lat in (0, 45, 60, 75):
    print(f"latitude {lat:2d}° : 1° de longitude vaut {111.32 * np.cos(np.radians(lat)):.1f} km | 1°
          ↳ de latitude vaut environ 111 km")
```

```
latitude 0° : 1° de longitude vaut 111.3 km | 1° de latitude vaut environ 111 km
latitude 45° : 1° de longitude vaut 78.7 km | 1° de latitude vaut environ 111 km
latitude 60° : 1° de longitude vaut 55.7 km | 1° de latitude vaut environ 111 km
latitude 75° : 1° de longitude vaut 28.8 km | 1° de latitude vaut environ 111 km
```

Un graphique tracé directement en degrés (longitude en abscisse, latitude en ordonnée) étire donc les régions éloignées de l'équateur. Les outils de cartographie gèrent les projections pour vous, mais il faut savoir qu'elles existent et **choisir** celle qui convient à la question (surfaces, distances ou angles).

3.4.7 Les outils réels : à décrire, non exécutés ici

Avec de vraies données géographiques, on utilise des bibliothèques spécialisées : **geopandas** (des tableaux pandas dont une colonne est une géométrie, avec jointures spatiales et projections) et **folium** (cartes interactives dans le navigateur, sur fonds de carte). Voici à quoi ressemble une carte choroplèthe réelle ; ce bloc est **non exécuté** ici (aucune donnée géographique n'est utilisée dans ce livre et aucun accès réseau n'est fait).

```
# non exécuté : nécessite geopandas et un fichier de contours administratifs (hypothétique)
import geopandas as gpd
zones = gpd.read_file("contours_regions.geojson")          # une ligne par région, une colonne
↳ « geometry »
carte = zones.merge(ventes_par_region, on="region")         # jointure attributaire
carte.plot(column="ca_par_habitant", cmap="Blues", legend=True, edgecolor="white")
```

Deux questions de **droits** se posent dès que l'on sort du plan fictif. Les **tuiles** (les images du fond de carte que les cartes interactives téléchargent) viennent de services tiers, avec des **conditions d'utilisation** et des mentions d'**attribution** obligatoires, et parfois des limites d'usage ou un coût ; il faut les lire. Les **contours administratifs** et les **bases d'adresses** ont aussi une licence, qui peut interdire la redistribution : on la vérifie avant de publier une carte. Enfin, une carte de **personnes** (clients, patients) peut révéler des informations personnelles : des points précis sur une carte sont des **adresses** ; on les **agrége** (par zone, avec des seuils de petits effectifs) avant de les montrer (volume II, chapitre 5).

À retenir. Une carte est utile quand la **position** compte ; on proportionne la **surface** (cercles), on **normalise** les valeurs absolues (par habitant), on se méfie du **biais des grandes zones**, et l'on joint des **barres** à toute carte qui doit servir à comparer. Les projections déforment, les fonds de carte ont des licences, et des points précis sur des personnes sont des données personnelles.

Pour s'entraîner. Cahier, chapitre 3 : application 3.9, exercices 3.15 et 3.16.

Bilan du chapitre 3

Vous savez maintenant :

- **décrire** un graphique par ses trois couches (données, esthétiques, géométrie) et nommer les objets de matplotlib (Figure, Axes, Axis, Artist) ;
- **transformer** un graphique par défaut en graphique qui porte un message en cinq retouches : ordre, étiquettes directes, suppression du superflu, couleur avec intention, titre qui dit le message ;
- **tracer** barres, courbes, histogrammes (échelle linéaire et logarithmique), nuages de points et **petits multiples**, avec ou sans échelle partagée ;
- **ranger** vos réglages dans un thème, **mesurer** le contraste d'une palette, **choisir** le format d'enregistrement ;
- **utiliser** seaborn (boîtes, cartes thermiques) et **lire** ses intervalles de confiance sans les prendre pour une dispersion ;
- **enfermer** un graphique dans une fonction et **tester** les données qu'il dessine ;
- **écrire** la même figure en ggplot2 et passer d'une syntaxe à l'autre ;
- **construire** des figures plotly interactives (survol, zoom, légende), savoir quand elles aident ou gênent, et les **photographier** pour un support fixe ;

- (en option) **écrire** le même tableau de bord avec Streamlit, Dash et Shiny, le **tester** sans navigateur et le **photographier** ;
- (en option) **tracer** des cartes en cercles proportionnels et en zones colorées, **normaliser** par habitant, reconnaître le biais des grandes zones et savoir quand préférer des barres.

Le fil conducteur du chapitre tient en une phrase : **un graphique est un programme que l'on peut rejouer, tester et relire**. Ce que l'on a gagné en code (reproductibilité, tests, thème) n'a de valeur que si les choix de conception du chapitre 1 y sont appliqués : le bon type de graphique, une mise en page claire, des couleurs accessibles, un titre qui dit le message.

Le chapitre 4 passe du graphique au **récit** : comment assembler des graphiques, des chiffres et du texte pour **raconter une analyse** et rédiger un rapport qui sera lu.

Pour s'entraîner. Cahier, chapitre 3 : applications 3.1 à 3.9 (barres, séries, thème, seaborn, test et ggplot2, plotly, Streamlit et Dash, Shiny, cartes) et exercices 3.1 à 3.16.

Chapitre 4 : Storytelling et rédaction de rapports

« Une analyse n'a de valeur que le jour où quelqu'un sait quoi en faire. »

Un mardi matin, la gérante repose sur votre bureau le rapport que vous lui aviez remis la semaine précédente : douze pages, dix-sept figures, un tableau de régression en annexe. Elle l'a lu. Elle sourit poliment, puis elle pose la question que tout analyste finit par entendre : « C'est très complet. **Et alors, qu'est-ce que je fais ?** »

Vous avez fait, aux chapitres précédents, le plus dur : poser la question, nettoyer les données, estimer un effet, mesurer son incertitude. Vous savez que les promotions ajoutent environ 19 % de commandes et que, malgré cela, elles font perdre de la marge. Mais ce savoir est **dans votre tête et dans votre notebook**. Tant qu'il n'est pas **dans la tête de la gérante**, dans une forme qui lui permette de **décider**, il n'a produit aucune valeur. C'est le sujet de ce chapitre : transformer une analyse en **récit** (4.1), en **rapport** (4.2), en **synthèse d'une page** (4.3) et en **rapport qui se fabrique tout seul** chaque semaine (4.4).

Intuition. Une analyse répond à une question ; un récit répond à une question **pour quelqu'un qui doit agir**. Le récit n'ajoute pas un chiffre : il **choisit, ordonne et conclut**. Il est donc une partie de l'analyse, pas son habillage.

Ce chapitre est surtout un chapitre d'**écriture**. Vous y verrez peu de code et beaucoup de textes : des paragraphes **avant** et **après** réécriture, commentés, parce que l'on apprend à écrire en comparant. Les chiffres cités viennent du calcul, comme partout dans ce livre : un récit honnête est un récit dont chaque nombre se retrouve.

Le chemin de ce chapitre

Le chapitre suit la question de la gérante, de l'analyse brute à l'automatisation d'un rapport.

- **4.1 Structurer un récit de données.** Un récit a un **arc** (contexte, tension, preuves, résolution), commence par la **réponse** (la pyramide de Minto) et porte **un message par figure et par titre**. Nous bâtissons le storyboard de l'analyse des promotions en cinq pages, nous décidons ce qui va en **annexe**, et nous voyons comment un récit peut devenir **malhonnête** (cerises cueillies, causalité sous-entendue).
- **4.2 Rédiger des rapports d'analyse clairs.** La **structure** d'un rapport (résumé, question, données, méthode, résultats, limites, recommandations, annexes), l'art d'écrire pour un **lecteur pressé**, les **chiffres** (arrondis, comparés, avec leur unité), les **tableaux** et les **légendes**, une **liste de relecture**, et la **reproductibilité** du rapport lui-même.

- **4.3 ⊕ Synthèses de direction, rapports d'une page, présentations.** Le résumé de cinq lignes, la page unique et la présentation de huit diapositives : ce qu'on garde et ce qu'on coupe.
- **4.4 ⊕ Rapports récurrents automatisés.** Un script qui fabrique chaque lundi le rapport de la semaine, avec des **contrôles avant envoi** et un texte généré **qui n'ose pas conclure quand les données ne le permettent pas**.

Les données du chapitre

Les données. Les fichiers de la boutique des volumes précédents, **simulés**, propres : `jours_exploitation.csv` (1 096 jours : commandes, chiffre d'affaires, météo, promotion, publicité), `commandes.csv`, `lignes_commande.csv`, `produits.csv` et `livraisons.csv`. L'analyse racontée est celle du **projet du volume III** : l'effet des promotions sur les commandes et sur la marge. Les nombres de ce chapitre sont recalculés ici par `build/outils_ch04.py`, et l'on connaît la **vérité programmée** : la promotion augmente les commandes de **18 %**.

Un mot sur ce que nous ne faisons pas : aucun logiciel de présentation ni de traitement de texte n'est exécuté dans ce chapitre. Les maquettes de pages et de diapositives sont **dessinées avec matplotlib** ; les outils (PowerPoint, Keynote, Word, LibreOffice Impress...) sont cités sans que leurs écrans soient reproduits.

Pour s'entraîner. Cahier, chapitre 4 : applications 4.1 à 4.6 et exercices 4.1 à 4.12 ; chacun renvoie à la section du livre qui l'éclaire.

4.1 Structurer un récit de données

Un récit de données n'est pas un conte : c'est une **suite d'affirmations ordonnées pour que le lecteur arrive à la bonne décision**, chacune appuyée par une preuve. Cette section donne la charpente (l'arc, la pyramide, le titre qui conclut), l'applique à l'analyse des promotions, puis montre où un récit cesse d'être honnête.

4.1.1 Le rapport qu'on a écrit, et celui qu'on devrait écrire

Le réflexe de l'analyste est d'écrire dans l'ordre où il a **travaillé** : les données, le nettoyage, l'exploration, le modèle, le résultat, et à la fin, peut-être, une recommandation. C'est l'ordre du **journal de bord**. Il est naturel, il est honnête, et il est presque inutilisable par un lecteur qui n'a pas participé au travail : il doit traverser dix pages avant d'apprendre ce qu'on lui veut.

L'ordre du **récit** est inverse : on part de ce dont le lecteur a besoin (une décision à prendre), on lui donne la réponse, puis seulement les raisons de la croire. Tout ce que vous avez fait n'a pas la même place : l'effort passé n'est pas une raison de le montrer.

	Journal de bord	Récit de données
Ordre	celui du travail	celui du besoin du lecteur
Début	les données	la réponse
Fin	le résultat, parfois la recommandation	la recommandation et la suite

	Journal de bord	Récit de données
Contenu	tout ce qui a été fait	ce qui prouve la réponse, le reste en annexe
Lecteur idéal	un collègue qui refait	une personne qui décide

Intuition. Le journal de bord s'adresse à **celui qui vérifie** ; le récit, à **celui qui décide**. Il faut les deux, mais pas dans le même document ni dans le même ordre : le journal va dans les annexes, le notebook et le dépôt de code.

4.1.2 L'arc : contexte, tension, preuves, résolution

Les récits efficaces, du roman à la note de service, ont la même ossature en quatre temps.

1. **Le contexte** : ce que le lecteur sait déjà et partage. « La boutique fait trois promotions par an. »
2. **La tension** : la question qui gêne, l'écart entre ce que l'on croit et ce qui est. « Les ventes montent pendant les promotions : mais gagne-t-on de l'argent ? »
3. **Les preuves** : les chiffres qui tranchent, dans l'ordre où ils lèvent les doutes du lecteur.
4. **La résolution** : ce que l'on fait, par qui, pour quand.

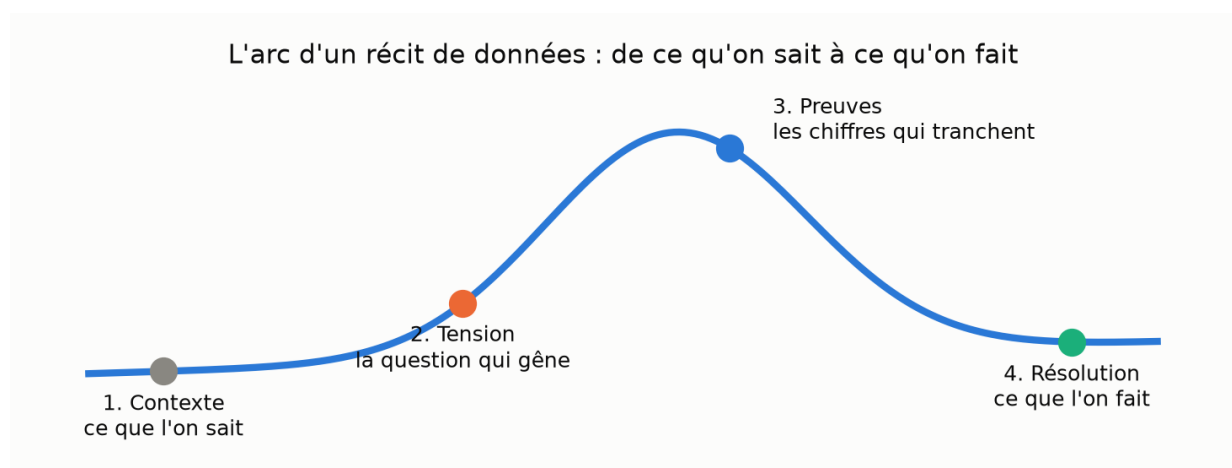


Figure 64: Les quatre temps d'un récit de données : on part de ce que l'on sait, on pose la question qui dérange, on apporte les preuves et l'on termine par l'action.

Dans une analyse, la tension est presque toujours la même : **le chiffre évident est trompeur**. Si le chiffre évident répondait à la question, la gérante n'aurait pas besoin de vous. Dans notre exemple, le chiffre évident est la comparaison brute (on vend plus les jours de promotion), et le travail de l'analyse consiste à montrer ce qui se cache derrière.

Piège. Un récit sans tension est un **inventaire** (« voici nos chiffres »). Un récit dont la tension est factice (« catastrophe ! ») est du **spectacle**. La bonne tension est celle qui existe déjà dans la tête du lecteur : une question qu'il se pose, ou qu'il devrait se poser.

4.1.3 Commencer par la réponse : la pyramide

Le principe, popularisé sous le nom de **pyramide de Minto**, est simple : on écrit **la réponse d'abord**, puis les deux ou trois arguments qui la soutiennent, puis les preuves de chaque argument. Le lecteur peut s'arrêter à n'importe quel niveau et en savoir assez pour son niveau de détail.

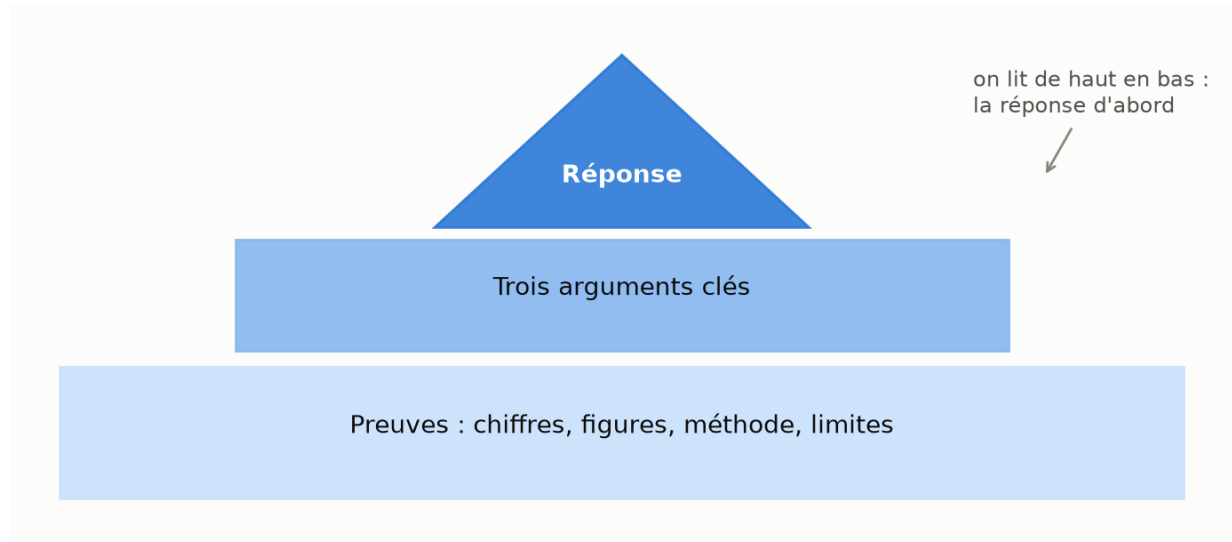


Figure 65: La pyramide : en haut la réponse, au milieu trois arguments, en bas les preuves. Chaque niveau résume celui du dessous.

Voici la même analyse racontée de deux façons. D'abord dans l'ordre du travail.

« Nous avons d'abord exploré les données de 2023 à 2025, puis nous avons constaté que les jours de promotion comptent en moyenne 7,8 % de commandes en plus. Nous avons ensuite construit une régression avec des effets de mois et de jour de semaine, une tendance, la pluie et la publicité, avec des erreurs robustes à l'autocorrélation. Le coefficient de la promotion correspond à un effet de 19,2 %, avec un intervalle de confiance de 13,5 à 25,1 %. Nous avons enfin calculé la marge contrefactuelle. Il en ressort que la marge est inférieure de 17 884 € à ce qu'elle aurait été sans promotion. »

Puis dans l'ordre de la pyramide.

« Les promotions font perdre de l'argent : environ 18 000 € de marge sur les 153 jours de promotion. Elles ajoutent bien des commandes (environ 19 %), mais chaque commande rapporte 24 € au lieu de 32 € et le gain de volume ne compense pas les remises. Il faudrait 36 % de commandes en plus pour s'en sortir. Nous recommandons de baisser la remise et de tester la prochaine promotion sur une partie des jours. »

Les deux textes disent la même chose, mais pas au même lecteur. Mesurons-le, sans prétendre que ces mesures remplacent le jugement.

```
t = pd.DataFrame({"journal de bord": 0.lisibilite(avant), "pyramide": 0.lisibilite(apres)})
print(t.to_string())
print("la réponse arrive au mot n° :", avant.split().index("inférieure") + 1, "contre",
      ↪ apres.split().index("perdre") + 1)
```

	journal de bord	pyramide
phrases	5.0	4.0
mots	105.0	70.0
mots_par_phrase	21.0	17.5

```
termes_techniques      5.0      0.0
chiffres                8.0      7.0
la réponse arrive au mot n° : 94 contre 4
```

Le second texte est plus court, ne contient aucun terme technique, et donne la réponse dès le **quatrième mot**, au lieu du quatre-vingt-quatorzième, et il est un tiers plus court. Ce n'est pas une affaire de style : c'est l'ordre dans lequel la gérante a besoin des informations.

En pratique : le test de l'ascenseur. Vous croisez la gérante dans l'ascenseur, vous avez trente secondes. Qu'est-ce que vous dites ? La réponse, la raison principale, ce que vous proposez. Le reste, vous le garderez pour la réunion.

Le texte de la pyramide cite des chiffres : vérifions qu'aucun n'est tapé de mémoire. La section 4.2.6 construira l'outil qui le fait systématiquement ; en voici l'esprit.

```
permis = [a["inc"] / -1000, a["e"] * 100, a["mo_np"], a["mo_p"], a["seuil"] * 100, a["jours"]]
print("nombres sans source :", 0.verifier_nombres(apres, permis))
```

```
nombres sans source : [18000.0]
```

4.1.4 Un message par figure, un titre qui conclut

Une figure bien faite, avec un titre qui décrit (« Chiffre d'affaires par mois en 2025 »), oblige le lecteur à **chercher** le message. Un titre qui **conclut** le lui donne ; la figure sert alors de preuve, et non d'énigme.

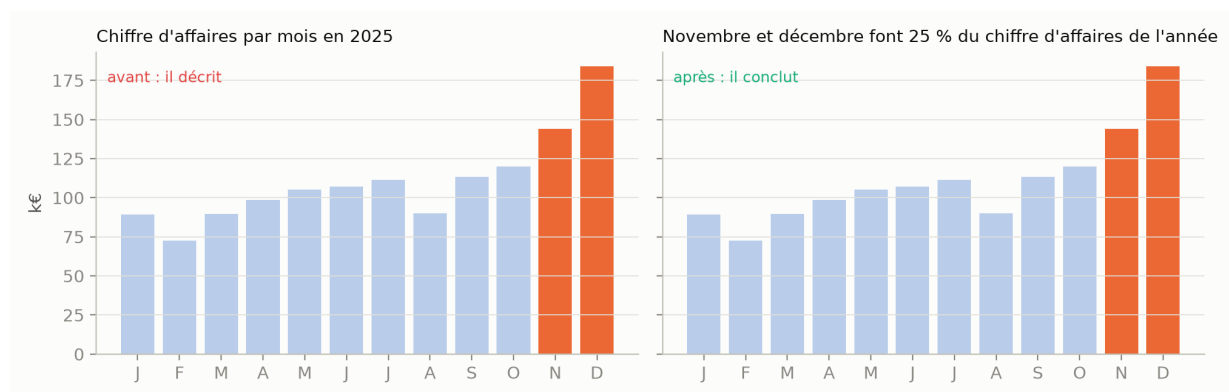


Figure 66: Le même graphique, avec deux titres. À gauche, le titre décrit ce que l'on voit ; à droite, il énonce la conclusion que la figure doit permettre de vérifier. La part de novembre et décembre est calculée sur les données.

Trois règles en découlent.

1. **Un message par figure.** Si vous avez besoin de deux phrases pour dire ce que la figure montre, c'est qu'il faut deux figures.
2. **Le titre est une phrase complète avec un verbe**, qui énonce le message et pas la variable. « Les remises mangent le gain de volume » plutôt que « Marge selon la promotion ».
3. **La figure doit prouver le titre sans aide.** Couleur et annotations guident l'œil vers ce qui compte (en orange sur la figure : les deux mois du titre), et tout le reste s'efface.

Cette dernière règle est une règle de **conception** : elle est détaillée au chapitre 1 (choisir et construire un graphique) et au chapitre 3 (le faire avec matplotlib et plotly). Ici, retenons seulement que **le titre est le premier texte du récit**, et souvent le seul que le lecteur lit.

Intuition. Lisez uniquement les titres de vos figures, dans l'ordre. S'ils forment une histoire cohérente qui conduit à la recommandation, le récit tient. S'ils ressemblent à une liste de noms de variables, il manque un récit.

4.1.5 L'analyse des promotions racontée en quatre figures

Appliquons tout cela à l'analyse du volume III. Le point de départ : la gérante demande si les promotions de la boutique « marchent ». La réponse tient en quatre figures, chacune avec un seul message. Le calcul est celui du volume précédent, regroupé dans la fonction `analyse_promo` (régression des commandes avec saison, jour de semaine, tendance, pluie et publicité, erreurs robustes ; marge contrefactuelle ; seuil de bascule).

Première figure : le chiffre évident

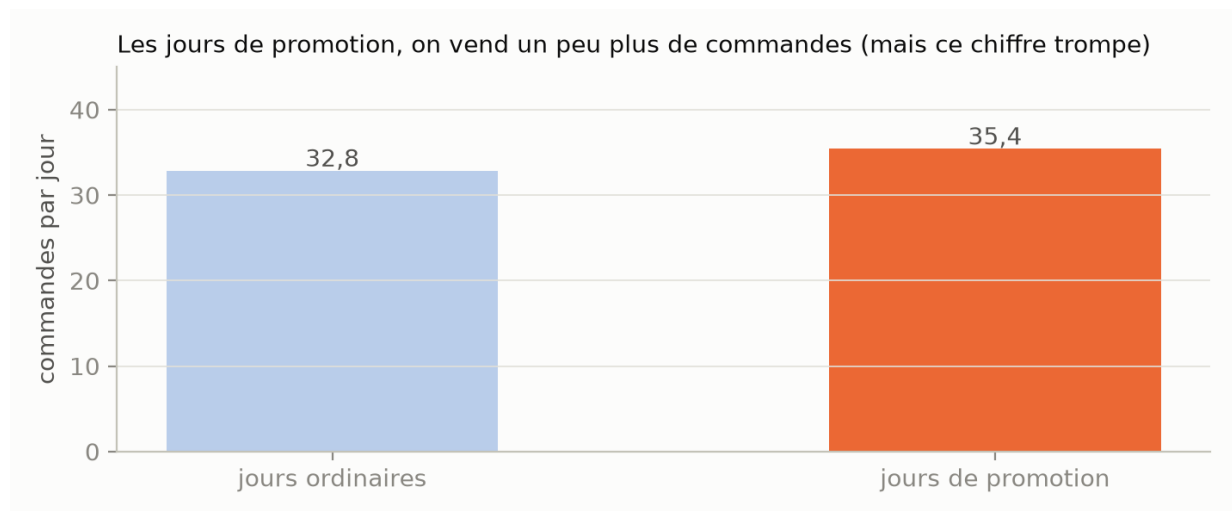


Figure 67: Le chiffre évident : les jours de promotion, la boutique prend un peu plus de commandes par jour que les autres jours.

C'est la **tension** : en moyenne, +7,8 % de commandes les jours de promotion, mais le chiffre d'affaires par jour **baisse** légèrement (−1,2 %). Voilà un premier doute pour la gérante : « on vend plus et on encaisse moins ? ». Ce chiffre trompe pour une raison que vous connaissez : les promotions tombent à des **saisons** particulières (janvier, juillet, fin novembre), et la comparaison brute mélange l'effet de la promotion et celui de la période.

```
print("commandes par jour : +", round(a["brut_cmd"] * 100, 1), "% | chiffre d'affaires par jour :",
      round(a["brut_ca"] * 100, 1), "%")
```

```
commandes par jour : + 7.8 % | chiffre d'affaires par jour : -1.2 %
```

Deuxième figure : l'effet réel

À saison égale, la promotion ajoute environ **19 %** de commandes (intervalle de confiance de 14 à 25 %). L'effet est **plus fort** que le chiffre évident. Le récit rassure d'abord : oui, les promotions fonctionnent pour attirer des commandes. On place cette bonne nouvelle **avant** la mauvaise : c'est plus honnête, et plus convaincant.

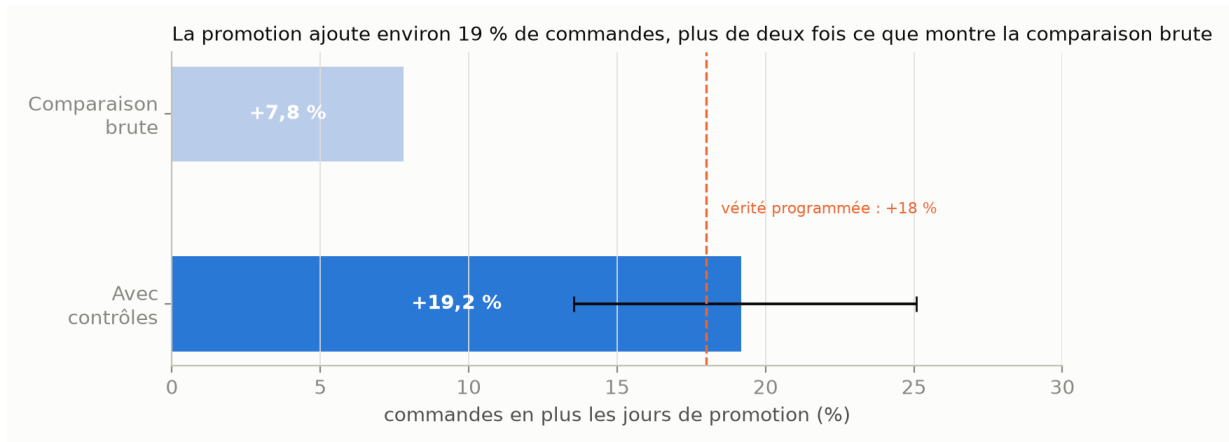


Figure 68: L'effet réel des promotions sur les commandes, une fois la saison, le jour de la semaine, la tendance, la pluie et la publicité pris en compte, avec son intervalle de confiance.

Troisième figure : le coût

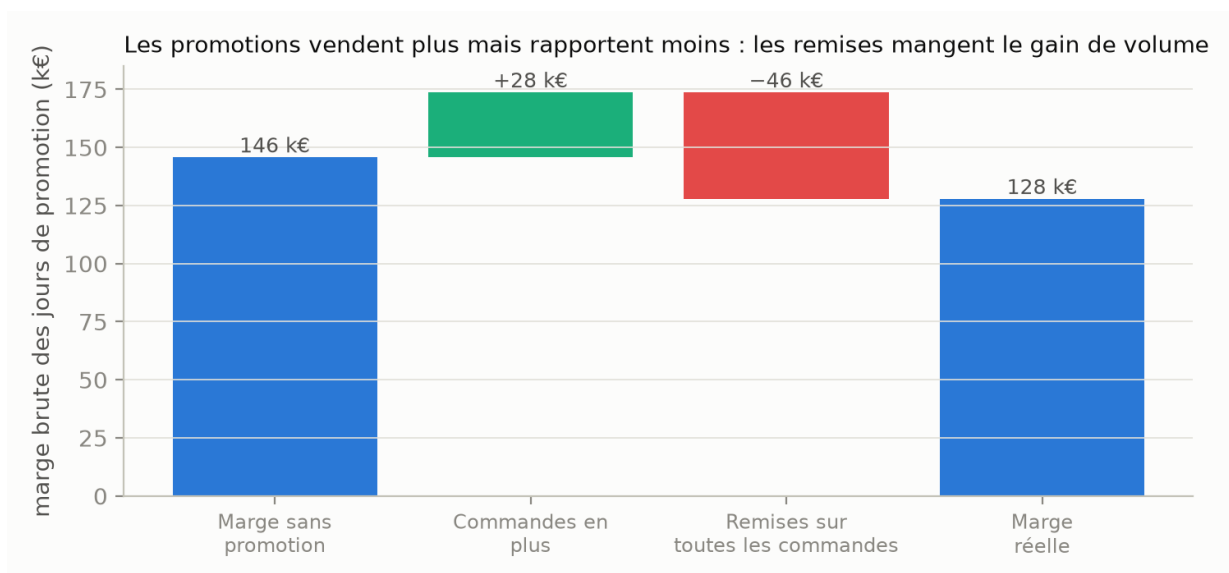


Figure 69: La marge des 153 jours de promotion. Sans promotion, elle aurait été de 146 k€ ; les commandes en plus apportent 28 k€, mais les remises accordées à toutes les commandes en retirent 46 k€.

Chaque commande vendue hors promotion rapporte 32 € de marge brute ; en promotion, 24 €. Les commandes en plus ne compensent pas la remise accordée à **toutes** les commandes, y compris celles qui auraient eu lieu de toute façon. La cascade ci-dessus le décompose, et l'on vérifie que ses morceaux se somment bien.

```
sans, gain, remise = casc["sans"], casc["gain"], casc["remise"]
print("marge sans promotion :", 0.fr(sans, 0), "€ | commandes en plus :", 0.fr(gain, 0, True), "€ |
↳ remises :", 0.fr(remise, 0, True), "€")
print("somme :", 0.fr(sans + gain + remise, 0), "€ = marge réelle :", 0.fr(a["marge_reelle"], 0), "€ |
↳ incrément :", 0.fr(a["inc"], 0), "€")
```

```
marge sans promotion : 145 861 € | commandes en plus : +27 969 € | remises : -45 854 €
somme : 127 977 € = marge réelle : 127 977 € | incrément : -17 884 €
```

Quatrième figure : le seuil

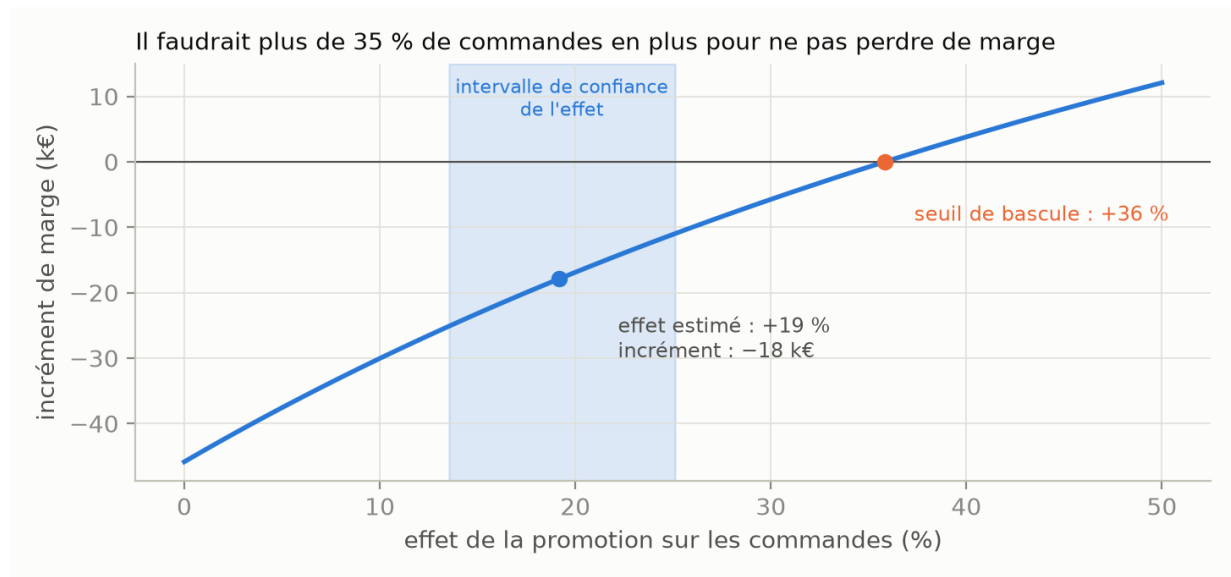


Figure 70: Incrément de marge des promotions selon l'effet réel sur les commandes. La courbe croise zéro au seuil de bascule, bien au-delà de l'intervalle de confiance de l'effet estimé.

Un lecteur sceptique demande : « et si l'effet était plus grand que vous ne dites ? » La quatrième figure répond à l'avance : même à la borne haute de l'intervalle (25 %), la marge est négative ; il faudrait **plus de 35 % de commandes en plus** pour que les promotions rapportent. C'est un argument plus fort qu'une estimation ponctuelle, parce qu'il survit à l'incertitude.

```
print("incrément de marge : borne haute de l'effet", 0.fr(a["inc_bas"], 0), "€ | estimation",
      0.fr(a["inc"], 0), "€ | borne basse", 0.fr(a["inc_haut"], 0), "€")
print("seuil de bascule : +", 0.fr(a["seuil"] * 100, 1), "% de commandes")
```

```
incrément de marge : borne haute de l'effet -10 986 € | estimation -17 884 € | borne basse -25 125 €
seuil de bascule : + 35,8 % de commandes
```

Ces quatre figures ne sont pas encore un récit : il manque la **résolution**. La cinquième page est la recommandation, et elle est aussi dans le résumé en tête de rapport.

À retenir. Un bon récit de données contient souvent la **bonne nouvelle avant la mauvaise, le chiffre évident avant le chiffre vrai, et l'objection du lecteur avant qu'il la formule**. Chaque figure répond à la question que la précédente a fait naître.

4.1.6 Le storyboard : décider avant de dessiner

Avant de produire la moindre figure définitive, on dessine le récit sur une feuille : **une case par page, un titre-conclusion par case, la preuve qu'on y mettra**. C'est le **storyboard**, emprunté au cinéma. Il est plus rapide à modifier qu'un rapport fini, et il permet de se faire relire (« est-ce que cette suite de messages convainc ? ») avant d'avoir investi des heures.

Écrit sous forme de tableau, il devient une liste de contrôle du récit.

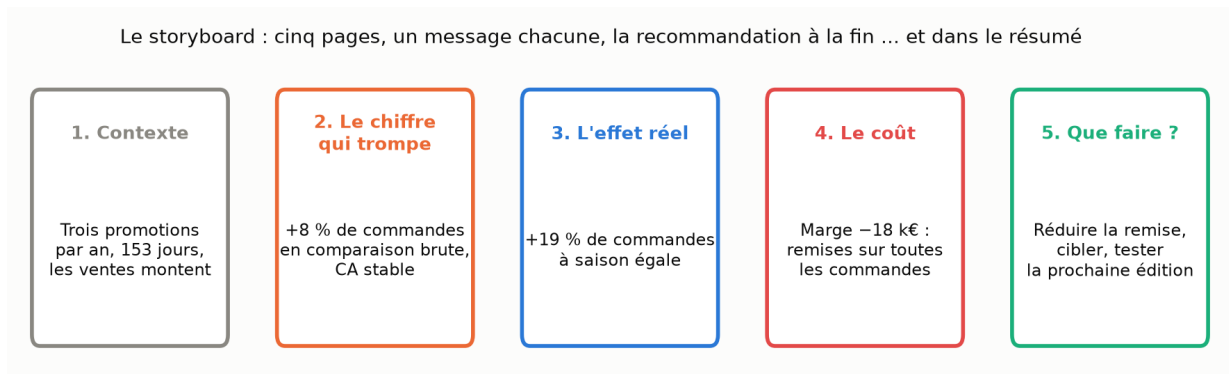


Figure 71: Le storyboard de l'analyse des promotions : cinq pages, un message par page, la recommandation en dernière.

Page	Titre-conclusion	Preuve	Question du lecteur à laquelle elle répond
1	Trois promotions par an, 153 jours, et les ventes montent	calendrier des promotions	« De quoi parle-t-on ? »
2	En comparaison brute, +8 % de commandes mais un chiffre d'affaires stable	figure 1	« Pourquoi se poser la question ? »
3	À saison égale, +19 % de commandes	figure 2 et son intervalle	« Les promotions marchent-elles ? »
4	Mais chaque commande rapporte 8 € de moins : -18 k€ de marge	figure 3	« Et l'argent ? »
5	Réduire la remise, cibler, tester	seuil (figure 4) et plan de mesure	« Que fait-on ? »

Ce qui va dans le récit, ce qui va en annexe

Vous avez fait beaucoup plus de calculs que le récit n'en montre : c'est normal et souhaitable. Pour trier, on se pose, pour chaque résultat, la question **« si je l'enlève, le lecteur change-t-il de conclusion ou de confiance ? »**.

Résultat de l'analyse	Dans le récit ?	Pourquoi
Effet de la promotion et son intervalle	oui	c'est la réponse
Marge par commande avec et sans promotion	oui	c'est l'argument
Seuil de bascule	oui	il répond à l'objection
Le choix des variables de contrôle	annexe	utile pour qui refait
Les diagnostics du modèle (résidus, autocorrélation)	annexe	donne confiance, ne donne pas le message
Les autres modèles essayés	annexe, brièvement	l'honnêteté demande de les mentionner
Le détail du nettoyage des données	annexe ou dépôt	déjà traité (volume II)

Résultat de l'analyse	Dans le récit ?	Pourquoi
Une corrélation intéressante mais hors sujet	non	elle détourne du message

Piège. « L'annexe » ne doit pas devenir **l'endroit où l'on cache ce qui gêne**. Un résultat qui affaiblit la conclusion (un modèle alternatif qui donne un effet différent) n'est pas à reléguer en annexe : il se dit dans les limites. L'annexe contient du détail, pas des surprises.

4.1.7 Quand un récit devient malhonnête

Un récit **choisit**, et c'est précisément ce qui le rend dangereux : chaque choix peut orienter le lecteur à son insu. Quatre dérives reviennent sans cesse.

Les cerises cueillies

On choisit la fenêtre de temps, le segment ou l'indicateur qui donne la meilleure image. Voici comment « on pourrait » écrire que les commandes explosent.

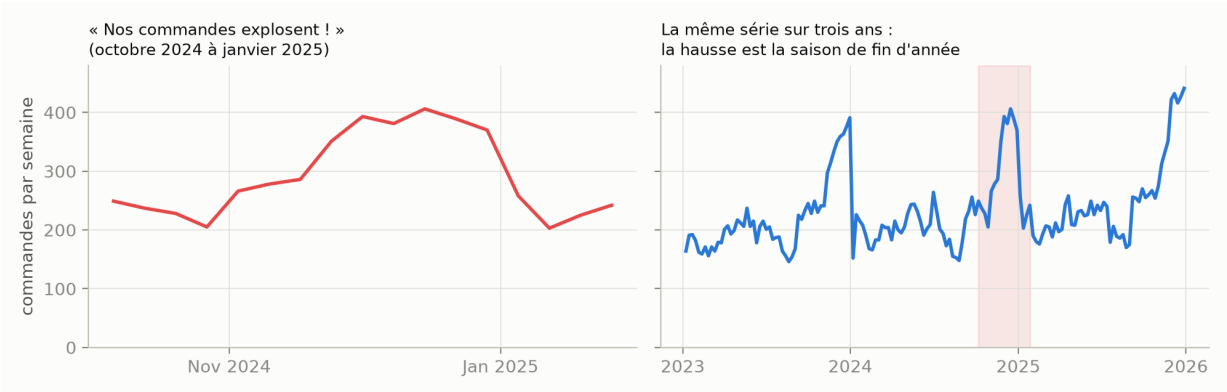


Figure 72: À gauche, une fenêtre bien choisie, qui commence en octobre : les commandes semblent exploser. À droite, la même série sur trois ans : la hausse de fin d'année revient chaque année.

```
for an in (2023, 2024, 2025):
    print(an, " : commandes par semaine, octobre", round(c[an]["octobre"]), "→ fin novembre-décembre",
          ↪ round(c[an]["decembre"]), "(", 0.fr(c[an]["hausse"] * 100, 0, True), "%)")
print("décembre 2024 contre décembre 2023 :", 0.fr(c["dec_sur_dec_2024"] * 100, 1, True), "%")

2023 : commandes par semaine, octobre 238 → fin novembre-décembre 351 ( +48 %)
2024 : commandes par semaine, octobre 230 → fin novembre-décembre 392 ( +71 %)
2025 : commandes par semaine, octobre 259 → fin novembre-décembre 424 ( +64 %)
décembre 2024 contre décembre 2023 : +11,7 %
```

Entre octobre et décembre 2024, les commandes hebdomadaires augmentent de 71 % : un titre alarmiste (« +71 % de commandes en deux mois ! ») serait **exact**. Mais c'est la **saison** : la même hausse, plus ou moins forte, revient chaque année, et la vraie comparaison (décembre 2024 contre décembre 2023) donne **+12 %**. La cerise cueillie n'est pas un mensonge sur les chiffres ; c'est un mensonge par **choix de la comparaison**.

En pratique : l'antidote. Avant d'écrire une variation, demandez-vous : « à quoi la compare-t-on, et qui a choisi ? » Comparer au **même moment de l'an dernier**, ou à une période fixée **avant** de voir les données, évite de se tromper soi-même.

La causalité sous-entendue

« Depuis le lancement de la nouvelle page d'accueil, la conversion a augmenté de 12 %. » La phrase ne dit pas que la page est la cause ; le lecteur le comprend tout seul. Pour garder la maîtrise de ce que le lecteur comprend, on choisit ses verbes avec une échelle.

Ce que vous avez établi	Verbes et tournures honnêtes	À éviter
Deux choses varient ensemble	« va de pair avec », « est associé à »	« a provoqué », « grâce à »
Une différence qui persiste à situation égale (régression)	« on observe, à saison égale, ... »	« prouve que »
Une explication plausible, non testée	« est probablement due à », « cause probable »	« est due à »
Une expérience aléatoire (test A/B)	« a causé », « a augmenté de ... (intervalle) »	(on peut l'affirmer)

Notre analyse des promotions est une **régression avec contrôles** : elle autorise « à saison égale, les jours de promotion comptent environ 19 % de commandes de plus », et elle n'autorise pas « la promotion cause 19 % de commandes en plus » sans réserve. Le volume III (chapitres 2 et 3) a détaillé pourquoi.

L'incertitude retirée

Un intervalle de confiance gêne un titre percutant. La tentation est de ne garder que l'estimation (« +19 % »). C'est une simplification acceptable si l'intervalle est dit quelque part et si la conclusion ne change pas en l'utilisant. Dans notre cas, elle ne change pas : tout l'intervalle donne une marge négative (figure 4). Mais si la conclusion changeait selon la borne, taire l'intervalle serait une faute.

Le graphique qui exagère

Axe tronqué, échelles différentes entre deux graphiques côte à côte, aire dont la surface ne correspond pas à la valeur : ce sont les erreurs de **conception** du chapitre 1. Dans un récit, elles ont un effet particulier : le lecteur les remarque rarement, et il se souvient de l'impression.

Piège : la règle d'honnêteté. Posez-vous cette question avant d'envoyer : « **si la gérante voyait tous les calculs que j'ai faits, serait-elle d'accord avec ma phrase ?** » Si non, la phrase est à corriger, pas le lecteur à convaincre.

À retenir. Un bon récit **simplifie sans fausser** : il choisit ce qu'il montre, mais il ne choisit pas ce qui est vrai. Les compromis (annexe, limites, intervalles) servent justement à rendre ces choix visibles.

Pour s'entraîner. Cahier, chapitre 4 : applications 4.1 et 4.2, exercices 4.1 à 4.4.

4.2 Rédiger des rapports d'analyse clairs

Le récit fixe l'ordre des messages ; le rapport est le **document** qui les porte. Écrire un rapport clair est un métier à part entière : cette section en donne la structure, les règles d'écriture pour un lecteur pressé, celles des chiffres et des tableaux, une liste de relecture, et la façon de rendre le rapport **reproductible**.

4.2.1 La structure d'un rapport d'analyse

Un rapport d'analyse a huit parties, dans un ordre qui sert deux lecteurs à la fois : le décideur, qui lit le début, et le collègue qui vérifie, qui lit le milieu et la fin.

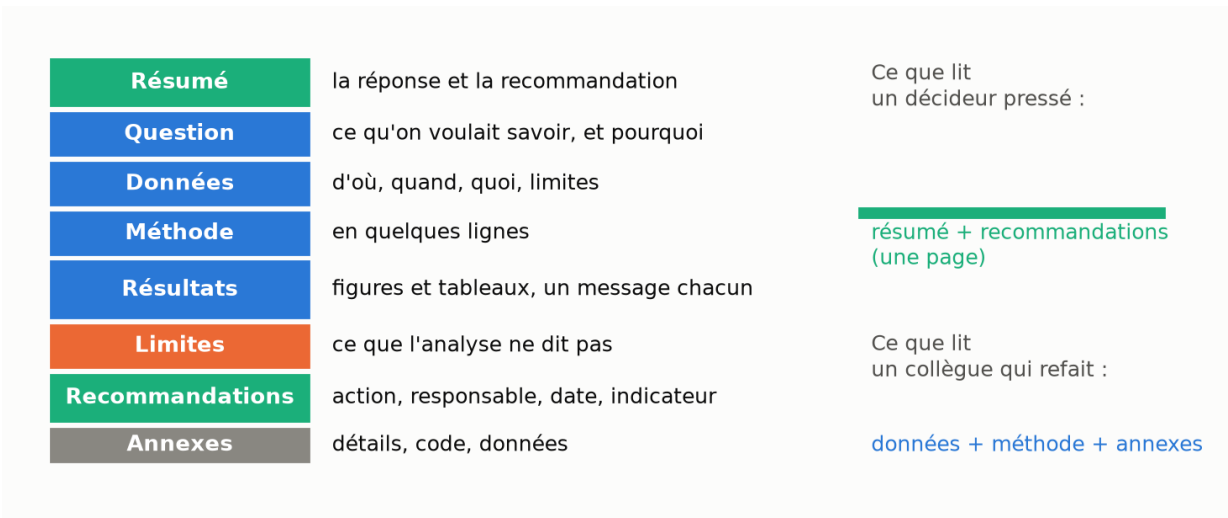


Figure 73: Les huit parties d'un rapport d'analyse. Le décideur lit le résumé et les recommandations (en vert) ; le collègue qui refait lit les données, la méthode et les annexes (en bleu).

Partie	Rôle	Longueur typique	Contenu
Résumé	la réponse et la recommandation	5 à 8 lignes	ce que l'on a trouvé, ce que l'on propose ; écrit en dernier
Question	pourquoi cette analyse	3 à 5 lignes	la décision à prendre, la question précise, le périmètre
Données	sur quoi on s'appuie	une demi-page	sources, période, volume, limites connues
Méthode	comment on a répondu	une demi-page	en langage simple ; le détail technique va en annexe
Résultats	ce que l'on a trouvé	1 à 3 pages	figures et tableaux, un message chacun (section 4.1)
Limites	ce que l'analyse ne dit pas	un quart de page	hypothèses, biais possibles, ce qui n'est pas mesuré
Recommandations	ce qu'on fait	une demi-page	action, responsable, échéance, indicateur de suivi

Partie	Rôle	Longueur typique	Contenu
Annexes	le détail pour qui vérifie	sans limite	code, diagnostics, tableaux complets, dictionnaire des variables

Deux conseils sur la structure.

1. **Le résumé s'écrit en dernier**, quand on sait ce que dit le rapport, mais il se **lit en premier**. Il doit pouvoir être lu seul, copié dans un courriel, et rester compréhensible.
2. **Les recommandations ne se cachent pas dans les résultats**. Un résultat décrit ce que l'on observe ; une recommandation dit **qui fait quoi**. Les mélanger, c'est laisser le lecteur deviner l'action.

Intuition. Le rapport est un **entonnoir d'attention** : la première page est lue par tous, la troisième par la moitié, les annexes par une personne. Placez l'information selon le nombre de personnes qui en ont besoin.

4.2.2 Écrire pour un lecteur pressé

Votre lecteur lit entre deux réunions, sur un écran, et ne connaît pas vos méthodes. Il n'a pas le temps de deviner. Voici les règles qui comptent le plus, et leur effet.

Règle	Avant	Après
Phrases courtes , une idée chacune	« Il ressort de l'analyse, qui a porté sur trois années de données, et compte tenu des variables de contrôle retenues, qu'un effet positif est observé. »	« Sur trois ans, les promotions ajoutent environ 19 % de commandes. »
Verbes actifs , sujet clair	« Une baisse de la marge est constatée. »	« Les promotions réduisent la marge de 18 000 €. »
Un terme pour une chose	« effet », « impact », « incidence », « coefficient » pour la même quantité	« effet » partout
Le jargon défini une fois ou évité	« Le coefficient de la régression log-linéaire est significatif. »	« À saison égale, les jours de promotion comptent 19 % de commandes de plus. »
Le chiffre avec sa comparaison	« La marge est de 24 €. »	« La marge est de 24 € par commande en promotion, contre 32 € sinon. »
La conclusion avant la preuve	« Nous avons calculé... Il en résulte que... »	« Les promotions font perdre de l'argent. Voici pourquoi. »

On peut repérer mécaniquement les phrases trop longues : au-delà d'environ 25 mots, une phrase demande en général à être coupée. Le premier texte de la section 4.1.3 en contient.

```
for p in 0.phrases(avant):
    n = len(p.split())
    if n > 25:
        print(n, "mots :", p[:70], "...")
```

28 mots : Nous avons d'abord exploré les données de 2023 à 2025, puis nous avons ...

29 mots : Nous avons ensuite construit une régression avec des effets de mois et ...

Ce contrôle ne remplace pas la relecture, mais il est objectif et rapide. Les mesures de lisibilité de la section 4.1.3 (mots par phrase, termes techniques, chiffres) servent de même : elles disent **où regarder**, pas si le texte est bon.

Le jargon : qui est le lecteur ?

Un terme technique n'est pas mauvais en soi : il est **mauvais pour ce lecteur-là**. « Intervalle de confiance » est un mot de travail pour un analyste, et du bruit pour la gérante. La règle : **traduire pour la direction, garder le terme exact pour le collègue** (annexe, notes). Pour la même idée :

Collègue analyste	Gérante
« Effet estimé de 19,2 % (IC à 95 % : 13,5 à 25,1 %) »	« Environ 19 % de commandes en plus ; l'estimation est sûre à quelques points près : entre 14 et 25 %. »
« L'effet est significatif au seuil de 5 % »	« Il est très improbable que l'effet soit nul. »
« Estimation robuste à l'autocorrélation »	(non mentionné ; en annexe)

Piège : la fausse simplicité. Simplifier ne veut pas dire affirmer ce qu'on ne sait pas. « Entre 14 et 25 % » est tout aussi simple que « 19 % exactement », et bien plus honnête. Le jargon se traduit, l'incertitude non.

4.2.3 Les chiffres : arrondir, comparer, nommer

Un chiffre mal écrit fait perdre au lecteur du temps et de la confiance. Cinq habitudes règlent l'essentiel.

1. **Arrondir à la précision que l'on connaît.** Un effet estimé à 19,17538 % avec un intervalle de ± 5 points se dit « environ 19 % ». Les décimales supplémentaires sont du **faux savoir**, et elles font croire à une exactitude qui n'existe pas.
2. **Écrire l'unité** et ne pas la changer en route (€, k€, %, points).
3. **Donner une comparaison** : sans référence (l'an dernier, hors promotion, le budget), un chiffre ne dit pas s'il est bon ou mauvais.
4. **Distinguer pourcentage et points de pourcentage.** Passer de 80,9 % à 77,7 % de livraisons à l'heure, c'est une baisse de **3,2 points**, soit 4 % en valeur relative : les deux se disent, mais pas l'un pour l'autre.
5. **Utiliser les conventions françaises** : espace insécable fine entre les milliers (17 884), virgule décimale (19,2), signe moins typographique (−18).

Une fonction suffit à arrondir à un nombre de **chiffres significatifs** choisi.

```
def sig(x, n=2):
    return round(x, n - 1 - int(np.floor(np.log10(abs(x)))))

print("marge perdue :", 0.fr(a["inc"], 0), "→", 0.fr(sig(a["inc"]), 0), "€ | avec la publicité :",
      0.fr(a["inc_pub"], 0), "→", 0.fr(sig(a["inc_pub"]), 0), "€")
print("effet :", 0.fr(a["e"] * 100, 2), "→", 0.fr(sig(a["e"] * 100), 0), "% | bornes :",
      0.fr(a["e_bas"] * 100, 1), "à", 0.fr(a["e_haut"] * 100, 1), "→", 0.fr(sig(a["e_bas"] * 100), 0),
      "à", 0.fr(sig(a["e_haut"] * 100), 0), "%")
```

marge perdue : -17 884 → -18 000 € | avec la publicité : -25 016 → -25 000 €
 effet : 19,18 → 19 % | bornes : 13,5 à 25,1 → 14 à 25 %

Pour un nombre à présenter à la gérante, « 18 000 € » vaut mieux que « 17 884 € » : le second suggère une exactitude que le modèle n'a pas. L'inverse est vrai dans un **tableau d'annexe**, où le nombre exact permet de vérifier. On adapte donc la précision **au rôle du chiffre**.

Absolu et relatif, ensemble

Un pourcentage seul peut cacher une bagatelle (+50 % de quelque chose de minuscule), un montant seul peut cacher une proportion (18 000 €, est-ce beaucoup ?). On donne les deux.

```
ecart = a["mo_p"] - a["mo_np"]
print("marge par commande :", 0.fr(a["mo_np"], 1), "€ hors promotion,", 0.fr(a["mo_p"], 1), "€ en
↳ promotion")
print("écart :", 0.fr(ecart, 1, True), "€ par commande, soit", 0.fr(ecart / a["mo_np"] * 100, 0,
↳ True), "%")
```

marge par commande : 32,1 € hors promotion, 23,6 € en promotion
 écart : -8,5 € par commande, soit -26 %

La phrase qui en résulte tient en une ligne : « chaque commande rapporte 8 € de moins en promotion, soit 26 % de moins ».

4.2.4 Tableaux, figures et légendes

Un tableau, comme une figure, porte **un message**. S'il en porte trois, c'est qu'il faut le couper. Quelques règles de lecture facile :

- **moins de sept lignes** dans le corps du rapport (le tableau complet va en annexe) ;
- **colonnes numériques alignées à droite**, avec le même nombre de décimales dans une colonne ;
- **l'unité dans l'en-tête**, pas dans chaque cellule ;
- **un ordre qui a un sens** (par valeur, par chronologie), pas l'ordre alphabétique par défaut ;
- **la ligne qui compte mise en évidence** (gras, ou une couleur et un signe, pour qui ne distingue pas les couleurs).

Voici le tableau qui accompagne la figure 3 : il met face à face les jours de promotion et les autres.

```
j = d["j"]
t = j.groupby("promo_active").agg(jours=("date", "count"), cmd_jour=("nb_commandes", "mean"),
↳ ca_jour=("chiffre_affaires", "mean"), marge_jour=("marge", "mean"))
t["marge_par_commande"] = j.groupby("promo_active")["marge"].sum() /
↳ j.groupby("promo_active")["nb_commandes"].sum()
t.index = ["hors promotion", "promotion"]
print(t.round(1).to_string())
```

	jours	cmd_jour	ca_jour	marge_jour	marge_par_commande
hors promotion	943	32.8	3338.5	1053.9	32.1
promotion	153	35.4	3300.1	836.4	23.6

On préférera, dans le rapport, une version épurée : trois lignes (commandes par jour, marge par commande, marge par jour), les deux colonnes, et la **différence** en dernière colonne. Les autres chiffres sont dans l'annexe.

La légende : décrire ou conclure

Sous une figure ou un tableau, la légende répond à trois questions : **qu'est-ce que c'est** (variable, période, unité), **comment le lire** (ce que signifie la couleur, la bande), et **d'où ça vient** (source, date). Elle peut ajouter le message si le titre ne l'a pas dit.

En pratique : la légende minimale. « Marge brute des 153 jours de promotion, en k€ (2023-2025). Les commandes en plus apportent 28 k€, les remises en retirent 46 k€. Source : lignes de commande de la boutique, TVA à 20 % retirée. » Une phrase pour la variable et la période, une pour la lecture, une pour la source.

4.2.5 Dire l'incertitude et les limites sans perdre le lecteur

C'est le point le plus difficile du rapport : être **honnête** sur ce qu'on ne sait pas, sans noyer le message. Trois principes aident.

1. **Faire le tri des limites**, en séparant celles qui **peuvent changer la conclusion** de celles qui n'y changent rien. On détaille les premières, on liste les secondes en annexe.
2. **Écrire les limites au présent et en positif** : ce qu'on sait, ce qu'on ne sait pas, ce qu'il faudrait pour trancher.
3. **Ne pas se couvrir** : une page de précautions n'est pas de l'honnêteté, c'est de la peur. Une limite précise (« la valeur à long terme des clients attirés par la promotion n'est pas comptée ») vaut mieux que dix vagues (« les résultats sont à interpréter avec prudence »).

La formule en trois temps donne, pour les promotions, un paragraphe de limites que la gérante peut utiliser.

Ce que nous savons	Ce que nous ne savons pas	Ce qu'il faudrait pour trancher
À saison égale, les promotions ajoutent environ 19 % de commandes (14 à 25 %).	Si les clients attirés reviennent ensuite : la valeur à long terme n'est pas comptée.	Suivre les clients acquis en promotion sur douze mois (cohortes, volume III, chapitre 4).
Chaque commande rapporte 24 € contre 32 €, remises comprises.	Si une partie des achats est simplement avancée : les ventes d'après-promotion ne sont pas étudiées.	Comparer les semaines suivant les promotions à une référence.
Même à la borne haute de l'effet, la marge baisse.	Si l'effet estimé est biaisé : l'analyse n'est pas une expérience.	Tester la prochaine édition sur une moitié des jours, tirés au hasard.

Les mots changent le message. « Il est possible que l'effet soit différent » ne dit rien ; « l'effet estimé est de 19 % et nous ne pouvons pas exclure qu'il soit 14 % ou 25 % » dit précisément ce qu'on ignore.

À retenir. Une limite utile est **précise, dite une fois** et suivie de **ce qu'on ferait pour la lever**. Elle donne au lecteur un moyen d'agir, pas seulement une raison de douter.

4.2.6 La relecture : une liste et un outil

Avant d'envoyer, on relit avec une liste, pas avec son impression. Voici les contrôles qui attrapent l'essentiel.

Contrôle	Question
Réponse	La réponse figure-t-elle dans les trois premières lignes ?
Décision	Le lecteur sait-il ce qu'on lui demande de décider, et pour quand ?
Titres	Les titres de figures, lus seuls, racontent-ils l'histoire ?
Chiffres	Chaque chiffre vient-il d'un calcul, avec la bonne unité et la bonne précision ?
Comparaisons	Chaque chiffre est-il comparé à quelque chose ?
Incertitude	Les estimations ont-elles leurs intervalles, ou sont-elles dites approximatives ?
Limites	Les limites qui pourraient changer la conclusion sont-elles dites ?
Jargon	Un lecteur non technicien comprend-il chaque phrase du résumé ?
Longueur	Le résumé tient-il en une demi-page ?
Reproduction	Un collègue peut-il retrouver chaque chiffre à partir de l'annexe ?

Certains contrôles s'automatisent. Celui des **chiffres** est le plus précieux : il compare les nombres écrits dans le texte à ceux que le code a calculés, et signale les orphelins. Prenons un brouillon où une erreur s'est glissée.

```
brouillon = ("Les promotions ajoutent environ 22 % de commandes et font perdre 18 000 € de marge sur
↳ 153 jours. "
             "Chaque commande rapporte 24 € contre 32 €. Il faudrait 36 % de commandes en plus pour
             ↳ s'en sortir.")
permis = [a["e"] * 100, a["e_bas"] * 100, a["e_haut"] * 100, -a["inc"], a["jours"], a["mo_p"],
↳ a["mo_np"], a["seuil"] * 100]
print("nombres sans source :", 0.verifier_nombres(brouillon, permis))
```

```
nombres sans source : [22.0]
```

Le « 22 % » est signalé : il ne correspond à aucune valeur calculée (l'effet est de 19 %, et sa borne haute de 25 %). On a retrouvé une erreur de recopie qu'aucune relecture rapide n'aurait vue. Le contrôle accepte les arrondis d'un nombre calculé (« 18 000 » pour 17 884, « 24 » pour 23,62) mais pas une valeur qui n'est l'arrondi d'aucun d'eux. Il ne dit pas si le texte est **vrai**, seulement s'il est **sourcé** ; c'est déjà beaucoup.

Piège. Un nombre qui passe le contrôle peut être mal employé (une borne basse présentée comme l'estimation, par exemple). L'outil écarte les erreurs de recopie, pas les erreurs de raisonnement.

4.2.7 Un rapport reproductible

Le plus sûr moyen d'éviter les erreurs de recopie est de ne **jamais recopier** : le texte du rapport est produit par le même code que les chiffres. On écrit une phrase à trous, que le code remplit.

```
resume = (f"Les promotions font perdre environ {0.fr(sig(-a['inc'], 2), 0)} € de marge sur
↳ {a['jours']} jours : elles ajoutent {0.fr(a['e'] * 100, 0)} % de commandes, "
        f"mais chaque commande rapporte {0.fr(a['mo_p'], 0)} € au lieu de {0.fr(a['mo_np'], 0)} €.
↳ Il faudrait {0.fr(a['seuil'] * 100, 0)} % de commandes en plus pour s'en sortir.")
print(resume)
print("nombres sans source :", 0.verifier_nombres(resume, permis))
```

```
Les promotions font perdre environ 18 000 € de marge sur 153 jours : elles ajoutent 19 % de commandes,
↳ mais chaque commande rapporte 24 € au lieu de 32 €. Il faudrait 36 % de commandes en plus pour
↳ s'en sortir.
nombres sans source : []
```

Si les données ou la méthode changent, **le texte change avec elles**, et le contrôle des chiffres continue de passer. C'est le principe de tout rapport reproductible, qu'il soit écrit avec un notebook (Jupyter), un document à code intégré (Quarto, R Markdown) ou un simple script qui remplit un modèle.

Quatre habitudes complètent le dispositif.

1. **Une seule commande** pour refaire le rapport depuis les données brutes.
2. **La date, la version du code et l'identité des données** écrites sur le rapport (une empreinte du fichier suffit, comme au volume II).
3. **Les graines aléatoires fixées** quand une simulation intervient.
4. **Le rapport et son code conservés ensemble**, dans un dépôt versionné, avec les décisions de choix de méthode.

```
import hashlib
empreinte = hashlib.sha256(open(os.path.join(D, "jours_exploitation.csv"),
↳ "rb").read()).hexdigest()[:12]
print("rapport produit le 2025-12-31 à partir de jours_exploitation.csv, empreinte", empreinte)
```

```
rapport produit le 2025-12-31 à partir de jours_exploitation.csv, empreinte 874abd43dd00
```

En pratique. Un rapport qui ne peut pas être refait dans six mois n'est pas un rapport : c'est une photo. Si votre lecteur vous demande « et si on enlevait le mois de décembre ? », vous devez pouvoir répondre en dix minutes, pas en deux jours.

Pour s'entraîner. Cahier, chapitre 4 : applications 4.3 et 4.4, exercices 4.5 à 4.8.

4.3 ⊕ Pour aller plus loin : synthèses de direction, rapports d'une page et présentations

Le rapport complet est un document de référence ; la plupart des décisions se prennent avec **moins** : cinq lignes dans un courriel, une page imprimée, dix minutes devant une équipe. Cette section montre comment réduire sans trahir, avec la même analyse (les promotions) comme fil conducteur.

4.3.1 Le résumé de direction en cinq lignes

Quand on n'a que cinq lignes, chacune doit jouer un rôle. Une structure fiable, reprise de la pyramide :

Ligne	Rôle	Pour les promotions
1. Contexte	ce que le lecteur sait déjà	La boutique fait trois promotions par an (153 jours).
2. Constat	la réponse	Elles ajoutent des commandes mais font perdre de la marge.
3. Pourquoi	le chiffre qui l'explique	Chaque commande rapporte 8 € de moins ; le gain de volume ne compense pas.
4. Recommandation	ce qu'on propose	Baisser la remise et tester la prochaine promotion sur une partie des jours.
5. Décision demandée	ce qu'on attend du lecteur, et quand	Accord sur le test avant le 15 novembre.

On peut les produire à partir des chiffres calculés, ce qui garantit leur cohérence avec le rapport.

```

cinq = [f"Contexte : la boutique fait trois promotions par an, soit {a['jours']} jours en trois ans.",
        f"Constat : elles ajoutent {0.fr(a['e'] * 100, 0)} % de commandes mais font perdre environ",
        f"↳ {0.fr(sig(-a['inc'], 2), 0)} € de marge.",
        f"Pourquoi : chaque commande rapporte {0.fr(a['mo_p'], 0)} € au lieu de {0.fr(a['mo_np'], 0)}",
        f"↳ € ; il faudrait {0.fr(a['seuil'] * 100, 0)} % de commandes en plus.",
        "Recommandation : baisser la remise et tester la prochaine promotion sur une partie des",
        f"↳ jours.",
        "Décision demandée : accord sur le test avant le 15 novembre."]
print("\n".join(cinq))

```

Contexte : la boutique fait trois promotions par an, soit 153 jours en trois ans.
 Constat : elles ajoutent 19 % de commandes mais font perdre environ 18 000 € de marge.
 Pourquoi : chaque commande rapporte 24 € au lieu de 32 € ; il faudrait 36 % de commandes en plus.
 Recommandation : baisser la remise et tester la prochaine promotion sur une partie des jours.
 Décision demandée : accord sur le test avant le 15 novembre.

Une dernière vérification : le texte tient-il dans une fenêtre de courriel, et chaque nombre a-t-il une source ?

```

texte = "\n".join(cinq)
print(0.lisibilite(texte))
print("nombres sans source :", 0.verifier_nombres(texte, permis + [15]))

```

```

{'phrases': 5, 'mots': 68, 'mots_par_phrase': 13.6, 'termes_techniques': 0, 'chiffres': 8}
nombres sans source : []

```

En pratique : l'objet du courriel est le premier résumé. « Promotions : –18 k€ de marge, test proposé, décision avant le 15 novembre » en dit plus que « Analyse des promotions ». Le lecteur qui n'ouvre pas le message en sait déjà l'essentiel.

4.3.2 La page unique

La note d'**une page** est l'un des formats les plus puissants et les plus difficiles : la contrainte oblige à choisir. Elle a un modèle presque universel.

Le plan se lit de haut en bas, dans l'ordre d'importance, et suit les règles suivantes.

1. **Le titre est une conclusion.** « Les promotions : vendre plus, gagner moins. »
2. **La recommandation est en haut**, dans un encadré : un lecteur qui s'arrête là a déjà l'essentiel.
3. **Trois chiffres, pas dix.** Chacun avec sa comparaison. Au-delà, le lecteur n'en retient aucun.
4. **Une seule figure**, qui prouve le titre. Si deux figures sont nécessaires, c'est que la note a deux messages.
5. **Les limites en pied de page**, brèves, avec la mention de ce qui n'est pas mesuré.
6. **Aucun jargon**, aucun détail de méthode : un renvoi vers le rapport complet suffit.

Comment la fabriquer ? Peu importe l'outil, pourvu que la page soit **reproductible** : un traitement de texte ou un éditeur de documents (Word, LibreOffice Writer) pour un document ponctuel ; un modèle HTML ou Markdown converti en PDF pour un document récurrent (section 4.4) ; ou une figure composée, comme celle ci-dessus, pour une note très courte. Quel que soit l'outil, **gardez la source** (le tableau et le code qui ont produit les chiffres).

Intuition. La contrainte d'une page est un **exercice de décision** : pour chaque élément, « si je l'enlève, le lecteur change-t-il de conclusion ? ». Ce qui ne change rien disparaît, et la note y gagne en force.

4.3.3 La présentation de huit diapositives

Pour une réunion de dix minutes, on compte **une à deux minutes par diapositive**, soit sept ou huit diapositives plus les annexes. Le récit de la section 4.1 s'y transpose presque directement.

Diapositive	Contenu	Durée
1	Titre-conclusion et décision attendue	30 s
2	Le chiffre évident, qui trompe (tension)	1 min
3	L'effet réel, avec son intervalle	1 min 30
4	Le coût : pourquoi chaque commande rapporte moins	1 min 30
5	Le résultat : -18 k€ de marge	1 min
6	La robustesse : même dans le meilleur cas, la marge baisse	1 min
7	La recommandation : que fait-on, qui, quand	2 min
8	Annexe : méthode et limites (non présentée , pour les questions)	

Quelques règles pratiques :

- **un message par diapositive**, formulé dans le titre (la règle de 4.1.4) ;
- **trois puces au maximum**, et chacune de moins de dix mots ; si une diapositive demande une longue explication, elle appartient au rapport ;

Les promotions : vendre plus, gagner moins

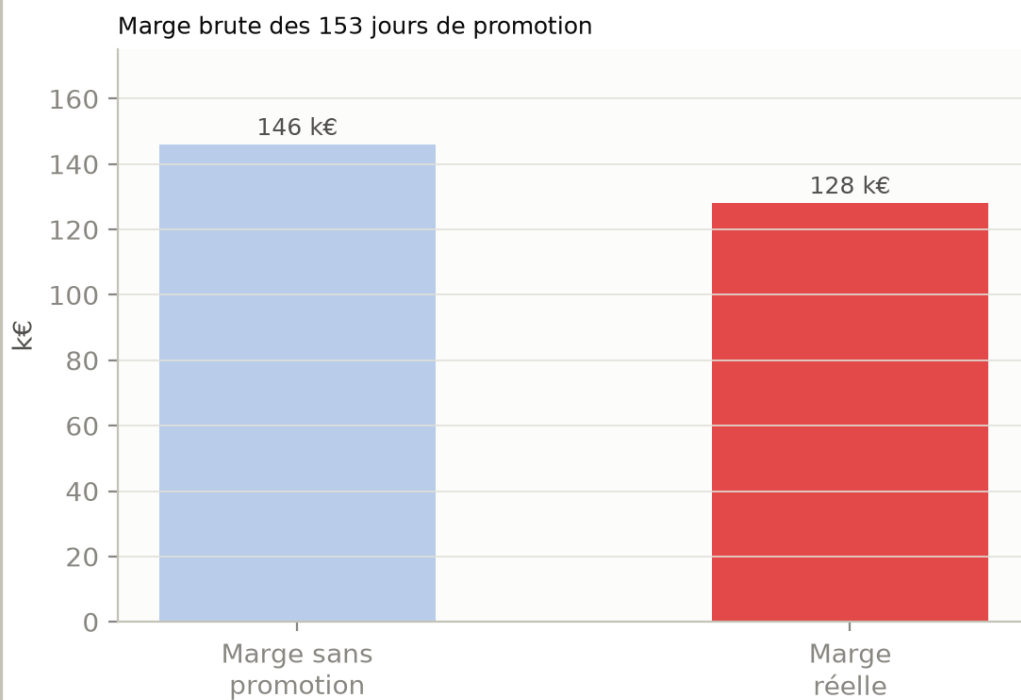
Note à la gérante : décision demandée avant le 15 novembre

Recommandation : ne pas reconduire la promotion telle quelle.

Baisser la remise, la limiter aux produits à forte marge, tester l'édition d'hiver sur une partie des jours.

Trois chiffres à retenir

- +19 % de commandes à saison égale (intervalle 14 à 25 %)
- marge par commande : 32 € hors promotion, 24 € en promotion
- perte de marge estimée : 18 k€ ; seuil de bascule : +36 % de commandes



Limites : effet estimé, non mesuré par expérience ;
valeur à long terme des clients acquis non comptée.

Figure 74: Une note d'une page (maquette dessinée) : le titre énonce le message, la recommandation est en haut dans un encadré, trois chiffres à retenir, une figure qui prouve le titre, et les limites en pied de page.



Figure 75: Huit diapositives pour dix minutes (maquette dessinée). Chaque titre est une conclusion ; les figures prouvent, les puces résument, et la méthode est en annexe.

- **la figure occupe l'espace** : une figure lisible vaut plus que trois puces ;
- **pas de lecture à voix haute du texte** de la diapositive : le lecteur lit plus vite que vous ne parlez ;
- **les notes de l'orateur** portent ce qu'on dit en plus, pas ce qu'on affiche ;
- **la recommandation arrive à l'avant-dernière position**, jamais noyée : la dernière diapositive de la présentation reste en général celle des questions et des annexes.

Les logiciels de présentation courants (PowerPoint, Keynote, LibreOffice Impress, Google Slides) conviennent tous ; ce qui compte est la **maîtrise du modèle** (une grille, deux polices, une palette) et non l'outil. Pour les présentations récurrentes dont les chiffres changent, on peut aussi écrire les diapositives en texte et les produire par un outil comme Marp ou Quarto. Voici ce que donne le début d'une présentation en Marp, sans l'exécuter ici.

```

---
marp: true
---
# Les promotions : vendre plus, gagner moins
Décision attendue : tester la prochaine édition avant le 15 novembre

---
# À saison égale, les promotions ajoutent 19 % de commandes
! [w:700](figures/ch04-recit-2-effet.png)

```

Piège : la diapositive-document. Une diapositive que l'on envoie par courriel sans présentation doit se comprendre seule ; une diapositive que l'on projette doit se comprendre en cinq secondes. Ce sont deux exigences incompatibles : choisissez, ou faites deux documents (la note d'une page est le bon document à envoyer).

4.3.4 Anticiper les questions

La présentation se joue souvent dans les **questions**. Les plus fréquentes se préparent, avec des chiffres déjà calculés, dans une feuille ou une diapositive d'annexe.

Question probable	Réponse préparée
« Et si l'effet était plus fort que vous ne dites ? »	Même à la borne haute de l'intervalle (+25 %), la marge baisse de 11 k€.
« Vous avez compté la publicité ? »	Les jours de promotion, la dépense publicitaire est supérieure d'environ 7 k€ : la perte monte à 25 k€.
« Combien de commandes en plus faudrait-il ? »	Plus de 35 % : près du double de l'effet estimé.
« Et si on baissait la remise de moitié ? »	Voir ci-dessous : si l'effet sur les commandes tenait, la marge redeviendrait positive en réduisant la remise d'environ 40 %. C'est une hypothèse, d'où le test.
« Et les clients que ça attire ? »	Non mesuré : c'est la première limite du rapport.

La quatrième réponse est un **scénario**, pas un résultat : on fait une hypothèse (l'effet sur les commandes reste le même avec une remise moindre) et l'on regarde ce qu'elle implique. On le calcule sans le présenter comme une prévision.

```
N, ecart = a["n_cmd"], a["mo_np"] - a["mo_p"]
sans = N / (1 + a["e"]) * a["mo_np"]
for part in (1.0, 0.75, 0.5, 0.25):
    marge = N * (a["mo_np"] - ecart * part)
    print(f"remise à {part * 100:3.0f} % de la remise actuelle : marge {0.fr(marge / 1000, 0)} k€,
    ↪   incrément {0.fr((marge - sans) / 1000, 0, True)} k€")
```

```
remise à 100 % de la remise actuelle : marge 128 k€, incrément -18 k€
remise à 75 % de la remise actuelle : marge 139 k€, incrément -6 k€
remise à 50 % de la remise actuelle : marge 151 k€, incrément +5 k€
remise à 25 % de la remise actuelle : marge 162 k€, incrément +17 k€
```

Le tableau montre que le **point d'équilibre** se situe vers 60 % de la remise actuelle (soit une remise réduite d'environ 40 %), et il faut le lire avec l'hypothèse qui le rend possible : plus la remise diminue, moins l'effet sur les commandes a de chances de rester à 19 %. C'est précisément ce que le test proposé mesurera.

À retenir. Préparer les questions, c'est **continuer l'analyse après le rapport**. Les réponses ont un statut : un résultat (tiré des données), un scénario (une hypothèse nommée), ou un inconnu (à dire franchement). Ne pas les confondre est la moitié de l'honnêteté.

4.3.5 Choisir le format selon le lecteur et le moment

Lecteur et moment	Format	Longueur
La gérante, entre deux rendez-vous	courriel de cinq lignes	100 mots
La gérante, avant une décision	note d'une page	1 page + rapport en pièce jointe
L'équipe, en réunion	présentation	7 à 8 diapositives + annexes
Un collègue analyste qui reprend	rapport complet et notebook	autant que nécessaire
Un lecteur futur (archives)	rapport complet daté, avec empreinte des données	idem

Le chapitre 5 revient sur la **connaissance du public** : comment recueillir ses besoins, adapter le niveau de détail et préparer la présentation orale. Retenons ici que **le même contenu change de forme selon la situation**, et que le travail de l'analyste est d'avoir les trois formes prêtes.

Pour s'entraîner. Cahier, chapitre 4 : application 4.5, exercices 4.9 et 4.10.

4.4 ⊕ Pour aller plus loin : rapports récurrents automatisés

Chaque lundi, la gérante veut les mêmes chiffres de la semaine. Les calculer à la main prend une heure, expose aux erreurs de recopie, et personne n'a envie de le faire le quarante-septième lundi. Cette section montre comment **produire le rapport par un programme**, en prenant garde à deux dangers : un rapport automatique qui **envoie des erreurs sans prévenir**, et un texte généré qui **raconte du hasard comme s'il s'agissait d'un événement**.

4.4.1 Quand automatiser, et quand ne pas le faire

Automatiser coûte du temps (écrire, tester, maintenir) et n'en rapporte que si certaines conditions sont réunies.

Condition	Pourquoi
Le rapport est récurrent (hebdomadaire, mensuel)	le gain se répète, le coût se paie une fois
Les définitions sont stables et écrites	un programme ne comprend pas qu'on a changé d'avis
Le format est stable	on ne passe pas son temps à retoucher le programme
Les données arrivent proprement (volume II)	un programme transmet vite les erreurs d'une source sale
Quelqu'un en est responsable	un programme sans propriétaire casse en silence

Si l'une manque, on commence par un notebook que l'on relance à la main : c'est déjà un rapport reproductible (section 4.2.7). L'automatisation arrive quand le notebook ne change plus.

Piège : automatiser une analyse qu'on ne comprend pas encore. Un rapport automatisé fige des choix de méthode. Si ces choix ne sont pas stabilisés, vous automatisez des erreurs et vous leur donnez en prime l'autorité de la machine.

4.4.2 Le rapport de la semaine

Notre rapport hebdomadaire tient en une page : les cinq indicateurs que la gérante suit (chiffre d'affaires, commandes, panier moyen, marge, livraisons à l'heure), chacun comparé à la **semaine précédente** et à la **même semaine de l'an dernier**, une phrase de synthèse, et la variation ordinaire en pied de page. Toute la chaîne est dans une fonction : charger les données, calculer, écrire.

```
md, k, bruit = 0.rapport_md(d, "2025-11-10")
lignes = [l for l in md.splitlines() if l.strip()]
print("\n".join(lignes))
```



```
# Rapport hebdomadaire : semaine du 10/11/2025 au 16/11/2025
**En une phrase.** Chiffre d'affaires de 31 312 €, stable par rapport à la même semaine de l'an
↳ dernier (+14,6 %, dans la variation ordinaire).
| Indicateur | Cette semaine | Semaine précédente | Même semaine N-1 |
|---|---|---|---|
| Chiffre d'affaires (€) | 31 312 | 30 240 | 27 320 |
| Commandes | 332 | 312 | 286 |
| Panier moyen (€) | 94,3 | 96,9 | 95,5 |
| Marge brute HT (€) | 10 216 | 9 815 | 8 583 |
| Livraisons à l'heure (%) | 77,7 | 77,6 | 80,9 |
**Lecture.** D'une semaine à l'autre, le chiffre d'affaires est stable par rapport à la semaine
↳ précédente (+3,5 %, dans la variation ordinaire).
*Variation ordinaire (écart-type sur 26 semaines) : 13,3 % d'une semaine à l'autre, 13,7 % d'une année
↳ à l'autre.*
```

On peut en faire un document lisible par tout le monde : ici, le Markdown est converti en page HTML (avec l'outil libre pandoc) et ouvert dans un navigateur.

Rapport hebdomadaire : semaine du 10/11/2025 au 16/11/2025

En une phrase. Chiffre d'affaires de 31 312 €, stable par rapport à la même semaine de l'an dernier (+14,6 %, dans la variation ordinaire).

Indicateur	Cette semaine	Semaine précédente	Même semaine N-1
Chiffre d'affaires (€)	31 312	30 240	27 320
Commandes	332	312	286
Panier moyen (€)	94,3	96,9	95,5
Marge brute HT (€)	10 216	9 815	8 583
Livraisons à l'heure (%)	77,7	77,6	80,9

Lecture. D'une semaine à l'autre, le chiffre d'affaires est stable par rapport à la semaine précédente (+3,5 %, dans la variation ordinaire).

Variation ordinaire (écart-type sur 26 semaines) : 13,3 % d'une semaine à l'autre, 13,7 % d'une année à l'autre.

Figure 76: Le rapport hebdomadaire de la semaine du 10 novembre 2025, produit par le programme : une phrase, un tableau de cinq indicateurs et la variation ordinaire en pied de page. Capture réelle d'une page HTML générée localement.

La conversion se fait en une commande, que l'on peut aussi lancer depuis un programme.

```
pandoc rapport.md --from gfm --to html --standalone --output rapport.html
```

4.4.3 Un texte qui sait se taire

La partie délicate d'un rapport automatique est la **phrase** : « le chiffre d'affaires est en hausse de 3,5 % ». Un programme naïf l'écrit chaque semaine, pour n'importe quelle variation, et le lecteur apprend à ne plus lire : ce qu'on lui dit change toutes les semaines sans jamais rien signifier.

Rappelons le principe (volume III, chapitre 6) : une variation n'est un signal que si elle **dépasse la variation ordinaire**. Notre générateur applique une règle simple : on ne commente une variation que si elle dépasse **une fois et demie** l'écart-type des variations passées (26 dernières semaines) ;

sinon, la phrase dit « stable... dans la variation ordinaire ». Comparons les deux générateurs sur les 51 semaines complètes de 2025.

```
res = []
for lundi in pd.date_range("2025-01-06", "2025-12-22", freq="7D"):
    s = 0.semaine_kpis(d, lundi)
    v = s["cur"]["ca"] / s["an"]["ca"] - 1
    res.append((lundi.date(), v, abs(v) >= 1.5 * 0.bruit_hebdo(d, lundi, annuel=True)))
r = pd.DataFrame(res, columns=["lundi", "variation", "commentee"])
print("semaines :", len(r), "| commentées par le générateur naïf :", len(r), "| par le générateur
↳ prudent :", int(r["commentee"].sum()))
print("variation annuelle médiane :", 0.fr(r["variation"].median() * 100, 1, True), "% | étendue :",
↳ 0.fr(r["variation"].min() * 100, 0, True), "à", 0.fr(r["variation"].max() * 100, 0, True), "%")
```

```
semaines : 51 | commentées par le générateur naïf : 51 | par le générateur prudent : 10
variation annuelle médiane : +12,0 % | étendue : -14 à +44 %
```

Sur 51 semaines, le générateur naïf aurait émis 51 commentaires, le prudent **dix**. Voici les deux phrases, pour une semaine ordinaire puis pour une semaine à examiner.

```
for lundi in ("2025-11-10", "2025-12-01"):
    print(lundi, ":", 0.rapport_md(d, lundi)[0].splitlines()[2].replace("***En une phrase.** ", ""))
```

```
2025-11-10 : Chiffre d'affaires de 31 312 €, stable par rapport à la même semaine de l'an dernier
↳ (+14,6 %, dans la variation ordinaire).
2025-12-01 : Chiffre d'affaires de 46 088 €, en hausse de 29,3 % par rapport à la même semaine de l'an
↳ dernier (au-delà de la variation ordinaire de ±20 %).
```

Deux remarques sur ces résultats, qui montrent la limite d'un générateur de texte.

1. **Les dix semaines signalées sont toutes en hausse**, or la variation annuelle médiane des 51 semaines est de +12 % : la boutique croît d'une année à l'autre, donc la référence « même semaine de l'an dernier » se trouve en moyenne en dessous. Une version plus fine retirerait cette croissance d'ensemble (exercice 4.11). Il reste que le rapport dit désormais **peu de choses, et plutôt les bonnes**.
2. **Le texte dit ce qui s'est passé, jamais pourquoi**. La semaine du 1er décembre est signalée (+29 %), mais le programme ne sait pas qu'elle suit la promotion de fin novembre. L'explication reste au lecteur ou à l'analyste, qui peut ajouter un commentaire de deux lignes. Un bon dispositif combine donc **des chiffres automatiques et un mot humain quand c'est signalé**.

Intuition. Un rapport automatique qui commente tout est un rapport qu'on cesse de lire. Un rapport qui **se tait** quand il n'y a rien à dire rend chaque phrase précieuse.

Le texte ne couvre ici que le chiffre d'affaires, mais le tableau montre un autre fait : la semaine du 1er décembre, les livraisons à l'heure tombent à 46 %, contre 78 % la semaine précédente. Un lecteur attentif s'inquiète. La colonne « même semaine N-1 » le calme : l'an dernier, c'était 40 %. Le tableau **contextualise** ce que le texte ne dit pas, d'où l'importance de garder la comparaison à l'année précédente à côté de chaque indicateur.

4.4.4 Les contrôles avant envoi

Un rapport manuel a un garde-fou : la personne qui le fait voit quand quelque chose cloche (« tiens, il manque mardi »). Un rapport automatique n'en a **aucun**, sauf ceux qu'on lui donne. Avant d'envoyer, le programme vérifie que les données sont complètes, fraîches et cohérentes ; au moindre doute, il **n'envoie pas** et il prévient.

```
for nom, ok in 0.controles_avant_envoi(d, "2025-11-10").items():
    print("OK " if ok else "ÉCHEC", nom)
```

```
OK sept jours présents
OK dernière date des données couvre la semaine
OK CA du fichier journalier = CA des lignes (à 1 €)
OK commandes du fichier journalier = commandes distinctes
OK aucune valeur manquante dans la semaine
```

Mettons ces contrôles à l'épreuve en abîmant les données : un jour manquant dans le fichier journalier, puis un fichier qui s'arrête trop tôt.

```
def envoyer(d, lundi):
    echecs = [nom for nom, ok in 0.controles_avant_envoi(d, lundi).items() if not ok]
    if echecs:
        raise RuntimeError("rapport NON envoyé : " + " ; ".join(echecs))
    return "rapport envoyé"

j = d["j"]
cas = {"données intactes": d, "un jour manquant": {**d, "j": j[j["date"] != "2025-11-12"]}, "fichier
↳ arrêté le 14 novembre": {**d, "j": j[j["date"] <= "2025-11-14"]}}
for nom, dd in cas.items():
    try:
        print(nom, "→", envoyer(dd, "2025-11-10"))
    except RuntimeError as e:
        print(nom, "→", e)
```

```
données intactes → rapport envoyé
un jour manquant → rapport NON envoyé : sept jours présents ; CA du fichier journalier = CA des lignes
↳ (à 1 €) ; commandes du fichier journalier = commandes distinctes
fichier arrêté le 14 novembre → rapport NON envoyé : sept jours présents ; dernière date des données
↳ couvre la semaine ; CA du fichier journalier = CA des lignes (à 1 €) ; commandes du fichier
↳ journalier = commandes distinctes
```

Avec les données abîmées, le programme refuse d'envoyer et dit pourquoi. Un seul jour manquant fait échouer trois contrôles à la fois : c'est l'**accumulation** de contrôles indépendants (complétude, fraîcheur, concordance des totaux) qui rend l'erreur difficile à manquer.

En pratique : quelques contrôles utiles. Les sept jours de la semaine sont présents ; la dernière date des données couvre la semaine ; les totaux de deux sources concordent (le chiffre d'affaires du fichier journalier et celui des lignes de commande) ; aucune valeur manquante ; aucun chiffre ne varie de plus de dix fois (probable erreur d'unité). Les mêmes idées sont développées au volume II, chapitre 3.

4.4.5 Les notebooks paramétrés

Une façon simple d'automatiser consiste à écrire un **notebook** dont la première cellule contient les paramètres (ici, le lundi de la semaine), puis à l'exécuter pour chaque valeur. Des outils spécialisés (papermill, Quarto) le font ; la mécanique est de toute façon accessible par les bibliothèques `nbformat` et `nbclient`, que l'on utilise ici sur un notebook construit à la volée.

```
import nbformat, nbclient
def executer(debut):
    nb = nbformat.v4.new_notebook()
    nb.cells = [nbformat.v4.new_code_cell(f'debut = "{debut}"'),
                nbformat.v4.new_code_cell("import sys, os; sys.path.insert(0, 'build'); import
↳ outils_ch04 as O\nd = 0.charger(os.environ['DONNEES'])\nprint(0.rapport_md(d,
↳ debut)[0].splitlines()[2]))"]
    nbclient.NotebookClient(nb, timeout=120, resources={"metadata": {"path": "."}}).execute()
    return nb.cells[1].outputs[0].text.strip().replace("***En une phrase.** ", "")
for lundi in ("2025-11-10", "2025-12-01"):
    print(lundi, ":", executer(lundi))
```

```
2025-11-10 : Chiffre d'affaires de 31 312 €, stable par rapport à la même semaine de l'an dernier
↳ (+14,6 %, dans la variation ordinaire).
2025-12-01 : Chiffre d'affaires de 46 088 €, en hausse de 29,3 % par rapport à la même semaine de l'an
↳ dernier (au-delà de la variation ordinaire de ±20 %).
```

Le notebook produit le même texte que la fonction directe : c'est le but. Son avantage est qu'il **conserve les sorties** (figures, tableaux) et peut être transformé en page HTML ou en PDF ; son inconvénient est qu'il est plus lourd à tester qu'une fonction. On choisit en pratique : **une fonction testée** pour le calcul, un notebook (ou un modèle) pour la mise en forme.

4.4.6 Planifier l'exécution

Un programme qui doit tourner chaque lundi à sept heures n'est pas lancé par une personne. Sous Linux, c'est le rôle de **cron** (ou d'un minuteur `systemd`) ; il existe des équivalents sous Windows (planificateur de tâches) et dans les outils de données (les ordonnanceurs des plateformes décisionnelles, les tâches planifiées d'un dépôt de code). Le principe est le même : une ligne qui dit **quand** et **quoi**.

```
# crontab -e : chaque lundi à 7 h 00, produire le rapport, garder la trace de l'exécution
0 7 * * 1 cd /srv/rapports && ./produire_rapport.sh >> journal.log 2>&1
```

Le script lui-même doit se comporter proprement : s'arrêter à la première erreur, ne jamais envoyer un rapport en cas d'échec d'un contrôle, et **dire** quand il échoue.

```
#!/usr/bin/env bash
set -euo pipefail                                # s'arrêter à la première erreur
lundi=$(date -d "last monday" +%F)                # la semaine qui vient de finir
python produire_rapport.py --lundi "$lundi" --sortie "rapports/$lundi.html"
python envoyer.py "rapports/$lundi.html"          # n'est appelé que si les contrôles ont réussi
echo "$(date -Is) rapport $lundi envoyé"
```

Reste l'inverse du problème : un programme qui **plante en silence** est pire qu'un programme qui n'existe pas, car personne ne sait qu'il faut y regarder. Deux parades.

1. **Prévenir en cas d'échec** : le planificateur ou le script envoie un message à un responsable quand il se termine mal.
2. **Prévenir en cas de silence** : un « test de présence » vérifie chaque lundi que le rapport est bien arrivé ; son absence est elle-même une alerte.

Ce qui peut mal tourner	Parade
Les données arrivent en retard	contrôle de fraîcheur, nouvel essai plus tard, alerte
Une source change de format	contrôles de colonnes et de totaux (volume II)
Une définition d'indicateur change	fiche de KPI versionnée ; le programme lit la fiche
L'envoi échoue	journal, alerte au responsable
Le texte généré devient absurde	règles prudentes (4.4.3), relecture trimestrielle
La personne responsable part	documentation, propriétaire désigné, dépôt partagé
Plus personne ne lit	question à la gérante : « ce rapport vous sert-il encore ? »

4.4.7 Ce qu'un rapport automatique ne remplace pas

Un rapport automatique donne **les chiffres de la semaine** ; il ne donne ni la décision ni l'analyse du problème nouveau. Trois habitudes l'empêchent de devenir une routine aveugle.

1. **Un champ de commentaire humain**, facultatif, que l'on remplit quand une variation est signalée : « semaine du 1er décembre : suite à la promotion de fin novembre ».
2. **Une revue trimestrielle** : les indicateurs sont-ils toujours les bons, les seuils toujours justes, la mise en page toujours claire ?
3. **Un moyen de dire « stop »** : tout lecteur doit pouvoir demander qu'un rapport cesse, change ou se complète.

À retenir. Automatiser un rapport, c'est **écrire ses règles une fois pour toutes** : quels chiffres, quelle comparaison, quel seuil de commentaire, quels contrôles avant envoi. Les règles sont le vrai livrable ; l'exécution n'est qu'une répétition.

Pour s'entraîner. Cahier, chapitre 4 : application 4.6, exercices 4.11 et 4.12.

Bilan du chapitre 4

Vous savez maintenant :

- **distinguer le journal de bord du récit** : le premier suit l'ordre du travail et s'adresse à qui vérifie, le second suit l'ordre du besoin du lecteur et s'adresse à qui décide ;
- **structurer un récit en quatre temps** (contexte, tension, preuves, résolution), **commencer par la réponse** (pyramide de Minto) et **donner un message par figure** avec un titre qui conclut ;
- **raconter l'analyse des promotions en quatre figures et un storyboard** : le chiffre évident (+7,8 % de commandes, chiffre d'affaires stable), l'effet réel (+19 %, intervalle de 14 à 25 %), le coût (24 € de marge par commande au lieu de 32 €, soit -18 k€ sur 153 jours), le seuil de bascule (+36 % de commandes), puis la recommandation ;
- **trier ce qui va dans le récit et ce qui va en annexe** (question du lecteur : « si je l'enlève, change-t-il de conclusion ? ») ;
- **reconnaître un récit malhonnête** : cerises cueillies (+71 % de commandes entre octobre et décembre 2024, mais +12 % de décembre à décembre), causalité sous-entendue, incertitude retirée, graphique qui exagère ; et **choisir ses verbes** selon ce qui est établi ;

- **structurer un rapport en huit parties, écrire pour un lecteur pressé** (phrases courtes, verbes actifs, un terme pour une chose, jargon traduit, 94 mots avant la réponse contre 4) ;
- **écrire les chiffres** : arrondir à la précision connue, nommer l'unité, comparer, distinguer pourcentage et points, appliquer les conventions françaises ;
- **dire l'incertitude et les limites** sans perdre le lecteur (ce que nous savons, ce que nous ne savons pas, ce qu'il faudrait pour trancher) ;
- **relire avec une liste et un outil** qui compare les nombres du texte aux nombres calculés, et **produire le texte par le code** pour qu'il ne puisse pas diverger ;
- **⊕ réduire sans trahir** : résumé en cinq lignes, note d'une page, présentation de huit diapositives, questions anticipées (même à la borne haute de l'effet, -11 k€ ; avec la publicité, -25 k€) ;
- **⊕ automatiser un rapport récurrent** avec un texte qui se tait quand la variation reste dans l'ordinaire (10 semaines commentées sur 51 au lieu de 51), des **contrôles avant envoi** qui bloquent un envoi sur des données abîmées, un notebook paramétré et une planification qui prévient quand elle échoue.

Le tableau suivant résume ce que nous avons mesuré dans ce chapitre.

Question	Résultat
Effet des promotions sur les commandes (brut, puis à saison égale)	+7,8 % puis +19 % (intervalle de 14 à 25 %)
Marge par commande, hors promotion et en promotion	32 € et 24 € (-26 %)
Incrément de marge des 153 jours de promotion	-18 k€ (-11 k€ à la borne haute de l'effet, -25 k€ avec la publicité)
Effet nécessaire pour ne pas perdre de marge	+36 % de commandes
Hausse d'octobre à décembre 2024, puis décembre contre décembre	+71 %, puis +12 %
Résumé « récit » contre résumé « journal de bord »	70 mots contre 105, 0 terme technique contre 5, réponse au 4 ^e mot contre le 94 ^e
Semaines de 2025 signalées par le rapport automatique	10 sur 51 (toutes en hausse)

Le fil conducteur du chapitre tient en une phrase : **une analyse ne vaut que par ce qu'en comprend et en fait son lecteur**, et cela se prépare : on choisit ce que l'on montre, dans l'ordre où le lecteur en a besoin, avec des chiffres que l'on peut retrouver. Trois habitudes à retenir :

En pratique : trois habitudes.

1. **La réponse d'abord, la preuve ensuite, le détail en annexe.** Chaque niveau du document doit pouvoir être lu seul.
2. **Jamais de chiffre recopié à la main.** Le texte est produit par le code, ou contrôlé contre lui.
3. **Un rapport automatique doit savoir se taire et savoir s'arrêter.** Se taire quand la variation est ordinaire, s'arrêter quand les données sont douteuses.

Rappel d'honnêteté. Les données sont **simulées** : la « vérité programmée » (+18 % de commandes) n'est connue que parce que nous avons écrit le simulateur. Dans la vraie vie, l'effet des promotions est **estimé**, et l'analyse n'est pas une expérience : c'est précisément ce que la section 4.2.5 apprend à dire. Les maquettes de pages et de diapositives sont dessinées ; aucune application de présentation n'a été exécutée.

Le chapitre 5 traite de la **présentation aux décideurs** : comprendre son public, recueillir ses besoins, présenter résultats et recommandations. Vous y retrouverez, sous l'angle de l'oral et de la relation, ce que ce chapitre a posé sous l'angle de l'écrit.

Pour s'entraîner. Cahier, chapitre 4 : applications 4.1 à 4.6 (arc et storyboard, titres et figures, relecture des chiffres, rapport reproductible, note d'une page, rapport automatique) et exercices 4.1 à 4.12.

Chapitre 5 : Présenter à des interlocuteurs non techniques

« Ce que l'on a trouvé importe moins que ce que l'autre a compris et décidé. »

Un mardi matin, la gérante passe la tête dans votre bureau. « La réunion de direction est **jeudi de la semaine prochaine**. J'ai inscrit le point des soldes à l'ordre du jour. **Tu as dix minutes**. Il y aura le responsable logistique, la comptable, et le représentant de la banque qui suit notre dossier. Dis-leur ce que tu as trouvé, et dis-leur ce qu'on doit faire. »

Vous avez fait le plus dur : l'analyse est terminée, vérifiée par plusieurs méthodes, et vous en êtes fier. Elle tient en quarante pages de résultats, dont une régression, un contrefactuel de marge, une simulation et un seuil de bascule. Et c'est ici que beaucoup d'analystes **perdent** ce qu'ils ont gagné : ils présentent **ce qu'ils ont fait** au lieu de ce que **l'autre doit en retenir**, ils parlent de la méthode alors qu'on leur demande une décision, ou ils disent « il y a une incertitude » d'un air gêné alors qu'on attend « voici ce qu'il faut faire, et voici la marge d'erreur ».

Ce chapitre ne contient presque pas de calcul : il traite de la **parole**, de l'**écoute** et de la **préparation**. Les deux premières sections suivent votre semaine de préparation ; les deux dernières, facultatives, regardent en amont (comprendre ce qu'on vous demande) et plus largement (négocier, collaborer, rester honnête).

Le chemin de ce chapitre

- **5.1 Comprendre son public.** Qui est dans la salle, ce que chacun sait, veut et craint ; comment dire **le même résultat** à trois personnes différentes ; comment traduire le jargon en phrases claires ; comment rendre un support lisible (daltonisme, projection) ; ce qui change à distance.
- **5.2 Présenter résultats et recommandations.** Une présentation de dix minutes : la **réponse d'abord**, trois preuves, une recommandation que l'on peut exécuter, la décision demandée ; comment parler de l'**incertitude** sans perdre la salle ; comment traiter les questions, les objections, le chiffre qui déplaît et l'erreur découverte en séance ; le compte rendu en cinq lignes.
- **5.3 ☉ Recueil des besoins, entretiens avec les parties prenantes, formulation de la question métier.** De « je veux un tableau de bord » à une question que l'on peut tester ; l'entretien structuré ; la fiche de cadrage.
- **5.4 ☉ Compétences transversales.** Raconter, négocier (dire non sans fermer la porte), collaborer avec les équipes métier, gérer un désaccord sur un chiffre, éthique professionnelle, retours d'expérience, développement de carrière.

Intuition. Présenter, c'est **traduire** : de votre langue (la méthode, l'incertitude, les hypothèses) vers celle de l'autre (la décision, le risque, l'argent, le temps). Une bonne traduction ne perd rien d'important, mais elle ne garde que ce qui est nécessaire pour décider.

Les données du chapitre

Ce chapitre ne produit pas de nouvelle analyse : il **présente** celles des volumes précédents. Les nombres qui servent d'exemples viennent de la boutique simulée (volume III) : l'effet des promotions sur les commandes et sur la marge (volume III, projet du volume), les retards de livraison par transporteur et par mois (volume III, chapitre 11), la conversion du site par source (volume III, chapitre 10) et le budget 2025 (volume III, chapitre 7). Tout est **simulé** ; la vérité programmée est celle des générateurs du volume III. Chaque nombre cité est recalculé par un bloc exécuté, souvent caché.

Pour les dialogues et les scénarios, les personnes sont désignées **par leur fonction** (la gérante, le responsable logistique, la comptable, le financeur) : aucun nom, aucune entreprise réelle n'apparaît.

5.1 Comprendre son public

Avant de préparer une seule diapositive, il faut savoir **à qui** l'on parle. La même analyse ne se présente pas de la même façon à la gérante, au responsable logistique ou à un banquier, parce qu'ils ne décident pas des mêmes choses, ne parlent pas la même langue et ne craignent pas les mêmes erreurs. Cette section donne une méthode simple : se poser trois questions, préparer **une phrase par personne**, traduire le jargon, soigner la lisibilité.

5.1.1 Qui est dans la salle ?

Dans presque toute réunion, on retrouve quatre rôles. Une même personne peut en cumuler deux, mais il est utile de les distinguer.

Le tableau suivant précise, pour chacun, ce qu'il sait, ce qu'il attend et ce qu'il craint. Il vaut pour la réunion de jeudi : la gérante décide, le responsable logistique utilisera les résultats, la comptable est l'experte des chiffres, et le représentant de la banque est le sceptique.

Rôle	Ce qu'il sait	Ce qu'il attend	Ce qu'il craint
Décideur (la gérante)	le métier, pas la statistique	une réponse, un ordre de grandeur, une action à décider	de décider sur un chiffre fragile
Expert (la comptable)	les chiffres de l'entreprise, parfois la méthode	les définitions, les sources, l'accord avec ses propres chiffres	un chiffre qui contredit ses comptes sans explication
Utilisateur (le responsable logistique)	son activité au quotidien	ce que cela change pour lui, dès demain	une consigne irréaliste ou non chiffrée
Sceptique (le représentant de la banque)	peu de détails, beaucoup de dossiers	une preuve, la solidité du résultat, le risque	d'être vendu un bon résultat qui ne tient pas

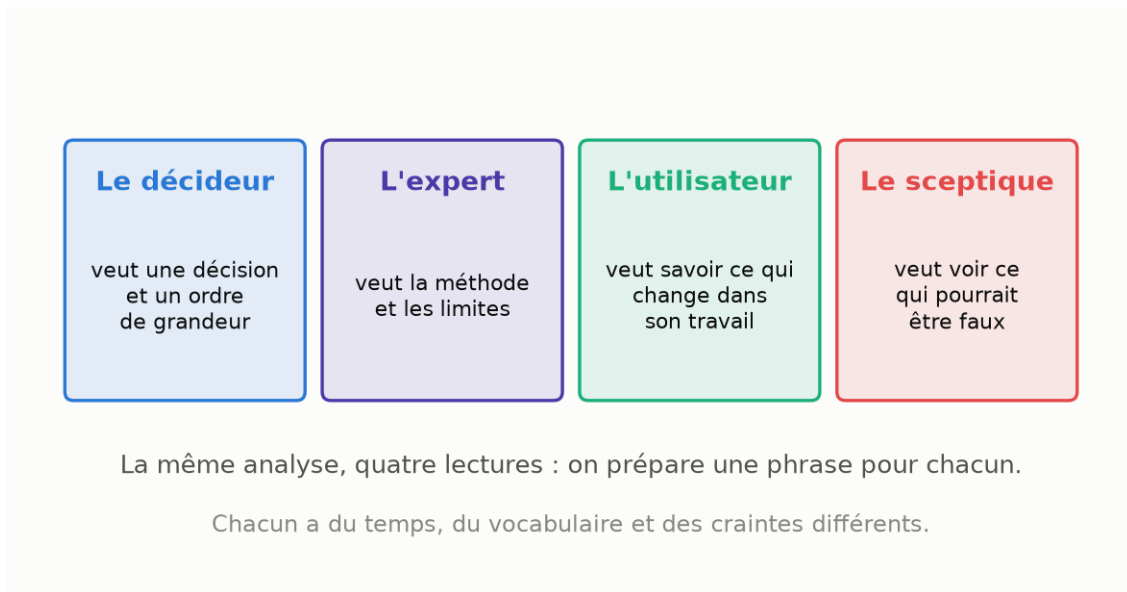


Figure 77: Quatre rôles dans une salle : le décideur veut une décision et un ordre de grandeur, l'expert la méthode et les limites, l'utilisateur ce qui change dans son travail, le sceptique ce qui pourrait être faux. Schéma dessiné avec matplotlib.

Piège. Préparer sa présentation **pour soi-même** (ou pour son meilleur collègue analyste). On y met tout ce que l'on a appris, dans l'ordre où on l'a appris. Or l'ordre de la découverte n'est presque jamais l'ordre de la compréhension.

5.1.2 Trois questions avant de préparer quoi que ce soit

Posez-vous, et posez à la personne qui vous invite, ces trois questions :

1. **Qui décide, et de quoi ?** Une présentation sans décision à prendre n'est qu'une information ; une présentation avec une décision a une direction.
2. **Que fera la personne de votre réponse ?** Si la gérante doit signer le plan des soldes de l'an prochain, il faut un chiffre sur la marge et une recommandation ; si c'est le responsable logistique qui prépare le personnel, il faut un nombre de commandes par jour.
3. **Combien de temps, et quel niveau de détail ?** Dix minutes avec questions ne laissent pas la place à une démonstration de régression.

Un échange de dix minutes avec la gérante, avant de préparer, change tout :

L'analyste. Pour jeudi, qu'attendez-vous exactement de moi ? **La gérante.** Que je sache si je reconduis les soldes d'hiver comme cette année. **L'analyste.** Et si la réponse est « pas tels quels », vous voulez une alternative ou seulement le diagnostic ? **La gérante.** Une alternative. Mais si je dois baisser les remises, je dois savoir de combien. **L'analyste.** D'accord. Et le représentant de la banque, qu'attend-il ? **La gérante.** Que je ne perde pas d'argent sur une opération que je lui ai présentée comme un succès.

En trois minutes, vous avez appris que la **décision** est « reconduire ou non, et à quelle profondeur de remise », que la salle comprend un **sceptique** avec un enjeu, et que le sujet sensible est la **marge** : la présentation se construira autour de cela, pas autour de la régression.

5.1.3 Un résultat, trois publics

Voici le résultat de l'analyse des soldes, tel que le donnent les volumes précédents : à jours comparables, les soldes ajoutent environ **19 %** de commandes (intervalle plausible : de 14 à 25 %), mais la marge par commande tombe de **32,1 €** à **23,6 €** à cause des remises ; au total, la marge des jours de soldes est inférieure d'environ **17 900 €** à ce qu'elle aurait été sans soldes ; il aurait fallu environ **36 %** de commandes supplémentaires pour ne rien perdre, et dans 99 % des simulations, les soldes font perdre de la marge.

C'est un seul résultat. On le dit pourtant **trois fois différemment**.

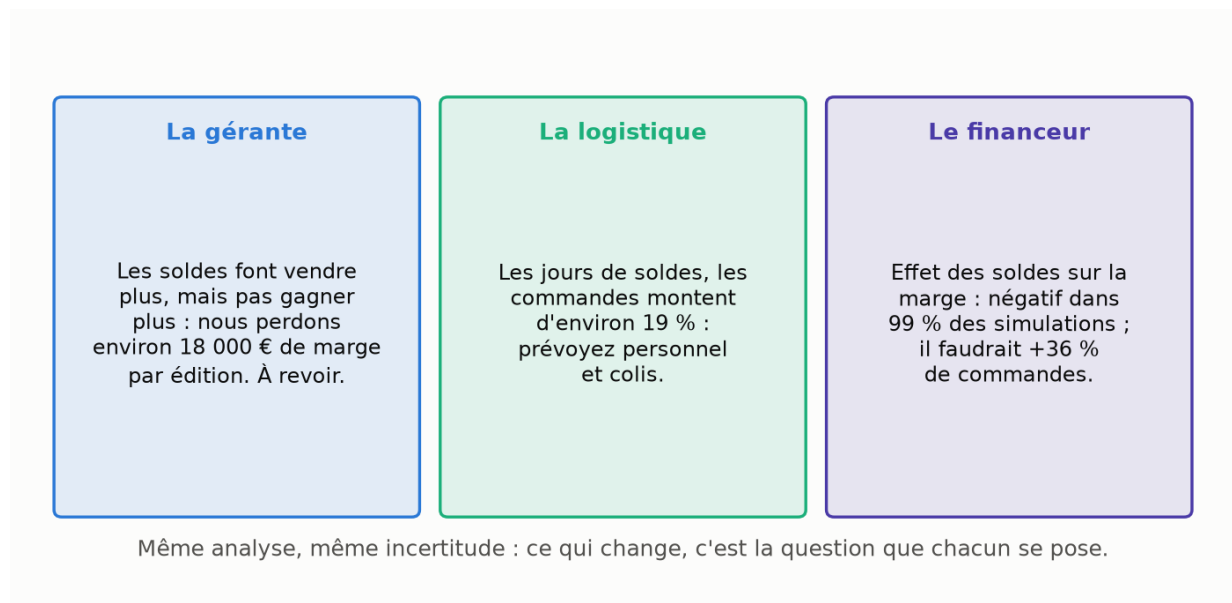


Figure 78: Le même résultat dit à la gérante (la marge perdue), au responsable logistique (la hausse du nombre de commandes) et au financeur (la proportion de simulations négatives et le seuil de rentabilité). Schéma dessiné avec matplotlib.

- **À la gérante** : « Les soldes font vendre plus, mais pas gagner plus : nous perdons environ 18 000 € de marge par édition. À revoir. » C'est une **décision** et un **montant**.
- **Au responsable logistique** : « Les jours de soldes, les commandes montent d'environ 19 % : prévoyez le personnel et les colis. » C'est une **conséquence opérationnelle** ; la marge ne l'intéresse pas pour ce qu'il doit faire.
- **Au financeur** : « L'effet des soldes sur la marge est négatif dans 99 % des simulations ; il faudrait 36 % de commandes en plus pour ne pas perdre d'argent. » C'est une **mesure de risque** et un **seuil**.

Aucune de ces phrases ne ment, aucune n'est complète. L'art consiste à choisir **celle qui répond à la question que la personne se pose**, en gardant les autres en réserve pour les questions.

5.1.4 Le jargon, traduit

Le jargon d'un analyste est un raccourci entre pairs ; devant un public non technique, c'est un mur. Le tableau suivant propose des **traductions** que vous pouvez adapter. La règle générale : dire **ce que cela veut dire pour la décision**, pas le nom de la méthode.

Jargon	Phrase claire
« La p-valeur est de 0,02. »	« Si les soldes n'avaient aucun effet, on verrait un écart aussi grand dans environ 2 cas sur 100 : l'écart est très probablement réel. »
« Intervalle de confiance à 95 % de 14 à 25 %. »	« Nous sommes presque certains que l'effet est entre 14 et 25 % ; notre meilleure estimation est 19 %. »
« Corrélation de 0,53. »	« Quand la publicité monte, les commandes montent aussi, mais surtout parce que les deux montent en fin d'année : cela ne prouve pas que la publicité les fait monter. »
« Toutes choses égales par ailleurs. »	« En comparant des jours de la même saison, du même jour de la semaine. »
« Régression. »	« Un calcul qui sépare l'effet de chaque facteur (soldes, saison, météo) pour ne pas attribuer aux soldes ce qui vient de la saison. »
« Écart-type. »	« L'écart habituel entre un jour et un jour moyen. »
« Médiane. »	« La valeur du milieu : la moitié des commandes est en dessous, la moitié au-dessus. »
« Significatif. »	« Assez net pour qu'on ne l'attribue pas au hasard » (et, séparément : « assez grand pour compter ? »).
« Contrefactuel. »	« Ce qui se serait passé sans les soldes ; on ne peut pas l'observer, on l'estime. »
« Cohorte. »	« Les clients qui nous ont rejoints le même mois. »
« Taux de conversion de 4,8 %. »	« Environ 5 visites sur 100 se terminent par une commande. »
« Saisonnalité. »	« Les mêmes hauts et bas qui reviennent chaque année (le creux de janvier, le pic de décembre). »
« Intervalle de prévision. »	« Une fourchette dans laquelle nous attendons les ventes, 4 fois sur 5. »

Intuition. Une bonne traduction ne remplace pas la rigueur, elle la **déplace** : au lieu d'annoncer la méthode, on garantit la phrase. Gardez la définition exacte à portée de main (une diapositive en annexe) pour l'expert qui la demandera.

5.1.5 La culture des chiffres : points, pour cent, pour mille

Le piège le plus fréquent n'est pas le jargon, c'est le **chiffre** lui-même : un pourcentage sans base, une variation sans point de départ, une probabilité mal lue. Quelques règles.

Points ou pour cent ? Entre l'e-mail (8,7 % de conversion) et les réseaux sociaux (2,2 %), l'écart est de **6,5 points** ; en relatif, l'e-mail convertit **quatre fois plus**. Les deux phrases sont exactes, elles ne disent pas la même chose, et un auditoire qui entend « 6,5 % » croit à une petite différence.

Les petits taux se disent « sur 100 » ou « sur 1 000 ». Un taux de conversion global de 4,78 % se dit « environ 48 commandes pour 1 000 visites », ou « une visite sur vingt et une ». Un taux de retard de livraison de 26,6 % se dit « environ un colis sur quatre ». Le cerveau retient mieux **une personne sur quatre** qu'un pourcentage.

Donnez toujours la base de comparaison. « 55,5 % des colis sont en retard » ne dit rien ; « en décembre, plus d'un colis sur deux est en retard, contre un peu plus d'un sur cinq le reste de l'année » dit **ce qui a changé**.

Arrondissez, mais au bon endroit. Une présentation de direction n'a pas besoin de 17 884 € : « environ 18 000 € » suffit, et dit en passant que le chiffre est une estimation. En revanche, ne mélangez pas des arrondis différents dans la même phrase.

Un ordre de grandeur vaut mieux qu'une précision fausse. « 19,2 % » promet une précision que l'intervalle (de 13,5 à 25,1 %) dément ; « environ un cinquième » ou « entre 14 et 25 % » est plus honnête.

5.1.6 Rendre le support lisible : daltonisme, contraste, projection

Un résultat que l'on ne **voit** pas ne se retient pas. Trois contraintes pratiques.

Le daltonisme. Environ un homme sur douze (et beaucoup moins de femmes) confond certaines couleurs, surtout le rouge et le vert, mais aussi, selon le cas, l'orange, le rouge et le vert olive. La palette du livre se comporte ainsi pour trois formes de daltonisme.

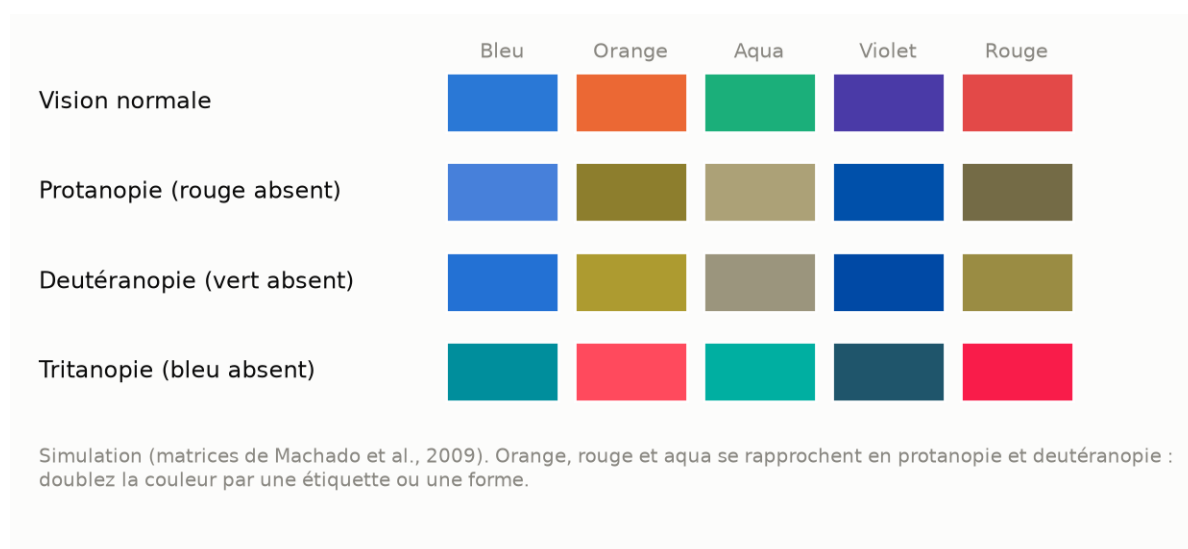


Figure 79: Les cinq couleurs de la palette du livre telles que les voit une vision normale, puis une protanopie, une deutéranopie et une tritanopie (simulation). Schéma calculé avec matplotlib.

On voit que l'orange, le rouge et l'aqua se rapprochent dans les deux premières formes : si votre graphique distingue des séries uniquement par ces couleurs, une partie de la salle ne les distingue pas. **Remède** : doubler la couleur par une **étiquette directe** (le nom de la série écrit au bout de la courbe), une **forme** ou un **motif**.

Le contraste. Un texte clair sur fond clair se lit mal, surtout à la projection. Le **rapport de contraste** entre deux couleurs (de 1 à 21) se calcule ; les recommandations d'accessibilité usuelles demandent au moins 4,5 pour le texte courant et 3 pour les grands textes et les éléments graphiques (à vérifier dans la version en vigueur du référentiel que vous suivez). Pour la palette du livre sur fond blanc :

couleur	sur blanc
bleu	4.4
orange	3.2
aqua	2.8
violet	8.6
rouge	4.0
gris du texte	7.9
gris muet	3.6

Le bleu (4,4) est juste sous le seuil du texte courant : on le réserve aux titres, aux traits et aux

grandes étiquettes ; l'orange et l'aqua (3,2 et 2,8) ne servent **jamais** à écrire en petit sur fond blanc ; le **texte courant** s'écrit en gris foncé (7,9) ou en noir.

La projection. Une salle, un écran partagé et des yeux fatigués : **18 points au minimum** pour le texte, **24 ou plus** pour les titres ; pas plus d'**une idée par diapositive** ; une figure par diapositive, avec un **titre qui est une phrase** (« Les soldes font perdre 18 000 € de marge », et non « Marge, soldes, 2025 »). Imprimez en noir et blanc pour vérifier que la lecture tient sans la couleur.

5.1.7 Présenter à distance

La visioconférence change trois choses. L'attention est plus **fragile** : annoncez au début le plan et la durée, découpez en blocs de trois minutes, posez une question au milieu. Le **retour visuel** disparaît : vous ne voyez plus les visages, donc demandez explicitement « est-ce clair jusqu'ici ? » et laissez un silence. Le **partage d'écran** est trompeur : la qualité varie, les petites polices disparaissent, et une figure à dix éléments devient illisible : réduisez, grossissez, et envoyez le support **avant** la réunion. Enfin, prévoyez une **deuxième voie** (le support envoyé par message) au cas où l'image ou le son tomberait.

À retenir. Avant de présenter, posez trois questions (qui décide, que fera-t-il de la réponse, de combien de temps dispose-t-on) ; préparez **une phrase par personne** ; traduisez le jargon ; donnez toujours une base de comparaison ; vérifiez la lisibilité (couleurs doublées, contraste, 18 points au minimum).

Pour s'entraîner. Cahier, chapitre 5 : applications 5.1 à 5.3, exercices 5.1 à 5.4.

5.2 Présenter résultats et recommandations

Vous savez à qui vous parlez ; reste à construire dix minutes qui **servent** la décision. Cette section suit la préparation de la réunion de jeudi : l'ordre des idées, la façon de parler de l'incertitude, la rédaction d'une recommandation qu'on peut exécuter, puis tout ce qui arrive **pendant** (questions, objections, mauvaise nouvelle, erreur découverte en séance) et **après** (le compte rendu).

5.2.1 La réponse d'abord

Dans une analyse, on part des données, on teste, on conclut. Dans une présentation, on **inverse** : on commence par la conclusion, puis on donne les preuves, comme dans un article de journal. La raison est simple : une personne qui a dix minutes et trois autres sujets en tête ne lira pas jusqu'au bout si la réponse arrive à la fin.

Comparez deux ouvertures pour la même réunion :

Ouverture de l'analyste (ordre de la découverte). « J'ai commencé par regarder les données de commandes, j'ai constaté une forte saisonnalité, j'ai donc construit un modèle de régression avec des variables de mois et de jour de la semaine... »

Ouverture de la réponse d'abord. « Reconduire les soldes tels quels ferait perdre environ 18 000 € de marge. Voici pourquoi en trois chiffres, puis ce que je propose. »

La seconde ne cache rien (la méthode est dans l'annexe), mais elle place la décision au centre et **donne envie d'écouter la suite**. Un bon test : si votre auditoire s'en va après la première phrase, a-t-il quand même la réponse ?

Intuition. Le titre de chaque diapositive est une phrase qui énonce la conclusion de la diapositive, pas le sujet. « Marge, soldes, 2025 » est un sujet ; « Les soldes font perdre 18 000 € de marge » est une conclusion. En lisant seulement les titres, on doit retrouver l'histoire.

5.2.2 Dix minutes, trois preuves

Dix minutes ne se découpent pas au hasard. Voici un plan éprouvé, en sept blocs.

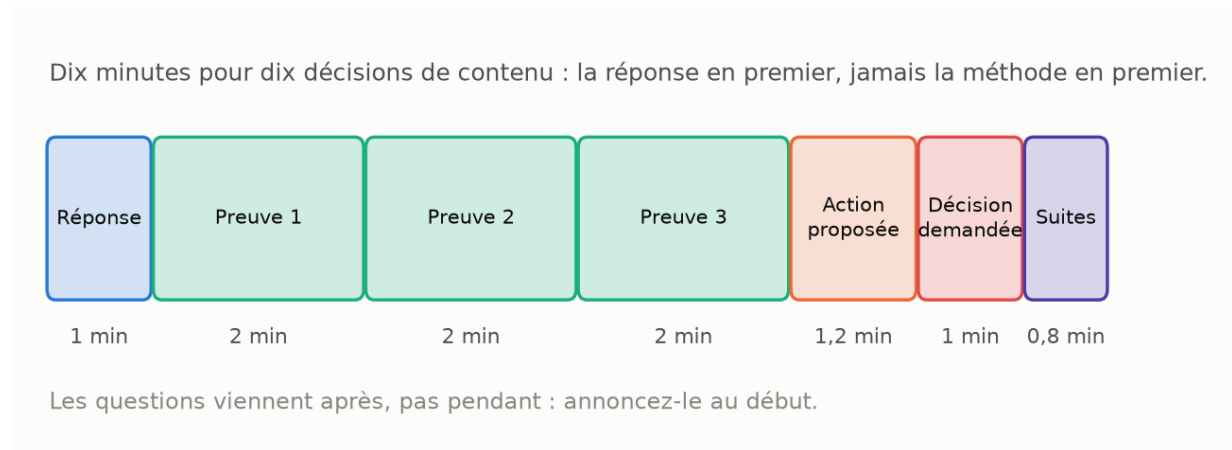


Figure 80: Une présentation de dix minutes en sept blocs : réponse (1 minute), trois preuves (2 minutes chacune), action proposée (1,2 minute), décision demandée (1 minute) et suites (0,8 minute). Schéma dessiné avec matplotlib.

Pour l'exemple des soldes, voici le **squelette** que vous présenteriez, avec un titre-phrase par bloc :

1. **Réponse** : « Reconduire les soldes tels quels ferait perdre environ 18 000 € de marge. »
2. **Preuve 1** : « À jours comparables, les soldes ajoutent environ 19 % de commandes. » (*le succès apparent : on le reconnaît*)
3. **Preuve 2** : « Mais chaque commande rapporte plus d'un quart de marge en moins. » (*32,1 € contre 23,6 € : les remises*)
4. **Preuve 3** : « Il faudrait 36 % de commandes en plus pour ne rien perdre, et la perte se confirme dans 99 % des simulations. » (*le seuil et la robustesse*)
5. **Action proposée** : « Réduire la profondeur des remises, concentrer l'opération, tester la prochaine édition. »
6. **Décision demandée** : « Je vous demande d'approuver le test de la prochaine édition avant le 15 novembre. »
7. **Suites** : « Le test dure environ deux mois ; je vous remets les résultats à telle date. »

Trois idées sont à retenir. **Trois preuves, pas dix** : le cerveau retient trois points, pas sept ; choisissez les trois qui répondent au sceptique. **Une idée par bloc**, chacune appuyée sur **un seul graphique** lisible en cinq secondes. Et **une décision demandée explicite** : « pour information » est une phrase de réunion qui ne mène à rien.

Voici la **preuve centrale**, telle qu'on la montrerait à l'écran : deux barres, un écart, un montant.

Le graphique ne montre **qu'un** message, son titre le dit, la perte est annotée en rouge, et la mention « estimé » rappelle honnêtement que le contrefactuel n'est pas observé.

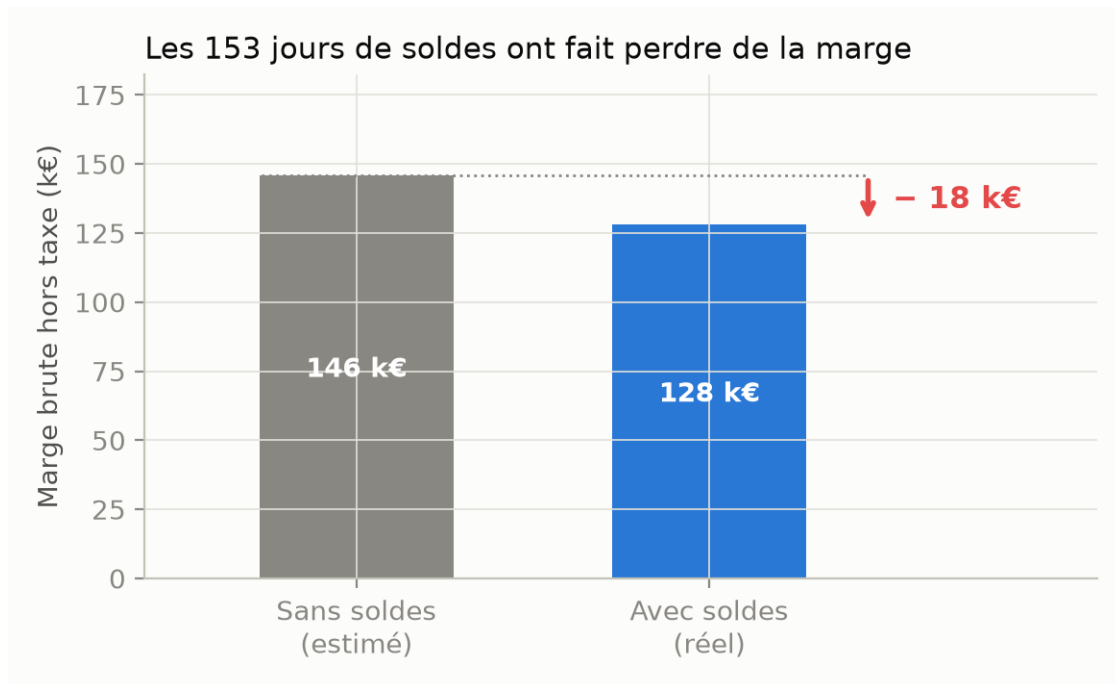


Figure 81: La marge brute hors taxe des 153 jours de soldes (128 k€, réel) comparée à celle qu'ils auraient dégagée sans soldes (146 k€, estimé) : une perte d'environ 18 k€. Graphique matplotlib.

Piège. Mettre la **méthode** en preuve 1. La méthode n'est pas une preuve pour un décideur ; c'est une **garantie** que l'on garde en annexe pour l'expert et le sceptique. Dites « j'ai comparé des jours de la même saison et du même jour de la semaine » en une phrase, pas en une diapositive.

5.2.3 Parler d'incertitude sans perdre la salle

Dire l'incertitude est une obligation d'honnêteté ; la dire mal fait croire que l'on ne sait rien. Deux principes.

Séparez ce que l'on sait de ce que l'on ignore. Une formule utile : « **nous sommes sûrs de la direction, moins de l'ampleur** ». Ici : les soldes font perdre de la marge (la direction : 99 % des simulations), pour un montant compris entre 3 400 et 32 500 € (l'ampleur : une fourchette large). Cela donne à l'auditoire de quoi décider (« il ne faut pas reconduire tel quel ») sans lui faire croire à un chiffre exact.

Choisissez la représentation à la mesure de la salle. Le même effet des soldes sur les commandes, montré de trois façons :

- **Un chiffre seul** (19 %) est clair mais promet une précision que l'on n'a pas.
- **Une fourchette** (de 14 à 25 %) dit la prudence sans noyer.
- **Une fourchette face à un seuil** (36 %) est la plus utile à un décideur : elle répond à la vraie question « est-ce que cela suffit ? ». Même la borne haute de la fourchette (25 %) reste loin du seuil : la conclusion ne dépend pas de l'incertitude.

Quelques **formulations** à adopter et à éviter :

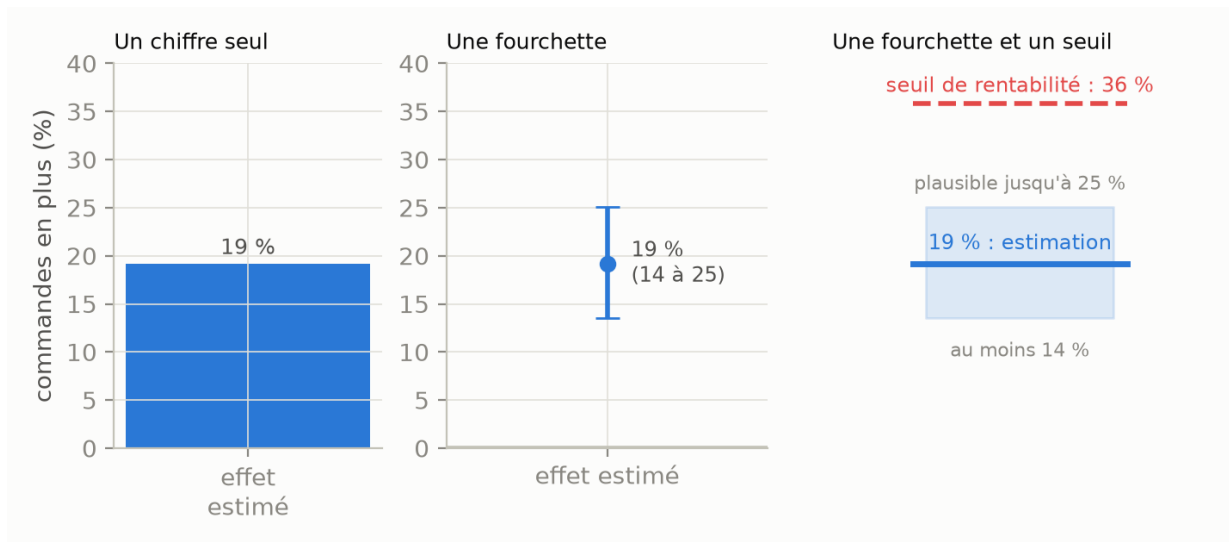


Figure 82: Trois manières de montrer l'effet des soldes sur les commandes : un chiffre seul (19 %), une fourchette (de 14 à 25 %), une fourchette comparée au seuil de rentabilité (36 %). Graphique matplotlib.

À éviter	À dire
« Il y a une incertitude. » (flou, inquiétant)	« Notre meilleure estimation est 19 %, et c'est très probablement entre 14 et 25 %. »
« Le résultat est significatif. » (jargon)	« L'écart est assez net pour que nous ne l'attribuons pas au hasard. »
« On ne peut rien conclure. » (exagéré)	« Nous ne pouvons pas dire si c'est +1 % ou +5 % ; nous savons que c'est inférieur à 10 %. »
« C'est sûr à 99 %. » (confond les deux)	« Dans 99 simulations sur 100, le résultat est une perte ; l'estimation reste incertaine sur son ampleur. »

Enfin, **annoncez ce que vous n'avez pas mesuré** : ici, la valeur à long terme des clients acquis pendant les soldes. L'honnêteté sur les limites **renforce** la confiance dans le reste.

5.2.4 Une recommandation qu'on peut exécuter

Une recommandation du type « il faudrait revoir la politique de soldes » ne fait rien faire à personne. Une bonne recommandation répond à **cinq questions** : **qui** fait, **quoi**, **quand**, **combien** (coût, effet attendu) et **comment on saura** si cela a marché.

Question	Réponse pour les soldes
Qui ?	La gérante décide ; le responsable des achats choisit les produits ; vous mesurez.
Quoi ?	Pour la prochaine édition, remise à 10 % au lieu de 20 % sur la moitié des produits (tirés au hasard), 20 % sur l'autre moitié, pour mesurer l'effet réel de la profondeur de la remise.
Quand ?	Décision avant le 15 novembre pour le Vendredi noir ; résultats deux semaines après la fin de l'opération.

Question	Réponse pour les soldes
Combien ?	Coût de l'expérience : une partie des ventes à marge réduite ; risque borné par la moitié des produits.
Comment saura-t-on ?	Marge par commande et nombre de commandes, groupe contre groupe, sur toute la durée ; critère fixé à l'avance : on garde la remise la plus faible si la marge totale n'est pas inférieure.

Cette proposition n'est pas arbitraire : elle découle du fait que **l'effet des soldes a été estimé sans randomisation** (comparaison de jours) ; une expérience tranchera. Elle a aussi un **coût d'incertitude** que l'on chiffre : pour détecter +10 % de commandes avec des jours, il faut environ 57 jours par groupe (volume III, section 2.5), ce qui est long ; en randomisant par **produit** plutôt que par jour, on obtient beaucoup plus de comparaisons en une seule édition.

Piège. Une recommandation que **vous** ne pouvez pas exécuter, vous ne devez pas l'imposer. Dites-le : « cela demande une décision de la gérante », « cela dépend du responsable logistique ». Une recommandation sans propriétaire est une opinion.

5.2.5 Les questions et les objections

Les questions sont **la partie la plus utile** de la réunion : elles montrent ce que l'auditoire n'a pas compris, ou ne croit pas. Préparez-les comme vous préparez la présentation. Voici les objections les plus probables après l'exposé sur les soldes, et des **réponses types**.

Objection	Réponse type
« Les soldes attirent de nouveaux clients qui reviendront. »	« C'est possible, et ce n'est pas mesuré ici : mon chiffre ne compte que la marge des jours de soldes. Je peux mesurer combien des clients acquis pendant les soldes rachètent à six mois. »
« Et si l'effet sur les commandes était plus fort que vous ne le dites ? »	« Même au bord haut de la fourchette (+25 %), les soldes perdent encore environ 11 000 € ; il faudrait +36 % pour ne rien perdre. »
« La comptable trouve un autre chiffre. »	« Je vérifie les définitions avec elle : hors taxe ou toutes taxes comprises, remises déduites ou non, même période. Je reviens vers vous demain avec la réconciliation. »
« On a toujours fait des soldes. »	« Je ne dis pas d'arrêter, je dis de changer la profondeur et de la tester ; la marge des soldes actuelles est inférieure à celle des jours ordinaires. »
« Je ne comprends pas votre méthode. »	« J'ai comparé des jours de soldes à des jours sans soldes de la même saison et du même jour de la semaine , pour ne pas confondre soldes et saison. »
« Pourquoi ne pas simplement comparer à l'an dernier ? »	« L'an dernier, les promotions tombaient à des dates voisines : on ne sépare pas l'effet des soldes de la tendance. J'ai contrôlé la tendance et la saison. »

Deux règles. **Une réponse courte** (deux phrases) puis on s'arrête : le silence est un outil. Et « **je ne sais pas, je vérifie** » est une réponse **professionnelle** : « Je ne sais pas, je vérifie et je vous

réponds demain » vaut mieux qu'une improvisation qu'il faudra corriger. Notez la question, la date et le nom de la personne.

5.2.6 Le chiffre qui déplaît

La gérante tenait les soldes pour un succès ; le chiffre dit le contraire. Quelques principes.

- **Ne personnalisez pas.** « Les soldes font perdre de la marge » parle d'une **opération**, pas d'une décision de la gérante. N'écrivez jamais « vous avez eu tort ».
- **Reconnaissez ce qui est vrai dans le point de vue adverse.** Les soldes **ont** augmenté les commandes de 19 %, comme on le croyait : c'est la marge qui pose problème.
- **Présentez toujours une issue.** Un mauvais chiffre sans option est une accusation ; avec un test à faire, c'est un plan.
- **Ne cachez pas, ne dramatisez pas.** Le même ton pour les bonnes et les mauvaises nouvelles, les mêmes chiffres à la même place.

Pour la logistique, l'exercice est le même avec des chiffres moins agréables au responsable logistique : plus d'un colis sur deux est en retard en décembre (55,5 %), contre un peu plus d'un sur cinq le reste de l'année (21,6 %) ; un transporteur concentre le problème (51 % de retards, contre 16 % pour le plus fiable) et 4,1 % de colis abîmés (contre 0,9 %). On le dit sans chercher de coupable : « le transporteur C est trois fois plus souvent en retard ; voici trois options, avec leur coût. »

5.2.7 L'erreur découverte en séance

Cela arrive : en pleine réunion, la comptable remarque que la marge que vous citez est **hors taxe**, alors que le chiffre d'affaires projeté est toutes taxes comprises. Ou vous vous apercevez vous-même que vous avez montré la mauvaise figure. Quatre gestes.

1. **Arrêtez-vous et dites-le simplement** : « Vous avez raison, il y a un mélange entre hors taxe et toutes taxes comprises sur cette ligne. »
2. **Évaluez l'impact sans inventer** : « Je vérifie si cela change la conclusion. » Si vous pouvez le faire en direct (un calcul simple), faites-le ; sinon dites quand vous répondrez.
3. **Ne vous excusez pas à l'excès** : une phrase, puis on avance. Trois excuses font plus de mal que l'erreur.
4. **Corrigez par écrit après la réunion** : envoyez le chiffre corrigé, ce qui a changé, et ce qui n'a pas changé (souvent, la conclusion).

La confiance se perd quand on **cache** une erreur, pas quand on la corrige proprement. Un analyste qui annonce ses propres erreurs est un analyste à qui l'on croit le reste du temps.

5.2.8 Supports, répétition, chronométrage, suivi

Quel support ? Selon la situation :

Support	Quand	Précaution
Une page (synthèse de direction)	décision à prendre, lecteurs pressés, trace écrite	titre-réponse, trois chiffres, une figure, la décision demandée ; voir la section 4.3
Quelques diapositives	réunion, discussion en direct	une idée par diapositive, titre-phrase, annexe pour la méthode

Support	Quand	Précaution
Démonstration (un tableau de bord, un outil)	utilisateurs qui vont s'en servir	script et données de démonstration figés, plan B si cela plante

Répétez, et chronométrez. Une présentation de dix minutes se parle en dix minutes **en vrai**. On parle à un rythme d'environ 130 mots par minute (variable selon les personnes) : un texte de 1 300 mots tient à peu près dans le temps. Un script court permet de le vérifier.

```
script = " ".join(["mot"] * 1150)          # remplacez par votre texte
minutes = len(script.split()) / 130
print(f"{len(script.split())} mots : environ {minutes:.1f} minutes")
```

```
1150 mots : environ 8.8 minutes
```

Répétez à voix haute devant une personne qui ne connaît pas le sujet : si elle ne peut pas répéter votre conclusion, la présentation est à refaire. Prévoyez **deux minutes de marge** pour les imprévus.

Après la réunion : le compte rendu en cinq lignes. Envoyez, le jour même, un message court : la **décision prise, qui fait quoi, pour quand**, comment on **mesurera**, et ce qui reste **ouvert**. Exemple :

Objet : soldes d'hiver, décisions du jeudi.

1. Décision : on ne reconduit pas les soldes tels quels ; on teste une remise à 10 % sur la moitié des produits.
2. Responsable : la gérante valide la liste de produits d'ici le 15 novembre.
3. Mesure : marge par commande et nombre de commandes, groupe contre groupe ; critère fixé avant l'opération.
4. Résultats : présentés deux semaines après la fin de l'opération.
5. Ouvert : valeur à long terme des clients acquis en soldes (à mesurer à six mois).

À retenir. Commencez par la réponse ; trois preuves, pas dix ; une idée et un graphique par bloc ; l'incertitude se dit en séparant la direction de l'ampleur et en la comparant à un seuil ; une recommandation répond à qui, quoi, quand, combien, comment mesurer ; « je ne sais pas, je vérifie » est une bonne réponse ; une erreur avouée vaut mieux qu'une erreur cachée ; terminez par un compte rendu en cinq lignes.

Pour s'entraîner. Cahier, chapitre 5 : applications 5.4 à 5.5, exercices 5.5 à 5.9.

5.3 ⊕ Pour aller plus loin : recueil des besoins, entretiens avec les parties prenantes, formulation de la question métier

Section complémentaire. Elle remonte **en amont** de la présentation : avant de présenter une réponse, il faut avoir compris la **question**. C'est là que se jouent la moitié des échecs d'un projet d'analyse : on répond très bien à une question que personne n'avait vraiment posée.

La plupart des demandes arrivent sous forme de **solutions** (« je veux un tableau de bord », « fais-moi une segmentation », « il me faut un modèle ») ou de **symptômes** (« les ventes baissent »). Votre premier travail est de remonter au **besoin**, puis à la **décision**, puis à une **question que les données peuvent traiter**.

5.3.1 De « je veux un tableau de bord » à une question

Voici une demande réelle de la gérante : « *Je voudrais un tableau de bord pour mes stocks.* » Si vous y répondez à la lettre, vous livrerez des graphiques que personne ne regardera. Si vous posez trois questions, vous trouverez ce qu'il fallait vraiment faire.

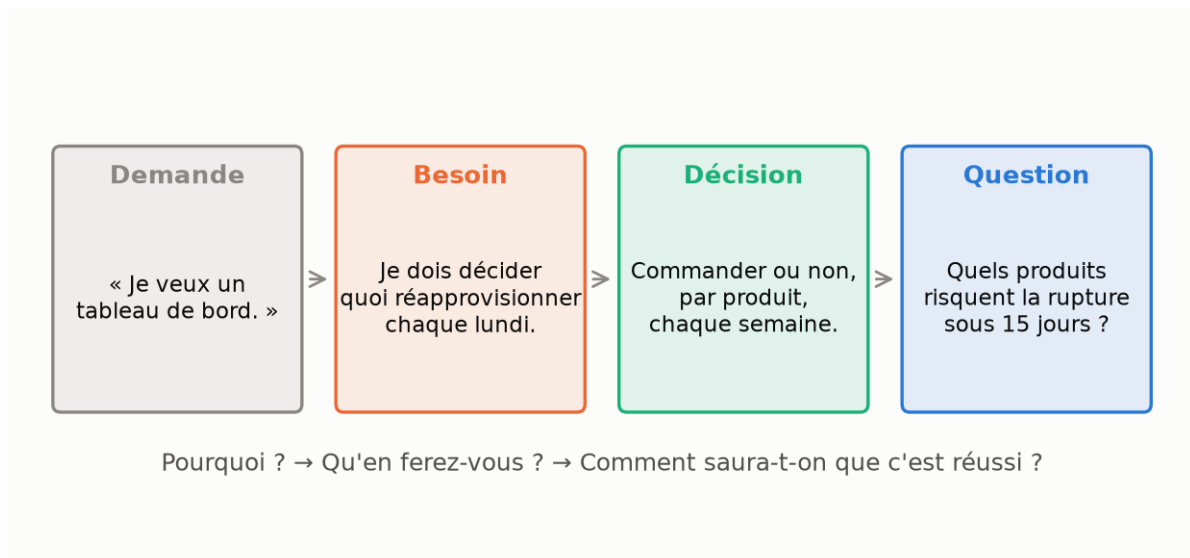


Figure 83: De la demande floue à la question : une demande (« un tableau de bord »), un besoin (réapprovisionner chaque lundi), une décision (commander ou non, par produit) et une question testable (quels produits risquent la rupture sous 15 jours ?). Schéma dessiné avec matplotlib.

Un échange court suffit :

L'analyste. Pourquoi voulez-vous ce tableau de bord ? **La gérante.** Parce que je me retrouve parfois en rupture sur un produit qui marchait bien. **L'analyste.** Qu'en feriez-vous, si vous le voyiez à temps ? **La gérante.** Je commanderais plus tôt. Tous les lundis, je décide quoi réapprovisionner. **L'analyste.** Et comment saurons-nous que c'est réussi ? **La gérante.** Si je n'ai plus de rupture sur mes vingt produits les plus vendus.

Les trois questions à retenir sont « **Pourquoi ?** » (le besoin), « **Qu'en feriez-vous ?** » (la décision) et « **Comment saura-t-on que c'est réussi ?** » (le critère de succès). Elles transforment une demande d'outil en une **question** : « *quels produits risquent la rupture dans les quinze jours ?* ». La réponse n'est peut-être même pas un tableau de bord : un message du lundi matin avec cinq produits à commander peut suffire.

5.3.2 L'entretien structuré

Un entretien de cadrage dure trente à quarante-cinq minutes ; il se prépare comme une réunion. Voici une trame, avec ce que chaque question cherche.

Question à poser	Ce qu'elle cherche
« Racontez-moi la dernière fois que ce problème est arrivé. »	un cas concret , plus fiable qu'une description générale
« Qu'est-ce qui vous a poussé à demander cela maintenant ? »	le déclencheur , donc l'urgence réelle
« Que ferez-vous de la réponse ? Quelle décision change selon le résultat ? »	la décision : sans décision, pas d'analyse utile
« Quel serait un bon résultat ? Un mauvais ? »	les critères de succès et les seuils
« Qui d'autre utilisera ou contestera ce résultat ? »	les parties prenantes
« Quelles données avez-vous déjà ? Quelles sont leurs limites ? »	la faisabilité (volume II)
« Pour quand en avez-vous besoin ? Qu'est-ce qui se passe si c'est en retard ? »	le délai réel, pas le délai affiché
« Qu'est-ce qui est hors sujet ? »	le périmètre

Quelques règles d'écoute. **Posez des questions ouvertes** (« comment », « pourquoi », « racontez-moi ») plutôt que fermées (« voulez-vous un graphique ? »). **Laissez des silences** : la personne ajoute souvent le plus important après un temps. **Reformulez** (« si je comprends bien, vous voulez... ») pour vérifier, et **demandez des exemples chiffrés** (« combien de produits ? combien de ruptures par mois ? »). **Ne proposez pas la solution pendant l'entretien** : écoutez d'abord.

Intuition. Le meilleur indicateur d'un bon entretien est que la personne dise « c'est vrai, je n'y avais pas pensé comme ça » : vous avez ajouté de la clarté, pas seulement recueilli une demande.

5.3.3 Cartographier les parties prenantes

Un projet d'analyse touche d'autres personnes que celle qui le demande. Une **carte pouvoir-intérêt** les classe selon deux axes : leur **pouvoir de décision** et leur **intérêt pour le sujet**.

On en tire une stratégie de communication : **associer étroitement** les acteurs à fort pouvoir et fort intérêt (ils valident le cadrage et reçoivent les brouillons) ; **tenir informés** ceux qui ont du pouvoir mais peu d'intérêt (un résumé court, à l'avance) ; **informer régulièrement** ceux qui sont très concernés mais décident peu (ils connaissent le terrain, ils vous donneront les contre-exemples) ; **surveiller** les autres. Cette carte est un outil de **préparation**, jamais un document à montrer.

5.3.4 Critères de succès, périmètre, délais, données

Le cadrage se termine par quatre vérifications, que l'on écrit.

- **Critères de succès** : comment saura-t-on que l'analyse a servi ? Par exemple « la décision est prise le jeudi » ou « le nombre de ruptures baisse ».
- **Périmètre** : ce qui est dedans (les vingt produits les plus vendus, 2025) et ce qui est **dehors** (les produits saisonniers, les autres canaux). Un périmètre non écrit grossit toujours.
- **Délais** : une date réelle et une date de **réunion** qui la justifie.
- **Données** : où sont-elles, sont-elles fiables, complètes, accessibles ? Un cadrage honnête dit « cette question ne peut pas être traitée avec les données actuelles » quand c'est le cas.

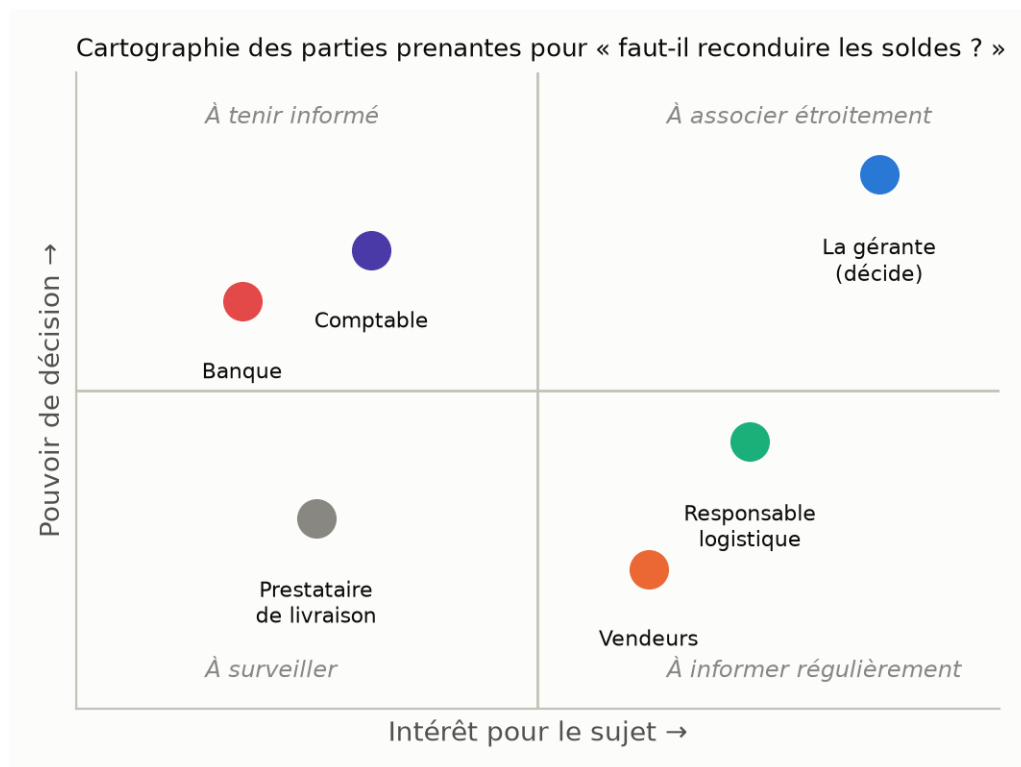


Figure 84: Cartographie des parties prenantes pour la question « faut-il reconduire les soldes ? » : la gérante (fort pouvoir, fort intérêt) à associer étroitement ; la comptable et la banque (fort pouvoir, intérêt moindre) à tenir informées ; le responsable logistique et les vendeurs (fort intérêt, pouvoir moindre) à informer régulièrement ; le prestataire de livraison à surveiller. Schéma dessiné avec matplotlib.

5.3.5 La fiche de cadrage

La fiche de cadrage tient sur une page ; c'est le **contrat** entre l'analyste et la personne qui demande. Voici celle d'une demande de la logistique : les clients se plaignent des retards de décembre.

Rubrique	Contenu
Demande d'origine	« Les clients se plaignent des livraisons tardives en décembre. Fais quelque chose. »
Décision à éclairer	Faut-il changer de transporteur, en ajouter un, ou avancer les dates limites de commande avant les fêtes ?
Question testable	Quelle part des retards de décembre est due au transporteur, et quelle part à la charge de fin d'année ?
Critère de succès	Une recommandation chiffrée avant la réunion du mois de septembre ; en décembre suivant, moins d'un colis sur trois en retard.
Périmètre	Commandes du Site et des Réseaux, 2023 à 2025 ; hors retraits en boutique.
Données	livraisons (une ligne par colis : dates, transporteur, mode, retard) ; limites : le délai promis est fixe (6 jours).
Parties prenantes	La gérante (décide), le responsable logistique (exécute), la comptable (coûts), le transporteur (informé après).
Délai	Résultats à la réunion de direction de septembre ; point d'étape dans trois semaines.
Livvable	Une page : réponse, trois chiffres, une recommandation ; annexe méthodologique.
Hors périmètre	Les retours de colis, les délais fournisseurs (autre analyse).

Avant de signer une telle fiche, vérifiez que les **données existent** et ont la forme annoncée. Un contrôle de quelques lignes suffit.

```
liv = pd.read_csv(os.path.join(D, "livraisons.csv"))
print(len(liv), "livraisons du", liv["date_commande"].min(), "au", liv["date_commande"].max())
print(sorted(liv["canal"].unique()), "| transporteurs :", liv["transporteur"].unique(), "| délai
↳ promis :", liv["delai_promis_j"].unique().tolist())
```

```
19420 livraisons du 2023-01-01 au 2025-12-31
['Réseaux', 'Site'] | transporteurs : 3 | délai promis : [6]
```

La fiche dit « 2023 à 2025, hors boutique, délai promis fixe » : les données le confirment. **Faites valider la fiche par écrit** (un message de quatre lignes suffit : « voici ce que j'ai compris ; si c'est exact, je lance »). Cette validation est votre meilleure protection contre le « ce n'est pas ce que je voulais » de la fin.

5.3.6 Trois pièges du recueil des besoins

La demande qui cache une autre demande. « Peux-tu me faire un graphique des ventes par ville ? » Derrière : « Je veux décider où ouvrir un point de retrait. » La première demande se livre en une heure ; la seconde exige une analyse. Demandez toujours « pour quoi faire ? ».

La solution déguisée en besoin. « Je veux un modèle de prévision. » Or la gérante veut surtout savoir combien commander en novembre ; une moyenne de l'an dernier majorée de 10 % lui suffit peut-être. Ne construisez pas un outil que vous n'avez pas justifié : volume III, section 5.2, sur les prévisions de référence.

Des besoins contradictoires. La comptable veut un chiffre d'affaires **hors taxe** ; le responsable des ventes veut **toutes taxes comprises** ; la gérante dit « un seul chiffre ». Ne tranchez pas seul : proposez **les deux** avec leurs libellés, expliquez la différence (ici, un facteur de 1,2) et demandez à la gérante de désigner **le chiffre de référence**, qui ira dans le dictionnaire.

À retenir. Une demande est un symptôme ; le besoin est derrière, et la décision derrière le besoin. Trois questions (pourquoi, qu'en ferez-vous, comment saura-t-on que c'est réussi), une cartographie des parties prenantes, une fiche de cadrage d'une page **validée par écrit** : voilà un cadrage. Si les données ne permettent pas de répondre, dites-le avant de commencer.

Pour s'entraîner. Cahier, chapitre 5 : application 5.6, exercices 5.10 et 5.11.

5.4 ⊕ Pour aller plus loin : compétences transversales

Section complémentaire. Ce qui distingue un analyste très bon techniquement d'un analyste **utile**, ce sont rarement les outils : ce sont des compétences de communication et de collaboration. Cette section en rassemble six : raconter, négocier, collaborer, gérer un désaccord sur un chiffre, rester honnête, et progresser.

5.4.1 Raconter

Un récit de données (chapitre 4 de ce volume) n'est pas réservé aux rapports écrits : à l'oral, **une histoire en trois phrases** suffit à fixer un résultat.

- **Situation** : « Les soldes d'hiver font monter les commandes chaque année. »
- **Complication** : « Mais ce que nous avons mesuré, c'est que chaque commande rapporte moins, et que le total perd de la marge. »
- **Résolution** : « Je propose de réduire la remise et de tester la prochaine édition. »

Ce schéma (situation, complication, résolution) fonctionne pour une réunion de dix minutes comme pour un message de cinq lignes : il donne le contexte que tout le monde partage, la **tension** qui justifie l'attention, et la **sortie**. Entraînez-vous à le dire **sans support** : si vous n'y arrivez pas, c'est que vous n'avez pas encore trouvé votre message.

5.4.2 Négocier : priorités, délais, périmètre

Les demandes d'analyse dépassent toujours le temps disponible. Négocier n'est pas dire non : c'est **choisir ensemble ce que l'on sacrifie**. Trois leviers existent, que l'on peut représenter par un triangle.

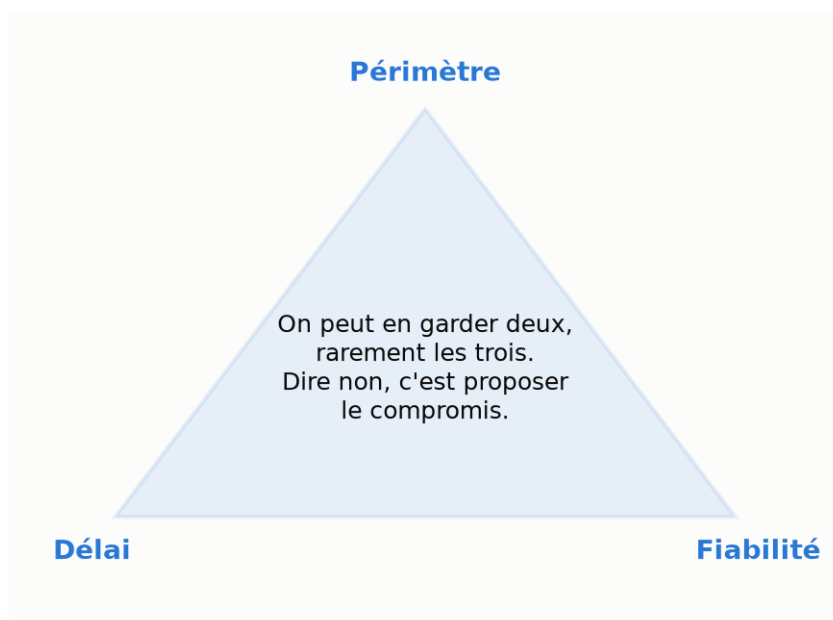


Figure 85: Le triangle périmètre, délai, fiabilité : on peut en garder deux, rarement les trois. Schéma dessiné avec matplotlib.

Si l'on vous demande **plus** (un périmètre plus large) **plus vite** (un délai plus court), la **fiabilité** en pâtit. Votre rôle est de **rendre visible** ce compromis. Voici comment **dire non sans fermer la porte** :

Situation	Réponse qui garde la relation
Un délai irréaliste : « pour demain »	« Pour demain, je peux vous donner un ordre de grandeur sur la base des chiffres de l'an dernier ; l'analyse complète, avec les intervalles, sera prête jeudi. Que préférez-vous ? »
Une demande de plus : « ajoute aussi les autres canaux »	« Je peux le faire ; cela repousse la livraison d'une semaine, ou je retire la comparaison saisonnière. Qu'est-ce qui compte le plus ? »
Un résultat souhaité : « j'aimerais que cela montre que les soldes marchent »	« Je regarde ce que disent les données ; si elles ne montrent pas cela, je vous le dirai, avec ce qui pourrait changer le résultat. »
Une demande hors de votre rôle	« Ce n'est pas mon rôle de trancher entre les deux options ; voici les chiffres de chacune pour que vous décidiez. »

La formule est toujours la même : **reconnaître la demande, énoncer le coût, proposer une alternative, demander un choix**. Une analyste qui dit « oui » à tout livre en retard et à moitié juste ; une qui dit « oui, si... » livre ce qu'elle a promis.

5.4.3 Collaborer avec les équipes métier

Votre travail dépend de personnes qui connaissent le terrain mieux que vous. Trois pratiques simplifient la collaboration.

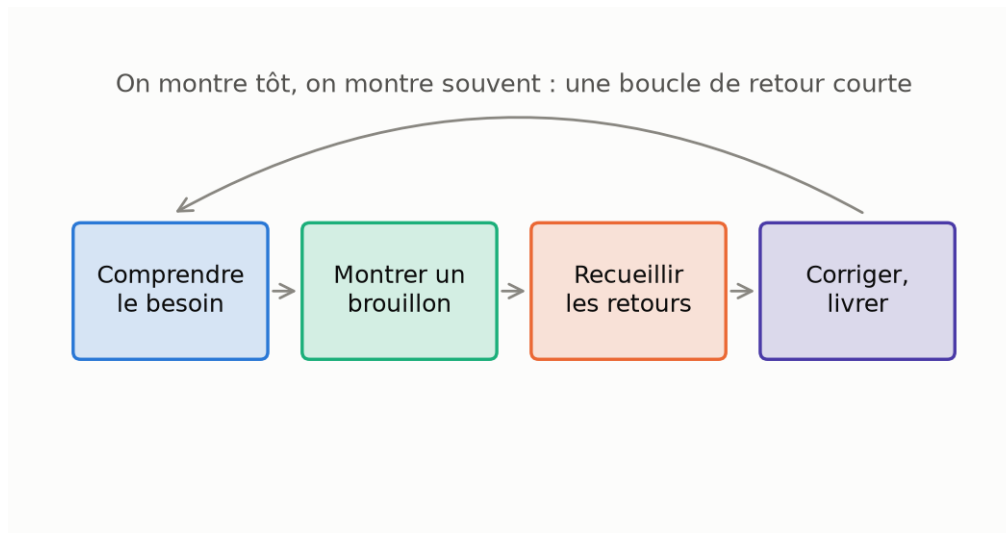


Figure 86: Une boucle de retour courte : comprendre le besoin, montrer un brouillon, recueillir les retours, corriger et livrer, puis recommencer. Schéma dessiné avec matplotlib.

Un langage commun. Tenez un **glossaire** partagé des termes ambigus : « client actif », « commande », « retard », « chiffre d'affaires ». Les dictionnaires du volume II (section 4.2) sont faits pour cela. Quand deux personnes utilisent le même mot pour deux choses, le désaccord sur les chiffres est assuré.

Une boucle de retour courte. Montrez un **brouillon** tôt : une table brute et un graphique vaut mieux qu'un rapport parfait livré au bout d'un mois. Un utilisateur découvre ce qu'il veut en voyant ce qu'il n'a pas demandé.

Des rituels légers. Un point de dix minutes par semaine, un canal de messages pour les questions, une liste des décisions prises : cela évite que les décisions se perdent dans les conversations.

5.4.4 Quand deux chiffres divergent

La situation la plus fréquente en entreprise : deux personnes citent deux chiffres pour la même chose. La **méthode** ne change pas, elle suit celle de la réconciliation du volume II (section 3.3) : (1) **mêmes définitions** ? (2) **mêmes périodes** ? (3) **mêmes données** ? (4) **expliquer l'écart** jusqu'à zéro. Quelques exemples de la boutique.

Un chiffre d'affaires de **1 324 764 €** pour la gérante, de **1 240 295 €** pour la comptable : l'écart de **84 469 €** (6,4 %) est exactement le montant des remboursements. Personne n'a fait d'erreur : on a **deux définitions** du « chiffre d'affaires » (brut ou net de retours). La solution est de **nommer** les deux (« CA brut », « CA net de retours »), d'indiquer lequel fait foi pour quelle décision, et de l'écrire dans le dictionnaire. Le même mécanisme explique bien d'autres désaccords : un « retard » est-il une livraison après la date promise ou après la date d'expédition annoncée ? Une « commande » inclut-elle les commandes annulées ?

Piège. Ne dites jamais « c'est votre chiffre qui est faux ». Dites « nos chiffres diffèrent ; cherchons pourquoi ». Un désaccord traité comme un problème commun se règle en une heure ; traité comme un procès, il dure des semaines.

5.4.5 Éthique professionnelle

L'analyste a un pouvoir discret : il choisit **ce qu'il montre**. Quelques principes, qui ne sont pas facultatifs.

- **Ne pas embellir.** La gérante espère que les soldes marchent ; si les données disent le contraire, vous le dites (avec tact : section 5.2.6). Choisir l'échelle, la période ou le graphique pour arranger le message est une faute professionnelle, pas un détail de style (voir le chapitre 1, sur les graphiques trompeurs).
- **Signaler les conflits d'intérêts.** Si vous avez un intérêt personnel dans le résultat (votre prime dépend de la hausse des ventes, par exemple), dites-le, et faites relire.
- **Respecter la confidentialité.** Les données de clients ou de collaborateurs (volume II, chapitre 5) ne sortent pas du cadre de l'analyse ; les petits groupes ne se publient pas ; les données de collaborateurs exigent encore plus de prudence (volume III, chapitre 12).
- **Dire ce que l'on ne sait pas**, et ce que l'on n'a pas mesuré : c'est un devoir, et c'est aussi ce qui fait votre crédibilité.
- **Ne pas laisser un chiffre faux circuler.** Si vous découvrez une erreur après la réunion, corrigez-la par écrit, même si personne ne l'a vue.

5.4.6 Donner et recevoir un retour

Un retour utile porte sur un **fait précis**, pas sur la personne : « le graphique de la diapositive 3 mélange hors taxe et toutes taxes comprises » plutôt que « tu es négligent ». Donner un retour : décrire, expliquer l'effet, proposer. Recevoir un retour : écouter sans se défendre, reformuler, remercier, puis décider ce qu'on en fait. Demandez-en : après chaque présentation, deux questions à une personne de confiance (« qu'est-ce qui était clair ? qu'est-ce qui t'a perdu ? ») valent dix heures de formation.

5.4.7 Votre développement, en une page

Un plan de progression tient sur une page. Trois colonnes suffisent : **technique** (un outil ou une méthode à approfondir ce trimestre), **métier** (un domaine à mieux comprendre : la logistique, la finance), **communication** (une compétence à travailler : présenter à l'oral, rédiger une page). Pour chacune : un **objectif mesurable**, un **moyen** (un cours, un projet, un mentor) et une **échéance**. Gardez un **dossier de réalisations** (volume VI) : chaque projet, avec la question, la méthode, le résultat et ce qu'il a changé. Relisez ce plan tous les trois mois.

À retenir. Une histoire en trois phrases (situation, complication, résolution) ; négocier, c'est choisir ensemble ce qu'on sacrifie ; un glossaire commun et une boucle de retour courte évitent les désaccords ; deux chiffres qui divergent ont presque toujours deux définitions ; ne jamais embellir ; un retour porte sur un fait ; un plan de progression tient sur une page.

Pour s'entraîner. Cahier, chapitre 5 : application 5.7, exercices 5.12 et 5.13.

Bilan du chapitre 5

Vous savez maintenant :

- **identifier** les rôles d'une salle (décideur, expert, utilisateur, sceptique) et poser les **trois questions** qui orientent une présentation : qui décide, que fera-t-il de la réponse, de combien de temps dispose-t-on ;
- **dire le même résultat** à trois publics, **traduire le jargon** en phrases claires, **manier points, pourcentages et « sur 100 »** avec une base de comparaison ;
- **rendre un support lisible** : couleurs doublées (daltonisme), contraste calculé, 18 points au minimum, un titre qui est une phrase ;
- **construire dix minutes** : la réponse d'abord, trois preuves, une action, une décision demandée, des suites ;
- **parler d'incertitude** sans perdre la salle (la direction, puis l'ampleur, face à un seuil) ;
- **formuler une recommandation exécutable** (qui, quoi, quand, combien, comment mesurer) ;
- **répondre aux questions et aux objections**, annoncer **un chiffre qui déplaît**, corriger **une erreur découverte en séance**, conclure par un **compte rendu en cinq lignes** ;
- (en option) **remonter d'une demande floue à une question testable** par un entretien structuré, une carte des parties prenantes et une fiche de cadrage validée par écrit ;
- (en option) **raconter, négocier, collaborer**, régler un désaccord sur un chiffre par la méthode de réconciliation, rester honnête et progresser.

Le tableau suivant résume la **préparation d'une réunion** en cinq questions, que vous pouvez recopier.

Question	Votre réponse pour la réunion de jeudi
Qui décide, de quoi ?	La gérante : reconduire ou non les soldes, et à quelle profondeur.
Quelle phrase pour chacun ?	Gérante : 18 000 € de marge perdus ; logistique : 19 % de commandes en plus ; financeur : 99 % de simulations négatives, 36 % nécessaires.
Quelle est ma réponse en une phrase ?	« Reconduire les soldes tels quels ferait perdre environ 18 000 € de marge. »
Quelles trois preuves ?	+19 % de commandes ; marge par commande en baisse de plus d'un quart ; seuil de 36 %.
Quelle décision demande-je, et comment la mesurer ?	Approuver un test de remise à 10 % sur la moitié des produits avant le 15 novembre.

Le fil conducteur du chapitre tient en une phrase : **ce qui compte n'est pas ce que vous avez trouvé, mais ce que l'autre a compris et décidé**. Il vous reste, dans le volume, à voir **comment fabriquer** les supports : le projet du volume (cahier) réunit un tableau de bord et une présentation pour un décideur.

Pour s'entraîner. Cahier, chapitre 5 : applications 5.1 à 5.7 (trois lectures d'un résultat, traduction du jargon, contraste et daltonisme, recommandation exécutable, plan minuté, fiche de cadrage, compte rendu et désaccord de chiffres) et exercices 5.1 à 5.13.

Points clés

« Un graphique est un argument : il est bon quand le lecteur en tire la bonne décision, pas quand il est beau. »

Ce court chapitre fixe l'essentiel du volume, chapitre par chapitre, puis les idées qui les relient. Le **projet du volume** (un tableau de bord d'une page et une courte présentation pour décider de l'avenir d'un transporteur) et une **auto-évaluation** de quarante questions se trouvent dans le cahier d'exercices.

Pour s'entraîner. Cahier, projet du volume et auto-évaluation : dix étapes, de la fiche de cadrage au message de cinq lignes et aux questions attendues, une variante sur les promotions, puis quarante questions avec corrigés.

Chapitre	Ce que vous devez emporter
----------	----------------------------

Les points clés, chapitre par chapitre

Chapitre	Ce que vous devez emporter
1. Principes de visualisation	On part de la question , pas du graphique : comparer, évoluer, composer, distribuer, relier, classer. L'œil lit mieux une position ou une longueur qu'un angle, une aire ou une couleur. Un bon graphique retire ce qui ne dit rien, ordonne , annote directement et porte un titre qui conclut . ☉ Palettes et accessibilité (le contraste se calcule, le daltonisme se simule, jamais la couleur seule) ; graphiques trompeurs : axe tronqué, double axe, période choisie, pourcentages sans effectifs, camembert à neuf parts.
2. Tableaux de bord	Un tableau de bord est un outil de décision : on part de la décision et de l'utilisateur, on choisit cinq à huit indicateurs, on organise trois niveaux (vue d'ensemble, analyse, détail) et on valide par le test des cinq secondes . Power BI et Tableau reposent sur un modèle en étoile et des mesures ; les mêmes calculs se reproduisent en pandas. ☉ DAX, sécurité par lignes, déploiement ; autres outils (aucun n'est exécuté ici).
3. Visualisation avec Python	matplotlib (figure, axes, trois couches), seaborn (statistique), plotly (interactif, qui ne remplace pas une figure imprimable). Une figure est une fonction (reproductible). ☉ Dash, Streamlit et Shiny : trois modèles d'exécution pour le même tableau de bord ; cartes (sur un plan fictif) : cercles proportionnels (rayon en racine carrée), normalisation par habitant, tableau parfois meilleur qu'une carte.
4. Storytelling et rapports	Un récit de données a un arc (contexte, tension, preuves, résolution) et commence par la réponse (pyramide). Un message par figure, des titres qui concluent, un rapport structuré, des chiffres arrondis et comparés, les limites dites. ☉ Résumé de direction en cinq lignes, page unique, huit diapositives, rapports automatiques (un texte qui sait se taire, des contrôles avant envoi).
5. Présenter à des non-techniciens	Comprendre le public (qui, ce qu'il sait, ce qu'il veut décider), traduire le jargon, dix minutes, trois preuves , une recommandation exécutable, l'incertitude dite en fourchette, les questions et le chiffre qui déplaît préparés, l'erreur découverte en séance avouée. ☉ Recueil des besoins (de « je veux un tableau de bord » à une question), fiche de cadrage ; négocier, collaborer, rester intègre, concilier deux chiffres divergents.

Chapitre	Ce que vous devez emporter
Projet (cahier)	Fiche de cadrage → indicateurs calculés une fois → graphiques choisis par question → tableau de bord d'une page → vérification (contraste, cinq secondes) → scénario chiffré avec incertitude et coût → storyboard de cinq diapositives → message de cinq lignes et questions attendues → limites.

Six idées qui traversent tout le volume

Les six idées.

1. **Le lecteur et la décision d'abord.** Chaque choix (graphique, couleur, ordre, titre, texte) se justifie par la personne qui lit et par ce qu'elle doit décider.
2. **La réponse avant la preuve.** Un graphique, une page, une présentation commencent par ce qu'il faut retenir.
3. **Montrer l'incertitude, pas la cacher.** Intervalles, fourchettes, ce que l'analyse ne dit pas : la crédibilité en dépend.
4. **Ne pas tromper, même sans le vouloir.** Axe, période, échelle, effectifs : on vérifie ce qu'un lecteur pressé comprendrait.
5. **L'accessibilité n'est pas une option.** Contraste, daltonisme, projection : si la moitié de la salle ne lit pas, le message n'existe pas.
6. **Un chiffre, un seul calcul.** Le tableau de bord, le rapport et la présentation reposent sur les mêmes valeurs, calculées une fois.

Et maintenant ?

Vous savez maintenant **choisir et construire une visualisation, concevoir un tableau de bord, raconter une analyse et la présenter** à des personnes qui décident. Le **volume V** va plus loin vers l'**analytique avancée et l'automatisation** : entrepôts de données, ETL, automatisation, introduction au prédictif, reporting de risque. Pour vous entraîner d'ici là, reprenez le projet avec une autre décision (les promotions, les ruptures de stock) et refaites le tableau de bord d'une page.

À retenir, tout simplement. Un enseignement que personne ne comprend n'a aucune valeur : le travail de l'analyste se termine quand **quelqu'un d'autre** a compris et peut décider.